CertyIQ



CertyIQ

Premium exam material

Get certification quickly with the CertyIQ Premium exam material.
Everything you need to prepare, learn & pass your certification exam easily. Lifetime free updates
First attempt guaranteed success.

https://www.CertyIQ.com

About CertyIQ

We here at CertyIQ eventually got enough of the industry's greedy exam paid for. Our team of IT professionals comes with years of experience in the IT industry Prior to training CertIQ we worked in test areas where we observed the horrors of the paywall exam preparation system.

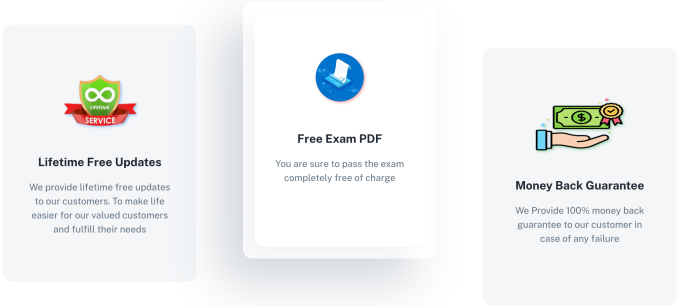
The misuse of the preparation system has left our team disillusioned. And for that reason, we decided it was time to make a difference. We had to make In this way, CertyIQ was created to provide quality materials without stealing from everyday people who are trying to make a living.

Doubt Support

We have developed a very scalable solution using which we are able to solve 400+ doubts every single day with an average rating of 4.8 out of 5.

https://www.certyiq.com

Mail us on - certyiqofficial@gmail.com



John

October 19, 2022

★★★★★

Thanks you so much for your help. I scored 972 in my exam today. More than 90% were from your PDFs!

Dana

September 04, 2022

★★★★★

Thanks a lot for this updated AZ-900 Q&A. I just passed my exam and got 974, I followed both of your Az-900 videos and the 6 PDF, the PDFs are very much valid, all answers are correct. Could you please create a similar video/PDF for DP900, your content/PDF's is really awesome. The team did a really good job. Thank You 😊.

Ahamed Shibly

2 months ago

★★★★★

Customer support is realy fast and helpful, I just finished my exam and this video along with the 6 PDF helped me pass! Definitely recommend getting the PDFs. Thank you!

October 22, 2022

★★★★★

Passed my exam today with 891 marks. Out of 52 questions, 51 were from certyiq PDFs including Contoso case study.
Thank You certyiq team!

Henry Rome

2 months ago

★★★★★

These questions are real and 100 % valid. Thank you so much for your efforts, also your 4 PDFs are awesome, I passed the DP900 exam on 1 Sept. With 968 marks. Thanks a lot, buddy!

Esmaria

2 months ago

★★★★★

Simple easy to understand explanations. To anyone out there wanting to write AZ900, I highly recommend 6 PDF's.Thank you so much, appreciate all your hard work in having such great content. Passed my exam Today - 3 September with 942 score.

Databricks

(Certified Data Engineer Associate)

Certified Data Engineer Associate

Total: **414 Questions**

Link: https://certyiq.com/papers/databricks/certified-data-engineer-associate

Question: 1 CertyIQ

A data organization leader is upset about the data analysis team's reports being different from the data engineering team's reports. The leader believes the siloed nature of their organization's data engineering and data analysis architectures is to blame.
Which of the following describes how a data lakehouse could alleviate this issue?

A. Both teams would autoscale their work as data size evolves
B. Both teams would use the same source of truth for their work
C. Both teams would reorganize to report to the same department
D. Both teams would be able to collaborate on projects in real-time
E. Both teams would respond more quickly to ad-hoc requests

Answer: B

Explanation:

B. Both teams would use the same source of truth for their work.

The core problem described is inconsistent reports caused by data silos between:

Data engineering

Data analysis teams

A data lakehouse architecture solves this by providing:

A unified data platform

A single source of truth

Shared access to the same, consistent datasets

This ensures:

Both teams work on identical data

Reports become consistent and reliable

Eliminates discrepancies caused by separate systems

Why the other options are incorrect:

A. Autoscaling

→ Related to performance, not data consistency.

C. Reorganizing teams

→ Organizational change, not a technical solution.

D. Real-time collaboration

→ Helpful, but doesn't directly fix data inconsistency.

E. Faster ad-hoc responses

→ Performance benefit, not the root issue.

Question: 2 CertyIQ

Which of the following describes a scenario in which a data team will want to utilize cluster pools?

A. An automated report needs to be refreshed as quickly as possible.
B. An automated report needs to be made reproducible.
C. An automated report needs to be tested to identify errors.
D. An automated report needs to be version-controlled across multiple collaborators.
E. An automated report needs to be runnable by all stakeholders.

Answer: A

Explanation:

The correct answer is **A, "An automated report needs to be refreshed as quickly as possible."** Here's why:

Cluster pools significantly reduce the time it takes to start a Databricks cluster. This is because pre-allocated, idle instances are maintained in the pool. When a job, like refreshing an automated report, requires a cluster, it can attach to an instance from the pool almost instantly, bypassing the usual cluster provisioning time. This dramatically improves the speed of the refresh process, meeting the requirement of getting the report refreshed "as quickly as possible."

Option B (reproducible reports) is more about consistent environments and configurations, which can be achieved through Databricks Repos, Delta Lake time travel, and specifying library versions. Option C (testing to identify errors) involves testing frameworks and unit testing best practices within the Databricks environment. Option D (version control) relies on Git integration via Databricks Repos. Option E (runnable by all stakeholders) is more about permissions and access control within Databricks and how you share notebooks/dashboards.

Cluster pools are primarily designed to accelerate cluster startup, focusing on speed rather than the features described in the other options. They offer a performance advantage when cluster initialization time is a bottleneck. Other approaches, like using serverless compute, could also provide rapid startup, but cluster pools are a direct solution offered within the Databricks environment for this specific problem.

Further reading on Databricks Cluster Pools can be found here: https://docs.databricks.com/clusters/instance-pools/index.html and https://www.databricks.com/blog/2020/07/08/introducing-instance-pools-reduce-cluster-start-times-and-cloud-costs.html.

Question: 3 CertyIQ

Which of the following is hosted completely in the control plane of the classic Databricks architecture?

A. Worker node
B. JDBC data source
C. Databricks web application
D. Databricks Filesystem
E. Driver node

Answer:

C

Explanation:

The Databricks architecture separates the control plane and the data plane. The control plane manages the overall Databricks environment, while the data plane is where the data processing occurs.

Option A, Worker nodes, reside in the data plane. They are virtual machines that execute the distributed computations.

Option B, JDBC data sources, are external systems, and their location is independent of the Databricks control plane. JDBC connections are configured within Databricks, but the database itself (the data source) resides outside.

Option C, the Databricks web application, is hosted entirely within the control plane. It is the user interface for interacting with Databricks, managing clusters, notebooks, jobs, and other administrative tasks. The control plane handles the front-end and back-end services to manage the Databricks environment. This includes features like workspace administration, security, user management, and job scheduling, which are all core elements of the control plane's responsibility.

Option D, the Databricks File System (DBFS), while accessible through the control plane, has its underlying storage typically residing within the data plane in cloud storage services such as AWS S3 or Azure Blob Storage. So it is not completely hosted in the control plane.

Option E, the Driver node, although interacting closely with the control plane for job submission and coordination, is part of the data plane as it is responsible for executing the main application code and distributing tasks to worker nodes.

Therefore, only the Databricks web application is fully hosted within the control plane, making option C the correct answer. The control plane is responsible for orchestration, monitoring, and management, and the web application serves as the primary interface for these functions.

Further research:

Databricks Architecture: https://docs.databricks.com/en/getting-started/concepts.html

Question: 4 CertyIQ

Which of the following benefits of using the Databricks Lakehouse Platform is provided by Delta Lake?

A. The ability to manipulate the same data using a variety of languages
B. The ability to collaborate in real time on a single notebook
C. The ability to set up alerts for query failures
D. The ability to support batch and streaming workloads
E. The ability to distribute complex data operations

Answer:

D

Explanation:

The correct answer is D: The ability to support batch and streaming workloads. Delta Lake is a storage layer that brings reliability to data lakes. A key characteristic of Delta Lake is its ACID (Atomicity, Consistency, Isolation, Durability) transaction support. This enables Delta Lake to handle both batch and streaming data

reliably. Batch processing involves large volumes of data processed at once, whereas streaming processes data in near real-time. Delta Lake's ability to handle both types of workloads provides flexibility. Options A, B, and C are features of the Databricks platform but aren't directly enabled by Delta Lake. Option E describes a general function that could be executed on the data itself, not a fundamental benefit provided by Delta Lake. The ability to support both batch and streaming is a direct feature of the Delta Lake storage layer.

Further Research:

Databricks Delta Lake: https://www.databricks.com/product/delta-lake
Delta Lake Documentation: https://docs.databricks.com/delta/index.html

Question: 5 CertyIQ

Which of the following describes the storage organization of a Delta table?

A. Delta tables are stored in a single file that contains data, history, metadata, and other attributes.
B. Delta tables store their data in a single file and all metadata in a collection of files in a separate location.
C. Delta tables are stored in a collection of files that contain data, history, metadata, and other attributes.
D. Delta tables are stored in a collection of files that contain only the data stored within the table.
E. Delta tables are stored in a single file that contains only the data stored within the table.

Answer: C

Explanation:

The correct answer is C because Delta Lake, the technology behind Delta tables, is built on top of cloud storage like AWS S3, Azure Blob Storage, or Google Cloud Storage. It leverages the distributed nature of these object storage systems for scalability and reliability.

Delta Lake doesn't store all data in a single file. Instead, data is partitioned into multiple data files (typically Parquet format), and these files are stored in the underlying cloud storage. This allows for parallel processing and efficient querying.

The "history" and "metadata" are critical components of Delta Lake's functionality, enabling features like time travel, ACID transactions, and schema evolution. This metadata isn't stored within each data file but rather as a transaction log alongside the data files. The transaction log is an ordered record of every operation performed on the Delta table.

Specifically, each commit to a Delta table creates a new entry (a metadata file) in the _delta_log directory, which is typically located alongside the data files in the same storage location. This log tracks changes to the data, including additions, deletions, and updates. These metadata files (often stored in JSON format) contain information about the data files involved in each transaction, enabling Delta Lake to track the table's state over time.

Therefore, a Delta table isn't a single file. It's a directory containing data files and a _delta_log directory containing metadata files. Answer C accurately reflects this distributed storage model, encompassing both the data and the necessary metadata for transaction management and data versioning. The distributed storage enables efficient processing and scaling provided by the underlying cloud storage layer. Options A, B, D, and E are incorrect because they imply a single-file storage or an incomplete view of how metadata is managed within a Delta table.

Here are some authoritative links for further research:

1. **Delta Lake Documentation (Databricks):** https://docs.databricks.com/delta/index.html - This is the

official documentation for Delta Lake and contains detailed information on its architecture and features.

2. **Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores:** https://databricks.com/blog/2019/04/24/delta-lake-high-performance-acid-table-storage-over-cloud-object-stores.html - Databricks blog explaining the architecture and benefits of Delta Lake.
3. **Understanding the Delta Lake Transaction Log:** https://medium.com/datascale/understanding-the-delta-lake-transaction-log-416503e29f64 - A deep dive into how Delta Lake manages transactions using the transaction log.

Question: 6 CertyIQ

Which of the following code blocks will remove the rows where the value in column age is greater than 25 from the existing Delta table my_table and save the updated table?

A. SELECT * FROM my_table WHERE age > 25;
B. UPDATE my_table WHERE age > 25;
C. DELETE FROM my_table WHERE age > 25;
D. UPDATE my_table WHERE age <= 25;
E. DELETE FROM my_table WHERE age <= 25;

Answer: C

Explanation:

The correct answer is C, DELETE FROM my_table WHERE age > 25;. This SQL command directly and efficiently removes rows from the my_table Delta table where the age column's value exceeds 25. Delta Lake, built on Apache Spark, supports standard SQL DELETE operations for modifying data within Delta tables. This functionality is crucial for data governance, regulatory compliance (like GDPR), and overall data quality management.

Option A (SELECT * FROM my_table WHERE age > 25;) only selects rows meeting the condition; it doesn't modify the table. Options B and D use UPDATE, which is designed to modify existing values in specified columns. UPDATE isn't used for row deletion. Option E (DELETE FROM my_table WHERE age <= 25;) is incorrect because it deletes the wrong rows; it removes rows where the age is less than or equal to 25, which is the opposite of what the question asks for.

The DELETE command with the WHERE clause provides the precise filtering logic required to remove the specific rows based on the given condition. Using Delta Lake transactions, this DELETE operation ensures atomicity, consistency, isolation, and durability (ACID) properties. If the DELETE command encounters an error, the entire operation is rolled back, maintaining data integrity. This contrasts sharply with non-ACID data storage systems where partial updates might leave the data in an inconsistent state. Delta Lake also automatically maintains a version history, enabling auditing and rollback to previous versions if needed after the DELETE operation.

For further reading on Delta Lake DELETE operations, refer to the official Databricks documentation:

Databricks Delta Lake Documentation: https://docs.databricks.com/delta/index.html
Delta Lake SQL Commands: https://docs.databricks.com/sql/language-manual/delta-lake-sql.html

These links provide comprehensive information on using Delta Lake and its SQL commands, including DELETE, with transaction guarantees.

Question: 7 CertyIQ

A data engineer has realized that they made a mistake when making a daily update to a table. They need to use Delta time travel to restore the table to a version that is 3 days old. However, when the data engineer attempts to time travel to the older version, they are unable to restore the data because the data files have been deleted. Which of the following explains why the data files are no longer present?

A. The VACUUM command was run on the table
B. The TIME TRAVEL command was run on the table
C. The DELETE HISTORY command was run on the table
D. The OPTIMIZE command was nun on the table
E. The HISTORY command was run on the table

Answer: A

Explanation:

The reason the data engineer is unable to time travel to a version of the Delta table that is 3 days old is most likely due to the VACUUM command having been run on the table. The VACUUM command removes data files from the Delta Lake table's underlying storage that are no longer needed by the Delta Lake transaction log. Its primary purpose is to reduce storage costs and improve query performance by eliminating outdated files. By default, Delta Lake retains historical versions of the data for 30 days, allowing users to time travel to any point within that window. However, if the VACUUM command is executed with a retention period shorter than 3 days (or if the default 7-day retention is already applied automatically), the data files required for the 3-day-old version would have been physically removed from the storage.

The TIME TRAVEL command itself is used to restore a table to a previous version, so it cannot be the cause. The DELETE HISTORY command, while related to removing versions, is not a standard Delta Lake command. Optimizing the table with the OPTIMIZE command reorganizes data files for better performance but doesn't inherently delete historical data. Similarly, the HISTORY command simply displays the table's history and doesn't modify or remove any data files. Therefore, VACUUM, with its function of physically removing data files based on a retention policy, directly explains why the data files are no longer available for time travel to a version older than the retention period. If VACUUM has been executed with a retention interval less than 3 days, restoring the table to a version 3 days old will be impossible, as the data files needed for that version are already purged.

Authoritative Links:

Databricks Delta Lake Vacuum: https://docs.databricks.com/delta/optimizations/vacuum.html
Databricks Delta Lake Time Travel: https://docs.databricks.com/delta/versioning.html

Question: 8 CertyIQ

Which of the following Git operations must be performed outside of Databricks Repos?

A. Commit
B. Pull
C. Push
D. Clone
E. Merge

Answer: E

Explanation:

The correct answer is E, Merge. Here's why:

Databricks Repos provide a Git-integrated development environment directly within the Databricks workspace. This allows data engineers to perform common Git operations such as committing (A), pulling (B), pushing (C), and even cloning (D) within the Databricks UI. These actions are directly supported and streamlined by the Databricks Repos functionality.

However, the merge operation is typically more complex and involves resolving potential conflicts between different branches. While Databricks Repos allows you to create merge requests or pull requests, the actual process of resolving conflicts and completing the merge often requires a more comprehensive Git client or integrated development environment (IDE) outside of Databricks. This is because resolving merge conflicts is usually best handled with tools offering visual diffing and conflict resolution capabilities, something not natively built into Databricks Repos.

Specifically, while Databricks repos can visually show you what code differences exist, the tool to resolve those conflicts can be cumbersome. It may involve downloading the impacted file, making the changes locally in a more robust IDE, and uploading the changed file back. It is more common to resolve conflicts in a common git tool like VS Code, or using a code repository like Github or Gitlab. Databricks repos focuses on facilitating collaboration and version control within the Databricks environment, but defers conflict resolution to dedicated Git tools. Therefore, the actual merge operation with conflict resolution is often, and more efficiently, handled externally.

In summary, commit, pull, push, and clone are core operations handled directly by Databricks Repos, while merge, particularly when involving conflict resolution, is usually performed externally for a more seamless workflow.

Further reading:

Databricks Repos Documentation: https://docs.databricks.com/repos/index.html (Explains core functionality)
Git Documentation: https://git-scm.com/doc (General Git concepts)
Github Documentation: https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/addressing-merge-conflicts/resolving-a-merge-conflict-on-github

Question: 9 CertyIQ

Which of the following data lakehouse features results in improved data quality over a traditional data lake?

A. A data lakehouse provides storage solutions for structured and unstructured data.
B. A data lakehouse supports ACID-compliant transactions.
C. A data lakehouse allows the use of SQL queries to examine data.
D. A data lakehouse stores data in open formats.
E. A data lakehouse enables machine learning and artificial Intelligence workloads.

Answer: B

Explanation:

The correct answer is **B. A data lakehouse supports ACID-compliant transactions.**

Here's why:

Data quality directly benefits from ACID (Atomicity, Consistency, Isolation, Durability) transactions. ACID properties ensure that data operations are reliable and consistent, preventing data corruption and maintaining data integrity.

Atomicity: Guarantees that a transaction is treated as a single, indivisible unit of work. Either all changes within the transaction are applied, or none are. This prevents partial updates that could lead to inconsistent data.
Consistency: Ensures that a transaction moves the database from one valid state to another. Constraints, rules, and validations are enforced, preventing invalid data from being written.
Isolation: Dictates that concurrent transactions do not interfere with each other. Each transaction operates as if it were the only transaction running on the system, preventing data corruption from concurrent updates.
Durability: Ensures that once a transaction is committed, it is permanently recorded and will survive system failures (e.g., power outages, crashes).

Traditional data lakes often lack ACID transaction support, leading to "dirty reads" and inconsistent data during concurrent writes or failures. Data lakehouses, by incorporating ACID transactions, provide a more reliable and consistent foundation for data, thus improving data quality. Features like Delta Lake are used to provide the ACID properties to Databricks Data Lakehouse.

The other options are less directly related to improved data quality compared to a traditional data lake, although they provide benefits to the general functionality. For example, storing data in open formats or using SQL are also typical of a data lake, and they don't ensure the correctness of the data being stored.

Supporting Resources:

Databricks Documentation on Delta Lake (ACID Transactions): https://www.databricks.com/product/delta-lake
ACID Transactions in Data Warehousing: https://www.ibm.com/docs/en/db2/11.5?topic=concepts-acid-properties

Question: 10 CertyIQ

A data engineer needs to determine whether to use the built-in Databricks Notebooks versioning or version their project using Databricks Repos.
Which of the following is an advantage of using Databricks Repos over the Databricks Notebooks versioning?

A. Databricks Repos automatically saves development progress
B. Databricks Repos supports the use of multiple branches
C. Databricks Repos allows users to revert to previous versions of a notebook
D. Databricks Repos provides the ability to comment on specific changes
E. Databricks Repos is wholly housed within the Databricks Lakehouse Platform

Answer: B

Explanation:

The correct answer is B: Databricks Repos supports the use of multiple branches. Here's a detailed justification:

Databricks Notebooks versioning offers a basic level of version control, primarily focused on tracking changes within a single notebook. While it allows users to revert to previous versions (option C) and automatically saves progress, it lacks the robust capabilities of a full-fledged Git-based version control system. Databricks Repos, on the other hand, integrates directly with Git providers like GitHub, GitLab, Azure DevOps, or

Bitbucket. This integration enables a full suite of version control features.

The key advantage highlighted in option B, "Databricks Repos supports the use of multiple branches," is crucial for collaborative development workflows. Branching allows data engineers to work on new features or bug fixes in isolation without disrupting the main codebase (e.g., the main or master branch). This is vital for larger projects and teams where multiple developers are working concurrently. Each developer or feature can have its own branch, promoting parallel development and reducing the risk of conflicts. Once changes are tested and approved, the branch can be merged back into the main branch.

Databricks Notebooks versioning does not provide branch management; changes are linear and applied directly to the existing notebook version. While both methods enable reverting (C), Databricks Repos via Git offers a more granular and auditable change history. Neither Notebook Versioning nor Repos provide in-platform commenting on specific changes directly as described in option D. Repos depend on the features offered in the connected git provider, which may include commenting features. Automatic saves (A) also apply to built-in Notebook versioning. Therefore, branching is the distinguishing advantage of Repos. Repos, by leveraging Git, aren't wholly housed within the Databricks Lakehouse Platform (E) but rather integrate with external Git repositories, making it a Git-based version control system integrated into Databricks.

For more information, refer to the official Databricks documentation:

Databricks Repos
Notebook Versioning

Question: 11 CertyIQ

A data engineer has left the organization. The data team needs to transfer ownership of the data engineer's Delta tables to a new data engineer. The new data engineer is the lead engineer on the data team.
Assuming the original data engineer no longer has access, which of the following individuals must be the one to transfer ownership of the Delta tables in Data Explorer?

A. Databricks account representative
B. This transfer is not possible
C. Workspace administrator
D. New lead data engineer
E. Original data engineer

Answer: C

Explanation:

The correct answer is C: Workspace administrator. Here's why:

Delta tables in Databricks, like other securable objects, have an owner. Ownership determines who has full control over the object, including the ability to grant permissions to others and, crucially, transfer ownership. When an employee leaves, their access is typically revoked, making them unable to transfer ownership. The new lead data engineer, while designated for future ownership, doesn't automatically inherit the privileges needed to perform the transfer unless they already possess them. Databricks account representatives lack the necessary workspace-level privileges to directly modify object ownership.

The key to resolving this lies with the workspace administrator. Workspace administrators possess elevated privileges within a Databricks workspace. They have the authority to manage users, groups, and, importantly, control access to data objects, including Delta tables. Because of these elevated permissions, a workspace administrator can reassign ownership of the Delta tables to the new lead data engineer. The command used would be ALTER TABLE <table_name> OWNER TO <new_owner>. This command requires the MODIFY

permission on the table, which is implicitly held by the table owner or a workspace administrator.

Transferring ownership is a crucial step in ensuring continued data governance and operational continuity. By assigning ownership to the appropriate person, the organization maintains control over its data assets and can ensure that the new lead data engineer has the necessary permissions to manage and maintain the tables. Thus, the workspace administrator is the only individual with the required privilege after the original owner has departed the organization.

Databricks Documentation on PrivilegesDatabricks Documentation on Workspace Admin

Question: 12 CertyIQ

A data analyst has created a Delta table sales that is used by the entire data analysis team. They want help from the data engineering team to implement a series of tests to ensure the data is clean. However, the data engineering team uses Python for its tests rather than SQL.
Which of the following commands could the data engineering team use to access sales in PySpark?

A. SELECT * FROM sales
B. There is no way to share data between PySpark and SQL.
C. spark.sql("sales")D. spark.delta.table("sales")
E. spark.table("sales")

Answer: E

Explanation:

E. spark.table("sales")

A Delta table created in Databricks (or any Spark environment using Delta Lake) is stored in the Metastore.

This means the table can be accessed both:

Through SQL:

SELECT * FROM sales;

And through PySpark using the SparkSession API.

To load a table in PySpark, the correct command is:

df = spark.table("sales")

This returns the Delta table as a DataFrame that the data engineering team can use for testing, transformations, or validation in Python.

Why the Other Options Are Incorrect:

A. SELECT * FROM sales

This is SQL only, not PySpark. It returns a result within the SQL interpreter, not a PySpark DataFrame.

B. "No way to share data"

Incorrect. Spark provides full interoperability between SQL and PySpark via the Metastore.

C. spark.sql("sales")

Incorrect. spark.sql() expects a full SQL query. The correct usage would require: spark.sql("SELECT * FROM sales").

D. spark.delta.table("sales")

Not valid syntax. The correct form would be: deltaTable = DeltaTable.forName(spark, "sales"), which returns a DeltaTable object (not a DataFrame).

Question: 13 CertyIQ

Which of the following commands will return the location of database customer360?

A. DESCRIBE LOCATION customer360;
B. DROP DATABASE customer360;
C. DESCRIBE DATABASE customer360;
D. ALTER DATABASE customer360 SET DBPROPERTIES ('location' = '/user' ;
E. USE DATABASE customer360;

Answer: C

Explanation:

The correct command to retrieve the location of a database in Databricks is DESCRIBE DATABASE customer360;. This command provides detailed metadata about the specified database, including its location. Options A, B, D, and E perform different operations. DESCRIBE LOCATION is not a valid command in Databricks SQL for retrieving database metadata. DROP DATABASE removes the database entirely, which is destructive and not intended for information retrieval. ALTER DATABASE modifies the database properties; while it can set the location, it doesn't retrieve it, and the example provided sets a new property rather than querying the existing one. USE DATABASE simply sets the current database context for subsequent operations.

DESCRIBE DATABASE provides a comprehensive overview of the database's properties. Databricks leverages a metastore (either the default Hive metastore or Unity Catalog) to manage the metadata for databases, tables, and other data assets. The database location is a crucial piece of this metadata, indicating where the database's managed tables and other associated data are physically stored in cloud storage (e.g., AWS S3, Azure Data Lake Storage, or Google Cloud Storage). The DESCRIBE DATABASE command retrieves this metadata from the metastore. The output includes the database name, description (if any), location (specifying the storage path), and other relevant database properties. By parsing the output of the DESCRIBE DATABASE command, you can programmatically determine the database's storage location.

For further information on Databricks SQL commands and database management, you can refer to the official Databricks documentation:

Databricks SQL Reference: https://docs.databricks.com/sql/language-manual/index.html
DESCRIBE DATABASE Command: (This would be in the SQL Reference above, search for "DESCRIBE DATABASE")
Databricks Unity Catalog: https://docs.databricks.com/data-governance/unity-catalog/index.html (for understanding metastore concepts).

Question: 14 CertyIQ

A data engineer wants to create a new table containing the names of customers that live in France.
They have written the following command:

```
CREATE TABLE customersInFrance
_____ AS
SELECT id,
       firstName,
       lastName,
FROM customerLocations
WHERE country = 'FRANCE';
```

A senior data engineer mentions that it is organization policy to include a table property indicating that the new table includes personally identifiable information (PII).
Which of the following lines of code fills in the above blank to successfully complete the task?

A. There is no way to indicate whether a table contains PII.
B. "COMMENT PII"
C. TBLPROPERTIES PII
D. COMMENT "Contains PII"
E. PII

Answer: D

Explanation:

D. COMMENT "Contains PII"

In SQL (especially when working with data platforms like Databricks, Hive, or Spark SQL), metadata can be added to tables using the COMMENT clause to describe the contents or purpose of the table.

If your organization requires tagging tables that contain Personally Identifiable Information (PII), adding a comment like "Contains PII" is a standard and accepted way to do it.

Question: 15 CertyIQ

Which of the following benefits is provided by the array functions from Spark SQL?

A. An ability to work with data in a variety of types at once
B. An ability to work with data within certain partitions and windows
C. An ability to work with time-related data in specified intervals
D. An ability to work with complex, nested data ingested from JSON files
E. An ability to work with an array of tables for procedural automation

Answer: D

Explanation:

The correct answer is D because Spark SQL's array functions are specifically designed to handle complex, nested data structures, commonly encountered when ingesting data from sources like JSON files. JSON often contains arrays of values or nested objects, which can be challenging to process with traditional SQL. Array

functions provide the means to manipulate and extract information from these array structures, such as accessing elements by index, transforming array contents, and performing operations on array elements.

Options A, B, and C are less relevant to the primary purpose of array functions. While Spark SQL can handle a variety of data types (A), that's not the exclusive domain of array functions. Window functions (B) handle calculations across sets of rows, not array manipulation, and date/time functions (C) are designed for time-series data, not array processing. Option E refers to handling an array of tables, which can be accomplished via other Spark SQL features like UNION or dynamic views, but doesn't directly represent the capability of array functions.

Therefore, the most direct and accurate benefit offered by Spark SQL's array functions is the ability to efficiently process and transform complex, nested data structures, especially those originating from JSON or similar semi-structured formats. They simplify the task of working with these intricate data structures and provide a more accessible way to extract insights from the data.

Reference:

Databricks documentation on array functions: https://docs.databricks.com/sql/language-manual/functions/array_contains.html (Example - illustrates usage)
Apache Spark SQL documentation: https://spark.apache.org/docs/latest/sql-ref.html (Provides a complete overview of all SQL functions, including array functions)

Question: 16 CertyIQ

Which of the following commands can be used to write data into a Delta table while avoiding the writing of duplicate records?

A. DROP
B. IGNORE
C. MERGE
D. APPEND
E. INSERT

Answer: C

Explanation:

The correct answer is **C. MERGE**. Here's a detailed justification:

The MERGE command in Delta Lake is specifically designed for performing upserts - combining update and insert operations based on a specified condition. This is precisely what's needed to avoid writing duplicate records. The MERGE command essentially allows you to specify a source dataset (the new data you're trying to write) and a target Delta table. You then define a condition that determines when a row from the source matches a row in the target.

If a match is found based on this condition, you can specify how to update the existing row in the target table (the "update" part of upsert). If no match is found, you can specify how to insert the new row from the source into the target table (the "insert" part of upsert).

By using MERGE with an appropriate matching condition (e.g., based on a unique key or combination of fields), you can effectively prevent the creation of duplicate records. If a record with the same key already exists, it will be updated; otherwise, a new record will be inserted. This ensures that your Delta table maintains data integrity.

Options A, B, D, and E are incorrect:

DROP: This command is used to delete entire tables or partitions, not to avoid writing duplicates during data ingestion.
IGNORE: While some database systems might have an IGNORE option, it's not a standard Delta Lake or Spark command for handling duplicate records. It wouldn't provide the necessary logic for an upsert operation.
APPEND: This command simply adds new data to the end of the table without checking for duplicates. It's the opposite of what we need.
INSERT: Similar to APPEND, INSERT adds new rows without any de-duplication logic. Using INSERT alone is a guaranteed way to potentially introduce duplicate records.

In summary, MERGE provides the fine-grained control needed to update existing records or insert new ones only when necessary, avoiding duplication and maintaining data integrity within your Delta table.

Further reading on Delta Lake MERGE can be found here:

Databricks Documentation: MERGE INTO

Question: 17 CertyIQ

A data engineer needs to apply custom logic to string column city in table stores for a specific use case. In order to apply this custom logic at scale, the data engineer wants to create a SQL user-defined function (UDF).
Which of the following code blocks creates this SQL UDF?

A.
```
CREATE FUNCTION combine_nyc(city STRING)
RETURNS STRING
RETURN CASE
  WHEN city = "brooklyn" THEN "new york"
  ELSE city
END;
```

B.
```
CREATE UDF combine_nyc(city STRING)
RETURNS STRING
CASE
  WHEN city = "brooklyn" THEN "new york"
  ELSE city
END;
```

C.
```
CREATE UDF combine_nyc(city STRING)
RETURN CASE
  WHEN city = "brooklyn" THEN "new york"
  ELSE city
END;
```

D.
```
CREATE FUNCTION combine_nyc(city STRING)
RETURN CASE
  WHEN city = "brooklyn" THEN "new york"
  ELSE city
END;
```

E.

```
CREATE UDF combine_nyc(city STRING)
RETURNS STRING
RETURN CASE
  WHEN city = "brooklyn" THEN "new york"
  ELSE city
END;
```

Answer: A

Explanation:

```
CREATE FUNCTION combine_nyc(city STRING)
RETURNS STRING
RETURN CASE
  WHEN city = "brooklyn" THEN "new york"
  ELSE city
END;
```

CREATE FUNCTION combine_nyc(city STRING): This defines a new function named combine_nyc that accepts a single input argument named city, which must be a string.

RETURNS STRING: This specifies that the function will output a string value.

RETURN CASE ... END: This is a conditional logic block (similar to an if-else statement in other programming languages):

WHEN city = "brooklyn" THEN "new york": If the input string is exactly "brooklyn", the function returns "new york".

ELSE city: For any other input (e.g., "queens", "manhattan", or "bronx"), the function returns the original input string unchanged.

Question: 18 CertyIQ

A data analyst has a series of queries in a SQL program. The data analyst wants this program to run every day. They only want the final query in the program to run on Sundays. They ask for help from the data engineering team to complete this task.
Which of the following approaches could be used by the data engineering team to complete this task?

A. They could submit a feature request with Databricks to add this functionality.

B. They could wrap the queries using PySpark and use Python's control flow system to determine when to run the final query.

C. They could only run the entire program on Sundays.

D. They could automatically restrict access to the source table in the final query so that it is only accessible on Sundays.

E. They could redesign the data model to separate the data used in the final query into a new table.

Answer: B

Explanation:

The correct answer is B. Here's why:

Option B leverages the flexibility of PySpark combined with Python's control flow capabilities to precisely meet the requirement. The data engineering team can encapsulate the entire SQL program within a PySpark script. Within this script, Python's datetime module can be used to determine the current day of the week. Based on this, a conditional statement (e.g., an if statement checking if the day is Sunday) can be used to decide whether to execute the final SQL query or skip it. The other queries will execute regardless of the day, fulfilling the requirement of running daily. This approach ensures that the entire process is automated, scheduled, and maintains the initial SQL logic provided by the data analyst.

Option A, submitting a feature request, is not a practical immediate solution. It relies on Databricks implementing the feature, which is uncertain and time-consuming. Option C, running the entire program only on Sundays, contradicts the requirement that the initial queries must run every day. Option D, restricting table access, is a poor design choice. It is not recommended to control query execution through data access restrictions, which would be very difficult to manage and could impact other processes. Option E, redesigning the data model, is an unnecessarily complex solution for this requirement. Redesigning data models requires significant effort, testing, and potential changes to other parts of the system. It is not justified by the simple requirement of conditionally executing one query.

Therefore, using PySpark's ability to run SQL queries and Python's control flow mechanisms is the most efficient and direct way to automate the analyst's request without causing unnecessary disruption or adding complex overhead to the system.

Further research:

PySpark SQL: https://spark.apache.org/docs/latest/sql-getting-started.html
Python datetime module: https://docs.python.org/3/library/datetime.html

Question: 19 CertyIQ

A data engineer runs a statement every day to copy the previous day's sales into the table transactions. Each day's sales are in their own file in the location "/transactions/raw".
Today, the data engineer runs the following command to complete this task:

```
COPY INTO transactions
FROM "/transactions/raw"
FILEFORMAT = PARQUET;
```

After running the command today, the data engineer notices that the number of records in table transactions has not changed.
Which of the following describes why the statement might not have copied any new records into the table?

A. The format of the files to be copied were not included with the FORMAT_OPTIONS keyword.
B. The names of the files to be copied were not included with the FILES keyword.
C. The previous day's file has already been copied into the table.
D. The PARQUET file format does not support COPY INTO.
E. The COPY INTO statement requires the table to be refreshed to view the copied rows.

Answer:

C

Explanation:

The previous day's file has already been copied into the table.

Reference:

https://docs.databricks.com/ingestion/copy-into/tutorial-notebook.html

Question: 20 CertyIQ

A data engineer needs to create a table in Databricks using data from their organization's existing SQLite database.
They run the following command:

```
CREATE TABLE jdbc_customer360
USING _____
OPTIONS (
  url "jdbc:sqlite:/customers.db",
  dbtable "customer360"
)
```

Which of the following lines of code fills in the above blank to successfully complete the task?

A. org.apache.spark.sql.jdbc
B. autoloader
C. DELTA
D. sqlite
E. org.apache.spark.sql.sqlite

Answer: A

Explanation:

A. org.apache.spark.sql.jdbc

To read from or write to a JDBC-compatible database (such as MySQL, PostgreSQL, SQL Server, etc.) in Apache Spark, you use the format("jdbc") API.

Question: 21 CertyIQ

A data engineering team has two tables. The first table march_transactions is a collection of all retail transactions in the month of March. The second table april_transactions is a collection of all retail transactions in the month of April. There are no duplicate records between the tables.
Which of the following commands should be run to create a new table all_transactions that contains all records from march_transactions and april_transactions without duplicate records?

A. CREATE TABLE all_transactions AS
SELECT * FROM march_transactions
INNER JOIN SELECT * FROM april_transactions;

B. CREATE TABLE all_transactions AS
SELECT * FROM march_transactions
UNION SELECT * FROM april_transactions;

C. CREATE TABLE all_transactions AS
SELECT * FROM march_transactions

OUTER JOIN SELECT * FROM april_transactions;

D. CREATE TABLE all_transactions AS
SELECT * FROM march_transactions
INTERSECT SELECT * from april_transactions;

E. CREATE TABLE all_transactions AS
SELECT * FROM march_transactions
MERGE SELECT * FROM april_transactions;

Answer: B

Explanation:

The correct command to combine all records from march_transactions and april_transactions into a new table all_transactions without duplicates is option B: CREATE TABLE all_transactions AS SELECT *FROM march_transactions UNION SELECT* FROM april_transactions;

Here's why:

UNION: The UNION operator in SQL is specifically designed to combine the result-sets of two or more SELECT statements. Importantly, UNION automatically removes duplicate rows from the final result, ensuring each record is unique. This directly addresses the requirement of creating a table without duplicate records from the two source tables.

INNER JOIN, OUTER JOIN, INTERSECT, MERGE: These other options are not suitable for combining records from two separate tables in the way described in the prompt.

INNER JOIN and OUTER JOIN are used to combine rows from two tables based on a related column. Since the goal is to combine all rows regardless of a shared key, these are incorrect.
INTERSECT returns only the rows that are present in both tables, not all rows from both.
MERGE is typically used for complex data manipulations, such as synchronizing two tables based on a condition, and it is not a standard SQL command in all database systems. Databricks may support it through variations in spark SQL, but it is not the simplest solution.

CREATE TABLE AS SELECT (CTAS): The CREATE TABLE all_transactions AS SELECT ... syntax allows creating a new table based on the results of a SELECT query. This is an efficient way to directly populate the new table with the combined and de-duplicated data.

Therefore, the UNION operator within a CREATE TABLE AS SELECT statement is the most appropriate way to create the all_transactions table with unique records from both input tables.

Supporting Documentation:

Databricks SQL Reference - UNION: https://docs.databricks.com/sql/language-manual/sql-ref-syntax-qry-select-union.html
Databricks SQL Reference - CREATE TABLE AS SELECT: https://docs.databricks.com/sql/language-manual/sql-ref-syntax-ddl-create-table-as-select.html

Question: 22 CertyIQ

A data engineer only wants to execute the final block of a Python program if the Python variable day_of_week is equal to 1 and the Python variable review_period is True.
Which of the following control flow statements should the data engineer use to begin this conditionally executed code block?

A. if day_of_week = 1 and review_period:

B. if day_of_week = 1 and review_period = "True":

C. if day_of_week == 1 and review_period == "True":

D. if day_of_week == 1 and review_period:

E. if day_of_week = 1 & review_period: = "True":

Answer: D

Explanation:

The correct answer is D: if day_of_week == 1 and review_period:.

The if statement in Python is used to execute a block of code only if a certain condition is met. This condition is evaluated as a boolean (True or False). In this scenario, the data engineer wants the code to execute only when day_of_week is equal to 1 and review_period is True.

Let's break down why other options are incorrect:

A. if day_of_week = 1 and review_period:: The = operator is an assignment operator, not a comparison operator. Using it in an if statement like this would attempt to assign the value 1 to day_of_week, which is syntactically incorrect and will raise an error. You need the comparison operator == to check for equality.
B. if day_of_week = 1 and review_period = "True":: This suffers from the same assignment error as option A. Additionally, even if corrected to use ==, comparing a boolean variable review_period to the string "True" is generally incorrect and will likely not produce the desired result. A boolean variable is already True or False.
C. if day_of_week == 1 and review_period == "True":: Correctly uses the == operator, however this incorrectly compares a boolean variable review_period to a string "True". This will only evaluate True if review_period is the string value "True" which is unlikely to be the case.
E. if day_of_week = 1 & review_period: = "True":: Contains both the assignment error (=) and uses the bitwise AND operator (&) instead of the logical AND operator (and). Additionally, has the same boolean comparison issue as option B and C.

Option D, if day_of_week == 1 and review_period:, correctly uses the equality operator == to compare day_of_week to 1. It also correctly checks if review_period is True. Because review_period is already a boolean variable (either True or False), simply stating its name after the and implicitly checks if it's True. The whole condition will only be true if both conditions are satisfied.

In short, using and ensures that both conditions, day_of_week == 1 and review_period, must be True for the conditional code block to execute.

Relevant links for more information:

Python if statement: https://www.w3schools.com/python/python_conditions.asp
Python comparison operators: https://www.w3schools.com/python/python_operators.asp
Python boolean data type: https://www.w3schools.com/python/python_booleans.asp

Question: 23 CertyIQ

A data engineer is attempting to drop a Spark SQL table my_table. The data engineer wants to delete all table metadata and data.
They run the following command:
DROP TABLE IF EXISTS my_table -
While the object no longer appears when they run SHOW TABLES, the data files still exist.
Which of the following describes why the data files still exist and the metadata files were deleted?

A. The table's data was larger than 10 GB

B. The table's data was smaller than 10 GB
C. The table was external
D. The table did not have a location
E. The table was managed

Answer: C

Explanation:

The reason the data files still exist despite dropping the table is that the table was an external table. In Spark SQL, when you drop a managed table (also known as an internal table), Spark removes both the metadata and the underlying data from the metastore and storage location, respectively. However, when you drop an external table, only the metadata is removed from the metastore. The data files themselves, residing in the external storage location (e.g., cloud object storage like AWS S3 or Azure Blob Storage), remain untouched. The DROP TABLE command only removes the table's definition from the metastore, disconnecting the table name from the actual data location. The IF EXISTS clause simply prevents an error if the table does not exist, and it doesn't affect the behavior regarding data deletion. The size of the data (options A and B) is irrelevant. Option D, "The table did not have a location," is contradictory, as external tables are defined by their location, unlike managed tables that default to the metastore's managed location. In summary, the defining characteristic of external tables is that their data lives outside the control of the metastore and is unaffected by table dropping operations.

Further research can be done here:https://spark.apache.org/docs/latest/sql-data-sources-hive-tables.htmlhttps://docs.databricks.com/en/sql/language-manual/sql-ref-syntax-ddl-drop-table.html

Question: 24 CertyIQ

A data engineer wants to create a data entity from a couple of tables. The data entity must be used by other data engineers in other sessions. It also must be saved to a physical location.
Which of the following data entities should the data engineer create?

A. Database
B. Function
C. View
D. Temporary view
E. Table

Answer: E

Explanation:

The correct answer is **E. Table**. Here's why:

The requirement is to create a data entity accessible across multiple sessions and persistently stored in a physical location. Let's examine each option:

A. Database: A database is a container for schemas, tables, views, and functions. While necessary for organizing data, it doesn't directly represent a data entity derived from other tables. It's more like the overall storage structure.
B. Function: Functions are reusable code blocks, not data entities derived from tables. They perform operations, not store data.

C. View: Views are virtual tables based on the result-set of a query. While they can combine data from multiple tables, they are **not physically stored**. Views are simply stored queries.
D. Temporary View: Temporary views are session-scoped. This means they exist only for the duration of the Databricks session in which they were created. This directly contradicts the requirement of being accessible across multiple sessions.
E. Table: Tables are physical representations of data stored in a structured format. They persist across sessions and are stored in a defined physical location (e.g., cloud storage like AWS S3 or Azure Data Lake Storage). A table can be created by selecting data from other tables, fulfilling the data engineer's requirement to combine data. Thus, other data engineers can access it in other sessions. This makes it the correct option. A managed table stores the data and metadata in the metastore while an unmanaged table only stores metadata in the metastore, and references the physical storage location that is manually defined.

In summary, only a table provides the required combination of persistence, cross-session availability, and physical storage for the derived data entity.

Further Research:

Databricks Tables: https://docs.databricks.com/en/tables/index.html
Databricks Views: https://docs.databricks.com/en/sql/language-manual/sql-ref-syntax-aux-show-views.html

Question: 25 CertyIQ

A data engineer is maintaining a data pipeline. Upon data ingestion, the data engineer notices that the source data is starting to have a lower level of quality. The data engineer would like to automate the process of monitoring the quality level.
Which of the following tools can the data engineer use to solve this problem?

A. Unity Catalog
B. Data Explorer
C. Delta Lake
D. Delta Live Tables
E. Auto Loader

Answer: D

Explanation:

The correct answer is **D. Delta Live Tables (DLT)**. Here's why:

Delta Live Tables is the best choice for automating data quality monitoring in a Databricks data pipeline. DLT allows you to define data quality constraints directly within your data pipeline code using expectations. Expectations are boolean expressions that, when evaluated against your data, determine whether the data meets a certain quality threshold. DLT automatically monitors these expectations during pipeline execution and tracks data quality metrics over time. This allows data engineers to create a continuous data quality monitoring system without manual intervention.

DLT provides features like expect and expect_or_drop that can enforce data quality. You can set actions based on expectation results, such as dropping records that fail a quality check or simply recording the violations. This active monitoring and enforcement of data quality rules during the transformation process ensures that only clean, reliable data proceeds through the pipeline.

Furthermore, DLT's UI provides detailed metrics and dashboards that visualize data quality trends. You can readily identify data quality issues, track improvements after implementing fixes, and set alerts to be notified when data quality metrics fall below acceptable levels. The pipeline's lineage and data flow are visualized,

making it easier to understand the impact of data quality issues.

While other options might play roles in a data pipeline, they don't directly provide the automated data quality monitoring features of DLT. Unity Catalog (A) focuses on data governance and discovery. Data Explorer (B) is more for interactive data browsing and exploration. Delta Lake (C) provides reliable storage but doesn't actively monitor data quality during pipeline execution. Auto Loader (E) efficiently ingests data but doesn't enforce or monitor data quality.

Therefore, Delta Live Tables is purpose-built for automating data quality monitoring within a Databricks data pipeline, making it the optimal solution.

Relevant links:

Delta Live Tables documentation
Declarative Data Quality with Delta Live Tables

Question: 26 CertyIQ

A Delta Live Table pipeline includes two datasets defined using STREAMING LIVE TABLE. Three datasets are defined against Delta Lake table sources using LIVE TABLE.
The table is configured to run in Production mode using the Continuous Pipeline Mode.
Assuming previously unprocessed data exists and all definitions are valid, what is the expected outcome after clicking Start to update the pipeline?

A. All datasets will be updated at set intervals until the pipeline is shut down. The compute resources will persist to allow for additional testing.

B. All datasets will be updated once and the pipeline will persist without any processing. The compute resources will persist but go unused.

C. All datasets will be updated at set intervals until the pipeline is shut down. The compute resources will be deployed for the update and terminated when the pipeline is stopped.

D. All datasets will be updated once and the pipeline will shut down. The compute resources will be terminated.

E. All datasets will be updated once and the pipeline will shut down. The compute resources will persist to allow for additional testing.

Answer: C

Explanation:

The correct answer is C because of the combination of STREAMING LIVE TABLE, LIVE TABLE, Continuous pipeline mode, and Production mode.

Let's break down why:

STREAMING LIVE TABLE: This keyword signifies datasets that continuously ingest and process new data as it arrives. They are designed for incremental updates.
LIVE TABLE: These datasets are also updated by the DLT pipeline, but may or may not be updated incrementally.
Continuous Pipeline Mode: This mode instructs the pipeline to run continuously, processing data as it becomes available. In contrast to Triggered mode, which runs once and then shuts down, Continuous mode aims for near real-time data processing.
Production Mode: In production mode, the DLT pipeline is optimized for efficiency and cost. This means compute resources are deployed when needed for updates and then terminated when the pipeline is stopped. This differs from development mode, where compute may persist for faster iterations.

Therefore, in this scenario, both the streaming and non-streaming tables will be updated continuously at set

intervals as governed by the DLT pipeline's internal mechanisms. Because it is production mode, the infrastructure will shut down when you hit the stop button.

Here are some relevant resources to enhance your understanding:

Delta Live Tables Modes: https://docs.databricks.com/workflows/delta-live-tables/delta-live-tables-concepts.html#pipeline-modes
STREAMING LIVE TABLE: https://docs.databricks.com/workflows/delta-live-tables/delta-live-tables-syntax.html#streaming-live-table
Delta Live Tables Concepts: https://docs.databricks.com/workflows/delta-live-tables/delta-live-tables-concepts.html

Question: 27 CertyIQ

In order for Structured Streaming to reliably track the exact progress of the processing so that it can handle any kind of failure by restarting and/or reprocessing, which of the following two approaches is used by Spark to record the offset range of the data being processed in each trigger?

A. Checkpointing and Write-ahead Logs
B. Structured Streaming cannot record the offset range of the data being processed in each trigger.
C. Replayable Sources and Idempotent Sinks
D. Write-ahead Logs and Idempotent Sinks
E. Checkpointing and Idempotent Sinks

Answer: A

Explanation:

The correct answer is **A. Checkpointing and Write-ahead Logs**. Here's a detailed justification:

Structured Streaming's fault tolerance and ability to guarantee exactly-once or at-least-once processing relies heavily on tracking the progress of data ingestion and processing in a reliable manner. This involves recording the offset range of data processed in each trigger. This enables the system to recover from failures and resume processing from the correct point.

Checkpointing: Checkpointing involves periodically saving the state of the streaming application to a reliable storage system, such as cloud storage (e.g., S3, ADLS). This state includes information about the processed offsets, aggregations, and any other relevant information. In case of a failure, the application can restart from the last checkpoint, effectively rewinding the state to a known good point. Checkpointing is used to store metadata about the streaming query itself, including information about the progress of data consumption.
Write-Ahead Logs (WAL): The Write-Ahead Log is a persistent record of all changes made to the state. Before any changes are applied, they are first written to the WAL. This ensures that even if a failure occurs during processing, the system can replay the WAL to reconstruct the exact state of the data processing, including the offsets that have been processed. The WAL provides durability for the per-micro-batch changes.

Together, checkpointing and WAL guarantee fault tolerance. The WAL provides immediate recovery in case of minor failures, while checkpoints allow for recovery from more significant failures. Without checkpointing or write-ahead logs, Structured Streaming would be unable to reliably resume processing after a failure, potentially leading to data loss or duplication.

The other options are incorrect for the following reasons:

B: Structured Streaming does record offset ranges for fault tolerance.
C: Replayable Sources and Idempotent Sinks are related to fault tolerance but do not, on their own, provide

the mechanism for tracking offsets. While ideal, not all sources are replayable. Sinks being idempotent prevents duplicate writes, but this is not directly related to tracking offsets.
D: Write-ahead Logs are important, and Idempotent Sinks are beneficial to avoid duplicates, but checkpoints are essential for long-term fault tolerance.
E: Checkpointing alone is insufficient; a Write-Ahead Log provides a finer-grained level of fault tolerance between checkpoints.

Authoritative Links:

Spark Structured Streaming Programming Guide - Fault Tolerance Semantics: https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html#fault-tolerance-semantics
Databricks - Fault tolerance in Structured Streaming: https://www.databricks.com/glossary/structured-streaming (Search for the term Fault tolerance)

Question: 28 CertyIQ

Which of the following describes the relationship between Gold tables and Silver tables?

A. Gold tables are more likely to contain aggregations than Silver tables.
B. Gold tables are more likely to contain valuable data than Silver tables.
C. Gold tables are more likely to contain a less refined view of data than Silver tables.
D. Gold tables are more likely to contain more data than Silver tables.
E. Gold tables are more likely to contain truthful data than Silver tables.

Answer: A

Explanation:

The correct answer is **A. Gold tables are more likely to contain aggregations than Silver tables.** This relates to the Medallion Architecture (Bronze, Silver, Gold), a common data lake design pattern used in Databricks and other cloud data platforms.

Bronze tables represent the raw data ingested from source systems, often without transformation. Silver tables are refined versions of Bronze tables, typically cleansed, filtered, and joined. Gold tables sit at the top of the pyramid, designed for specific business use cases, reporting, and analysis.

Therefore, Gold tables often involve aggregations, such as calculating daily sales totals, customer lifetime value, or inventory summaries. Silver tables, while refined, are more likely to be at a row-level detail, providing the foundation for these aggregations. Options B, D, and E aren't correct because Silver tables contain refined and valuable data, though perhaps not tailored to a specific end-user use case, so they would not necessarily contain "more data" or "more truthful data" than Silver tables. While Gold tables are valuable, the type of value is different. Option C is incorrect because Gold tables provide a more refined view of the data tailored to answering specific business questions. The Medallion architecture is all about progressive refinement as data moves from Bronze to Gold.

Here are some authoritative links for further research:

Databricks Medallion Architecture: https://www.databricks.com/glossary/medallion-architecture
Understanding the Medallion Architecture: https://towardsdatascience.com/understanding-the-medallion-architecture-d566f8364164
Delta Lake Documentation (related to table structure): https://docs.delta.io/latest/index.html

Question: 29 CertyIQ

Which of the following describes the relationship between Bronze tables and raw data?

A. Bronze tables contain less data than raw data files.
B. Bronze tables contain more truthful data than raw data.
C. Bronze tables contain aggregates while raw data is unaggregated.
D. Bronze tables contain a less refined view of data than raw data.
E. Bronze tables contain raw data with a schema applied.

Answer: E

Explanation:

The correct answer is E: Bronze tables contain raw data with a schema applied. Let's break down why.

Bronze tables in the Databricks Lakehouse architecture represent the first layer of data ingestion. Think of them as landing zones for your data. The primary purpose of a Bronze table is to reliably and incrementally ingest raw data from various sources into your data lake.

Importantly, data in Bronze tables is not transformed or cleaned. It is persisted in its original form, ensuring a complete and auditable record of the incoming information. The key difference between the raw files themselves and the Bronze table is the imposition of a schema. This schema provides structure to the raw data, making it queryable and discoverable within the Databricks environment. Without a schema, the raw data files would be difficult to query directly using SQL or other data processing tools.

Options A, B, C, and D are incorrect because they describe transformations or refinements that occur in later stages of the medallion architecture (Silver and Gold layers). Bronze tables should not be reducing data volume (A), improving data quality (B), aggregating data (C), or refining the view of the data (D). These are all responsibilities of subsequent layers in the data pipeline.

Therefore, the defining characteristic of a Bronze table is that it provides a structured view (through schema enforcement) of raw, untransformed data. It forms the foundation for building a robust and reliable data pipeline within the Databricks Lakehouse.

For further research, consult the official Databricks documentation on the Lakehouse architecture and Delta Lake:

Databricks Lakehouse Architecture: https://www.databricks.com/product/lakehouse-architecture
Delta Lake Documentation: https://docs.databricks.com/delta/index.html
Medallion Architecture: Search Databricks Documentation for "Medallion Architecture"

Question: 30 CertyIQ

Which of the following tools is used by Auto Loader process data incrementally?

A. Checkpointing
B. Spark Structured Streaming
C. Data Explorer
D. Unity Catalog
E. Databricks SQL

Answer: B

Explanation:

The correct answer is **B. Spark Structured Streaming**.

Auto Loader leverages Spark Structured Streaming as its underlying engine for processing data incrementally. This means instead of processing entire datasets at once, Auto Loader reads new data as it arrives, offering a near real-time data ingestion pipeline. Checkpointing (A) is a mechanism used by Spark Structured Streaming to provide fault tolerance and recover state, but it's not the tool driving the incremental processing itself. Data Explorer (C) and Databricks SQL (E) are tools for data discovery, visualization, and querying of data stored in a Databricks workspace. Unity Catalog (D) provides centralized data governance, security, and access control across Databricks workspaces, but is not directly involved in the incremental processing mechanism.

Spark Structured Streaming defines a stream, which is a continuously updating table that represents new data arriving. Auto Loader uses this stream to automatically discover new files in the source data location. As new files are discovered, Structured Streaming processes them and appends the data to the target table. This continuous processing model is what allows Auto Loader to process data incrementally. The entire process is designed to be fault-tolerant and scalable thanks to the capabilities inherited from Spark Structured Streaming. Without Spark Structured Streaming, Auto Loader would not be able to offer its core functionality of incremental data ingestion. In essence, Auto Loader is a specialized application built on top of Spark Structured Streaming for simplified incremental data loading from cloud storage.

For further research, consider exploring these resources:

Databricks Auto Loader Documentation: https://docs.databricks.com/ingestion/auto-loader/index.html
Spark Structured Streaming Programming Guide: https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html

Question: 31 CertyIQ

A data engineer has configured a Structured Streaming job to read from a table, manipulate the data, and then perform a streaming write into a new table.
The cade block used by the data engineer is below:

```
(spark.table("sales")
    .withColumn("avg_price", col("sales") / col("units"))
    .writeStream
    .option("checkpointLocation", checkpointPath)
    .outputMode("complete")
    ._____
    .table("new_sales")
)
```

If the data engineer only wants the query to execute a micro-batch to process data every 5 seconds, which of the following lines of code should the data engineer use to fill in the blank?

A. trigger("5 seconds")
B. trigger()
C. trigger(once="5 seconds")
D. trigger(processingTime="5 seconds")
E. trigger(continuous="5 seconds")

Answer: D

Explanation:

D. trigger(processingTime="5 seconds")

In Structured Streaming with Apache Spark, if a data engineer wants to run the stream as a micro-batch every 5 seconds, they need to specify a processing time trigger.

Question: 32 CertyIQ

A dataset has been defined using Delta Live Tables and includes an expectations clause:
CONSTRAINT valid_timestamp EXPECT (timestamp > '2020-01-01') ON VIOLATION DROP ROW
What is the expected behavior when a batch of data containing data that violates these constraints is processed?

A. Records that violate the expectation are dropped from the target dataset and loaded into a quarantine table.

B. Records that violate the expectation are added to the target dataset and flagged as invalid in a field added to the target dataset.

C. Records that violate the expectation are dropped from the target dataset and recorded as invalid in the event log.

D. Records that violate the expectation are added to the target dataset and recorded as invalid in the event log.

E. Records that violate the expectation cause the job to fail.

Answer: C

Explanation:

The correct answer is C because it accurately describes the behavior of Delta Live Tables (DLT) when handling expectation violations with the ON VIOLATION DROP ROW clause. When this clause is specified, any record that fails to meet the defined expectation (in this case, timestamp > '2020-01-01') is removed from the target dataset during the DLT pipeline execution. Crucially, DLT provides a way to track these violations without failing the entire pipeline. It does this by logging the details of the dropped records in the DLT event log. This log provides valuable insights into data quality issues and allows for monitoring and auditing of data transformations.

Option A is incorrect because DLT does not automatically load violating records into a quarantine table when ON VIOLATION DROP ROW is used. The records are simply dropped. Option B is incorrect as DLT does not add records to the target dataset and flag them as invalid when using DROP ROW. Option D is partially correct as invalid records are indeed recorded in the event log. However, it incorrectly suggests the records are added to the target dataset, which is not the case when using DROP ROW. Finally, option E is incorrect because the DROP ROW clause is specifically designed to prevent the job from failing due to data quality issues. Instead, DLT gracefully handles the violations by dropping the problematic records. The overall goal of expectations within DLT is to maintain data quality without interrupting the continuous flow of data processing.

For further research on DLT expectations, you can refer to the official Databricks documentation:

Expectations in Delta Live Tables: https://docs.databricks.com/delta-live-tables/expectations.html
Monitor Delta Live Tables Pipelines: https://docs.databricks.com/delta-live-tables/monitor.html

Question: 33 CertyIQ

Which of the following describes when to use the CREATE STREAMING LIVE TABLE (formerly CREATE INCREMENTAL LIVE TABLE) syntax over the CREATE LIVE TABLE syntax when creating Delta Live Tables (DLT) tables using SQL?

A. CREATE STREAMING LIVE TABLE should be used when the subsequent step in the DLT pipeline is static.
B. CREATE STREAMING LIVE TABLE should be used when data needs to be processed incrementally.
C. CREATE STREAMING LIVE TABLE is redundant for DLT and it does not need to be used.
D. CREATE STREAMING LIVE TABLE should be used when data needs to be processed through complicated aggregations.
E. CREATE STREAMING LIVE TABLE should be used when the previous step in the DLT pipeline is static.

Answer: B

Explanation:

The correct answer is **B. CREATE STREAMING LIVE TABLE should be used when data needs to be processed incrementally.**

Here's a detailed justification:

Delta Live Tables (DLT) is a framework for building reliable, maintainable, and testable data pipelines on Databricks. It automatically manages data dependencies, optimizes execution, and handles data quality. Within DLT, the CREATE LIVE TABLE and CREATE STREAMING LIVE TABLE commands serve distinct purposes.

CREATE LIVE TABLE is used for batch processing. It's suitable when you need to process the entire input dataset at once. The entire table is recomputed upon each update. If the underlying data changes, the entire table will be reprocessed during the next pipeline update. This is ideal for scenarios where you want to ensure the table always reflects the complete and accurate view of the underlying data, irrespective of incremental changes.

CREATE STREAMING LIVE TABLE, on the other hand, is specifically designed for incremental data processing. This is crucial when dealing with continuously arriving data streams or when you only want to process new data since the last update, avoiding full refreshes. The command leverages Structured Streaming for continuous data ingestion and processing. It maintains state and only processes new records as they arrive, significantly improving efficiency and reducing processing time for streaming data or large datasets with frequent updates.

When using CREATE STREAMING LIVE TABLE, DLT understands that the source data is continuously changing. Hence, it's optimized to incrementally update the table based on the changes in the source data. Using this with batch data that doesn't incrementally change will lead to inefficient pipeline execution. The statement correctly relates to processing of continuously arriving data stream or when only processing new data since the last update.

Option A is incorrect because the nature of the subsequent step (static or not) doesn't dictate whether to use CREATE STREAMING LIVE TABLE. The key factor is whether the input is a continuous stream.

Option C is incorrect because CREATE STREAMING LIVE TABLE has a very specific and important function in incremental data processing.

Option D is incorrect. Complicated aggregations can be performed with both CREATE LIVE TABLE and CREATE STREAMING LIVE TABLE, so this isn't a defining factor. CREATE STREAMING LIVE TABLE can handle aggregations too, albeit in a streaming fashion.

Option E is incorrect. The nature of the previous step is not the primary determinant. The main consideration is whether the data feeding into the current table is a stream.

Therefore, CREATE STREAMING LIVE TABLE excels when the underlying data is constantly evolving and you only need to process the new or changed data incrementally to update the table.

Further Reading:

Delta Live Tables: https://docs.databricks.com/delta-live-tables/index.html
CREATE LIVE TABLE: https://docs.databricks.com/delta-live-tables/delta-live-tables-sql-ref.html#create-live-table
CREATE STREAMING LIVE TABLE: https://docs.databricks.com/delta-live-tables/delta-live-tables-sql-ref.html#create-streaming-live-table

Question: 34 CertyIQ

A data engineer is designing a data pipeline. The source system generates files in a shared directory that is also used by other processes. As a result, the files should be kept as is and will accumulate in the directory. The data engineer needs to identify which files are new since the previous run in the pipeline, and set up the pipeline to only ingest those new files with each run.
Which of the following tools can the data engineer use to solve this problem?

A. Unity Catalog
B. Delta Lake
C. Databricks SQL
D. Data Explorer
E. Auto Loader

Answer: E

Explanation:

The correct answer is E, Auto Loader, because it's specifically designed for incrementally and efficiently ingesting new files as they arrive in a cloud storage location (like a shared directory). Auto Loader automatically detects new files without requiring you to manually track which files have already been processed. It does this by leveraging either directory listing, file notification, or a combination of both, optimizing for different scale and latency requirements.

Here's a breakdown of why the other options are not suitable:

A. Unity Catalog: Unity Catalog is a data governance solution for managing data assets across Databricks workspaces. While it's important for data discovery and access control, it doesn't directly address the problem of identifying and ingesting new files.

B. Delta Lake: Delta Lake is a storage layer that brings reliability to data lakes by providing ACID transactions, scalable metadata handling, and unified streaming and batch data processing. While Delta Lake is excellent for storing and managing ingested data, it doesn't solve the initial problem of identifying new files for ingestion.

C. Databricks SQL: Databricks SQL is a serverless data warehouse on Databricks that allows you to run SQL queries on data stored in your data lake. It's useful for analyzing the data after it's been ingested, but it's not a tool for incremental file ingestion.

D. Data Explorer: Data Explorer allows users to explore the Databricks filesystem, display table metadata, and display sample data. This does not solve the ingestion challenge.

Auto Loader's ability to handle the continuous arrival of new files in a shared directory, especially in scenarios where other processes are also writing to the same location, makes it the ideal choice for this data pipeline

requirement. It's built to handle the scale and complexity of such scenarios without requiring manual intervention or custom scripting to track file ingestion status.

Authoritative Links:

Auto Loader: https://docs.databricks.com/ingestion/auto-loader/index.html

Question: 35 CertyIQ

Which of the following Structured Streaming queries is performing a hop from a Silver table to a Gold table?

A.
```
(spark.readStream.load(rawSalesLocation)
    .writeStream
    .option("checkpointLocation", checkpointPath)
    .outputMode("append")
    .table("newSales")
)
```

B.
```
(spark.read.load(rawSalesLocation)
    .writeStream
    .option("checkpointLocation", checkpointPath)
    .outputMode("append")
    .table("newSales")
)
```

C.
```
(spark.table("sales")
    .withColumn("avgPrice", col("sales") / col("units"))
    .writeStream
    .option("checkpointLocation", checkpointPath)
    .outputMode("append")
    .table("newSales")
)
```

D.
```
(spark.table("sales")
    .filter(col("units") > 0)
    .writeStream
    .option("checkpointLocation", checkpointPath)
    .outputMode("append")
    .table("newSales")
)
```

E.

```
(spark.table("sales")
    .groupBy("store")
    .agg(sum("sales"))
    .writeStream
    .option("checkpointLocation", checkpointPath)
    .outputMode("complete")
    .table("newSales")
)
```

Answer: E

Explanation:

E is the right answer. The "gold layer" is used to store aggregated clean data, E is the only answer in which aggregation is performed.

Question: 36 CertyIQ

A data engineer has three tables in a Delta Live Tables (DLT) pipeline. They have configured the pipeline to drop invalid records at each table. They notice that some data is being dropped due to quality concerns at some point in the DLT pipeline. They would like to determine at which table in their pipeline the data is being dropped.
Which of the following approaches can the data engineer take to identify the table that is dropping the records?

A. They can set up separate expectations for each table when developing their DLT pipeline.
B. They cannot determine which table is dropping the records.
C. They can set up DLT to notify them via email when records are dropped.
D. They can navigate to the DLT pipeline page, click on each table, and view the data quality statistics.
E. They can navigate to the DLT pipeline page, click on the “Error” button, and review the present errors.

Answer: D

Explanation:

The correct approach for identifying the table in a Delta Live Tables (DLT) pipeline where records are being dropped due to quality concerns is option D: "They can navigate to the DLT pipeline page, click on each table, and view the data quality statistics."

Here's why this is the best solution and why the other options are less suitable:

Delta Live Tables (DLT) Observability: DLT provides built-in monitoring and observability features specifically designed to track data quality metrics. This is a core benefit of using DLT for data transformation pipelines. The DLT UI provides a visual representation of the pipeline's data flow, including information about data quality.
Data Quality Statistics per Table: DLT automatically tracks data quality statistics for each table in the pipeline. These statistics include metrics like the number of records processed, the number of records that passed quality checks (expectations), and the number of records that were dropped or failed expectations.
Navigation and Inspection: By navigating to the DLT pipeline UI in Databricks and clicking on a specific table node in the graph, the data engineer can access detailed statistics for that table. The "Data Quality" section displays metrics related to expectations that were defined and how many records satisfied or violated each

expectation. This allows direct inspection of the data quality results at each stage of the pipeline.
Option A (Separate Expectations): Setting up separate expectations is a good practice when developing the pipeline, but it doesn't directly help after the fact if you just observe that data is being dropped. Separate expectations are necessary to enable the tracking that option D relies on. Having defined specific expectations for each table's data quality will then allow you to see the resulting statistics in the DLT UI.
Option B (Cannot Determine): This is incorrect. DLT is designed to provide this type of data lineage and quality tracking information.
Option C (Email Notifications): While DLT does offer the ability to configure email notifications based on pipeline status, it wouldn't directly point to which table is dropping the data. Custom code would be needed to generate the proper notification content.
Option E (Error Button): The "Error" button in the DLT UI typically displays errors that prevented the pipeline from running or errors caused by syntax issues. Errors related to data quality are not necessarily indicated by the "Error" button, especially when configured to drop invalid records using constraint clause or expectations using expect_or_drop or expect_all_or_drop functions. Data quality issues are generally tracked as data quality statistics related to expectation results.

Authoritative Links:

Delta Live Tables documentation: https://docs.databricks.com/delta-live-tables/index.html
Monitor Delta Live Tables pipelines: https://docs.databricks.com/delta-live-tables/monitor.html
Manage data quality with Delta Live Tables: https://docs.databricks.com/delta-live-tables/expectations.html

Question: 37 CertyIQ

A data engineer has a single-task Job that runs each morning before they begin working. After identifying an upstream data issue, they need to set up another task to run a new notebook prior to the original task.
Which of the following approaches can the data engineer use to set up the new task?

A.They can clone the existing task in the existing Job and update it to run the new notebook.
B.They can create a new task in the existing Job and then add it as a dependency of the original task.
C.They can create a new task in the existing Job and then add the original task as a dependency of the new task.
D.They can create a new job from scratch and add both tasks to run concurrently.
E.They can clone the existing task to a new Job and then edit it to run the new notebook.

Answer: B

Explanation:

The correct answer is B: "They can create a new task in the existing Job and then add it as a dependency of the original task." Here's why:

The scenario requires executing a new notebook before the existing one within the same automated daily schedule. Jobs in Databricks provide the capability to define dependencies between tasks, enabling a directed acyclic graph (DAG) workflow.

Option B leverages this dependency feature perfectly. Creating a new task that executes the new notebook, and then configuring the original task to depend on the new task, ensures the new notebook finishes successfully before the original one starts. This achieves the required sequential execution.

Option A is incorrect because cloning and updating the existing task would simply replace the notebook it executes, not run one before it. Option C has the dependency relationship reversed. The original task depending on the new task means the original task would run after the new one, contradicting the

requirement. Option D proposes a new job, which is unnecessary. Creating a new job would decouple the original task and introduce scheduling complexities. Option E is also incorrect because moving a task to a new job does not solve the dependency problem. The data engineer aims to execute two notebooks sequentially within a single workflow.

In summary, Databricks Jobs' task dependency feature is specifically designed to handle sequential task execution within a workflow. Option B correctly applies this feature to meet the scenario's requirements.

For more information, refer to the Databricks documentation on Jobs: https://docs.databricks.com/workflows/jobs/index.html and task dependencies: https://docs.databricks.com/workflows/jobs/jobs-concepts.html#dependencies.

Question: 38 CertyIQ

An engineering manager wants to monitor the performance of a recent project using a Databricks SQL query. For the first week following the project's release, the manager wants the query results to be updated every minute. However, the manager is concerned that the compute resources used for the query will be left running and cost the organization a lot of money beyond the first week of the project's release.
Which of the following approaches can the engineering team use to ensure the query does not cost the organization any money beyond the first week of the project's release?

A.They can set a limit to the number of DBUs that are consumed by the SQL Endpoint.

B.They can set the query's refresh schedule to end after a certain number of refreshes.

C.They cannot ensure the query does not cost the organization money beyond the first week of the project's release.

D.They can set a limit to the number of individuals that are able to manage the query's refresh schedule.

E.They can set the query's refresh schedule to end on a certain date in the query scheduler.

Answer: E

Explanation:

The correct answer is **E: They can set the query's refresh schedule to end on a certain date in the query scheduler.**

Here's a detailed justification:

Databricks SQL provides a query scheduler that allows users to define how frequently a query should be executed and its results updated. The key to controlling costs in this scenario is to ensure that the query stops running after the first week.

Option E directly addresses this requirement. By setting an end date in the query scheduler, you can ensure that the scheduled refreshes will automatically stop after the specified date, preventing further compute resource consumption and associated costs. This provides a precise and automated way to stop the query from running beyond the intended period. The Databricks SQL UI enables users to specify the end date during schedule creation/modification.

Options A, B, C, and D are not suitable solutions:

A: Limiting DBUs consumed by the SQL Endpoint may prevent a single query from consuming excessive resources, but it does not guarantee that the query will stop running after the first week. It only controls the resource usage per run, not the overall duration of the query's execution.
B: Setting a limit on the number of refreshes might work if the query is refreshed at exact one-minute

intervals. However, it is not a robust solution because intervals may vary and will cause the query to stop early or run late and cost additional money.
C: This statement is incorrect. The query execution can be precisely controlled.
D: Limiting the individuals that manage the refresh schedule doesn't automatically ensure that the query costs are limited.

Ultimately, the best way to limit query costs is to schedule the query to stop running automatically after the needed period, which is achieved by specifying an end date in the scheduler.

Here are some useful links for further information:

Databricks SQL scheduling
Optimize Databricks SQL costs

Question: 39 CertyIQ

A data analysis team has noticed that their Databricks SQL queries are running too slowly when connected to their always-on SQL endpoint. They claim that this issue is present when many members of the team are running small queries simultaneously. They ask the data engineering team for help. The data engineering team notices that each of the team's queries uses the same SQL endpoint.
Which of the following approaches can the data engineering team use to improve the latency of the team's queries?

A.They can increase the cluster size of the SQL endpoint.

B.They can increase the maximum bound of the SQL endpoint's scaling range.

C.They can turn on the Auto Stop feature for the SQL endpoint.

D.They can turn on the Serverless feature for the SQL endpoint.

E.They can turn on the Serverless feature for the SQL endpoint and change the Spot Instance Policy to "Reliability Optimized."

Answer: B

Explanation:

The problem described involves slow Databricks SQL query performance due to concurrent small queries overwhelming a single SQL endpoint. The goal is to improve query latency under high concurrency.

Option B, increasing the maximum bound of the SQL endpoint's scaling range, is the most appropriate solution. A SQL endpoint is a compute resource, and if multiple users simultaneously submit queries, the endpoint may become overloaded. By increasing the maximum bound of the scaling range, the endpoint can automatically scale up to a larger size (more compute resources) to handle the increased load. This allows more queries to be processed concurrently, reducing latency.

Option A, increasing the cluster size, could help temporarily but doesn't address the dynamic nature of the workload. Increasing the cluster size directly sets a fixed size and doesn't automatically adapt to fluctuations in query load. A larger fixed size might lead to underutilization and unnecessary costs during periods of lower activity.

Option C, turning on Auto Stop, is counterproductive. Auto Stop is designed to shut down the endpoint after a period of inactivity to save costs. This would only exacerbate the problem because the endpoint would need to restart frequently, adding significant latency to the initial queries after periods of inactivity.

Option D, turning on the Serverless feature is also a very good answer and, depending on Databricks account configuration, may be the best answer. Databricks Serverless SQL Endpoints (if available) automatically scale

resources based on workload demands and are managed entirely by Databricks. This removes the need to manually configure and manage cluster sizes. Serverless SQL is designed for exactly this type of scenario: concurrent small queries from many users.

Option E is redundant. Turning on Serverless inherently involves Databricks managing the infrastructure. Changing the Spot Instance Policy is a more fine-grained optimization option, but not the primary solution for this issue. While "Reliability Optimized" can improve things, Serverless as a whole is more impactful.

The most appropriate solution is therefore either B or D, depending on the context and the specific architecture of the Data Engineering Team's Databricks platform. B is acceptable as the answer if Serverless is not an option, or considered too risky to enable immediately.

Here are some resources:

Databricks SQL Endpoints: https://docs.databricks.com/sql/admin/sql-endpoints.html
Databricks Serverless SQL: https://docs.databricks.com/sql/admin/serverless.html
Autoscaling Databricks Clusters: https://docs.databricks.com/clusters/auto-scaling.html

Question: 40 CertyIQ

A data engineer wants to schedule their Databricks SQL dashboard to refresh once per day, but they only want the associated SQL endpoint to be running when it is necessary.
Which of the following approaches can the data engineer use to minimize the total running time of the SQL endpoint used in the refresh schedule of their dashboard?

A.They can ensure the dashboard's SQL endpoint matches each of the queries' SQL endpoints.
B.They can set up the dashboard's SQL endpoint to be serverless.
C.They can turn on the Auto Stop feature for the SQL endpoint.
D.They can reduce the cluster size of the SQL endpoint.
E.They can ensure the dashboard's SQL endpoint is not one of the included query's SQL endpoint.

Answer: C

Explanation:

The correct answer is C, enabling the Auto Stop feature for the SQL endpoint. Here's a detailed justification:

The goal is to minimize the total running time (and thus cost) of the SQL endpoint used by the dashboard's refresh schedule. The most direct way to achieve this is to automatically shut down the endpoint when it's not actively processing queries for the dashboard refresh. The "Auto Stop" feature does precisely that. When enabled, the SQL endpoint will automatically terminate after a specified period of inactivity. This prevents the endpoint from running unnecessarily when the dashboard doesn't need to be refreshed, drastically reducing compute costs.

Option A is incorrect because matching SQL endpoints of queries to the dashboard doesn't inherently minimize runtime. The endpoint still needs to run to execute the queries.

Option B (serverless) is not applicable to Databricks SQL. Serverless compute for dashboards and SQL workloads is a feature available in other cloud platforms but is not a direct option in Databricks SQL as of the current available knowledge base.

Option D (reducing cluster size) might reduce costs per hour of runtime, but it doesn't guarantee minimizing total runtime. A smaller cluster might take longer to execute the queries, potentially offsetting the cost savings. It can also cause performance issues and query timeouts if the cluster is too small to handle the

workload.

Option E is incorrect. There's no performance reason for the dashboard's SQL endpoint not to match the queries.

In summary, the Auto Stop feature directly addresses the requirement to minimize the running time of the SQL endpoint by automatically shutting it down when not in use, leading to significant cost savings.

Authoritative Link:

While a direct link focusing solely on the 'Auto Stop' feature within Databricks documentation is limited (the interface and location of that setting can change), the core concept of auto-scaling and auto-stopping compute resources is a fundamental principle in cloud computing cost optimization. Search the Databricks documentation for "SQL endpoint configuration" and "cost management" for relevant details, as well as looking for topics on "Auto Stop." The UI for auto-stopping is generally located in the SQL endpoint settings configuration.

Question: 41 CertyIQ

A data engineer has been using a Databricks SQL dashboard to monitor the cleanliness of the input data to an ELT job. The ELT job has its Databricks SQL query that returns the number of input records containing unexpected NULL values. The data engineer wants their entire team to be notified via a messaging webhook whenever this value reaches 100.
Which of the following approaches can the data engineer use to notify their entire team via a messaging webhook whenever the number of NULL values reaches 100?

A. They can set up an Alert with a custom template.
B. They can set up an Alert with a new email alert destination.
C. They can set up an Alert with a new webhook alert destination.
D. They can set up an Alert with one-time notifications.
E. They can set up an Alert without notifications.

Answer: C

Explanation:

The correct answer is C: They can set up an Alert with a new webhook alert destination.

Here's why: The requirement is to notify the team via a messaging webhook when a specific threshold (100 NULL values) is reached. Databricks SQL Alerts are specifically designed to trigger notifications based on query results meeting certain conditions. Alert destinations define where and how these notifications are sent.

Option A is incorrect because while custom templates allow formatting the alert message, they don't dictate where the alert is sent. A webhook destination is needed for that.

Option B is incorrect because email alerts are designed for email notifications, not for sending data to messaging webhooks.

Option D is incorrect because "one-time notifications" is not a standard feature within Databricks SQL Alerts. Alerts are designed to trigger repeatedly based on the specified condition being met.

Option E is incorrect because the goal is to notify the team; therefore, notifications are essential.

Webhooks are a standard way for applications to send real-time information to other applications. In this context, Databricks SQL can send data (specifically, the query result showing NULL values) to a messaging

platform like Slack, Microsoft Teams, or any other system that accepts webhooks. Setting up a "new webhook alert destination" configures Databricks SQL to post data to a specific URL whenever the query result triggers the alert condition (NULL values >= 100). The messaging service listens at this URL and processes the data to send a notification to the team.

Therefore, the most appropriate solution is to configure an Alert with a new webhook alert destination to achieve real-time notifications to the team when the threshold is met.

Further research:

Databricks SQL Alerts: https://docs.databricks.com/sql/user/alerts/index.html
Databricks SQL Alert Destinations: https://docs.databricks.com/sql/user/alerts/destinations.html
Webhooks: Search for "webhooks" on the Databricks website or general cloud computing resources to learn more about them and how they are used for integration.

Question: 42 CertyIQ

A single Job runs two notebooks as two separate tasks. A data engineer has noticed that one of the notebooks is running slowly in the Job's current run. The data engineer asks a tech lead for help in identifying why this might be the case.
Which of the following approaches can the tech lead use to identify why the notebook is running slowly as part of the Job?

A. They can navigate to the Runs tab in the Jobs UI to immediately review the processing notebook.

B. They can navigate to the Tasks tab in the Jobs UI and click on the active run to review the processing notebook.

C. They can navigate to the Runs tab in the Jobs UI and click on the active run to review the processing notebook.

D. There is no way to determine why a Job task is running slowly.

E. They can navigate to the Tasks tab in the Jobs UI to immediately review the processing notebook.

Answer: C

Explanation:

C. They can navigate to the Runs tab in the Jobs UI and click on the active run to review the processing notebook.

To investigate why a notebook (task) is running slowly in a Databricks Job, the best approach is:

Go to the Jobs UI → Runs tab

Select the specific active run

Drill down into the task (notebook)

This allows you to:

View execution details

Check logs and Spark UI

Analyze performance bottlenecks

Question: 43 CertyIQ

A data engineer has a Job with multiple tasks that runs nightly. Each of the tasks runs slowly because the clusters take a long time to start.
Which of the following actions can the data engineer perform to improve the start up time for the clusters used for the Job?

A. They can use endpoints available in Databricks SQL
B. They can use jobs clusters instead of all-purpose clusters
C. They can configure the clusters to be single-node
D. They can use clusters that are from a cluster pool
E. They can configure the clusters to autoscale for larger data sizes

Answer: D

Explanation:

The correct answer is D: They can use clusters that are from a cluster pool. Here's why:

Cluster startup time is a common bottleneck in Databricks Jobs, especially when clusters are created from scratch for each job run. Cluster pools address this issue by pre-allocating a set of idle instances ready for use. When a job requests a cluster that's configured to use a pool, Databricks attempts to allocate an instance from the pool first. This is significantly faster than provisioning a completely new cluster.

Options A and B are incorrect. Endpoints in Databricks SQL are for serving SQL queries, not for accelerating job startup times. Jobs clusters versus all-purpose clusters doesn't affect startup time, but rather lifecycle management (job clusters are automatically terminated after the job completes).

Option C is incorrect. Single-node clusters might be faster to provision, but they are generally inadequate for data engineering workloads as they lack the distributed processing capabilities needed to efficiently handle large datasets. Moreover, reducing resources doesn't directly address the initial startup delay.

Option E is related to scaling an already-running cluster. Autoscaling helps adjust the cluster size during a job based on workload demands, but doesn't affect the initial cluster startup time.

Cluster pools optimize the allocation of resources, significantly reducing the time required for jobs to begin execution. By reusing pre-warmed instances, the overhead associated with provisioning and configuring new clusters is minimized.

For more information on cluster pools, refer to the official Databricks documentation:
https://docs.databricks.com/clusters/instance-pools/index.html

Question: 44 CertyIQ

A new data engineering team team. has been assigned to an ELT project. The new data engineering team will need full privileges on the database customers to fully manage the project.
Which of the following commands can be used to grant full permissions on the database to the new data engineering team?

A. GRANT USAGE ON DATABASE customers TO team;
B. GRANT ALL PRIVILEGES ON DATABASE team TO customers;
C. GRANT SELECT PRIVILEGES ON DATABASE customers TO teams;
D. GRANT SELECT CREATE MODIFY USAGE PRIVILEGES ON DATABASE customers TO team;
E. GRANT ALL PRIVILEGES ON DATABASE customers TO team;

Answer: E

Explanation:

The correct answer is E: GRANT ALL PRIVILEGES ON DATABASE customers TO team;

Here's why:

The goal is to grant the data engineering team full privileges on the customers database. This implies the ability to perform all possible operations on the database, including creating, reading, updating, deleting, and managing its objects. The GRANT ALL PRIVILEGES command is specifically designed for this purpose.

Option A, GRANT USAGE ON DATABASE customers TO team;, only grants the ability to access the database, not to create or modify objects within it. It's a necessary but insufficient privilege for full management.

Option B, GRANT ALL PRIVILEGES ON DATABASE team TO customers;, reverses the subject and object. It attempts to give privileges to the customers database based on the existence of a team database, which is incorrect syntax and doesn't fulfill the requirement.

Option C, GRANT SELECT PRIVILEGES ON DATABASE customers TO teams;, grants only read access (SELECT) and misuses the word teams (plural). It does not provide the other necessary privileges for managing the database.

Option D, GRANT SELECT CREATE MODIFY USAGE PRIVILEGES ON DATABASE customers TO team;, grants several important permissions but does not grant all possible permissions. For example, it excludes DROP, ALTER, or OWNERSHIP capabilities, which are essential for full management control.

Therefore, only option E grants all the privileges necessary for the data engineering team to fully manage the customers database. ALL PRIVILEGES is a shorthand to specify all possible permissions allowed on the specified securable (in this case, a database).

For further information on Databricks SQL privileges and GRANT syntax, consult the official Databricks documentation:

Databricks SQL Privileges and Securable Objects
GRANT statement (Databricks SQL)

Question: 45 CertyIQ

A new data engineering team has been assigned to work on a project. The team will need access to database customers in order to see what tables already exist. The team has its own group team.
Which of the following commands can be used to grant the necessary permission on the entire database to the new team?

A. GRANT VIEW ON CATALOG customers TO team;
B. GRANT CREATE ON DATABASE customers TO team;
C. GRANT USAGE ON CATALOG team TO customers;
D. GRANT CREATE ON DATABASE team TO customers;
E. GRANT USAGE ON DATABASE customers TO team;

Answer: E

Explanation:

The correct answer is **E. GRANT USAGE ON DATABASE customers TO team;**

Here's why:

The question asks how to grant the team the permission to see what tables exist within the customers database. The USAGE privilege on a database (in Databricks, which operates on the Apache Spark SQL engine and leverages its metastore) is the key to allowing a principal (in this case, the team group) to access the database's metadata. Without USAGE, even if a user or group has specific permissions on tables within the database, they will not be able to list the tables present. USAGE is a prerequisite for accessing any objects within the database.

Option A, GRANT VIEW ON CATALOG customers TO team; is incorrect because there is no VIEW privilege on a catalog or database.

Option B, GRANT CREATE ON DATABASE customers TO team; is wrong because CREATE permission would allow the team to create new tables in the database, which isn't needed only to see existing tables.

Option C, GRANT USAGE ON CATALOG team TO customers; is incorrect because it reverses the objects involved in the grant statement. It also tries to grant access on catalog named team to the object called customers, which is likely not what you intended.

Option D, GRANT CREATE ON DATABASE team TO customers; is also incorrect because it is trying to grant permissions on a database named team to something called customers. The logic of the objects involved also makes no sense.

Essentially, USAGE on a database enables the principal to traverse the database and see its contents (tables, views, etc.). Other permissions, like SELECT or MODIFY, control what the principal can do with the data within those objects. To explore the database structure without modifying data, the USAGE privilege is sufficient and necessary.

For more information, review the Databricks SQL documentation on GRANT statements and access control:

Databricks SQL GRANT statement
Databricks Access Control

Question: 46 CertyIQ

A data engineer is running code in a Databricks Repo that is cloned from a central Git repository. A colleague of the data engineer informs them that changes have been made and synced to the central Git repository. The data engineer now needs to sync their Databricks Repo to get the changes from the central Git repository.
Which of the following Git operations does the data engineer need to run to accomplish this task?

A.Merge
B.Push
C.Pull
D.Commit
E.Clone

Answer: C

Explanation:

The correct answer is **C. Pull**. Here's a detailed justification:

When a Databricks Repo is cloned from a central Git repository, it becomes a local copy of the remote repository. The data engineer's goal is to update their local Repo with the latest changes from the central (remote) repository. Git provides several operations for managing these changes, and the appropriate one depends on the desired outcome.

Pull: The git pull command is used to fetch changes from a remote repository and then automatically merge them into the current branch. It's essentially a combination of git fetch (retrieving the remote changes) and git merge (integrating those changes into the local branch). In this scenario, the data engineer needs to get the changes made by their colleague from the central Git repository and integrate them into their local Databricks Repo. Therefore, git pull is the correct operation.

Merge: The git merge command integrates changes from one branch into another. While merging is part of the pull operation, it's not the sole command needed to retrieve changes from the remote repository in the first place. A pull automatically does a fetch + merge.

Push: The git push command is used to upload local repository content to a remote repository. It's the opposite of pulling; the data engineer is trying to receive changes, not contribute them in this scenario.

Commit: The git commit command records changes to the local repository. It prepares changes to be pushed to the remote repo later. However, it doesn't retrieve any remote changes.

Clone: The git clone command creates a copy of a remote repository. The data engineer already has a cloned Repo and only needs to sync it with the latest changes. Cloning is a one-time operation to initially create a local copy.

In summary, the data engineer should use git pull to download and merge the changes from the central Git repository into their Databricks Repo.

Authoritative Links for further research:

Git Basics - Getting a Git Repository: https://git-scm.com/book/en/v2/Git-Basics-Getting-a-Git-Repository
Git Branching - Basic Branching and Merging: https://git-scm.com/book/en/v2/Git-Branching-Basic-Branching-and-Merging
Git Remote - Working with Remotes: https://git-scm.com/book/en/v2/Git-Basics-Working-with-Remotes

Question: 47 CertyIQ

Which of the following is a benefit of the Databricks Lakehouse Platform embracing open source technologies?

A.Cloud-specific integrations
B.Simplified governance
C.Ability to scale storage
D.Ability to scale workloads
E.Avoiding vendor lock-in

Answer: E

Explanation:

The correct answer is **E. Avoiding vendor lock-in**.

Here's a detailed justification:

Databricks' embrace of open-source technologies like Delta Lake, Apache Spark, and MLflow is a cornerstone of its Lakehouse platform. Vendor lock-in occurs when using proprietary technologies that tightly couple users to a specific vendor's ecosystem, making it difficult and costly to migrate data, code, and processes to another platform.

By leveraging open-source technologies, Databricks allows users to maintain greater control over their data and processing. They are not locked into proprietary formats or APIs. Users can leverage their skills and knowledge across different environments, increasing portability. Open standards promote interoperability and reduce the risk of being dependent on a single vendor's roadmap or pricing.

Specifically:

Delta Lake, an open-source storage layer, provides ACID transactions, scalable metadata handling, and unified streaming and batch data processing. Using Delta Lake avoids proprietary data formats.
Apache Spark, an open-source distributed processing engine, is the core compute framework of the Lakehouse. Its wide adoption and open nature mean that data engineers have a large pool of skills and can run Spark workloads on various infrastructures.
MLflow is an open-source platform to manage the ML lifecycle, promoting reproducibility and collaboration across different platforms.

Options A, B, C, and D are relevant benefits of the Databricks platform in general, but they are not direct consequences of embracing open-source technologies. While the Lakehouse platform leverages cloud-specific integrations (A) and allows for scaling storage (C) and workloads (D), these capabilities are provided via Databricks' infrastructure. Simplified governance (B) can be facilitated by specific open-source tools, but is also a function of the governance tools Databricks incorporates, not the open-source nature.

The key differentiator is that the open-source component directly addresses vendor lock-in because it provides alternatives to completely proprietary solutions. The other options are more about the platform's functionalities.

Further resources:

Delta Lake: https://delta.io/
Apache Spark: https://spark.apache.org/
MLflow: https://mlflow.org/

Question: 48 CertyIQ

A data engineer needs to use a Delta table as part of a data pipeline, but they do not know if they have the appropriate permissions.
In which of the following locations can the data engineer review their permissions on the table?

A.Databricks Filesystem
B.Jobs
C.Dashboards
D.Repos
E.Data Explorer

Answer: E

Explanation:

The correct answer is **E. Data Explorer**. Here's why:

The Data Explorer in Databricks is the primary user interface for discovering, exploring, and managing data objects within the Databricks environment. It provides a centralized location to view tables, databases, and their associated metadata. Crucially, this includes access control information. Specifically, Data Explorer displays the permissions that a user or group has on a particular table, including SELECT, MODIFY, CREATE, etc. A data engineer can easily navigate to the specific Delta table in Data Explorer and review the "Permissions" tab or similar section to understand their access rights.

The other options are incorrect because:

A. Databricks Filesystem (DBFS): DBFS is a distributed filesystem, and while Delta tables are stored as data files within DBFS, the filesystem itself doesn't inherently display access control lists (ACLs) related to Databricks SQL permissions or Delta table-specific permissions. While POSIX-style permissions exist on the underlying files, these are not the Databricks-level permissions that control access to the table via Spark SQL or Delta Lake APIs.

B. Jobs: Jobs are for scheduling and running notebooks or other tasks; they don't provide information about data object permissions.

C. Dashboards: Dashboards are visualization tools and do not expose table-level permissions.

D. Repos: Repos are used for version control of code and have nothing to do with data permissions.

Data Explorer, specifically, is designed to provide a graphical user interface to manage and inspect metadata, including security and access control. It provides a centralized place to discover, explore, and grant/revoke access to data assets managed by Databricks.

Authoritative links for further research:

Databricks Data Explorer documentation: https://docs.databricks.com/data-governance/data-explorer/index.html
Databricks Permissions Model: https://docs.databricks.com/security/access-control/index.html

Question: 49 CertyIQ

Which of the following describes a scenario in which a data engineer will want to use a single-node cluster?

A.When they are working interactively with a small amount of data
B.When they are running automated reports to be refreshed as quickly as possible
C.When they are working with SQL within Databricks SQL
D.When they are concerned about the ability to automatically scale with larger data
E.When they are manually running reports with a large amount of data

Answer: A

Explanation:

The correct answer is A: "When they are working interactively with a small amount of data." Here's why:

Single-node clusters in Databricks are designed for development, testing, and interactive data exploration with small datasets. They are not intended for production workloads or scenarios requiring high availability, fault tolerance, or scalability. The driver node acts as both the driver and worker, streamlining resource allocation for smaller tasks.

Option B is incorrect because automated reports often benefit from parallel processing for speed, which a

single-node cluster lacks. Option C pertains to Databricks SQL, which leverages optimized engines and doesn't necessarily dictate the need for a single-node cluster; it's more about query performance. Option D is the opposite of what single-node clusters offer; they explicitly don't scale. Option E is wrong because large datasets necessitate distributed processing to be handled efficiently, which is beyond the capabilities of a single-node setup.

Single-node clusters offer a cost-effective and simple environment for initial data exploration, prototyping, and learning Databricks features. They're ideal for smaller datasets where the overhead of distributed computing would outweigh the performance gains. Interactivity is improved as there is no need to distribute the data across multiple nodes for simple analysis.

For further reading on Databricks clusters and their use cases:

Databricks documentation on cluster configuration: https://docs.databricks.com/clusters/index.html
Databricks documentation on single-node clusters: https://docs.databricks.com/clusters/single-node.html

Question: 50 CertyIQ

A data engineer has been given a new record of data:
id STRING = 'a1'
rank INTEGER = 6
rating FLOAT = 9.4
Which of the following SQL commands can be used to append the new record to an existing Delta table my_table?

A.INSERT INTO my_table VALUES ('a1', 6, 9.4)
B.my_table UNION VALUES ('a1', 6, 9.4)
C.INSERT VALUES ('a1' , 6, 9.4) INTO my_table
D.UPDATE my_table VALUES ('a1', 6, 9.4)
E.UPDATE VALUES ('a1', 6, 9.4) my_table

Answer: A

Explanation:

The correct answer is A: INSERT INTO my_table VALUES ('a1', 6, 9.4). This is because it follows the standard SQL syntax for inserting new rows into a table.

Here's a breakdown:

INSERT INTO my_table: This specifies the target table (my_table) where the new record will be inserted. The INSERT INTO clause is the foundation for adding data to a table.

VALUES ('a1', 6, 9.4): This provides the values for the new record. The values are specified in the same order as the columns defined in the my_table schema: id (STRING), rank (INTEGER), and rating (FLOAT). String values are enclosed in single quotes, while numeric values are not. The number of provided values must match the number of columns you are populating in your table (unless default values are defined for some columns).

Let's examine why the other options are incorrect:

B. my_table UNION VALUES ('a1', 6, 9.4): UNION is used to combine the result sets of two or more SELECT statements. It does not insert data directly into a table. It also requires SELECT statements on both sides of the UNION operator.

C. INSERT VALUES ('a1' , 6, 9.4) INTO my_table: The SQL syntax is incorrect. The INTO my_table part should come directly after INSERT.

D. UPDATE my_table VALUES ('a1', 6, 9.4): UPDATE is used to modify existing rows in a table, not to insert new rows. Furthermore, the UPDATE statement requires a WHERE clause to specify which rows should be updated. VALUES keyword is not used with UPDATE statement.

E. UPDATE VALUES ('a1', 6, 9.4) my_table: The syntax is completely incorrect for an UPDATE statement.

Delta Lake and SQL:

Delta Lake, which Databricks uses extensively, supports standard SQL commands. The INSERT INTO command works seamlessly with Delta tables for appending new data. Delta Lake extends the capabilities of Apache Spark with ACID transactions, scalable metadata handling, and unified streaming and batch data processing. It ensures data reliability when writing and reading data. Using the correct SQL syntax ensures that the data insertion operation is consistent and reliable within the Delta Lake environment.

Further Research:

Delta Lake Documentation: https://docs.databricks.com/delta/index.html
SQL INSERT Statement: https://www.w3schools.com/sql/sql_insert.asp
Apache Spark SQL Documentation: https://spark.apache.org/docs/latest/sql-ref.html

Question: 51 CertyIQ

A data engineer has realized that the data files associated with a Delta table are incredibly small. They want to compact the small files to form larger files to improve performance.
Which of the following keywords can be used to compact the small files?

A.REDUCE
B.OPTIMIZE
C.COMPACTION
D.REPARTITION
E.VACUUM

Answer: B

Explanation:

The correct answer is **B. OPTIMIZE**. Here's a detailed justification:

The primary purpose of the OPTIMIZE command in Databricks Delta Lake is to compact small files within a Delta table into larger files. This process, often referred to as bin-packing, aims to reduce the number of files that the query engine needs to read when querying the table. Fewer, larger files generally result in improved query performance, particularly for workloads that involve scanning large portions of the dataset.

Delta Lake stores data in Parquet format, and having numerous small Parquet files can lead to increased metadata overhead and slower query execution. The OPTIMIZE command addresses this issue by rewriting these small files into fewer, larger ones, thereby optimizing the data layout for better performance.

The REDUCE keyword isn't a standard Delta Lake command. COMPACTION is a general term describing the process, but OPTIMIZE is the specific command used in Databricks Delta Lake to perform file compaction. REPARTITION is used to change the number of partitions in a DataFrame, and while it can affect file size, it's

not the direct tool for compacting existing small files. VACUUM removes files that are no longer part of the current table state (old versions), it doesn't compact files.

Therefore, OPTIMIZE is the most suitable keyword to compact small files in a Databricks Delta table, enhancing query performance by reducing the overhead associated with numerous small files.

Here are some authoritative links for further research:

Databricks Documentation on OPTIMIZE: https://docs.databricks.com/optimizations/optimize.html
Delta Lake Documentation: https://delta.io/

Question: 52 CertyIQ

In which of the following file formats is data from Delta Lake tables primarily stored?

A.Delta
B.CSV
C.Parquet
D.JSON
E.A proprietary, optimized format specific to Databricks

Answer: C

Explanation:

The correct answer is Parquet (C). Delta Lake leverages Parquet as its underlying storage format for data within its tables. While Delta Lake introduces a transactional layer and other features on top, the actual data itself is stored in Parquet files.

Parquet is a columnar storage format. This characteristic offers significant advantages for analytical workloads common in data engineering. Columnar storage enables efficient data retrieval because only the necessary columns for a query need to be read, minimizing I/O operations. This significantly accelerates query performance compared to row-oriented formats like CSV or JSON, where entire rows must be read even if only a few columns are relevant.

Delta Lake's transactional guarantees, versioning, and schema evolution are implemented by storing metadata alongside the Parquet data files. This metadata, including a transaction log, tracks changes to the data and allows for ACID properties. However, the core data persistence relies on Parquet's efficient storage and retrieval capabilities. Delta Lake doesn't create its own proprietary data format; it optimizes and extends the functionality of Parquet for data lake use cases. While Delta Lake can ingest data from other formats (like CSV or JSON), when that data is stored in a Delta Lake table, it is converted to Parquet. The Delta format then wraps the data for transactional integrity.

Therefore, while Delta Lake provides enhanced capabilities, Parquet remains the fundamental file format for data storage within Delta Lake tables.

For more information, refer to the official Databricks documentation:

Delta Lake Overview: https://docs.databricks.com/delta/index.html
Parquet Format: Understanding Parquet will involve researching various resources, as the Databricks documentation is more centered on Delta Lake usage than on a deep dive into Parquet itself. General resources on Parquet are beneficial, such as those from Apache.

Question: 53 CertyIQ

Which of the following is stored in the Databricks customer's cloud account?

A.Databricks web application
B.Cluster management metadata
C.Repos
D.Data
E.Notebooks

Answer: D

Explanation:

The correct answer is D (Data). Here's a detailed justification:

Databricks operates within the customer's cloud account (AWS, Azure, or GCP), employing a control plane/data plane architecture. The control plane, managed by Databricks, handles cluster management, job orchestration, and the web application. This includes components like cluster metadata (B), Repos (C), and Notebooks (E) which are essentially metadata or code pointers, not the raw data itself. These items are stored securely and efficiently in Databricks-managed infrastructure, enabling scalable computation.

The crucial distinction is where the actual data resides. Databricks doesn't store customer's data in its managed control plane. Instead, Databricks clusters are configured to access data stored in the customer's own cloud storage services, such as AWS S3, Azure Blob Storage, or Google Cloud Storage. This means the underlying data (D) remains entirely within the customer's cloud account, providing them with control over data governance, security, and compliance. Databricks only processes and analyzes this data based on the instructions given in the code (Notebooks, for example), but it doesn't permanently store copies of it. The web application (A) is also part of the Databricks control plane and therefore not residing within the customer's cloud account. The benefit of this architecture is enhanced data security and governance because data never leaves the customer's environment. It also ensures cost optimization because the customers can leverage their existing cloud storage investments.

For more information, refer to the Databricks documentation on the Databricks architecture and security model:

Databricks Architecture: https://docs.databricks.com/en/getting-started/overview.html
Databricks Security: https://www.databricks.com/trust

Question: 54 CertyIQ

Which of the following can be used to simplify and unify siloed data architectures that are specialized for specific use cases?

A.None of these
B.Data lake
C.Data warehouse
D.All of these
E.Data lakehouse

Answer: E

Explanation:

The correct answer is **E. Data lakehouse**. Here's a detailed justification:

Siloed data architectures are common when organizations use specialized systems for different tasks. For example, a data warehouse might handle structured reporting, while a data lake stores raw, unstructured data for data science. This creates data duplication, complexity in data pipelines, and difficulties in consistent data governance.

A data lake can store diverse data types in a central repository, but it often lacks strong transactional support and governance features needed for reliable analytical use cases, leading to "data swamps." A data warehouse can provide structured, governed data, but typically struggles with the volume and variety of data found in modern enterprise environments.

A data lakehouse architecture aims to combine the best of both worlds. It offers the low-cost storage and scalability of a data lake with the data management and performance capabilities of a data warehouse. This unification allows organizations to work with all their data – structured, semi-structured, and unstructured – within a single system. Key features of a data lakehouse include:

Schema enforcement and governance: Metadata management and schema evolution ensure data quality and consistency.
ACID transactions: Support for transactional consistency across multiple data updates.
Direct access to data: Enables diverse workloads, including SQL analytics, data science, and machine learning, directly on the data in the lakehouse.
End-to-end streaming: Real-time data ingestion and processing.
Open formats: Usually based on open file formats like Parquet and Delta Lake to avoid vendor lock-in.

Therefore, a data lakehouse can significantly simplify and unify data architectures that are specialized for specific use cases by eliminating data silos and providing a single source of truth for all data-driven initiatives.

Authoritative links for further research:

Databricks Data Lakehouse: https://www.databricks.com/glossary/data-lakehouse
Microsoft: What is a data lakehouse?: https://learn.microsoft.com/en-us/azure/architecture/guide/technology-choices/data-lakehouse

Question: 55 CertyIQ

A data architect has determined that a table of the following format is necessary:

employeeId	startDate	avgRating
a1	2009-01-06	5.5
a2	2018-11-21	7.1
...	...	...

Which of the following code blocks uses SQL DDL commands to create an empty Delta table in the above format regardless of whether a table already exists with this name?

A.

```
CREATE TABLE IF NOT EXISTS table_name (
  employeeId STRING,
  startDate DATE,
  avgRating FLOAT
)
```

B.

```
CREATE OR REPLACE TABLE table_name AS
SELECT
  employeeId STRING,
  startDate DATE,
  avgRating FLOAT
USING DELTA
```

C.

```
CREATE OR REPLACE TABLE table_name WITH COLUMNS (
  employeeId STRING,
  startDate DATE,
  avgRating FLOAT
) USING DELTA
```

D.

```
CREATE TABLE table_name AS
SELECT
  employeeId STRING,
  startDate DATE,
  avgRating FLOAT
```

E.

```
CREATE OR REPLACE TABLE table_name (
  employeeId STRING,
  startDate DATE,
  avgRating FLOAT
)
```

Answer: E

Explanation:

```
CREATE OR REPLACE TABLE table_name (
  employeeId STRING,
  startDate DATE,
  avgRating FLOAT
)
```

Question: 56 CertyIQ

A data engineer has a Python notebook in Databricks, but they need to use SQL to accomplish a specific task within a cell. They still want all of the other cells to use Python without making any changes to those cells. Which of the following describes how the data engineer can use SQL within a cell of their Python notebook?

A.It is not possible to use SQL in a Python notebook
B.They can attach the cell to a SQL endpoint rather than a Databricks cluster
C.They can simply write SQL syntax in the cell
D.They can add %sql to the first line of the cell
E.They can change the default language of the notebook to SQL

Answer: D

Explanation:

The correct answer is **D. They can add %sql to the first line of the cell.**

Here's a detailed justification:

Databricks notebooks support polyglot programming, meaning you can use multiple languages (Python, SQL, R, Scala) within the same notebook. To achieve this, Databricks uses "magic commands" or "language magics". These are special commands that start with a percent sign (%) and tell the notebook to interpret the cell's content in a specific language.

In this scenario, the data engineer wants to use SQL within a Python notebook without altering the other cells. Prefixing a cell with %sql instructs Databricks to interpret the subsequent content of that specific cell as SQL code. The Databricks runtime then executes the SQL query against the attached Spark cluster, leveraging Spark SQL capabilities. This allows the data engineer to seamlessly incorporate SQL operations (like querying tables, creating views, or performing aggregations) within their predominantly Python-based workflow. The Python cells remain unaffected and continue to execute as Python code.

Option A is incorrect because Databricks notebooks are designed to support multiple languages. Option B is incorrect because attaching a cell to a SQL endpoint would require fundamental changes to the notebook structure and not allow for mixed language usage in other cells. Option C is wrong because writing SQL syntax directly in a Python cell will result in syntax errors, as the Python interpreter will not recognize the

SQL code. Option E is incorrect because you change the cell to SQL and not the notebook.

Therefore, using the %sql magic command provides a localized and efficient way to integrate SQL into a Python Databricks notebook, enabling the data engineer to complete their task without disrupting the existing Python code.

Authoritative links:

Databricks Documentation on Magic Commands: https://docs.databricks.com/notebooks/notebooks-use.html#mix-languages
Spark SQL: https://spark.apache.org/sql/

Question: 57 CertyIQ

Which of the following SQL keywords can be used to convert a table from a long format to a wide format?

A.TRANSFORM
B.PIVOT
C.SUM
D.CONVERT
E.WHERE

Answer: B

Explanation:

The correct SQL keyword to convert a table from a long (narrow and tall) format to a wide (short and wide) format is PIVOT.

PIVOT is a relational operator that transforms rows into columns. In the context of data warehousing and data transformation, pivoting is a common technique used to reshape data for analysis and reporting. A long format table typically has multiple rows representing different measurements or attributes for the same entity, while a wide format table consolidates these measurements into a single row per entity, with each measurement becoming a separate column.

The PIVOT operator takes values from one column and turns them into new columns. It then aggregates values from another column based on these new columns. For example, if a table has columns category, product, and sales, pivoting on category would create new columns for each category, and the sales column would be aggregated (e.g., summed) for each category.

The other options are incorrect because:

TRANSFORM is not a standard SQL keyword for reshaping data. It's more commonly associated with data transformation tools but doesn't directly perform pivoting in SQL.
SUM is an aggregate function used to calculate the sum of values but does not restructure the table. It's often used within a PIVOT query for aggregation.
CONVERT is used for data type conversions, not for reshaping data.
WHERE is used to filter rows based on specified conditions and doesn't change the table structure.

Databricks' implementation of PIVOT adheres to the standard SQL functionality, allowing for efficient and scalable data reshaping within their environment. This is crucial for preparing data for analysis and machine learning tasks on Databricks. Pivoting is a common data manipulation technique in Spark SQL, frequently

used when preparing data frames for analysis.

For further research, refer to the official Databricks documentation on Spark SQL, specifically examples related to data transformation and pivoting. Additionally, consult SQL tutorials focusing on the PIVOT operator for various database systems.

Authoritative Link:

Apache Spark SQL Programming Guide: https://spark.apache.org/docs/latest/sql-programming-guide.html (Look for examples utilizing PIVOT for data transformation)

Question: 58 CertyIQ

Which of the following describes a benefit of creating an external table from Parquet rather than CSV when using a CREATE TABLE AS SELECT statement?

A.Parquet files can be partitioned
B.CREATE TABLE AS SELECT statements cannot be used on files
C.Parquet files have a well-defined schema
D.Parquet files have the ability to be optimized
E.Parquet files will become Delta tables

Answer: C

Explanation:

The correct answer is C: Parquet files have a well-defined schema. Here's why:

When creating an external table using CREATE TABLE AS SELECT (CTAS), the source data's structure is critical. CSV files are inherently schema-less. While you can use CTAS on a CSV file (contradicting option B), you must explicitly define the schema during the table creation process. The CTAS statement will infer data types, but it relies on assumptions that may not always be accurate, leading to potential data type mismatches or incorrect data interpretation.

Parquet, on the other hand, is a columnar storage format that stores its schema within the file itself. When you use CTAS to create an external table from Parquet files, the Databricks runtime automatically infers the schema from the Parquet metadata. This eliminates the need to manually define the schema, reduces the risk of schema inconsistencies, and streamlines the table creation process.

While option A (Parquet files can be partitioned) is true in general, partitioning is a separate concern from the immediate benefit when using CTAS. Partitioning can be applied to both Parquet and CSV tables after creation. Option D (Parquet files have the ability to be optimized) is also true but less directly related to the benefit of CTAS. Parquet's columnar nature lends itself to optimization techniques, but the primary benefit in the context of CTAS is the automatic schema inference. Option E (Parquet files will become Delta tables) is incorrect; you would need to specifically create a Delta table, it doesn't happen automatically from a Parquet source with CTAS.

In essence, the self-describing nature of Parquet files, with their embedded schema, makes CREATE TABLE AS SELECT operations simpler, more reliable, and less prone to errors compared to using schema-less formats like CSV.Here are some resources for further reading:

Apache Parquet: https://parquet.apache.org/

Databricks CREATE TABLE AS SELECT: https://docs.databricks.com/sql/language-manual/sql-ref-syntax-ddl-create-table-as.html

Question: 59 CertyIQ

A data engineer wants to create a relational object by pulling data from two tables. The relational object does not need to be used by other data engineers in other sessions. In order to save on storage costs, the data engineer wants to avoid copying and storing physical data.
Which of the following relational objects should the data engineer create?

A.Spark SQL Table
B.View
C.Database
D.Temporary view
E.Delta Table

Answer: D

Explanation:

Here's a detailed justification for why the correct answer is D, Temporary View, along with explanations of why the other options are not the best choice:

The scenario requires creating a relational object based on two tables without physically storing or copying data and limiting its scope to the current session.

Temporary Views (D) perfectly satisfy these requirements. A temporary view is a view that exists only for the duration of the SparkSession in which it was created. It doesn't persist data to storage. It dynamically generates results based on the underlying tables each time it's queried. Moreover, it is not accessible to other sessions, aligning with the requirement that the object shouldn't be used by other data engineers.

Why other options are incorrect:

Spark SQL Table (A): Tables, by default, persist data to storage (e.g., cloud storage like AWS S3, Azure Data Lake Storage, or Google Cloud Storage). This contradicts the need to avoid physical storage and incur unnecessary cost.
View (B): While a regular view is also a logical representation without data duplication, it persists in the metastore and is available to other users and sessions. This contradicts the single-session requirement.
Database (C): A database is a container for tables, views, and functions. It doesn't directly address the requirement of creating a relational object by pulling data from two tables in a session-specific manner without storing data. Creating a database is not directly relevant to the described problem.
Delta Table (E): Delta Tables are an enhanced version of Spark tables and provide ACID transactions, scalable metadata handling, and time travel capabilities. However, they fundamentally persist data to storage as well and are not inherently session-specific.

In summary, a temporary view allows the data engineer to create a relational object derived from two tables without incurring storage costs or making it accessible to other sessions. This aligns perfectly with the problem's constraints.

Relevant Documentation:

Spark SQL Views: https://spark.apache.org/docs/latest/sql-ref-syntax-ddl-create-view.html
Databricks Views: https://docs.databricks.com/sql/language-manual/sql-ref-syntax-ddl-create-view.html

Question: 60 CertyIQ

A data analyst has developed a query that runs against Delta table. They want help from the data engineering team to implement a series of tests to ensure the data returned by the query is clean. However, the data engineering team uses Python for its tests rather than SQL.
Which of the following operations could the data engineering team use to run the query and operate with the results in PySpark?

A.SELECT * FROM sales
B.spark.delta.table
C.spark.sql
D.There is no way to share data between PySpark and SQL.
E.spark.table

Answer: C

Explanation:

The correct answer is C, spark.sql. Here's why:

The scenario requires running an SQL query (developed by the data analyst) within a PySpark environment for data quality testing. PySpark, which is the Python API for Apache Spark, provides several ways to interact with Spark's SQL engine.

Option A, SELECT * FROM sales, is a raw SQL query string. While syntactically correct SQL, it doesn't execute anything within a PySpark context on its own. It would need to be passed to a Spark execution function.

Option B, spark.delta.table, is specific to interacting with Delta tables. While the query operates on a Delta table, this option doesn't provide a mechanism to execute the analyst's arbitrary SQL query. It's used for specific Delta Lake operations like versioning or time travel.

Option C, spark.sql, is the most direct and appropriate way to run SQL queries from within PySpark. The spark object represents the SparkSession, and the sql method allows you to execute any valid SQL query against the Spark environment. The result of the query is a PySpark DataFrame, which can then be easily manipulated and tested using Python code. This directly allows the engineering team to work with results in PySpark.

Option D is incorrect because PySpark is specifically designed to bridge Python and SQL.

Option E, spark.table, is used to access an existing table registered in the Spark metastore as a DataFrame. While you could access the Delta table this way, it doesn't directly execute the arbitrary SQL the data analyst wrote. You'd need to write additional DataFrame transformations to recreate the query's logic. spark.sql is much simpler.

Therefore, spark.sql(query_string) allows the data engineering team to take the analyst's SQL query (represented as query_string), execute it within the Spark environment, and receive the results as a PySpark DataFrame, which is readily usable for further processing and testing using Python.

Authoritative Links:

Apache Spark SQL Programming Guide: https://spark.apache.org/docs/latest/sql-programming-guide.html
SparkSession: https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.sql.SparkSession.html
Delta Lake documentation: https://docs.delta.io/latest/

Question: 61 CertyIQ

Which of the following commands will return the number of null values in the member_id column?

A.SELECT count(member_id) FROM my_table;
B.SELECT count(member_id) - count_null(member_id) FROM my_table;
C.SELECT count_if(member_id IS NULL) FROM my_table;
D.SELECT null(member_id) FROM my_table;
E.SELECT count_null(member_id) FROM my_table;

Answer: C

Explanation:

The correct answer is C, SELECT count_if(member_id IS NULL) FROM my_table; because it accurately counts the number of null values within the member_id column.

Let's break down why each option is or isn't correct:

A. SELECT count(member_id) FROM my_table;: The count() function ignores null values. This command will return the number of non-null values in the member_id column, not the number of nulls.

B. SELECT count(member_id) - count_null(member_id) FROM my_table;: The count_null() function is not a standard SQL or Spark SQL function. Therefore, this command will likely result in an error. Furthermore, even if such a function existed, this logic subtracts something that might not be directly equivalent to the total count of non-null values.

C. SELECT count_if(member_id IS NULL) FROM my_table;: The count_if() function evaluates a boolean expression for each row and counts the number of times the expression is true. In this case, member_id IS NULL is a boolean expression that returns true if member_id is null and false otherwise. Therefore, count_if(member_id IS NULL) correctly returns the number of null values in the member_id column. This aligns with Databricks' and Spark SQL's functionality.

D. SELECT null(member_id) FROM my_table;: The null() function does not exist in SQL or Spark SQL. This would cause an error. Even conceptually, the null() keyword is used to represent a null value, not to count them.

E. SELECT count_null(member_id) FROM my_table;: Again, the count_null() function is not a standard SQL or Spark SQL function. It will likely cause an error.

Therefore, count_if(member_id IS NULL) is the only correct option because it uses a valid Spark SQL function (count_if) and a valid SQL comparison operator (IS NULL) to achieve the desired outcome of counting null values. This directly addresses the problem of identifying rows where member_id is missing.Here are some relevant links for further research:

Apache Spark SQL Functions: https://spark.apache.org/docs/latest/api/sql/ (Look for count_if in the documentation).
SQL IS NULL operator: https://www.w3schools.com/sql/sql_isnull.asp
Databricks SQL Documentation: https://docs.databricks.com/sql/language-manual/functions.html (Search for count_if)

Question: 62 CertyIQ

A data engineer needs to apply custom logic to identify employees with more than 5 years of experience in array column employees in table stores. The custom logic should create a new column exp_employees that is an array of all of the employees with more than 5 years of experience for each row. In order to apply this custom logic at scale, the data engineer wants to use the FILTER higher-order function.
Which of the following code blocks successfully completes this task?

A.
```
SELECT
    store_id,
    employees,
    FILTER (employees, i -> i.years_exp > 5) AS exp_employees
FROM stores;
```

B.
```
SELECT
    store_id,
    employees,
    FILTER (exp_employees, years_exp > 5) AS exp_employees
FROM stores;
```

C.
```
SELECT
    store_id,
    employees,
    FILTER (employees, years_exp > 5) AS exp_employees
FROM stores;
```

D.
```
SELECT
    store_id,
    employees,
    CASE WHEN employees.years_exp > 5 THEN employees
         ELSE NULL
    END AS exp_employees
FROM stores;
```

E.
```
SELECT
    store_id,
    employees,
    FILTER (exp_employees, i -> i.years_exp > 5) AS exp_employees
FROM stores;
```

Answer: A

Explanation:

```
SELECT
    store_id,
    employees,
    FILTER (employees, i -> i.years_exp > 5) AS exp_employees
FROM stores;
```

Question: 63 CertyIQ

A data engineer has a Python variable table_name that they would like to use in a SQL query. They want to construct a Python code block that will run the query using table_name.
They have the following incomplete code block:
____(f"SELECT customer_id, spend FROM table_name ")
Which of the following can be used to fill in the blank to successfully complete the task?

A.spark.delta.sql
B.spark.delta.table
C.spark.table
D.dbutils.sql
E.spark.sql

Answer: E

Explanation:

Here's a detailed justification for why option E, spark.sql, is the correct answer and why the other options are incorrect when constructing a dynamic SQL query using a Python variable in Databricks:

The core requirement is to execute a SQL query string that incorporates a Python variable, table_name. Databricks leverages Apache Spark for data processing. Spark provides several methods for interacting with data, including its own SQL engine.

spark.sql(query_string) is the standard and most direct way to execute a SQL query within a SparkSession (represented by spark in Databricks notebooks). It takes a string as input, which can be dynamically constructed using Python's f-strings to embed the table_name variable. The result of spark.sql is a Spark DataFrame, which can then be further processed.Apache Spark SQL Programming Guide - Look for the section describing running SQL queries programmatically.

spark.delta.sql is specific to Delta Lake functionality. While Delta Lake uses SQL syntax, spark.delta.sql isn't a standard function for executing arbitrary SQL queries with variable interpolation.

spark.delta.table is used to create or refer to a Delta table object, not to execute general SQL queries. It's used when you want to perform Delta Lake-specific operations (like time travel or schema evolution).

spark.table is used to access an existing table or view as a Spark DataFrame. It does not execute SQL queries.

dbutils.sql is not a valid method provided by Databricks dbutils module. dbutils provides a variety of utility functions within Databricks notebooks, mostly related to interacting with the Databricks environment (e.g., file

system, secrets).

Therefore, using spark.sql(f"SELECT customer_id, spend FROM table_name ") correctly constructs a dynamic SQL query using the value of table_name and executes it using Spark's SQL engine. This ensures the query will run against the appropriate table and return a Spark DataFrame.

Question: 64 CertyIQ

A data engineer has created a new database using the following command:
CREATE DATABASE IF NOT EXISTS customer360;
In which of the following locations will the customer360 database be located?

A.dbfs:/user/hive/database/customer360
B.dbfs:/user/hive/warehouse
C.dbfs:/user/hive/customer360
D.More information is needed to determine the correct response
E.dbfs:/user/hive/database

Answer: B

Explanation:

The correct answer is **B. dbfs:/user/hive/warehouse**. Here's why:

When a database is created in Databricks without explicitly specifying a LOCATION, it defaults to using the Hive metastore's managed location. The Hive metastore stores metadata about your tables and databases, including their location. The default location for managed tables and databases in Databricks is dbfs:/user/hive/warehouse.

The CREATE DATABASE IF NOT EXISTS customer360; command creates a database named customer360 if one doesn't already exist. Because no LOCATION clause is included in the command, the database's data files will be stored in the default Hive warehouse location.

Option A (dbfs:/user/hive/database/customer360) is incorrect because this path is usually where the database directory would be if the location were explicitly specified during the database creation. However, without a LOCATION clause, the database itself isn't created inside a 'database' subfolder.

Option C (dbfs:/user/hive/customer360) is incorrect because the default path does not directly append the database name to /user/hive/.

Option D is incorrect because sufficient information is provided in the command to determine the default location. The lack of a LOCATION clause is key.

Option E (dbfs:/user/hive/database) is incorrect because the tables will exist within the default directory(/user/hive/warehouse).

In summary, in absence of the LOCATION clause in the CREATE DATABASE statement, Databricks utilizes the default Hive warehouse location for storing metadata.

Further research:

Databricks documentation on Hive Metastore: https://docs.databricks.com/data/metastores/index.html
Databricks documentation on CREATE DATABASE: https://docs.databricks.com/sql/language-manual/sql-

ref-syntax-ddl-create-database.html - Check the section on the LOCATION clause.

Question: 65 CertyIQ

A data engineer is attempting to drop a Spark SQL table my_table and runs the following command:
DROP TABLE IF EXISTS my_table;
After running this command, the engineer notices that the data files and metadata files have been deleted from the file system.
Which of the following describes why all of these files were deleted?

A.The table was managed

B. The table's data was smaller than 10 GB

C.The table's data was larger than 10 GB

D.The table was external

E.The table did not have a location

Answer: A

Explanation:

The correct answer is A: The table was managed.

When a DROP TABLE command is executed in Spark SQL, the behavior differs significantly depending on whether the table is a managed (or internal) table or an external table. A managed table's data and metadata are completely under Spark SQL's control. This implies that Spark SQL manages the data's storage location and its schema. When a managed table is dropped, Spark SQL not only removes the table's metadata from the metastore (e.g., Hive metastore or Databricks metastore) but also deletes the underlying data files from the storage location (usually cloud storage like AWS S3, Azure Blob Storage, or ADLS Gen2).

In contrast, an external table points to data stored in an external location. The DROP TABLE command for an external table only removes the metadata from the metastore, leaving the data in the external location untouched. This allows for scenarios where you want to remove the table definition from Spark SQL without losing the underlying data. This is useful when the data is managed by another system or you want to retain the data for other purposes.

The question specifically states that the data files and metadata files were deleted. This behavior definitively points to the table being a managed table. Options B and C concerning table size are irrelevant; the size of the table data does not affect whether the data files are deleted upon dropping the table. Option D is incorrect because if the table was external, only the metadata would be dropped, leaving the data files untouched. Option E is incorrect because a table always has a defined type, so saying "the table did not have a location" doesn't accurately specify the table type. The key factor is that DROP TABLE deletes the data for a managed table.

Further reading:

Apache Spark SQL Documentation on Tables: https://spark.apache.org/docs/latest/sql-data-sources-hive-tables.html
Databricks documentation on creating tables: https://docs.databricks.com/en/sql/language-manual/sql-ref-syntax-ddl-create-table.html - Pay attention to the CREATE TABLE syntax and the difference between specifying LOCATION and not specifying it.

Question: 66 CertyIQ

A data engineer that is new to using Python needs to create a Python function to add two integers together and return the sum?
Which of the following code blocks can the data engineer use to complete this task?

A.
```
function add_integers(x, y):
    return x + y
```

B.
```
function add_integers(x, y):
    x + y
```

C.
```
def add_integers(x, y):
    print(x + y)
```

D.
```
def add_integers(x, y):
    return x + y
```

E.
```
def add_integers(x, y):
    x + y
```

Answer: D

Explanation:

```
def add_integers(x, y):
    return x + y
```

Question: 67 CertyIQ

In which of the following scenarios should a data engineer use the MERGE INTO command instead of the INSERT INTO command?

A.When the location of the data needs to be changed
B.When the target table is an external table
C.When the source table can be deleted
D.When the target table cannot contain duplicate records
E.When the source is not a Delta table

Answer: D

Explanation:

The correct answer is **D. When the target table cannot contain duplicate records.**

The MERGE INTO command in Databricks is specifically designed for performing upsert operations, which are combinations of update and insert operations. Upsert is crucial when maintaining data integrity and ensuring uniqueness in a target table. It allows you to update existing records in the target table based on matching conditions with the source data and insert new records from the source if no match is found in the target. This inherently addresses the scenario where the target table cannot contain duplicate records by either updating existing matching records or inserting only non-existing ones.

In contrast, the INSERT INTO command simply appends new records to the target table without checking for duplicates. If the source data contains records that already exist in the target table, INSERT INTO will create duplicates. Option A, changing data location, is typically handled through ALTER TABLE commands or data lake management features. Option B, concerning external tables, doesn't directly relate to the choice between MERGE INTO and INSERT INTO; both can be used with external tables. Option C, deleting the source table, is irrelevant to whether to use MERGE INTO or INSERT INTO. The decision hinges on the need to maintain data uniqueness in the target table.

Therefore, the key difference lies in MERGE INTO's ability to handle both updates and inserts while preventing duplicates in the target table, making it ideal when data uniqueness is a requirement. INSERT INTO only handles inserts and does not consider existing data, leading to potential duplication issues.

For more information, refer to the Databricks documentation on the MERGE INTO command: https://docs.databricks.com/sql/language-manual/delta-merge-into.html and for general DML commands: https://docs.databricks.com/sql/language-manual/delta-dml.html. These resources offer comprehensive details on the command's syntax, behavior, and best practices.

Question: 68 CertyIQ

A data engineer is working with two tables. Each of these tables is displayed below in its entirety.

sales

customer_id	spend	units
a1	28.94	7
a3	874.12	23
a4	8.99	1

favorite_stores

customer_id	store_id
a1	s1
a2	s1
a4	s2

The data engineer runs the following query to join these tables together:

```
SELECT
    sales.customer_id,
    sales.spend,
    favorite_stores.store_id
FROM sales
LEFT JOIN favorite_stores
ON sales.customer_id = favorite_stores.customer_id;
```

Which of the following will be returned by the above query?

A.

customer_id	spend	store_id
a1	28.94	s1
a4	8.99	s2

B.

customer_id	spend	units	store_id
a1	28.94	7	s1
a4	8.99	1	s2

C.

customer_id	spend	store_id
a1	28.94	s1
a3	874.12	NULL
a4	8.99	s2

D.

customer_id	spend	store_id
a1	28.94	s1
a2	NULL	s1
a3	874.12	NULL
a4	8.99	s2

E.

customer_id	spend	store_id
a1	28.94	s1
a2	NULL	s1
a4	8.99	s2

Answer: C

Explanation:

A typical LEFT JOIN scenario.

The LEFT JOIN keyword returns all records from the left table, even if there are no matches in the right table.

Question: 69 CertyIQ

A data engineer needs to create a table in Databricks using data from a CSV file at location /path/to/csv.
They run the following command:

```
CREATE TABLE new_table

_______
OPTIONS (
  header = "true",
  delimiter = "|"
)
LOCATION "path/to/csv"
```

Which of the following lines of code fills in the above blank to successfully complete the task?

A.None of these lines of code are needed to successfully complete the task
B.USING CSV
C.FROM CSV
D.USING DELTA
E.FROM "path/to/csv"

Answer: B

Explanation:

Correct answer is B:USING CSV.

Question: 70 CertyIQ

A data engineer has configured a Structured Streaming job to read from a table, manipulate the data, and then perform a streaming write into a new table.
The code block used by the data engineer is below:

```
(spark.readStream
    .table("sales")
    .withColumn("avg_price", col("sales") / col("units"))
    .writeStream
    .option("checkpointLocation", checkpointPath)
    .outputMode("complete")
    .______
    .table("new_sales")
)
```

If the data engineer only wants the query to process all of the available data in as many batches as required, which of the following lines of code should the data engineer use to fill in the blank?

A.processingTime(1)
B.trigger(availableNow=True)
C.trigger(parallelBatch=True)
D.trigger(processingTime="once")
E.trigger(continuous="once")

Answer: B

Explanation:

Correct answer is B:trigger(availableNow=True).

Question: 71 CertyIQ

A data engineer has developed a data pipeline to ingest data from a JSON source using Auto Loader, but the engineer has not provided any type inference or schema hints in their pipeline. Upon reviewing the data, the data engineer has noticed that all of the columns in the target table are of the string type despite some of the fields only including float or boolean values.
Which of the following describes why Auto Loader inferred all of the columns to be of the string type?

A.There was a type mismatch between the specific schema and the inferred schema
B.JSON data is a text-based format
C.Auto Loader only works with string data
D.All of the fields had at least one null value
E.Auto Loader cannot infer the schema of ingested data

Answer: B

Explanation:

The correct answer is **B. JSON data is a text-based format.**

Here's a detailed justification:

Auto Loader in Databricks is designed to automatically infer the schema of incoming data, simplifying data ingestion pipelines. However, schema inference isn't always perfect, especially when dealing with semi-structured data like JSON.

JSON (JavaScript Object Notation) is inherently a text-based data format. While JSON supports various data types like numbers, booleans, and arrays, these are all represented as text within the JSON structure. When Auto Loader encounters a JSON file without explicit schema hints, it analyzes the data to determine the most appropriate data type for each field.

Because everything in the original JSON is represented as text and without further information or configurations, Auto Loader plays it safe and interprets fields as strings. This is a common and expected behavior when dealing with schemaless or schema-on-read scenarios. Auto Loader doesn't "know" that a particular field should be an integer or a boolean simply by seeing textual representations like "123" or "true".

Option A is incorrect because there's no pre-defined schema in this scenario. The entire point is that Auto Loader is attempting to infer the schema. A type mismatch would only occur if you provide a schema that conflicts with the data.

Option C is incorrect because Auto Loader certainly works with data types other than string. The inference process aims to recognize various data types if possible.

Option D is incorrect as while null values can influence data type inference, they do not automatically force all columns to be strings. Typically, a column with null values will be nullable but of its inferred type if other non-null values can be used to ascertain the right type.

Option E is incorrect because Auto Loader does attempt to infer the schema, it's a core feature. This question specifically deals with a situation where schema inference leads to an unexpected result.

To ensure correct data types, users should use schema hints or schema evolution features of Auto Loader. Schema hints can be passed in as options when creating your stream. Using the .option("cloudFiles.schemaHints", "col1 string, col2 int") can define specific column types when the inferred types are not what is desired. You can enable schema evolution with .option("cloudFiles.schemaLocation", "<path-to-checkpoint>") which will allow it to use a schema from a location that contains the current structure.

Further Research:

Databricks Auto Loader Documentation: [https://docs.databricks.com/delta/autos loader.html] (https://docs.databricks.com/delta/autos loader.html)
JSON Data Format: https://www.json.org/

Question: 72 CertyIQ

A Delta Live Table pipeline includes two datasets defined using STREAMING LIVE TABLE. Three datasets are defined against Delta Lake table sources using LIVE TABLE.
The table is configured to run in Development mode using the Continuous Pipeline Mode.
Assuming previously unprocessed data exists and all definitions are valid, what is the expected outcome after clicking Start to update the pipeline?

A.All datasets will be updated once and the pipeline will shut down. The compute resources will be terminated.

B.All datasets will be updated at set intervals until the pipeline is shut down. The compute resources will persist until the pipeline is shut down.

C.All datasets will be updated once and the pipeline will persist without any processing. The compute resources will persist but go unused.

D.All datasets will be updated once and the pipeline will shut down. The compute resources will persist to allow for additional testing.

E.All datasets will be updated at set intervals until the pipeline is shut down. The compute resources will persist to allow for additional testing.

Answer: E

Explanation:

Here's a detailed justification for why option E is the correct answer:

The question specifies a Delta Live Tables (DLT) pipeline operating in Development mode with Continuous Pipeline Mode. This configuration significantly affects how the pipeline processes data and manages compute resources.

First, let's understand Development mode. In this mode, DLT is designed to facilitate iterative development and testing. This means it prioritizes speed and resource efficiency for quick feedback loops. A key characteristic is the intention to retain compute resources for interactive analysis and modifications.

Second, Continuous Pipeline Mode dictates that the pipeline will continuously process new data as it arrives, rather than running as a batch job. The STREAMING LIVE TABLE declarations further solidify this continuous processing behavior, as these tables are specifically designed to ingest and transform streaming data incrementally. Since the pipeline is set up as continuous, all datasets, regardless of their origin (streaming or static), will be updated at set intervals.

Now, let's address the different dataset types. STREAMING LIVE TABLE datasets will continuously ingest and transform data as it becomes available. LIVE TABLE datasets defined against Delta Lake sources will also be updated incrementally in Continuous mode, picking up any changes made to the source tables.

Therefore, the pipeline will update all datasets at set intervals. Because it's in Development mode, the compute resources will persist after the initial update, allowing data engineers to inspect the results, make adjustments to the pipeline, and re-run it as needed without incurring the overhead of spinning up new clusters each time. This enables interactive development and rapid iteration. When the pipeline is no longer needed, it should be manually stopped to deallocate the compute resources.

Other options are incorrect for the following reasons:

A and D: These options suggest that the pipeline shuts down after a single update, which is not the behavior of Continuous mode.
B: Although datasets update at set intervals, the mention of indefinite operation is not correct. Continuous pipeline will persist unless shut down.
C: Suggests the pipeline will persist without processing, is inconsistent with continuous mode.

In summary, Development mode in Continuous pipeline mode enables rapid iteration and testing, involving continuous updates, and the preservation of compute resources for interactive analysis and testing.

For further reading, refer to the official Databricks documentation on Delta Live Tables:

Delta Live Tables concepts
Delta Live Tables settings

Question: 73 CertyIQ

Which of the following data workloads will utilize a Gold table as its source?

A.A job that enriches data by parsing its timestamps into a human-readable format

B.A job that aggregates uncleaned data to create standard summary statistics
C.A job that cleans data by removing malformatted records
D.A job that queries aggregated data designed to feed into a dashboard
E.A job that ingests raw data from a streaming source into the Lakehouse

Answer: D

Explanation:

The correct answer is **D. A job that queries aggregated data designed to feed into a dashboard**.

Here's why: Gold tables in the medallion architecture (Bronze, Silver, Gold) represent the highest quality, most refined, and business-ready data. They are designed for consumption by end-users or applications that require highly reliable and aggregated information. Dashboards typically visualize summarized, aggregated data. Therefore, a dashboard directly benefits from the refined data provided by a Gold table.

Let's eliminate the other options:

A. A job that enriches data by parsing its timestamps into a human-readable format: Enrichment typically occurs in the Silver layer after initial cleansing and standardization.
B. A job that aggregates uncleaned data to create standard summary statistics: Aggregating uncleaned data doesn't belong in the Gold layer. Aggregation should happen after cleaning and transformation, using Silver data.
C. A job that cleans data by removing malformatted records: Cleaning is primarily done in the Silver layer (transforming Bronze data).
E. A job that ingests raw data from a streaming source into the Lakehouse: Ingestion of raw data goes directly into the Bronze layer.

In summary, Gold tables are the final stage in the data refinement process, providing clean, aggregated, and business-ready data specifically designed for consumption, such as feeding a dashboard. The medallion architecture prioritizes incremental refinement of data as it flows from Bronze (raw) to Silver (cleaned and transformed) to Gold (aggregated and business-ready). This makes Gold tables the ideal source for reporting and visualization tools that require pre-processed, reliable information.

Authoritative Links:

Databricks Medallion Architecture: https://www.databricks.com/blog/2019/08/15/diving-into-delta-lake-unpacking-the-medallion-architecture.html (This blog post provides a detailed explanation of the Medallion Architecture)
Delta Lake Documentation: https://docs.delta.io/latest/index.html (Official documentation for Delta Lake, the underlying technology for the Medallion Architecture)

Question: 74 CertyIQ

Which of the following must be specified when creating a new Delta Live Tables pipeline?

A.A key-value pair configuration
B.The preferred DBU/hour cost
C.A path to cloud storage location for the written data
D.A location of a target database for the written data
E.At least one notebook library to be executed

Answer: E

Explanation:

E. At least one notebook library to be executed.

When creating a Delta Live Tables (DLT) pipeline in Databricks, the system must know which code to run. This is specified by providing at least one notebook library (or source file) that contains the DLT SQL or Python code. This is the only essential element for a basic, functional pipeline.

Why other options are incorrect

A. A key-value pair configuration is used for optional parameters but not a required field to create the pipeline.

B. The preferred DBU/hour cost is not a parameter specified during pipeline creation. Cost is managed through other mechanisms like cluster policies and job configuration.

C. A path to cloud storage location for the written data is optional. DLT will use a default storage location if one is not specified.

D. A location of a target database for the written data is optional. Data can be processed and stored without immediately being registered in a specific target database.

Question: 75 CertyIQ

A data engineer has joined an existing project and they see the following query in the project repository:
CREATE STREAMING LIVE TABLE loyal_customers AS
SELECT customer_id -
FROM STREAM(LIVE.customers)
WHERE loyalty_level = 'high';
Which of the following describes why the STREAM function is included in the query?

A.The STREAM function is not needed and will cause an error.
B.The table being created is a live table.
C.The customers table is a streaming live table.
D.The customers table is a reference to a Structured Streaming query on a PySpark DataFrame.
E.The data in the customers table has been updated since its last run.

Answer: C

Explanation:

The correct answer is C: The customers table is a streaming live table.

Here's why:

The CREATE STREAMING LIVE TABLE syntax in Databricks SQL signifies that we are defining a table as part of a Delta Live Tables (DLT) pipeline designed for incremental data processing. DLT pipelines are specifically built to handle streaming data, allowing continuous data ingestion and transformation.

The STREAM function in STREAM(LIVE.customers) is crucial because it tells the DLT engine that LIVE.customers is a streaming live table itself, meaning it continuously receives updates. If customers were a regular, batch-oriented live table, then using STREAM would not be appropriate. The LIVE. prefix indicates the

table is managed within the DLT environment.

In a DLT pipeline, data flows from one streaming live table to another. In this case, loyal_customers is reading updates (streams) from LIVE.customers to identify high-loyalty customers, instead of reading all available LIVE.customers data at once.

Option A is incorrect because the STREAM function is essential when reading from a streaming live table within another streaming live table definition in a DLT pipeline. Removing it would disrupt the streaming data flow, which is the core function of DLT.

Option B is partially correct in that the table being created is a live table; however, it doesn't explain the usage of the STREAM function. The function's presence is directly linked to the input table's streaming nature.

Option D is incorrect because while Structured Streaming is related to streaming data processing in Spark, this syntax STREAM(LIVE.customers) specifically refers to interacting with DLT managed streaming live tables, not directly to Spark DataFrames.

Option E might be a characteristic of streaming data, but the STREAM function's main purpose isn't to indicate updates; instead, it's to access the continuous stream of updates from a streaming live table. The DLT engine internally manages the update handling.

In short, the STREAM function is used to read the streaming data from a source streaming live table within a Delta Live Tables pipeline, ensuring that the destination table (loyal_customers) is updated incrementally as new data arrives in the source table (customers).

For authoritative information, refer to the official Databricks documentation on Delta Live Tables:

Delta Live Tables
Create a Live Table
Streaming Live Tables

Question: 76 CertyIQ

Which of the following describes the type of workloads that are always compatible with Auto Loader?

A.Streaming workloads
B.Machine learning workloads
C.Serverless workloads
D.Batch workloads
E.Dashboard workloads

Answer: A

Explanation:

Auto Loader in Databricks is specifically designed to efficiently and incrementally ingest new data as it arrives in cloud storage (like AWS S3, Azure Data Lake Storage Gen2, or Google Cloud Storage). This "as it arrives" nature makes it fundamentally suited for streaming workloads. Streaming workloads involve the continuous flow of data, where new data points are constantly being added. Auto Loader automatically detects new files as they land in the storage location and processes them in a streaming fashion, without requiring manual triggering or scheduling.

Batch workloads, in contrast, involve processing a large, finite set of data all at once. While Auto Loader can

be used with batch workloads initially, its strengths lie in its ability to handle the continuous arrival of new data, not in processing a pre-existing, fixed dataset. Serverless workloads are more about the infrastructure aspect of not managing servers, and not directly related to the type of data ingestion. Machine learning workloads and dashboard workloads are application areas that may consume data ingested by Auto Loader, but Auto Loader itself isn't intrinsically tied to these workload types. The core capability of Auto Loader is optimized for the continuous processing of data streams, making it inherently compatible with streaming workloads. Its schema inference and evolution capabilities are particularly valuable in streaming scenarios where data schemas might change over time. Its incremental processing ensures that the data remains fresh and up-to-date, which is crucial in real-time applications.

Therefore, the characteristic of incremental and continuous data ingestion directly aligns with the definition of Streaming workloads.

Further reading on Auto Loader and its use cases:

Databricks Auto Loader documentation: https://docs.databricks.com/delta/auto-loader/index.html

Question: 77 CertyIQ

A data engineer and data analyst are working together on a data pipeline. The data engineer is working on the raw, bronze, and silver layers of the pipeline using Python, and the data analyst is working on the gold layer of the pipeline using SQL. The raw source of the pipeline is a streaming input. They now want to migrate their pipeline to use Delta Live Tables.
Which of the following changes will need to be made to the pipeline when migrating to Delta Live Tables?

A.None of these changes will need to be made
B.The pipeline will need to stop using the medallion-based multi-hop architecture
C.The pipeline will need to be written entirely in SQL
D.The pipeline will need to use a batch source in place of a streaming source
E.The pipeline will need to be written entirely in Python

Answer: A

Explanation:

The correct answer is A, "None of these changes will need to be made." Here's why:

Delta Live Tables (DLT) is designed to simplify the development and deployment of reliable data pipelines. It embraces and enhances existing best practices, not replaces them.

Medallion Architecture: DLT fully supports the medallion architecture (bronze, silver, gold). This approach of progressively refining data is fundamental to building robust data lakes and data warehouses. DLT streamlines the implementation by handling dependency management and data quality checks, but it doesn't force you to abandon the pattern.
Language Support: DLT pipelines can be built using both Python and SQL. This allows data engineers and data analysts to continue using their preferred languages and skillsets within the same pipeline. DLT simply provides a declarative framework for orchestrating these transformations.
Streaming Support: DLT natively supports both streaming and batch data sources. It automatically manages the complexities of incremental data processing for streaming sources. This means the existing streaming source can be seamlessly integrated into the DLT pipeline. DLT handles the necessary state management and checkpointing implicitly.

Therefore, the existing pipeline already aligns with the core principles of DLT. Migrating to DLT would

primarily involve refactoring the existing code into DLT-compatible notebooks or files and defining the data dependencies within the DLT framework. The underlying logic and data flow can remain largely unchanged.

For further research, consult the official Databricks Delta Live Tables documentation:

Delta Live Tables Introduction: https://docs.databricks.com/delta-live-tables/index.html
Delta Live Tables Python API: https://docs.databricks.com/delta-live-tables/delta-live-tables-python.html
Delta Live Tables SQL API: https://docs.databricks.com/delta-live-tables/delta-live-tables-sql.html

Question: 78 CertyIQ

A data engineer is using the following code block as part of a batch ingestion pipeline to read from a composable table:

```
transactions_df = (spark.read
      .schema(schema)
      .format("delta")
      .table("transactions")
)
```

Which of the following changes needs to be made so this code block will work when the transactions table is a stream source?

A.Replace predict with a stream-friendly prediction function
B.Replace schema(schema) with option ("maxFilesPerTrigger", 1)
C.Replace "transactions" with the path to the location of the Delta table
D.Replace format("delta") with format("stream")
E.Replace spark.read with spark.readStream

Answer: E

Explanation:

Replace spark.read with spark.readStream.

Question: 79 CertyIQ

Which of the following queries is performing a streaming hop from raw data to a Bronze table?

A.

```
(spark.table("sales")
    .groupBy("store")
    .agg(sum("sales"))
    .writeStream
    .option("checkpointLocation", checkpointPath)
    .outputMode("complete")
    .table("newSales")
)
```

B.

```
(spark.table("sales")
    .filter(col("units") > 0)
    .writeStream
    .option("checkpointLocation", checkpointPath)
    .outputMode("append")
    .table("newSales")
)
```

C.

```
(spark.table("sales")
    .withColumn("avgPrice", col("sales") / col("units"))
    .writeStream
    .option("checkpointLocation", checkpointPath)
    .outputMode("append")
    .table("newSales")
)
```

D.

```
(spark.read.load(rawSalesLocation)
    .write
    .mode("append")
    .table("newSales")
)
```

E.

```
(spark.readStream.load(rawSalesLocation)
      .writeStream
      .option("checkpointLocation", checkpointPath)
      .outputMode("append")
      .table("newSales")
)
```

Answer: E

Explanation:

```
(spark.readStream.load(rawSalesLocation)
      .writeStream
      .option("checkpointLocation", checkpointPath)
      .outputMode("append")
      .table("newSales")
)
```

Question: 80 CertyIQ

A dataset has been defined using Delta Live Tables and includes an expectations clause:
CONSTRAINT valid_timestamp EXPECT (timestamp > '2020-01-01') ON VIOLATION FAIL UPDATE
What is the expected behavior when a batch of data containing data that violates these constraints is processed?

A.Records that violate the expectation are dropped from the target dataset and recorded as invalid in the event log.

B.Records that violate the expectation cause the job to fail.

C.Records that violate the expectation are dropped from the target dataset and loaded into a quarantine table.

D.Records that violate the expectation are added to the target dataset and recorded as invalid in the event log.

E.Records that violate the expectation are added to the target dataset and flagged as invalid in a field added to the target dataset.

Answer: B

Explanation:

The correct answer is B because the ON VIOLATION FAIL UPDATE clause in the Delta Live Tables (DLT) pipeline specifies the action to be taken when an expectation (constraint) is violated. Let's break down why the other options are incorrect:

Option A: This behavior corresponds to ON VIOLATION DROP ROW, where violating records are dropped and logged.
Option C: Quarantine tables are not a standard feature for expectation failures in DLT. While you could

manually implement similar logic with more complex transformations, it's not the default behavior for built-in expectations.
Option D: Adding invalid records and logging them goes against the purpose of data quality checks, especially when FAIL UPDATE is specified.
Option E: DLT doesn't automatically add an "invalid" flag field to the dataset.

The ON VIOLATION FAIL UPDATE setting is designed for scenarios where data quality is critical, and any violation of the specified expectation should halt the pipeline's progress. This ensures that downstream processes are not affected by potentially incorrect or incomplete data. When a violation is detected, the DLT pipeline stops processing the current batch and raises an error, preventing further updates to the target dataset until the data quality issue is resolved. This approach emphasizes data correctness and allows data engineers to investigate and fix the problem before proceeding. This approach is useful for situations where the data quality must be guaranteed for downstream processes and decision-making.

Authoritative Links:

Delta Live Tables Expectations: https://docs.databricks.com/delta-live-tables/expectations.html (Specifically, see the section on "Handle expectation failures")
Delta Live Tables Error Handling: (Refer to general DLT documentation on error handling and pipeline monitoring on the Databricks website.)

The FAIL UPDATE option provides the strongest guarantee of data quality within a DLT pipeline.

Question: 81 CertyIQ

Which of the following statements regarding the relationship between Silver tables and Bronze tables is always true?

A.Silver tables contain a less refined, less clean view of data than Bronze data.
B.Silver tables contain aggregates while Bronze data is unaggregated.
C.Silver tables contain more data than Bronze tables.
D.Silver tables contain a more refined and cleaner view of data than Bronze tables.
E.Silver tables contain less data than Bronze tables.

Answer: D

Explanation:

The correct answer is D: Silver tables contain a more refined and cleaner view of data than Bronze tables. This is fundamentally based on the Medallion Architecture, a common data engineering paradigm.

In the Medallion Architecture (Bronze, Silver, Gold), data progresses through layers of increasing refinement and business readiness. Bronze tables, also known as raw tables, store data directly from source systems with minimal transformation. They essentially provide a historical, immutable record of the data received. They are often implemented using Delta Lake for its ACID properties and schema evolution capabilities.

Silver tables sit in the middle layer. The purpose of this layer is to clean, transform, and conform the data from the Bronze layer. This includes tasks such as data cleansing (handling missing values, correcting errors), data standardization (ensuring consistent formats), data deduplication, and joining data from multiple Bronze tables.

Therefore, Silver tables inherently hold a more refined and cleaner view compared to Bronze. Data quality

checks and enforcement are implemented in the transition from Bronze to Silver. Silver tables represent data ready for further processing and analysis, having been scrubbed of inconsistencies present in the raw Bronze data.

Options A, B, C, and E are incorrect because they misrepresent the relationships and characteristics of Bronze and Silver layers within a medallion architecture. Option A states the opposite of the truth: Silver tables are more refined. Option B is incorrect because Silver tables don't necessarily contain aggregates; aggregation usually happens at the Gold level. Options C and E's claims about the relative data size are not guaranteed; Silver tables might have fewer rows due to deduplication or filtering but could also have more columns due to joins and transformations. The crucial difference is the data quality and refinement aspect, making option D the only correct choice.

For more information, you can refer to these resources:

Databricks Medallion Architecture: https://www.databricks.com/glossary/medallion-architecture
Delta Lake Documentation: https://docs.delta.io/latest/index.html - while Delta Lake is an implementation detail it is the most common method for building the architecture.

Question: 82 CertyIQ

A data engineering team has noticed that their Databricks SQL queries are running too slowly when they are submitted to a non-running SQL endpoint. The data engineering team wants this issue to be resolved.
Which of the following approaches can the team use to reduce the time it takes to return results in this scenario?

A.They can turn on the Serverless feature for the SQL endpoint and change the Spot Instance Policy to "Reliability Optimized."

B.They can turn on the Auto Stop feature for the SQL endpoint.

C.They can increase the cluster size of the SQL endpoint.

D.They can turn on the Serverless feature for the SQL endpoint.

E.They can increase the maximum bound of the SQL endpoint's scaling range.

Answer: D

Explanation:

D. They can turn on the Serverless feature for the SQL endpoint.

Turning on the Serverless feature for the Databricks SQL endpoint enables the use of serverless compute for the workspace. This significantly reduces the time for queries to return results when the endpoint is inactive because the serverless infrastructure maintains a pool of warm instances, which allows for near-instantaneous startup times compared to the cold start of a standard (Pro or Classic) endpoint.

Why other options are incorrect

A. They can turn on the Serverless feature for the SQL endpoint and change the Spot Instance Policy to "Reliability Optimized." The "Reliability Optimized" policy primarily deals with cluster stability during runtime once started, but the core issue is the initial startup time of a non-running endpoint. The serverless feature itself addresses this startup latency.

B. They can turn on the Auto Stop feature for the SQL endpoint. Auto Stop is used to terminate the endpoint after a period of inactivity to save costs, which would increase the likelihood of a cold start and slow query return times, not decrease them.

C. They can increase the cluster size of the SQL endpoint. Increasing the cluster size improves query

performance after the endpoint is running, but does not solve the initial slow startup time when the endpoint is in a non-running state.

E. They can increase the maximum bound of the SQL endpoint's scaling range. Like increasing cluster size, increasing the scaling range's maximum bound helps with concurrency and performance when the endpoint is active and under heavy load, but does not affect the cold start time of a stopped endpoint.

Question: 83 CertyIQ

A data engineer has a Job that has a complex run schedule, and they want to transfer that schedule to other Jobs. Rather than manually selecting each value in the scheduling form in Databricks, which of the following tools can the data engineer use to represent and submit the schedule programmatically?

A.pyspark.sql.types.DateType
B.datetime
C.pyspark.sql.types.TimestampType
D.Cron syntax
E.There is no way to represent and submit this information programmatically

Answer: D

Explanation:

The correct answer is **D. Cron syntax**. Here's why:

Databricks Jobs can be scheduled to run automatically based on a defined schedule. While the Databricks UI offers a visual scheduling form, more complex or repetitive schedules are better managed programmatically. Cron syntax provides a standardized, concise way to express recurring time-based schedules. It's a widely used convention supported by various scheduling systems, including those found in cloud platforms. Cron expressions consist of fields representing minutes, hours, days of the month, months, and days of the week, allowing for precise schedule definitions (e.g., "Run every day at 5 AM").

Options A, B, and C represent data types used for representing dates and timestamps within PySpark. These are useful for handling date and time data within a Spark application but not for defining the schedule of the Databricks Job itself. Option E is incorrect because Databricks Jobs do support programmatic scheduling, particularly through the use of Cron expressions.

By using Cron syntax, the data engineer can define the complex schedule in a text-based format, allowing for easy transfer to other Jobs via scripts, APIs, or infrastructure-as-code deployments. This promotes repeatability and reduces the risk of manual configuration errors. The Databricks Jobs API also supports the use of Cron expressions, making it the preferred method for programmatic scheduling.

Further research:

Databricks Jobs API: https://docs.databricks.com/api/workspace/jobs
Cron Expressions: https://en.wikipedia.org/wiki/Cron or search for "Cron syntax" for numerous online generators and explanations.

Question: 84 CertyIQ

Which of the following approaches should be used to send the Databricks Job owner an email in the case that the

Job fails?

A.Manually programming in an alert system in each cell of the Notebook
B.Setting up an Alert in the Job page
C.Setting up an Alert in the Notebook
D.There is no way to notify the Job owner in the case of Job failure
E.MLflow Model Registry Webhooks

Answer: B

Explanation:

The correct answer is B: Setting up an Alert in the Job page. Here's why:

Databricks Jobs offer built-in alerting mechanisms directly within the Job configuration. These alerts are specifically designed to notify job owners or other designated recipients about job status changes, including failures. This approach provides a centralized and easily configurable way to manage job monitoring and notifications.

Option A (Manually programming an alert system in each cell) is highly inefficient, error-prone, and difficult to maintain. Embedding alert logic within individual notebook cells would scatter monitoring responsibilities across the codebase, making it hard to manage and update. Furthermore, this would significantly bloat the notebook code and make it less readable.

Option C (Setting up an Alert in the Notebook) is not the intended use of notebooks. While notebooks can generate outputs and logs, the core purpose is for data engineering tasks, not centralized job monitoring and notification. While some notebook functionality might be used to trigger external alerting, the native Databricks Jobs interface is the preferred method.

Option D (There is no way to notify the Job owner in the case of Job failure) is factually incorrect, as Databricks Jobs directly supports email alerts.

Option E (MLflow Model Registry Webhooks) is also incorrect. MLflow Model Registry Webhooks are designed to trigger actions (like retraining or deployment) based on changes in the model registry, such as a new model version being created. They are not directly tied to the status of Databricks Jobs that might use those models. While it is possible to create workflows that would detect a failed job and then trigger a model redeployment via Webhooks, this is an indirect and more complicated solution.

Therefore, using the Databricks Job page alerts provides a straightforward, native, and maintainable solution for notifying job owners of failures.

Here are some helpful resources:

Databricks Jobs documentation: https://docs.databricks.com/workflows/jobs/index.html
Configure email alerts for Databricks Jobs: https://docs.databricks.com/workflows/jobs/jobs-alerts.html

Question: 85 CertyIQ

An engineering manager uses a Databricks SQL query to monitor ingestion latency for each data source. The manager checks the results of the query every day, but they are manually rerunning the query each day and waiting for the results.
Which of the following approaches can the manager use to ensure the results of the query are updated each day?

A.They can schedule the query to refresh every 1 day from the SQL endpoint's page in Databricks SQL.
B.They can schedule the query to refresh every 12 hours from the SQL endpoint's page in Databricks SQL.
C.They can schedule the query to refresh every 1 day from the query's page in Databricks SQL.
D.They can schedule the query to run every 1 day from the Jobs UI.
E.They can schedule the query to run every 12 hours from the Jobs UI.

Answer: C

Explanation:

The correct answer is **C. They can schedule the query to refresh every 1 day from the query's page in Databricks SQL.**

Here's why:

Databricks SQL Queries and Scheduling: Databricks SQL provides the ability to schedule queries directly from the query editor. This allows for automated execution and refreshing of results. This functionality is designed specifically for Databricks SQL queries and makes it the most straightforward method for the manager's goal. The schedule can be set to a daily interval, addressing the requirement that the query results be updated each day.
Refreshing vs. Running: Scheduling a query to refresh is different from scheduling a job to run the query. Refreshing a query in Databricks SQL uses caching mechanisms and optimized execution plans to efficiently update the results. Running the query as a new job might incur more overhead. The question implied that it is enough for the results to be updated, making the "refresh" option in Databricks SQL the more suited option.
Location of Scheduling: The scheduling option for Databricks SQL queries is found within the query's page. This makes management and monitoring easier since the schedule is directly associated with the query itself.
Why other options are incorrect:
A & B: While scheduling might be available from the SQL endpoint, it would be less direct. The scheduling is intrinsically tied to the query's execution logic, making it most appropriate to manage the schedule from the query's page.
D & E: While you could execute the query from the Jobs UI, this would be more suited to ad-hoc tasks. It is also less convenient because the query is already saved in Databricks SQL. Databricks SQL is made to manage the running and scheduling of SQL queries. Using the Jobs UI adds unnecessary complexity.

Relevant Links for further research:

Databricks SQL Query Scheduling: https://docs.databricks.com/sql/user/queries/query-schedule.html
Databricks Jobs: https://docs.databricks.com/workflows/jobs/index.html (For contrasting with Databricks SQL Query Scheduling).

Question: 86 CertyIQ

In which of the following scenarios should a data engineer select a Task in the Depends On field of a new Databricks Job Task?

A.When another task needs to be replaced by the new task
B.When another task needs to fail before the new task begins
C.When another task has the same dependency libraries as the new task
D.When another task needs to use as little compute resources as possible
E.When another task needs to successfully complete before the new task begins

Answer: E

Explanation:

The correct answer, E, "When another task needs to successfully complete before the new task begins," precisely describes the purpose of the Depends On field in Databricks Jobs. This field establishes a task dependency, meaning the designated "new task" will only initiate execution if the task it depends on has finished with a successful (or, depending on settings, with specific success/failure configurations) status.

This mechanism allows for orchestrating a sequence of operations within a job, where the output or state of one task is a prerequisite for another. Task dependencies are a fundamental concept in workflow management and data pipelines. Cloud computing platforms such as Databricks provide such features for dependency management to efficiently handle complex workflows. Option A is incorrect because the Depends On field does not replace tasks. Option B is wrong as the new task won't trigger if the dependent task fails without configuring it to do so. Option C is irrelevant since dependency libraries are typically managed at the job or cluster level, not individual task dependencies. Option D has no relationship to task dependencies; resource management is separate.

For example, imagine a data pipeline where you first extract data from a source (Task 1) and then transform it (Task 2). Task 2 should only begin after Task 1 has successfully extracted the data. By setting Task 1 as a dependency for Task 2, you guarantee this order of execution. If Task 1 fails, Task 2 won't start, preventing potential errors caused by missing input data. This ensures data integrity and efficient resource utilization. Databricks Jobs uses directed acyclic graphs (DAGs) to handle dependencies, where each task is a node, and the dependencies create edges.

Further reading can be found in the Databricks documentation on Jobs: https://docs.databricks.com/workflows/jobs/index.html and Task dependencies specifically: https://docs.databricks.com/workflows/jobs/jobs-concepts.html#task-dependencies.

Question: 87 CertyIQ

A data engineer has been using a Databricks SQL dashboard to monitor the cleanliness of the input data to a data analytics dashboard for a retail use case. The job has a Databricks SQL query that returns the number of store-level records where sales is equal to zero. The data engineer wants their entire team to be notified via a messaging webhook whenever this value is greater than 0.
Which of the following approaches can the data engineer use to notify their entire team via a messaging webhook whenever the number of stores with $0 in sales is greater than zero?

A.They can set up an Alert with a custom template.
B.They can set up an Alert with a new email alert destination.
C.They can set up an Alert with one-time notifications.
D.They can set up an Alert with a new webhook alert destination.
E.They can set up an Alert without notifications.

Answer: D

Explanation:

Here's a detailed justification for why option D is the correct answer, along with supporting explanations and links:

The core requirement is to trigger a notification to the team via a messaging webhook whenever a specific condition in the data (number of stores with zero sales) is met. Databricks SQL Alerts are designed to monitor

query results and trigger actions when those results meet predefined thresholds.

Option D, setting up an Alert with a new webhook alert destination, directly addresses this need. Webhooks allow Databricks to send automated HTTP requests to external services (like messaging platforms such as Slack, Microsoft Teams, etc.) when an alert condition is triggered. This enables real-time notification to the team whenever the "number of stores with $0 sales" query returns a value greater than zero. The data engineer can configure the webhook payload to include relevant information about the alert, allowing the team to quickly understand the issue.

Let's examine why the other options are not ideal:

A. They can set up an Alert with a custom template: While custom templates are useful for formatting alert messages, they don't address the fundamental requirement of sending the notification to a messaging webhook. Custom templates enhance the content of the notification, but they don't determine the delivery mechanism.
B. They can set up an Alert with a new email alert destination: Email notifications might be a viable option in some scenarios, but the question specifically requests a messaging webhook. Email is less immediate and collaborative compared to messaging platforms commonly used by teams.
C. They can set up an Alert with one-time notifications: One-time notifications would only trigger the alert once, which is not suitable for continuous monitoring of data cleanliness. The requirement is to be notified every time the condition is met.
E. They can set up an Alert without notifications: Setting up an alert without notifications defeats the entire purpose. The goal is to notify the team when the specified condition is met.

Therefore, leveraging Databricks SQL Alerts with a webhook destination is the optimal solution. It enables automated, real-time notification to the team's messaging platform whenever the query result (number of stores with zero sales) exceeds zero, ensuring prompt attention to data quality issues.

Supporting Links:

Databricks SQL Alerts: https://docs.databricks.com/sql/user/alerts/index.html
Databricks SQL Alerts - Webhooks: https://docs.databricks.com/sql/user/alerts/webhooks.html

Question: 88 CertyIQ

A data engineer wants to schedule their Databricks SQL dashboard to refresh every hour, but they only want the associated SQL endpoint to be running when it is necessary. The dashboard has multiple queries on multiple datasets associated with it. The data that feeds the dashboard is automatically processed using a Databricks Job. Which of the following approaches can the data engineer use to minimize the total running time of the SQL endpoint used in the refresh schedule of their dashboard?

A.They can turn on the Auto Stop feature for the SQL endpoint.
B.They can ensure the dashboard's SQL endpoint is not one of the included query's SQL endpoint.
C.They can reduce the cluster size of the SQL endpoint.
D.They can ensure the dashboard's SQL endpoint matches each of the queries' SQL endpoints.
E.They can set up the dashboard's SQL endpoint to be serverless.

Answer: A

Explanation:

The correct answer is A: They can turn on the Auto Stop feature for the SQL endpoint.

Here's why: The goal is to minimize the total running time (and therefore cost) of the Databricks SQL endpoint while still ensuring the dashboard refreshes hourly. Let's analyze the options:

A. They can turn on the Auto Stop feature for the SQL endpoint. This is the most effective approach. The Auto Stop feature automatically shuts down the SQL endpoint after a period of inactivity. Since the dashboard only needs the endpoint to be running during the refresh, setting Auto Stop ensures it shuts down after the refresh is complete, minimizing idle run time and associated costs. The endpoint will automatically restart when the next scheduled refresh is triggered. This approach directly addresses the requirement of only running the endpoint when necessary. Databricks documentation on SQL endpoint auto-stop confirms this functionality.
B. They can ensure the dashboard's SQL endpoint is not one of the included query's SQL endpoint. This is incorrect. The dashboard needs to be pointed at a single SQL endpoint so that its queries can be executed. This suggestion conflicts with the core architecture of Databricks SQL Dashboards.
C. They can reduce the cluster size of the SQL endpoint. Reducing the cluster size can lower the cost per hour while the endpoint is running, but it doesn't address the problem of minimizing total running time. A smaller cluster might also increase query execution time, potentially negating cost savings and impacting dashboard responsiveness.
D. They can ensure the dashboard's SQL endpoint matches each of the queries' SQL endpoints. This is generally not possible or desirable. It implies having multiple, independent SQL endpoints for each query, which complicates management and defeats the purpose of having a consolidated endpoint for the dashboard. Dashboards are typically connected to a single endpoint to execute all of their queries.
E. They can set up the dashboard's SQL endpoint to be serverless. While serverless SQL endpoints offer automatic scaling and resource management, the key benefit here is cost optimization when the endpoint is idle. However, serverless SQL endpoints are only available on AWS and only on specific Databricks Workspaces. If the environment does not support serverless, this option isn't applicable, making option A a more reliable overall approach. Even if serverless is an option, Auto Stop would still be beneficial to further optimize cost. Databricks serverless compute automatically scales and shuts down when not in use (similar to auto-stop), reducing operational burden and costs.

Therefore, Auto Stop is the most direct and universally applicable solution to minimize the SQL endpoint's total running time and associated costs when using a scheduled refresh. It's a standard feature in Databricks SQL endpoints designed for this purpose.

Question: 89 CertyIQ

A data engineer needs access to a table new_table, but they do not have the correct permissions. They can ask the table owner for permission, but they do not know who the table owner is.
Which of the following approaches can be used to identify the owner of new_table?

A.Review the Permissions tab in the table's page in Data Explorer
B.All of these options can be used to identify the owner of the table
C.Review the Owner field in the table's page in Data Explorer
D.Review the Owner field in the table's page in the cloud storage solution
E.There is no way to identify the owner of the table

Answer: C

Explanation:

The correct answer is **C. Review the Owner field in the table's page in Data Explorer.**

Here's why:

Databricks Data Explorer provides a user interface to browse and manage data objects in the metastore. A key function is displaying metadata about tables, including information about the owner. The "Owner" field directly displays the user or service principal who owns the table. This is the most straightforward and accessible method for a data engineer to identify the owner when working within the Databricks environment.

Option A, "Review the Permissions tab in the table's page in Data Explorer," is incorrect because while the Permissions tab shows who has various permissions (SELECT, MODIFY, etc.), it doesn't explicitly label one user as the "owner." The owner generally has implicit full permissions, but they're not identified as the owner on this tab.

Option D, "Review the Owner field in the table's page in the cloud storage solution," is incorrect because the ownership in the underlying cloud storage (like AWS S3 or Azure Data Lake Storage) is separate from the ownership managed by the Databricks metastore. While the storage location has its own permissions and ownership settings, those don't necessarily translate directly to the metastore owner. Databricks manages access control on top of the underlying storage.

Option E, "There is no way to identify the owner of the table," is incorrect because, as discussed above, Data Explorer provides this functionality.

Option B, "All of these options can be used to identify the owner of the table," is incorrect because options A and D are not direct or reliable methods to find the Databricks metastore owner of the table.

In summary, the most direct and appropriate method to determine the owner of a table within a Databricks workspace is to check the "Owner" field displayed in the table's page within Data Explorer. This feature is specifically designed to provide metadata, including ownership information, for managed tables.

For further research, you can refer to the official Databricks documentation on Data Explorer and Access Control:

Databricks Data Explorer: https://docs.databricks.com/data-governance/data-explorer/index.html
Databricks Access Control: https://docs.databricks.com/security/access-control/index.html

Question: 90 CertyIQ

A new data engineering team team has been assigned to an ELT project. The new data engineering team will need full privileges on the table sales to fully manage the project.
Which of the following commands can be used to grant full permissions on the database to the new data engineering team?

A.GRANT ALL PRIVILEGES ON TABLE sales TO team;
B.GRANT SELECT CREATE MODIFY ON TABLE sales TO team;
C.GRANT SELECT ON TABLE sales TO team;
D.GRANT USAGE ON TABLE sales TO team;
E.GRANT ALL PRIVILEGES ON TABLE team TO sales;

Answer: A

Explanation:

The correct answer is A, GRANT ALL PRIVILEGES ON TABLE sales TO team;. This command provides the team with comprehensive control over the sales table. Here's why the other options are incorrect:

B. GRANT SELECT CREATE MODIFY ON TABLE sales TO team; This grants specific privileges (SELECT, CREATE, MODIFY) but may not cover all necessary permissions for full management, such as DROP, OWNER, or potentially other administrative tasks.

C. GRANT SELECT ON TABLE sales TO team; This only allows the team to read data from the sales table and nothing else. It's insufficient for full management.

D. GRANT USAGE ON TABLE sales TO team; The USAGE privilege is typically used for schemas (databases), not tables directly, to grant access to objects within the schema. It doesn't grant permissions for manipulating the data within the table.

E. GRANT ALL PRIVILEGES ON TABLE team TO sales; This is syntactically incorrect. The GRANT statement should grant privileges on the object (in this case, the sales table) to the principal (the team). The order is reversed. Furthermore, "team" is assumed to be a user or group, not an object.

GRANT ALL PRIVILEGES provides the new data engineering team the ability to select, insert, update, delete, alter, and drop the sales table. This level of access enables them to perform all the operations necessary for their ELT project. This command grants the team the highest level of access, which aligns with the requirement of having "full privileges" to fully manage the project. Databricks SQL uses ANSI SQL syntax for GRANT and REVOKE statements. Therefore, the statement grants all object-specific privileges associated with table sales to the group or user team. The concept of assigning privileges to teams (groups) streamlines permission management, ensuring consistent access control. * Databricks SQL Privileges and Grantable Privileges * Databricks SQL GRANT statement

Question: 91 CertyIQ

Which data lakehouse feature results in improved data quality over a traditional data lake?

A.A data lakehouse stores data in open formats.
B.A data lakehouse allows the use of SQL queries to examine data.
C.A data lakehouse provides storage solutions for structured and unstructured data.
D.A data lakehouse supports ACID-compliant transactions.

Answer: D

Explanation:

The correct answer is D, a data lakehouse supports ACID-compliant transactions. This feature significantly improves data quality compared to a traditional data lake because it ensures data consistency and reliability.

Traditional data lakes often lack ACID properties (Atomicity, Consistency, Isolation, Durability). This means that concurrent read and write operations can lead to data corruption or inconsistent query results. For example, if one process is updating a table while another is reading it, the reader might see partially updated data, leading to inaccurate analytics.

ACID transactions, on the other hand, guarantee that database operations are performed reliably. Atomicity ensures that a transaction is treated as a single, indivisible unit of work - either all changes are applied, or none. Consistency ensures that a transaction only brings the database from one valid state to another. Isolation ensures that concurrent transactions do not interfere with each other. Durability guarantees that once a transaction is committed, it remains committed even in the event of a system failure.

By supporting ACID transactions, a data lakehouse prevents data corruption and ensures that all users see a

consistent view of the data, leading to improved data quality and more reliable analytics. This is crucial for business intelligence, machine learning, and other data-driven applications. Features like Delta Lake, which provide ACID transactions on top of data lakes, are key to making data lakehouses a valuable tool for data engineering.

Further Research:

Databricks Delta Lake Documentation
What is a Data Lakehouse? - Databricks

Question: 92 CertyIQ

In which scenario will a data team want to utilize cluster pools?

A.An automated report needs to be version-controlled across multiple collaborators.
B.An automated report needs to be runnable by all stakeholders.
C.An automated report needs to be refreshed as quickly as possible.
D.An automated report needs to be made reproducible.

Answer: C

Explanation:

The correct answer, C (An automated report needs to be refreshed as quickly as possible), is the most fitting scenario for utilizing Databricks cluster pools. Cluster pools are designed primarily to accelerate cluster startup times. In a traditional Databricks setup, creating a new cluster requires allocating resources, installing libraries, and initializing the environment, which can take several minutes. This delay becomes a bottleneck when you have automated reports that need to be generated frequently, such as hourly or even more often.

Cluster pools pre-allocate and maintain a set of idle instances, ready to be used by new clusters. When an automated report job requests a cluster, Databricks can quickly grab an instance from the pool, drastically reducing the cluster startup time. This directly translates to a faster report refresh cycle. This speed improvement is crucial for time-sensitive reporting applications.

Option A is incorrect because version control is typically managed using tools like Git, not cluster pools. Option B is incorrect because running reports for all stakeholders is more about access control and sharing permissions than cluster startup speed. Option D is incorrect because reproducibility is addressed through defining consistent environments (Databricks Repos, Databricks CLI with job definitions) and managing dependencies, not primarily through cluster pools. While reproducibility could be indirectly assisted by using a cluster pool that consistently provides the same environment, it's not the primary reason to use pools.

Therefore, the speed enhancement offered by cluster pools is the key factor driving their use for quickly refreshing automated reports. The ability to rapidly allocate resources minimizes delays and ensures reports are generated and distributed in a timely manner, enabling faster decision-making based on near real-time data.

Refer to Databricks documentation for detailed information on cluster pools and their functionalities:

Databricks Cluster Pools
Best practices: Using cluster pools

Question: 93 CertyIQ

What is hosted completely in the control plane of the classic Databricks architecture?

A.Worker node
B.Databricks web application
C.Driver node
D.Databricks Filesystem

Answer: B

Explanation:

The correct answer is B, the Databricks web application. The control plane in the classic Databricks architecture is responsible for managing and coordinating the Databricks environment. This includes aspects like authentication, authorization, workspace management, job scheduling, and the web interface that users interact with.

The Databricks web application is the primary user interface for interacting with the Databricks service. It's where users manage their workspaces, notebooks, jobs, clusters, and data. This interface resides entirely within the control plane, managed and operated by Databricks. It doesn't run on the user's infrastructure.

On the other hand, worker nodes and driver nodes (A and C) are part of the data plane. These nodes are the computational resources where the actual data processing and Spark jobs execute. They are typically provisioned within the user's cloud environment (AWS, Azure, GCP) and are responsible for processing data and running Spark applications.

The Databricks File System (DBFS) (D) is a distributed file system mounted into a Databricks workspace and available within Databricks clusters. While DBFS has elements managed by the control plane (metadata and access control), the actual storage of data resides in the user's cloud storage account (e.g., AWS S3, Azure Blob Storage, or Google Cloud Storage). Therefore, DBFS is not completely in the control plane.

The control plane being managed entirely by Databricks allows them to handle infrastructure management, upgrades, and security patching for the web application seamlessly. This separation of concerns (control plane managed by Databricks, data plane in the user's cloud) is a core principle of the Databricks architecture, providing a managed service experience while giving users control over their data and compute resources.

For further reading:

Databricks Architecture: https://docs.databricks.com/en/getting-started/concepts.html#architecture
Control Plane vs Data Plane: (While not specific to Databricks, understanding the general concept is helpful): https://www.cloudflare.com/learning/ddos/what-is-the-ddos-control-plane/

Question: 94 CertyIQ

A data engineer needs to determine whether to use the built-in Databricks Notebooks versioning or version their project using Databricks Repos.
What is an advantage of using Databricks Repos over the Databricks Notebooks versioning?

A.Databricks Repos allows users to revert to previous versions of a notebook
B.Databricks Repos is wholly housed within the Databricks Data Intelligence Platform
C.Databricks Repos provides the ability to comment on specific changes

D.Databricks Repos supports the use of multiple branches

Answer: D

Explanation:

The correct answer is D: Databricks Repos supports the use of multiple branches. Here's why:

Databricks Notebooks versioning, while helpful for basic version control, offers limited functionality compared to Git-based version control systems like those used in Databricks Repos. The crucial advantage lies in **branching**. Branching allows data engineers to work on different features, bug fixes, or experiments in isolation without affecting the main codebase (usually the main or master branch). This is a fundamental principle of modern software development and facilitates parallel development, collaboration, and easier rollback of changes. Notebook versioning, on the other hand, primarily provides a linear history of changes to a single notebook.

Databricks Repos leverages Git for version control, thus inheriting Git's powerful branching capabilities. This means developers can create new branches (e.g., feature/new-data-pipeline) to develop new features, and subsequently merge these changes back into the main branch via pull requests after review. This workflow promotes code quality and collaboration.

Option A is incorrect because Databricks Notebook versioning does allow reverting to previous versions of a notebook. Option B is incorrect because both are housed within the Databricks platform, and this isn't a distinguishing advantage. Option C is inaccurate, as code review and commenting are possible via pull requests with Databricks Repos, but not built into native Notebook versioning. Databricks Repos, through Git integration, provides more comprehensive collaboration and code management features.

In summary, the ability to create and manage multiple branches for parallel development and feature isolation is a significant advantage of Databricks Repos compared to the built-in Notebook versioning.

Supporting Links:

Databricks Repos: https://docs.databricks.com/repos/index.html
Git Branching: https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell

Question: 95 CertyIQ

What is a benefit of the Databricks Lakehouse Architecture embracing open source technologies?

A.Avoiding vendor lock-in
B.Simplified governance
C.Ability to scale workloads
D.Cloud-specific integrations

Answer: A

Explanation:

The correct answer is A, avoiding vendor lock-in. The Databricks Lakehouse Architecture leverages open-source technologies like Apache Spark, Delta Lake, and MLflow. Choosing open-source formats and frameworks allows organizations to avoid being tied to a single vendor's proprietary solutions. This gives them the freedom to switch providers or adopt best-of-breed tools without extensive data migration or code

refactoring. Vendor lock-in can lead to increased costs, limited flexibility, and reduced innovation as businesses become dependent on a particular provider's services and pricing. Open source reduces dependency and enables broader community support and improvements. Simplified governance (B) is a potential benefit but isn't directly inherent to open-source. Scaling workloads (C) is possible due to Spark, but open-source alone doesn't guarantee scalability. Cloud-specific integrations (D) may be present in Databricks, but this isn't a primary benefit of using open-source technologies within the Lakehouse Architecture itself. The freedom of choice and control offered by open-source are key advantages for long-term data strategy. By using open standards and technologies, organizations can ensure that their data and analytical pipelines remain portable and interoperable across different environments.Here are a few resources for further research:

Apache Spark: https://spark.apache.org/
Delta Lake: https://delta.io/
MLflow: https://mlflow.org/
Databricks Lakehouse: https://www.databricks.com/product/lakehouse-architecture

Question: 96 CertyIQ

A data engineer needs to use a Delta table as part of a data pipeline, but they do not know if they have the appropriate permissions.
In which location can the data engineer review their permissions on the table?

A.Jobs
B.Dashboards
C.Catalog Explorer
D.Repos

Answer: C

Explanation:

The correct answer is C, Catalog Explorer. Here's a detailed justification:

Catalog Explorer within Databricks is the primary tool for discovering and managing data assets, including Delta tables. A core function of Catalog Explorer is to provide users with detailed information about tables, including their schema, metadata, and importantly, access control lists (ACLs). These ACLs define the permissions granted to users and groups on the table. A data engineer can navigate to the specific Delta table within Catalog Explorer and then review the "Permissions" tab or section. This section clearly lists who has what type of access (e.g., SELECT, MODIFY, CREATE).

Jobs (A) are used for scheduling and managing data pipelines, not for inspecting table-level permissions. While a job might fail due to insufficient permissions, it wouldn't be the place to review what those permissions are. Dashboards (B) are for visualizing data and don't provide information about table permissions. Repos (D) are for version controlling code, which is unrelated to data permissions on Delta tables.

Therefore, Catalog Explorer is the only tool of the options presented that directly addresses the data engineer's need to understand their permissions on a Delta table within the Databricks environment. The permissions information displayed enables them to understand if they can read, write, or modify the table.

For further information, you can refer to the Databricks documentation on Catalog Explorer and data governance:

Databricks Documentation on Unity Catalog: https://docs.databricks.com/en/unity-catalog/index.html (Note:

Unity Catalog is used to manage permissions centrally)
Catalog Explorer: https://docs.databricks.com/en/data-governance/catalog-explorer/index.html

Question: 97 CertyIQ

A data engineer is running code in a Databricks Repo that is cloned from a central Git repository. A colleague of the data engineer informs them that changes have been made and synced to the central Git repository. The data engineer now needs to sync their Databricks Repo to get the changes from the central Git repository.
Which Git operation does the data engineer need to run to accomplish this task?

A.Clone
B.Pull
C.Merge
D.Push

Answer: B

Explanation:

The correct Git operation for the data engineer to synchronize their Databricks Repo with the central Git repository after changes have been made remotely is **B. Pull**.

Here's why:

Git repositories maintain a local copy and a remote copy of the codebase. When a repository is initially set up, Clone is used to copy the remote repository to the local environment. Subsequent updates from the remote repository require a different operation.

Clone: This is used to create a local copy of a remote repository. It is a one-time operation usually done at the start. Since the data engineer already has a cloned repo, Clone isn't appropriate.

Pull: This operation fetches changes from a remote repository and merges them into the current branch of the local repository. It's essentially a combination of fetch (downloading the remote changes) and merge (integrating them into the local branch). This is exactly what the data engineer needs to do – get the latest changes from the central repository.

Merge: While merge is part of the pull operation, it specifically integrates changes from one branch into another. A simple merge operation, without first fetching, will not retrieve updates from the remote repository.

Push: This operation uploads local changes to the remote repository. The data engineer wants to receive changes, not send them.

Therefore, to get the latest changes made by the colleague in the central Git repository and integrate those changes into the local Databricks Repo, the data engineer needs to execute a pull operation. This ensures their local copy is synchronized with the latest version in the central repository. A pull keeps local repo up-to-date. Databricks repos use standard git commands under the hood, so standard git workflows apply.

Git Basics - Getting a Git RepositoryGit Branching - Basic Branching and Merging

Question: 98 CertyIQ

Which file format is used for storing Delta Lake Table?

A.CSV
B.Parquet
C.JSON
D.Delta

Answer: B

Explanation:

Delta Lake, an open-source storage layer, brings ACID (Atomicity, Consistency, Isolation, Durability) transactions to Apache Spark and big data workloads. The foundation of a Delta Lake table lies in its underlying storage format. The primary format used for storing data in a Delta Lake table is **Parquet**.

While Delta Lake provides additional features like versioning, time travel, and schema evolution, it leverages the efficient storage capabilities of Parquet. Parquet is a columnar storage format that optimizes for query performance, especially when dealing with large datasets, as it enables efficient encoding and compression, leading to faster data retrieval and reduced storage costs. CSV and JSON are row-oriented formats that are not optimized for the analytical workloads that Delta Lake targets. The "Delta" format itself isn't a data storage format but rather refers to the Delta Lake transaction log which stores metadata about changes to the Parquet files. The Delta log provides a versioned history of the Delta Lake table, enabling features like time travel and rollback capabilities.

Therefore, the actual data within a Delta Lake table resides in Parquet files, while the Delta Lake transaction log manages the metadata and changes to those files. Other file formats like CSV or JSON are not natively integrated for storing the actual underlying data in a Delta Lake Table. Using Parquet allows Delta Lake to provide ACID compliance and enhanced query performance, making it ideal for data warehousing and large-scale data processing scenarios. Choosing Parquet as the storage format ensures efficient data storage, retrieval, and processing, which is essential for building reliable and performant data pipelines on platforms like Databricks.

Here's a useful link for further reading:

What is Delta Lake?

Question: 99 CertyIQ

A data architect has determined that a table of the following format is necessary:

```
employeeIdstartDate avgRating

a1          2009-01-06  5.5
a2          2018-11-21  7.1
...         ...             ...
```

Which code block is used by SQL DDL command to create an empty Delta table in the above format regardless of whether a table already exists with this name?

A.CREATE OR REPLACE TABLE table_name (employeeId STRING, startDate DATE, avgRating FLOAT)
B.CREATE OR REPLACE TABLE table_name WITH COLUMNS (employeeId STRING, startDate DATE, avgRating FLOAT) USING DELTA

C.CREATE TABLE IF NOT EXISTS table_name (employeeId STRING, startDate DATE, avgRating FLOAT)
D.CREATE TABLE table_name AS SELECT employeeId STRING, startDate DATE, avgRating FLOAT

Answer: A

Explanation:

CREATE OR REPLACE TABLE table_name (employeeId STRING, startDate DATE, avgRating FLOAT)

Question: 100

CertyIQ

A data engineer has been given a new record of data:
id STRING = 'a1'
rank INTEGER = 6
rating FLOAT = 9.4
Which SQL commands can be used to append the new record to an existing Delta table my_table?

A.INSERT INTO my_table VALUES ('a1', 6, 9.4)
B.INSERT VALUES ('a1', 6, 9.4) INTO my_table
C.UPDATE my_table VALUES ('a1', 6, 9.4)
D.UPDATE VALUES ('a1', 6, 9.4) my_table

Answer: A

Explanation:

The correct SQL command to append a new record to an existing Delta table my_table is INSERT INTO my_table VALUES ('a1', 6, 9.4). Let's break down why and why the others are incorrect.

INSERT INTO is the standard SQL command for adding new rows to a table. It's fundamental to database manipulation. The syntax is INSERT INTO table_name VALUES (value1, value2, value3, ...). In our case, my_table is the table we're inserting into, and ('a1', 6, 9.4) represents the values for the id, rank, and rating columns, respectively. These values directly correspond to the data types defined for those columns (STRING, INTEGER, and FLOAT).

Option B, INSERT VALUES ('a1', 6, 9.4) INTO my_table, reverses the expected syntax. The INTO clause should precede the table name. SQL syntax requires a specific order, and reversing it will cause a syntax error.

Options C and D use UPDATE. UPDATE commands are used to modify existing rows in a table, not to create new ones. You use UPDATE when you need to change the values in certain columns of a row that already exists based on some condition (e.g., UPDATE my_table SET rating = 9.5 WHERE id = 'a1'). Attempting to use UPDATE without a WHERE clause to add a new row is incorrect and will likely result in errors or unexpected behavior, as it's not designed for record insertion. UPDATE also always needs a SET clause.

Delta Lake, built on top of Apache Spark, fully supports standard SQL commands, including INSERT INTO. Delta Lake extends the functionality of Spark SQL, providing features like ACID transactions, schema enforcement, and time travel, but it does not alter the basic SQL syntax. Therefore, the familiar INSERT INTO command works seamlessly with Delta tables.

Therefore, INSERT INTO my_table VALUES ('a1', 6, 9.4) is the only option that aligns with correct SQL syntax and the purpose of adding a new row to a Delta table.

For further research, you can consult the following resources:

Databricks SQL Reference: https://docs.databricks.com/sql/language-manual/index.html
Apache Spark SQL Documentation: https://spark.apache.org/docs/latest/sql-ref.html
Delta Lake Documentation: https://docs.delta.io/latest/index.html

Question: 101 CertyIQ

A data engineer has realized that the data files associated with a Delta table are incredibly small. They want to compact the small files to form larger files to improve performance.
Which keyword can be used to compact the small files?

A.OPTIMIZE
B.VACUUM
C.COMPACTION
D.REPARTITION

Answer: A

Explanation:

The correct keyword for compacting small files in a Delta table is OPTIMIZE. Here's why:

OPTIMIZE is a Delta Lake command specifically designed to consolidate small files within a Delta table into larger files. This process directly addresses the "small file problem," a common issue in data lakes that can degrade query performance. Having numerous small files increases metadata management overhead for the Spark engine and leads to excessive I/O operations as it must open, read, and close many small files instead of fewer, larger ones.

OPTIMIZE improves read performance by reducing the number of files that need to be accessed during queries. It achieves this by rewriting existing data into fewer, larger Parquet files, which are the standard format for Delta tables. You can optionally specify a partitioning column with OPTIMIZE, enabling the command to perform bin-packing and co-locate data within partitions based on specified criteria, further enhancing query performance.

VACUUM, on the other hand, is used for removing files that are no longer referenced by the Delta table's transaction log (older versions of the data). It is essential for cost optimization by deleting obsolete data files but does not consolidate existing small files.

COMPACTION isn't a standard Delta Lake keyword, though the concept of compaction is what OPTIMIZE achieves. The documentation uses the term "compaction" to describe what OPTIMIZE does.

REPARTITION is a Spark operation used to change the number of partitions in a DataFrame, which can influence the number of files written when the DataFrame is saved as a Delta table. While using REPARTITION before writing a Delta table can prevent small file issues initially, it's not the solution for compacting existing small files within an already created Delta table. OPTIMIZE is specifically created to tackle already created small file issue.

In summary, OPTIMIZE is the only keyword designed to address the stated problem of compacting small files in an existing Delta table. Therefore, using OPTIMIZE is the most effective way to improve performance in this scenario.

Delta Lake Optimize CommandDelta Lake Vacuum Command

Question: 102 CertyIQ

A data engineer wants to create a data entity from a couple of tables. The data entity must be used by other data engineers in other sessions. It also must be saved to a physical location.
Which of the following data entities should the data engineer create?

A.Table
B.Function
C.View
D.Temporary view

Answer: A

Explanation:

The correct answer is A (Table) because it best addresses all the requirements outlined in the problem. Here's a breakdown of why:

Data Entity: Tables, views, functions, and temporary views are all valid data entities.
Used by Other Data Engineers in Other Sessions: This rules out temporary views. Temporary views are session-scoped. They are only available within the session in which they were created and disappear once the session ends. Other users/sessions cannot access them. Tables, views, and functions persist across sessions.
Saved to a Physical Location: This is a key differentiator. Tables are physically stored in a cloud storage location (like Azure Blob Storage or AWS S3, configured in the Databricks workspace). When a table is created, the data is persistently written to the defined storage path. Views, while persistent metadata-wise, are logical representations of data from one or more tables/views. They don't store data on their own; they are queries stored as objects. Functions, similarly, are code blocks that perform a specific task and don't contain actual data. The physical data continues to be available in the underlying storage and not in the functions.
Benefits of Tables: Creating a table allows other data engineers to query and manipulate the data in a structured format. The table acts as a source of truth and enforces a schema, ensuring data consistency.

Therefore, a table satisfies all the requirements: it's a data entity, can be accessed by other users in different sessions, and is physically saved to a storage location.

Authoritative Links for Further Research:

Databricks Tables: https://docs.databricks.com/en/data/tables.html
Databricks Views: https://docs.databricks.com/en/sql/language-manual/sql-ref-syntax-ddl-create-view.html
Databricks Temporary Views: https://spark.apache.org/docs/latest/sql-ref-syntax-ddl-create-view.html
Databricks User-Defined Functions (UDFs): https://docs.databricks.com/en/sql/udf/index.html

Question: 103 CertyIQ

A data engineer runs a statement every day to copy the previous day's sales into the table transactions. Each day's sales are in their own file in the location "/transactions/raw".
Today, the data engineer runs the following command to complete this task:

```
COPY INTO transactions
FROM "/transactions/raw"
FILEFORMAT = PARQUET;
```

After running the command today, the data engineer notices that the number of records in table transactions has not changed.
What explains why the statement might not have copied any new records into the table?

A.The format of the files to be copied were not included with the FORMAT_OPTIONS keyword.
B.The COPY INTO statement requires the table to be refreshed to view the copied rows.
C.The previous day's file has already been copied into the table.
D.The PARQUET file format does not support COPY INTO.

Answer: C

Explanation:

The previous day's file has already been copied into the table.

Question: 104 CertyIQ

Which command can be used to write data into a Delta table while avoiding the writing of duplicate records?

A.DROP
B.INSERT
C.MERGE
D.APPEND

Answer: C

Explanation:

The correct answer is MERGE because it provides a powerful mechanism for updating Delta tables while handling duplicate records intelligently.

The MERGE command allows you to conditionally insert, update, or delete rows in a target Delta table based on conditions evaluated against a source table or DataFrame. It avoids the writing of duplicate records because it can identify existing records based on a JOIN condition and either update them or ignore new records if they are duplicates. This operation is idempotent, meaning it can be run multiple times without creating further duplicates beyond the initial run.

Compared to other options, DROP is used to remove tables or views entirely, not to manage duplicate records during data writing. INSERT blindly appends new records, leading to duplicates if the same data is inserted repeatedly. APPEND is essentially the same as INSERT, and therefore also contributes to the problem of duplicates.

MERGE offers flexibility in how duplicate records are handled. You can specify conditions to update existing records that match the criteria or insert new records only if they don't already exist, effectively preventing duplicates. You can also handle updates and inserts simultaneously within a single MERGE statement. This is crucial for scenarios where data sources can produce updates to existing records or entirely new records, some of which might already be present. It's also important for slowly changing dimensions.

The atomicity and transactionality of Delta Lake guarantee that the MERGE operation either completes fully, with all changes applied, or rolls back entirely in case of failures, ensuring data consistency and preventing

partial updates that could lead to inconsistencies related to duplicate records.

Further reading:

Delta Lake MERGE: https://docs.databricks.com/delta/delta-update.html
Databricks documentation on Delta Lake: https://docs.databricks.com/delta/index.html

Question: 105 CertyIQ

A data analyst has created a Delta table sales that is used by the entire data analysis team. They want help from the data engineering team to implement a series of tests to ensure the data is clean. However, the data engineering team uses Python for its tests rather than SQL.
Which command could the data engineering team use to access sales in PySpark?

A.SELECT * FROM sales
B.spark.table("sales")
C.spark.sql("sales")
D.spark.delta.table("sales")

Answer: B

Explanation:

The correct way to access a Delta table in PySpark is using spark.table("sales"). Here's why:

Option A, SELECT * FROM sales, is SQL syntax. While you can execute SQL queries in PySpark using spark.sql(), directly using a SQL query string isn't the preferred method for simply accessing an existing table as a DataFrame.

Option C, spark.sql("sales"), is incorrect. spark.sql() is used to execute a SQL query string. Passing just the table name "sales" as a string won't retrieve the table's content. It would be trying to interpret "sales" as a query, which is invalid.

Option D, spark.delta.table("sales"), is also incorrect. While spark.delta is related to Delta Lake, spark.delta.table() is primarily used when you want to get a DeltaTable object, not a Spark DataFrame. The DeltaTable object provides Delta Lake-specific operations like versioning and time travel. If the goal is merely to read the data for testing purposes, retrieving a DataFrame is simpler and more efficient.

spark.table("sales") directly loads the Delta table named "sales" into a PySpark DataFrame, which is the standard and most straightforward way to access it within PySpark. This creates a DataFrame that the data engineering team can then use with Python-based data quality testing libraries like Great Expectations or Deequ. The DataFrame representation makes the data accessible through PySpark's API, enabling various transformations and analysis, essential for implementing their data validation procedures. Using spark.table() ensures that the tests operate directly on the data, ensuring the accuracy of the data validation process.

Relevant documentation:

SparkSession.table() documentation:
https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.sql.SparkSession.html
Delta Lake documentation: https://docs.delta.io/latest/index.html

Question: 106 CertyIQ

A data engineer has created a new database using the following command:
CREATE DATABASE IF NOT EXISTS customer360;
In which location will the customer360 database be located?

A.dbfs:/user/hive/database/customer360

B.dbfs:/user/hive/warehouse

C.dbfs:/user/hive/customer360

D.dbfs:/user/hive/database

Answer: B

Explanation:

The correct answer is B. dbfs:/user/hive/warehouse. Here's why:

When a database is created in Databricks without specifying a LOCATION, the database's metadata and associated tables are stored in a default location managed by the Hive metastore. The default location for Hive metastore managed databases and tables is dbfs:/user/hive/warehouse. This directory acts as the central repository for all managed data in the Hive metastore.

Option A, dbfs:/user/hive/database/customer360, is incorrect. While it might seem logical that a database named customer360 would be placed in a similarly named directory under /database, the standard Hive metastore convention places databases directly under the warehouse directory. The specific location is not under the "/database" folder.

Option C, dbfs:/user/hive/customer360, is incorrect. Again, this is a plausible path, but is not the default managed location. The database and it's tables will be located under /user/hive/warehouse.

Option D, dbfs:/user/hive/database is incomplete. When creating a database, specifying the LOCATION is optional, but the default location should be /user/hive/warehouse.

The IF NOT EXISTS clause in the CREATE DATABASE statement ensures that the command will not fail if a database with the same name already exists.

Understanding the Hive metastore and how Databricks leverages it for managing data and metadata is critical for data engineers working on the platform. It allows for consistent data discovery and management.

For more information, you can refer to the official Databricks documentation on database management and Hive metastore integration.

Databricks documentation for CREATE DATABASE: https://docs.databricks.com/sql/language-manual/sql-ref-syntax-ddl-create-database.html
Databricks documentation on managed tables: https://docs.databricks.com/data/tables.html#managed-and-unmanaged-tables

Question: 107 CertyIQ

A data engineer is attempting to drop a Spark SQL table my_table and runs the following command:
DROP TABLE IF EXISTS my_table;
After running this command, the engineer notices that the data files and metadata files have been deleted from the file system.
What is the reason behind the deletion of all these files?

A.The table was managed
B.The table's data was smaller than 10 GB
C.The table did not have a location
D.The table was external

Answer: A

Explanation:

Here's a detailed justification for why the correct answer is A (The table was managed) when a DROP TABLE IF EXISTS my_table command results in the deletion of both data and metadata files.

When a Spark SQL table is created, it can be either a managed table (also known as an internal table) or an external table. The crucial difference lies in how the data is managed and where it resides.

Managed Tables: In the case of a managed table, Spark SQL controls both the data and the metadata. The data files physically reside within the Spark warehouse directory (e.g., /user/hive/warehouse by default) or a location managed by Spark. Crucially, when you DROP a managed table, Spark completely removes the table, including the data files and the metadata that defines the table's schema and properties within the metastore.

External Tables: Conversely, an external table's data resides in a location outside the Spark warehouse. You explicitly define the location of the data files when creating the external table (using the LOCATION clause in the CREATE TABLE statement). When you DROP an external table, Spark only removes the metadata from the metastore. The underlying data files in the specified location remain untouched. This is because Spark only "points" to the external data; it does not own or manage the storage.

Therefore, the DROP TABLE command's behavior of deleting both data and metadata definitively indicates that the table in question, my_table, was a managed table. The other options are not relevant: table size (option B) and the absence of a location (option C) would not lead to data deletion on a managed table. In fact, managed tables do not require a user-specified location; Spark manages their location internally.

Because the engineer observed the deletion of both data and metadata, it provides conclusive evidence that my_table was a managed table.Here is supporting documentation:

Apache Hive Documentation (Concepts of Managed vs. External Tables): While the concept originated in Hive, it's fundamental to Spark SQL's table management. You can search for "managed vs external tables" in the Apache Hive documentation to understand the core distinction. For instance, consult: https://cwiki.apache.org/confluence/display/Hive/Managed+vs+External+Tables
Databricks Documentation (Create Table): Look for sections detailing how to create managed and external tables, noting the significance of the LOCATION keyword. Search for "create table external" in Databricks documentation. https://docs.databricks.com/sql/language-manual/sql-ref-syntax-ddl-create-table.html

Question: 108 CertyIQ

A data engineer needs to create a table in Databricks using data from a CSV file at location /path/to/csv.
They run the following command:

```
CREATE TABLE new_table

_______
OPTIONS (
  header = "true",
  delimiter = "|"
)
LOCATION "path/to/csv"
```

Which of the following lines of code fills in the above blank to successfully complete the task?

A.FROM "path/to/csv"
B.USING CSV
C.FROM CSV
D.USING DELTA

Answer: B

Explanation:

Correct answer is B:USING CSV.

Question: 109 CertyIQ

What is a benefit of creating an external table from Parquet rather than CSV when using a CREATE TABLE AS SELECT statement?

A.Parquet files can be partitioned
B.Parquet files will become Delta tables
C.Parquet files have a well-defined schema
D.Parquet files have the ability to be optimized

Answer: C

Explanation:

Here's a detailed justification for why option C, "Parquet files have a well-defined schema," is the most accurate benefit when creating an external table from Parquet versus CSV using a CREATE TABLE AS SELECT (CTAS) statement in Databricks.

When using CTAS, you are essentially creating a new table based on the result of a SELECT query. The source data for this query can be either CSV or Parquet files. CSV (Comma Separated Values) is a plain text format where data fields are separated by commas. While widely used, CSV lacks a built-in schema definition. This

means the data types for each column are not explicitly specified within the file itself. The schema must be inferred or externally defined.

Parquet, on the other hand, is a columnar storage format optimized for data warehousing and analytics. A key advantage of Parquet is that it stores schema information (data types for each column, column names) directly within the file's metadata. This self-describing nature is particularly beneficial when creating tables using CTAS.

When creating an external table from Parquet using CTAS, Databricks can directly read and leverage the schema embedded within the Parquet files. This ensures that the created table accurately reflects the data types and structure of the underlying data. In contrast, when using CSV files, the schema must be explicitly defined in the CTAS statement or inferred by Databricks, which can sometimes lead to incorrect data type assignments or parsing errors. The schema evolution capabilities also facilitate seamless upgrades.

Option A is incorrect because while partitioning can improve query performance, it's a separate feature and not inherent only to Parquet with CTAS. Partitioning can be applied regardless of the file format. Option B is incorrect because Parquet files will not automatically transform to Delta tables unless you explicitly convert them. Option D is incorrect because optimization is also separate and not automatically applied, nor is it a feature that solely benefits Parquet when used with CTAS.

Therefore, having a well-defined schema embedded within Parquet files simplifies the CTAS process, reduces the risk of data type mismatches, and ensures data integrity, making it the most significant benefit in this context.

For further reading and reference, please refer to the official Databricks documentation on:

Data Formats: https://docs.databricks.com/en/delta/optimizations/file-selection-optimization.html

CREATE TABLE AS SELECT: https://spark.apache.org/docs/latest/sql-ref-syntax-dml-insert.html

Apache Parquet: https://parquet.apache.org/

Question: 110 CertyIQ

Which SQL keyword can be used to convert a table from a long format to a wide format?

A.TRANSFORM
B.PIVOT
C.SUM
D.CONVERT

Answer: B

Explanation:

The correct answer is **B. PIVOT**.

The PIVOT operator in SQL is specifically designed to transform data from a long (or narrow) format, where data is stacked vertically, to a wide format, where data is spread horizontally. Essentially, PIVOT rotates rows into columns.

Here's why the other options are incorrect:

A. TRANSFORM: This is not a standard SQL keyword for data transformation from long to wide formats. While transformations are common in data engineering (often using Spark's transformations), "TRANSFORM" isn't a specific SQL command for pivoting.

C. SUM: SUM is an aggregate function used to calculate the sum of values within a group. It doesn't inherently change the structure of the table from long to wide. It might be used in conjunction with a PIVOT operation, but it isn't the keyword responsible for the transformation itself.

D. CONVERT: CONVERT is generally used for data type conversions (e.g., converting a string to an integer). It does not manipulate the structure of the table to pivot rows into columns.

The PIVOT operation reorganizes data based on values in one or more columns, turning those values into new column headers. The data associated with those values is then distributed across these newly created columns. This is the fundamental characteristic of a long-to-wide transformation. Within Databricks, which leverages Apache Spark SQL, the PIVOT keyword functions as it would in standard SQL databases, providing a straightforward way to reshape data. Data engineers frequently employ PIVOT to create summary reports or prepare data for machine learning models, where a wide format is often preferred. It reduces data redundancy by representing multiple measurements for a single entity within a single row, enhancing readability and facilitating analysis.

For further research, you can refer to:

Databricks SQL documentation on PIVOT: https://docs.databricks.com/sql/language-manual/sql-ref-syntax-qry-select-pivot.html
General SQL PIVOT documentation (applies conceptually): https://www.sqlservertutorial.net/sql-server-pivot/

Question: 111 CertyIQ

A data engineer has a Python variable table_name that they would like to use in a SQL query. They want to construct a Python code block that will run the query using table_name.
They have the following incomplete code block:
____(f"SELECT customer_id, spend FROM table_name ")
What can be used to fill in the blank to successfully complete the task?

A.spark.delta.sql
B.spark.sql
C.spark.table
D.dbutils.sql

Answer: B

Explanation:

The correct answer is **B. spark.sql**. Here's why:

The core objective is to execute a SQL query constructed dynamically using a Python variable (table_name) within a Databricks environment, which leverages Apache Spark. spark here represents the SparkSession, the entry point to Spark functionality. To run SQL queries against data in a Spark environment, you use the spark.sql() method. This method takes a SQL query string as input and returns a Spark DataFrame representing the result of that query.

The provided code snippet ____(f"SELECT customer_id, spend FROM table_name ") indicates that we need a

function or method that accepts a SQL query string. Using an f-string allows us to seamlessly inject the table_name variable into the query. spark.sql() perfectly fits this requirement.

Option A, spark.delta.sql, is incorrect because it's more specific to Delta Lake functionalities. While you might be querying a Delta table, spark.sql is the general method for executing SQL queries, even on Delta tables. Using spark.delta.sql might not work universally for all table types.

Option C, spark.table, is also incorrect. spark.table() is used to retrieve a table as a DataFrame, not to execute arbitrary SQL queries against it. It returns an existing table or view as a DataFrame.

Option D, dbutils.sql, is incorrect. While dbutils provides utility functions for interacting with Databricks, it doesn't directly offer a method to execute SQL queries and return DataFrames in the same way as spark.sql. Furthermore, dbutils typically focuses on file system operations, secret management, and notebook utilities rather than core Spark SQL functionality.

In summary, spark.sql() is the direct and appropriate method in Spark for executing SQL queries constructed as strings, making it the correct choice to complete the given code block.

For further reading, refer to the official Apache Spark documentation:

Spark SQL, DataFrames and Datasets Guide: https://spark.apache.org/docs/latest/sql-getting-started.html
SparkSession: https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.sql.SparkSession.html

Question: 112 CertyIQ

A data engineer is working with two tables. Each of these tables is displayed below in its entirety.

sales

customer_id	spend	units
a1	28.94	7
a3	874.1223	
a4	8.99	1

favorite_stores

customer_id	store_id
a1	s1
a2	s1
a4	s2

The data engineer runs the following query to join these tables together:

```
SELECT
    sales.customer_id,
    sales.spend,
    favorite_stores.store_id
FROM sales
LEFT JOIN favorite_stores
ON sales.customer_id = favorite_stores.customer_id;
```

A.

```
customer_idspendstore_id
a1          28.94s1
a2          NULL s1
a4          8.99 s2
```

B.

```
customer_idspendstore_id
a1          28.94 s1
a4          8.99  s2
```

C.

```
customer_idspend store_id
a1          28.94 s1
a3          874.12NULL
a4          8.99  s2
```

D.

```
customer_idspend store_id
a1          28.94 s1
a2          NULL  s1
a3          874.12NULL
a4          8.99  s2
```

Answer: C

Explanation:

```
customer_idspend store_id
a1          28.94 s1
a3          874.12NULL
a4          8.99  s2
```

Question: 113 CertyIQ

A data engineer needs to apply custom logic to identify employees with more than 5 years of experience in array column employees in table stores. The custom logic should create a new column exp_employees that is an array of all of the employees with more than 5 years of experience for each row. In order to apply this custom logic at scale, the data engineer wants to use the FILTER higher-order function.
Which code block successfully completes this task?

A.
```
SELECT
    store_id,
    employees,
    FILTER (employees, i -> i.years_exp > 5) AS exp_employees
FROM stores;
```

B.
```
SELECT
    store_id,
    employees,
    FILTER (exp_employees, i -> i.years_exp > 5) AS exp_employees
FROM stores;
```

C.
```
SELECT
    store_id,
    employees,
    FILTER (employees, years_exp > 5) AS exp_employees
FROM stores;
```

D.

```
SELECT
    store_id,
    employees,
    CASE WHEN employees.years_exp > 5 THEN employees
         ELSE NULL
    END AS exp_employees
FROM stores;
```

Answer: A

Explanation:

```
SELECT
    store_id,
    employees,
    FILTER (employees, i -> i.years_exp > 5) AS exp_employees
FROM stores;
```

Question: 114 CertyIQ

A data engineer that is new to using Python needs to create a Python function to add two integers together and return the sum?
Which code block can the data engineer use to complete this task?

A.
```
function add_integers(x, y):
    return x + y
```

B.
```
def add_integers(x, y):
    print(x + y)
```

C.
```
def add_integers(x, y):
    x + y
```

D.
```
def add_integers(x, y):
    return x + y
```

Answer: D

Explanation:

```
def add_integers(x, y):
    return x + y
```

Question: 115 CertyIQ

A data engineer has configured a Structured Streaming job to read from a table, manipulate the data, and then perform a streaming write into a new table.
The code block used by the data engineer is below:

```
(spark.table("sales")
    .withColumn("avg_price", col("sales") / col("units"))
    .writeStream
    .option("checkpointLocation", checkpointPath)
    .outputMode("complete")
    ._____
    .table("new_sales")
)
```

Which line of code should the data engineer use to fill in the blank if the data engineer only wants the query to execute a micro-batch to process data every 5 seconds?

A.trigger("5 seconds")
B.trigger(continuous="5 seconds")
C.trigger(once="5 seconds")
D.trigger(processingTime="5 seconds")

Answer: D

Explanation:

trigger(processingTime="5 seconds").

Question: 116 CertyIQ

A data engineer is maintaining a data pipeline. Upon data ingestion, the data engineer notices that the source data is starting to have a lower level of quality. The data engineer would like to automate the process of monitoring the quality level.
Which of the following tools can the data engineer use to solve this problem?

A.Auto Loader
B.Unity Catalog
C.Delta Lake
D.Delta Live Tables

Answer: D

Explanation:

The correct answer is D, Delta Live Tables (DLT). Here's why:

Delta Live Tables is a framework designed specifically for building and managing reliable data pipelines with automated data quality monitoring. It allows data engineers to define data quality expectations using constraints (e.g., expect_or_fail, expect_or_drop) directly within their data pipelines. These expectations are evaluated during the pipeline's execution. If data doesn't meet the defined quality thresholds, DLT can take actions like failing the pipeline, dropping the bad records, or quarantining the problematic data for further investigation. This built-in data quality management is a core feature of DLT.

Auto Loader (Option A) is an efficient way to incrementally and automatically ingest new data files as they arrive in cloud storage, but it doesn't inherently provide data quality monitoring. While Auto Loader can be part of a pipeline feeding into a system that does check data quality, it doesn't do it itself.

Unity Catalog (Option B) is Databricks' unified governance solution for data and AI assets. It provides centralized access control, auditing, and data discovery. It helps in organizing and securing data but is not a tool for actively monitoring data quality during pipeline execution. It helps manage lineage, which indirectly aids understanding potential quality issues.

Delta Lake (Option C) is a storage layer that brings reliability to data lakes. It provides ACID transactions, scalable metadata handling, and unified streaming and batch data processing. While Delta Lake improves data reliability in general, it doesn't have built-in automated data quality monitoring like DLT. DLT pipelines often use Delta Lake as the underlying storage format.

Therefore, DLT's features for defining data quality expectations and automatically monitoring data quality during pipeline execution make it the best solution for the data engineer's problem.

Further Research:

Delta Live Tables Overview: https://docs.databricks.com/delta-live-tables/index.html
Delta Live Tables expectations: https://docs.databricks.com/delta-live-tables/expectations.html

Question: 117 CertyIQ

A data engineer has three tables in a Delta Live Tables (DLT) pipeline. They have configured the pipeline to drop invalid records at each table. They notice that some data is being dropped due to quality concerns at some point in the DLT pipeline. They would like to determine at which table in their pipeline the data is being dropped.
Which approach can the data engineer take to identify the table that is dropping the records?

A. They can set up separate expectations for each table when developing their DLT pipeline.
B. They can navigate to the DLT pipeline page, click on the “Error” button, and review the present errors.
C. They can set up DLT to notify them via email when records are dropped.
D. They can navigate to the DLT pipeline page, click on each table, and view the data quality statistics.

Answer: D

Explanation:

Here's a detailed justification for why option D is the best approach to identify the table in a Delta Live Tables (DLT) pipeline where records are being dropped due to quality concerns:

Option D, navigating to the DLT pipeline page, clicking on each table, and viewing the data quality statistics, provides the most direct and granular way to identify where data is being dropped. DLT inherently captures and displays detailed data quality metrics for each table in the pipeline. This includes metrics like the number of records processed, the number of records that passed expectations, and the number of records that failed

expectations (dropped records). By inspecting these statistics for each table, the data engineer can immediately pinpoint the specific table where a significant number of records are being dropped.

Option A, setting up separate expectations, is good practice but primarily focuses on preventing and detecting data quality issues from the beginning. It doesn't directly help diagnose where existing drops are occurring within the current pipeline execution. It's a proactive approach, but not reactive for pinpointing the source of existing drops. Expectations help control the data quality, but the pipeline already has the drop invalid records configuration which is what is leading to the dropping and is being investigated.

Option B, clicking on the "Error" button, provides information about pipeline failures or exceptions, which might not always be directly related to data quality drops specifically. Errors can arise from other pipeline issues besides expectation failures leading to dropped records. "Error" button may show the end result error from the data not present at the end table, but not which table is dropping them along the way.

Option C, email notifications, are a reactive approach that can alert the data engineer when drops occur, but it doesn't automatically identify the problematic table. The notification would still require the data engineer to investigate the pipeline to determine the source of the drops.

In summary, analyzing the data quality statistics for each table in the DLT pipeline provides the most direct and detailed information needed to identify the table where records are being dropped due to quality concerns. This approach leverages the built-in monitoring capabilities of DLT for data quality.

Relevant Resources:

Databricks Documentation on Delta Live Tables: https://docs.databricks.com/delta-live-tables/index.html
Databricks Documentation on DLT Monitoring: https://docs.databricks.com/delta-live-tables/monitor.html
Databricks Blog on DLT: https://www.databricks.com/blog/2021/06/15/simplify-etl-with-delta-live-tables.html

Question: 118 CertyIQ

What is used by Spark to record the offset range of the data being processed in each trigger in order for Structured Streaming to reliably track the exact progress of the processing so that it can handle any kind of failure by restarting and/or reprocessing?

A.Checkpointing and Write-ahead Logs
B.Replayable Sources and Idempotent Sinks
C.Write-ahead Logs and Idempotent Sinks
D.Checkpointing and Idempotent Sinks

Answer: A

Explanation:

The correct answer is A: Checkpointing and Write-ahead Logs. Let's break down why.

Structured Streaming in Spark ensures fault tolerance and exactly-once semantics through a combination of mechanisms, with **Checkpointing** and **Write-ahead Logs (WAL)** being core to this capability.

Checkpointing: This mechanism periodically saves the state of the streaming query to reliable storage (like cloud storage such as AWS S3, Azure Blob Storage, or DBFS). This state includes information about the offset ranges of the input data that have been processed up to a certain point in time. In case of failure, the streaming application can restart from the last checkpoint and resume processing. This allows Spark to remember where it left off. Without checkpointing, Spark would lose track of its progress and potentially

reprocess data from the beginning, leading to data duplication.
Write-ahead Logs (WAL): Before any data is actually processed, the system writes the details of the intended operation (including the input data offset) to a persistent log. This log ensures that even if a failure occurs during processing, the system can replay the operations from the log and recover the state, preventing data loss. These logs provide fine-grained recovery capabilities in addition to the broader checkpoints.

Together, checkpointing and WAL enable Spark to reliably track the progress of streaming jobs, ensuring fault tolerance and consistent results. The offset ranges of the data being processed are specifically recorded and recovered using these two components. The combination ensures that Spark can handle any kind of failure by restarting and/or reprocessing from a known, consistent state.

Options B, C, and D are not as complete or accurate because:

Replayable Sources and Idempotent Sinks: While important, they don't address the fundamental need to track the progress of the streaming query itself. Replayable sources (like Kafka) allow re-reading data, and idempotent sinks ensure that writing the same data multiple times has the same effect as writing it once. But without checkpointing and WAL, the streaming job wouldn't know which data to replay or write.
Idempotent Sinks Idempotency is a desirable property for sinks, ensuring that writing the same data multiple times has the same effect as writing it once. This prevents data duplication in case of restarts. However, it does not address the tracking of processing progress; Checkpointing handles this task.
Write-ahead Logs and Idempotent Sinks: While WAL is crucial, it typically works in tandem with checkpointing for robust recovery in Structured Streaming. Idempotent sinks are helpful, but without checkpointing to determine what data needs to be written, idempotency alone is insufficient for complete fault tolerance.

Therefore, option A provides the most comprehensive and accurate explanation of how Spark Structured Streaming achieves fault tolerance in streaming data processing.

Relevant Documentation:

Apache Spark Structured Streaming Programming Guide
Databricks Documentation on Structured Streaming

Question: 119 CertyIQ

What describes the relationship between Gold tables and Silver tables?

A.Gold tables are more likely to contain aggregations than Silver tables.
B.Gold tables are more likely to contain valuable data than Silver tables.
C.Gold tables are more likely to contain a less refined view of data than Silver tables.
D.Gold tables are more likely to contain truthful data than Silver tables.

Answer: A

Explanation:

The correct answer, A, accurately reflects the typical relationship between Gold and Silver tables in a Medallion architecture (Bronze, Silver, Gold). Gold tables are indeed more likely to contain aggregations compared to Silver tables.

The Medallion architecture organizes data processing into distinct layers representing data quality and transformation stages. Bronze tables serve as the raw landing zone for data, mirroring the source data format. Silver tables represent data that has been cleaned, conformed, and enriched with joins/lookups, providing a

refined and reliable dataset. Gold tables, at the final stage, house data that's been transformed for specific business needs, often involving aggregations, summaries, and business-oriented views to power dashboards, reports, and downstream applications.

Silver tables focus on standardizing and cleaning the data, establishing a single source of truth with cleaned and validated records. Gold tables focus on deriving business insights. For example, a Silver table may contain individual transaction records. A corresponding Gold table could aggregate those transactions to show total sales by region or product category, representing a more summarized and insightful view. Gold tables are designed for efficient querying by business users, so aggregations and pre-calculated metrics are common. Options B, C, and D are incorrect because while Gold tables might contain more valuable or truthful data in some contexts, the primary difference from Silver tables is the level of data aggregation and business-specific transformation performed. Silver tables contain clean, validated data which is essential for both the Silver and Gold layer, meaning 'truthful data' is found in both layers.

Further research can be done on these topics:

Databricks Medallion Architecture: https://www.databricks.com/blog/2019/08/15/delta-lake-medallion-architecture.html
Delta Lake: https://delta.io/

Question: 120 CertyIQ

What describes when to use the CREATE STREAMING LIVE TABLE (formerly CREATE INCREMENTAL LIVE TABLE) syntax over the CREATE LIVE TABLE syntax when creating Delta Live Tables (DLT) tables using SQL?

A.CREATE STREAMING LIVE TABLE should be used when the subsequent step in the DLT pipeline is static.
B.CREATE STREAMING LIVE TABLE should be used when data needs to be processed incrementally.
C.CREATE STREAMING LIVE TABLE should be used when data needs to be processed through complicated aggregations.
D.CREATE STREAMING LIVE TABLE should be used when the previous step in the DLT pipeline is static.

Answer: B

Explanation:

The correct answer is **B. CREATE STREAMING LIVE TABLE should be used when data needs to be processed incrementally.**

Let's break down why:

Delta Live Tables (DLT) purpose: DLT simplifies building and managing reliable data pipelines on Databricks. It automatically manages infrastructure, scaling, data quality, and error handling. The core idea is to declare the desired end-state of your tables, and DLT figures out how to get there.

CREATE LIVE TABLE vs. CREATE STREAMING LIVE TABLE: The key difference lies in how they handle data updates. CREATE LIVE TABLE is designed for batch processing. It recomputes the entire table each time the pipeline runs. CREATE STREAMING LIVE TABLE, however, is designed for incremental processing. It processes only the new data that has arrived since the last pipeline update.

Incremental Processing: Incremental processing is critical when dealing with large, continuously updating datasets (streaming data). Instead of reprocessing the entire dataset every time new data arrives, only the new records are processed and appended to the existing table. This significantly improves efficiency and reduces processing time.

Why other options are incorrect:

A. CREATE STREAMING LIVE TABLE should be used when the subsequent step in the DLT pipeline is static. The type of the subsequent step is not the primary deciding factor. The need for incremental processing of the current step is what dictates the use of CREATE STREAMING LIVE TABLE.
C. CREATE STREAMING LIVE TABLE should be used when data needs to be processed through complicated aggregations. Complex aggregations can be performed on either type of table. The method of processing (batch vs. incremental) is the deciding factor. While aggregations on streaming data often benefit from incremental updates, that's a consequence, not the defining characteristic.
D. CREATE STREAMING LIVE TABLE should be used when the previous step in the DLT pipeline is static. Again, the nature of the previous step is not the key factor. A static previous step can feed data into either a batch or streaming table.

In summary, CREATE STREAMING LIVE TABLE is essential when you need to process continuously arriving data in an efficient, incremental manner. By only processing new data, it minimizes processing time and resources, making it ideal for streaming data scenarios within DLT pipelines.

Authoritative Links for further research:

Delta Live Tables documentation: https://docs.databricks.com/delta-live-tables/index.html
Creating a Streaming Live Table: https://docs.databricks.com/delta-live-tables/delta-live-tables-tutorial-sql.html#create-a-streaming-live-table
Delta Live Tables SQL Reference: https://docs.databricks.com/sql/language-manual/delta-live-tables/index.html

Question: 121 CertyIQ

A Delta Live Table pipeline includes two datasets defined using STREAMING LIVE TABLE. Three datasets are defined against Delta Lake table sources using LIVE TABLE.
The table is configured to run in Production mode using the Continuous Pipeline Mode.
What is the expected outcome after clicking Start to update the pipeline assuming previously unprocessed data exists and all definitions are valid?

A.All datasets will be updated at set intervals until the pipeline is shut down. The compute resources will persist to allow for additional testing.

B.All datasets will be updated once and the pipeline will shut down. The compute resources will persist to allow for additional testing.

C.All datasets will be updated at set intervals until the pipeline is shut down. The compute resources will be deployed for the update and terminated when the pipeline is stopped.

D.All datasets will be updated once and the pipeline will shut down. The compute resources will be terminated.

Answer: C

Explanation:

All datasets will be updated at set intervals until the pipeline is shut down. The compute resources will be deployed for the update and terminated when the pipeline is stopped."

The pipeline characteristics:

Uses STREAMING LIVE TABLE → supports continuous updates

Uses LIVE TABLE on Delta sources → refreshed as part of each update cycle

Runs in Production mode

Uses Continuous Pipeline Mode

What Continuous Pipeline Mode does:

The pipeline runs continuously, not once.

It reprocesses new data as it arrives.

It keeps processing until manually stopped.

Compute resources start when the pipeline starts and shut down when the pipeline stops.

Question: 122 CertyIQ

Which type of workloads are compatible with Auto Loader?

A.Streaming workloads
B.Machine learning workloads
C.Serverless workloads
D.Batch workloads

Answer: A

Explanation:

The correct answer is A, Streaming workloads. Here's a detailed justification:

Auto Loader is a Databricks feature specifically designed to incrementally and efficiently process new data files as they arrive in a cloud storage location (like AWS S3, Azure Data Lake Storage Gen2, or Google Cloud Storage). Its core functionality revolves around continuously detecting and ingesting new files as they become available. This real-time or near real-time ingestion characteristic directly aligns with the requirements of streaming workloads.

Streaming workloads, by definition, involve the continuous processing of data streams. Auto Loader addresses the challenge of managing the continuous influx of new data files that often characterize streaming applications. It automatically handles schema inference, schema evolution, and handles corrupted records, simplifying the data ingestion process for streaming pipelines.

Option B, Machine learning workloads, can utilize data ingested by Auto Loader, but Auto Loader itself isn't a machine learning-specific tool. Machine learning pipelines often consume data, regardless of how it was ingested.

Option C, Serverless workloads, refers to the deployment model of the compute infrastructure. While Auto Loader can be part of a serverless data pipeline (e.g., triggered by a serverless function when new files arrive), its primary role isn't being a serverless workload itself.

Option D, Batch workloads, are typically processed on a fixed schedule or on-demand, processing all available data at once. While Auto Loader can be used to load data for batch processing if the data is arriving in a continuous stream, it's primarily optimized for streaming. Batch processing is not the optimal use case because Auto Loader is meant for incremental loading.

In summary, Auto Loader's continuous, incremental processing nature makes it ideally suited for streaming

workloads, where timely data ingestion is crucial. It automatically detects and processes new files, making it a key component in building robust and efficient streaming pipelines.

For more information, consult the official Databricks documentation:

Auto Loader introduction: https://docs.databricks.com/ingestion/auto-loader/index.html

Question: 123 CertyIQ

A data engineer has developed a data pipeline to ingest data from a JSON source using Auto Loader, but the engineer has not provided any type inference or schema hints in their pipeline. Upon reviewing the data, the data engineer has noticed that all of the columns in the target table are of the string type despite some of the fields only including float or boolean values.
Why has Auto Loader inferred all of the columns to be of the string type?

A.Auto Loader cannot infer the schema of ingested data

B.JSON data is a text-based format

C.Auto Loader only works with string data

D.All of the fields had at least one null value

Answer: B

Explanation:

The correct answer is **B. JSON data is a text-based format**.

Auto Loader's schema inference behavior is crucial to understand. While Auto Loader excels at evolving schemas, its initial type inference in the absence of schema hints leans towards safety and minimal data loss. JSON is inherently a text-based format. Without explicit schema information, Auto Loader interprets all values initially as strings to prevent data loss. This is because a string representation can accommodate any possible value, be it a number, boolean, or complex object.

Consider the example: ["voted_answers": "B", "vote_count": 1, "is_most_voted": false]. While "vote_count" appears to be an integer and "is_most_voted" appears to be a boolean, Auto Loader sees them as strings "1" and "false" respectively, due to JSON's representation and the lack of schema hints. If the column was inferred as a numeric or boolean type, it might lead to errors if a non-numeric or non-boolean value were encountered later, causing the pipeline to break.

Option A is incorrect. Auto Loader can infer schemas, but its default behavior without guidance is to err on the side of safety. Option C is also incorrect. Auto Loader supports various data types, not just strings. Option D is incorrect. While null values can influence schema inference, they are not the sole reason for all columns to be strings. Auto Loader handles null values appropriately during schema inference and will only promote to nullable types when appropriate. The primary reason is the textual nature of JSON and the need for a safe initial schema that avoids data loss. If the schema is known beforehand it's highly recommended to specify the schema, this avoids potential data loss and can allow you to optimize the loading of your data. By providing a schema, you're able to avoid the schema inference and optimize the whole process.

For further reading on Auto Loader and schema inference, refer to the official Databricks documentation:

Auto Loader Schema Inference and Evolution
Auto Loader

Question: 124 CertyIQ

Which statement regarding the relationship between Silver tables and Bronze tables is always true?

A.Silver tables contain a less refined, less clean view of data than Bronze data.
B.Silver tables contain aggregates while Bronze data is unaggregated.
C.Silver tables contain more data than Bronze tables.
D.Silver tables contain less data than Bronze tables.

Answer: B

Explanation:

The correct answer, stating that Silver tables contain aggregates while Bronze tables are unaggregated, is rooted in the Medallion Architecture employed within Databricks. This architecture provides a structured approach to data lake management, classifying data into three distinct layers: Bronze, Silver, and Gold.

Bronze tables, also known as raw data tables, ingest data directly from source systems without significant transformation. They maintain the data's original format and granularity, acting as an immutable record of the data's journey into the lake. Therefore, Bronze tables contain unaggregated data.

Silver tables, in contrast, represent a refined and enriched version of the data found in the Bronze layer. They are often created through initial transformations, cleaning, and standardization processes. A common transformation applied at the Silver layer is aggregation, allowing for summaries and roll-ups of the raw data, hence storing aggregated data. Data deduplication, data cleansing, and standardization also occur at this stage.

Gold tables then utilize Silver tables to create data products designed for business intelligence (BI), reporting, and analytics. These will provide further transformation to the data.

Options A, C, and D are incorrect because they do not reflect the purpose or standard usage of the Medallion Architecture. Silver tables are more refined than Bronze tables (contradicting A). Silver tables generally contain less granular data than Bronze tables (contradicting C). Lastly, raw data will generally be more than what has been aggregated, meaning D is also incorrect. The key distinction lies in the data transformation phase and the aggregation that occurs when moving from raw (Bronze) to a refined stage (Silver).

Further Research:

Databricks Medallion Architecture: https://www.databricks.com/glossary/medallion-architecture
Delta Lake Documentation: Consult the official Delta Lake documentation, as it's the underpinning technology for the Medallion Architecture implementation in Databricks.

Question: 125 CertyIQ

Which query is performing a streaming hop from raw data to a Bronze table?

A.

```
(spark.table("sales")
    .groupBy("store")
    .agg(sum("sales"))
    .writeStream
    .option("checkpointLocation", checkpointPath)
    .outputMode("complete")
    .table("newSales")
)
```

B.

```
(spark.read.load(rawSalesLocation)
    .write
    .mode("append")
    .table("newSales")
)
```

C.

```
(spark.table("sales")
    .withColumn("avgPrice", col("sales") / col("units"))
    .writeStream
    .option("checkpointLocation", checkpointPath)
    .outputMode("append")
    .table("newSales")
)
```

D.

```
(spark.readStream.load(rawSalesLocation)
    .writeStream
    .option("checkpointLocation", checkpointPath)
    .outputMode("append")
    .table("newSales")
)
```

Answer: D

Explanation:

```
(spark.readStream.load(rawSalesLocation)
      .writeStream
      .option("checkpointLocation", checkpointPath)
      .outputMode("append")
      .table("newSales")
)
```

Question: 126 CertyIQ

A dataset has been defined using Delta Live Tables and includes an expectations clause:
CONSTRAINT valid_timestamp EXPECT (timestamp > '2020-01-01') ON VIOLATION DROP ROW
What is the expected behavior when a batch of data containing data that violates these constraints is processed?

A.Records that violate the expectation cause the job to fail.

B.Records that violate the expectation are added to the target dataset and flagged as invalid in a field added to the target dataset.

C.Records that violate the expectation are dropped from the target dataset and recorded as invalid in the event log.

D.Records that violate the expectation are added to the target dataset and recorded as invalid in the event log.

Answer: C

Explanation:

The correct answer is C: Records that violate the expectation are dropped from the target dataset and recorded as invalid in the event log.

Here's a detailed justification:

Delta Live Tables (DLT) is a declarative framework for building reliable, maintainable, and testable data pipelines. Expectations in DLT allow you to define data quality constraints. The ON VIOLATION clause specifies what should happen when an expectation is violated. In this case, the expectation CONSTRAINT valid_timestamp EXPECT (timestamp > '2020-01-01') ON VIOLATION DROP ROW dictates that rows where the timestamp is not greater than '2020-01-01' should be dropped from the output table.

When a DLT pipeline processes data, it evaluates these expectations. If a record violates the valid_timestamp constraint, the DROP ROW action is triggered, and the row is excluded from the target dataset. Furthermore, DLT tracks data quality metrics and expectation results. Rows dropped due to failed expectations are recorded as invalid data, and information about these violations is captured in the DLT event log. This event log provides valuable insights into data quality issues and helps with monitoring and debugging the pipeline. The event log allows you to understand the extent of data quality issues and implement corrective actions. Options A, B, and D are incorrect because they do not accurately describe the behavior of the DROP ROW action in DLT expectations. Option A is incorrect because the job does not fail; instead, violating records are dropped. Options B and D are wrong because the violating rows are dropped, not added to the target dataset.

Authoritative Links:

Delta Live Tables Documentation: https://docs.databricks.com/delta-live-tables/index.html

Expectations in Delta Live Tables: https://docs.databricks.com/delta-live-tables/expectations.html

Question: 127 CertyIQ

A data engineer has a Job with multiple tasks that runs nightly. Each of the tasks runs slowly because the clusters take a long time to start.
Which action can the data engineer perform to improve the start up time for the clusters used for the Job?

A.They can use endpoints available in Databricks SQL
B.They can use jobs clusters instead of all-purpose clusters
C.They can configure the clusters to autoscale for larger data sizes
D.They can use clusters that are from a cluster pool

Answer: D

Explanation:

The correct answer is **D. They can use clusters that are from a cluster pool.**

Here's a detailed justification:

The problem describes slow task execution due to long cluster startup times. Cluster startup time is a known bottleneck in Databricks, especially for frequent or short-lived jobs. Cluster pools address this issue directly by pre-allocating and maintaining a pool of idle instances. When a job requests a cluster, it can be provisioned from the pool much faster than creating a new cluster from scratch. This is because the instances are already up and running, bypassing the typical provisioning process (VM allocation, software installation, configuration).

Option A is incorrect because Databricks SQL endpoints are designed for SQL workloads, not general-purpose data engineering tasks within a Databricks Job.

Option B is incorrect because both job clusters and all-purpose clusters can benefit from cluster pools. Switching to job clusters alone won't inherently improve cluster startup time if new clusters are still being created for each job run. While job clusters are often a good practice for managed execution, they don't directly address the startup delay.

Option C is incorrect because autoscaling affects the runtime scaling of a cluster based on workload, not its initial startup time. Autoscaling adjusts the number of instances after the cluster is already running.

Cluster pools drastically reduce the time it takes to acquire a cluster because the instances are already available. The key benefit is reduced latency from cold starts. For nightly jobs, this can significantly decrease overall job duration and free up resources. Configuring clusters to draw from a cluster pool ensures that instances are readily available, leading to faster job startup times. The pool maintains a set of idle instances ready to be assigned to jobs, thereby avoiding the lengthy cluster creation process. The job is then scheduled on this quickly provisioned cluster and executed.

For further research, refer to the official Databricks documentation on cluster pools:

Databricks Cluster Pools

Question: 128 CertyIQ

A data engineer has a single-task Job that runs each morning before they begin working. After identifying an

upstream data issue, they need to set up another task to run a new notebook prior to the original task.
Which approach can the data engineer use to set up the new task?

A.They can clone the existing task in the existing Job and update it to run the new notebook.

B.They can create a new task in the existing Job and then add it as a dependency of the original task.

C.They can create a new task in the existing Job and then add the original task as a dependency of the new task.

D.They can create a new job from scratch and add both tasks to run concurrently.

Answer: B

Explanation:

The correct answer is B: **They can create a new task in the existing Job and then add it as a dependency of the original task**. Here's why:

The goal is to run a new notebook before the original task. This implies a sequential execution order within the existing daily job. Modifying the existing job is preferable to creating a new job, as it maintains the daily schedule and avoids unnecessary complexity.

Option A is incorrect because cloning the existing task and updating it would create a second, independent task that doesn't address the dependency requirement. The new notebook needs to run before the original one.

Option C correctly creates a new task in the existing job. However, it incorrectly makes the original task dependent on the new task. This would cause the original task to run after the new task, which is the reverse of the desired outcome.

Option D is inefficient. Creating a new job from scratch duplicates scheduling and job management unnecessarily. The objective is to integrate the new notebook into the existing workflow seamlessly, not to create a completely separate process. Running the tasks concurrently would also defeat the purpose of ensuring the new notebook runs before the original. Concurrent execution would introduce race conditions, which should be avoided when data dependencies exist.

By creating a new task running the new notebook within the existing job and designating it as a dependency of the original task, the job execution engine (Databricks in this case) will ensure the new task completes successfully before initiating the original task. This achieves the objective of running the data issue resolution notebook first, followed by the regularly scheduled process. This approach leverages the existing job's scheduling and environment, promoting code reuse and simplifying maintenance. The dependency relationship guarantees the necessary pre-processing completes before the original task begins.

Further research on Databricks Jobs and Task Dependencies can be found at:

Databricks Jobs: https://docs.databricks.com/workflows/jobs/index.html
Databricks Job Task Dependencies: https://docs.databricks.com/workflows/jobs/jobs.html#--job-task-dependencies

Question: 129 CertyIQ

A single Job runs two notebooks as two separate tasks. A data engineer has noticed that one of the notebooks is running slowly in the Job's current run. The data engineer asks a tech lead for help in identifying why this might be the case.
Which approach can the tech lead use to identify why the notebook is running slowly as part of the Job?

A.They can navigate to the Runs tab in the Jobs UI to immediately review the processing notebook.

B.They can navigate to the Tasks tab in the Jobs UI and click on the active run to review the processing notebook.

C.They can navigate to the Runs tab in the Jobs UI and click on the active run to review the processing notebook.

D.They can navigate to the Tasks tab in the Jobs UI to immediately review the processing notebook.

Answer: C

Explanation:

The correct answer is **C. They can navigate to the Runs tab in the Jobs UI and click on the active run to review the processing notebook.**

Here's a detailed justification:

The Databricks Jobs UI is the central place to manage and monitor the execution of jobs. When a job is running, it creates a "run" for that specific execution. To investigate a slow-running notebook within a job, the tech lead needs to access the details of the active, or currently executing, run.

The **Runs tab** provides a history of all job executions, including the active one. By selecting the active run from the Runs tab, the tech lead can drill down into the specific details of that execution.

Once inside the active run's details, Databricks provides granular information about each task within that run. This includes the status of the task (e.g., running, succeeded, failed), start and end times, duration, and most importantly, links to the notebook or task being executed.

Clicking on the notebook link will take the tech lead directly to the specific execution of that notebook within the context of the job run. From there, they can examine cell-level execution times, Spark UI logs (if the notebook is using Spark), and any error messages that might be causing the slowdown. They can also observe the resource utilization (CPU, memory) during the problematic cells' execution to identify potential bottlenecks.

Option A is incorrect because while the Runs tab is the correct starting point, merely reviewing the processing notebook "immediately" might not offer sufficient insight. Examining it within the context of the active run is crucial.

Option B and D are incorrect because the **Tasks tab** generally shows a more aggregate view of the tasks across all runs of the job. While useful for identifying trends or persistent issues, it's not the optimal first step to diagnose a slowdown in a specific active run. The Runs tab allows you to navigate to a specific run and see detailed information about its tasks.

Therefore, navigating to the Runs tab, selecting the active run, and then drilling into the specific notebook execution within that run provides the most direct and detailed information for troubleshooting the slow notebook.Here are some relevant links for further research:

Databricks Jobs documentation: https://docs.databricks.com/workflows/jobs/index.html
Monitor Databricks Jobs: https://docs.databricks.com/workflows/jobs/monitor-jobs.html
Databricks UI: https://docs.databricks.com/getting-started/index.html#databricks-ui

Question: 130 CertyIQ

A data analysis team has noticed that their Databricks SQL queries are running too slowly when connected to their always-on SQL endpoint. They claim that this issue is present when many members of the team are running small queries simultaneously. They ask the data engineering team for help. The data engineering team notices that each

of the team's queries uses the same SQL endpoint.
Which approach can the data engineering team use to improve the latency of the team's queries?

A.They can increase the cluster size of the SQL endpoint.
B.They can increase the maximum bound of the SQL endpoint's scaling range.
C.They can turn on the Auto Stop feature for the SQL endpoint.
D.They can turn on the Serverless feature for the SQL endpoint.

Answer: B

Explanation:

The correct answer is **B: They can increase the maximum bound of the SQL endpoint's scaling range.**

Here's a detailed justification:

The problem described is that numerous small queries are running concurrently against a single SQL endpoint, leading to performance bottlenecks and increased latency. This suggests that the existing SQL endpoint is resource-constrained and unable to handle the concurrent workload efficiently.

Option A, increasing the cluster size, could help, but it's a more static approach. While it provides more resources, it doesn't dynamically adjust to varying workloads. The SQL endpoint might be oversized during periods of low activity, wasting resources.

Option B, increasing the maximum bound of the SQL endpoint's scaling range, is the most appropriate solution. SQL endpoints in Databricks can be configured to automatically scale up (and down) the number of clusters based on the workload. By increasing the maximum bound, you allow the endpoint to dynamically allocate more resources (clusters) when a large number of concurrent queries are running, thus improving latency. The endpoint will scale out to meet demand during periods of high concurrency, and then scale back in during periods of low activity, optimizing cost and performance. This dynamic scaling directly addresses the issue of concurrent queries overloading the endpoint.

Option C, turning on Auto Stop, is counterproductive. Auto Stop shuts down the endpoint after a period of inactivity, meaning users would experience cold starts (increased latency) whenever they tried to run a query after the endpoint had been stopped. This would exacerbate the problem, not alleviate it.

Option D, turning on Serverless, could be beneficial but isn't necessarily the best immediate solution. Serverless SQL endpoints automatically manage compute resources, potentially improving efficiency. However, switching to Serverless might involve configuration changes and could introduce unforeseen complexities. Increasing the scaling range is a simpler, less disruptive change to address the existing problem.

Therefore, increasing the maximum bound of the SQL endpoint's scaling range allows it to dynamically adjust resources to handle the concurrent queries effectively, reducing latency and improving the overall performance of the data analysis team's queries.

Further reading on Databricks SQL endpoints and auto-scaling:

Databricks SQL endpoints: https://docs.databricks.com/sql/admin/sql-endpoints.html
Configure SQL endpoint compute: https://docs.databricks.com/sql/admin/configure-endpoints.html#configure-compute

Question: 131 CertyIQ

A data engineer has been using a Databricks SQL dashboard to monitor the cleanliness of the input data to an ELT job. The ELT job has its Databricks SQL query that returns the number of input records containing unexpected NULL values. The data engineer wants their entire team to be notified via a messaging webhook whenever this value reaches 100.
Which approach can the data engineer use to notify their entire team via a messaging webhook whenever the number of NULL values reaches 100?

A.They can set up an Alert with a custom template.
B.They can set up an Alert with a new email alert destination.
C.They can set up an Alert with a new webhook alert destination.
D.They can set up an Alert with one-time notifications.

Answer: C

Explanation:

C. They can set up an Alert with a new webhook alert destination.

In Databricks SQL, Alerts can be used to monitor query results and notify when specific conditions are met — in this case, when the number of records with unexpected NULL values reaches 100.

Question: 132 CertyIQ

A data engineer wants to schedule their Databricks SQL dashboard to refresh once per day, but they only want the associated SQL endpoint to be running when it is necessary.
Which approach can the data engineer use to minimize the total running time of the SQL endpoint used in the refresh schedule of their dashboard?

A.They can ensure the dashboard's SQL endpoint matches each of the queries' SQL endpoints.
B.They can set up the dashboard's SQL endpoint to be serverless.
C.They can turn on the Auto Stop feature for the SQL endpoint.
D.They can ensure the dashboard's SQL endpoint is not one of the included query's SQL endpoint.

Answer: C

Explanation:

The correct answer is **C: They can turn on the Auto Stop feature for the SQL endpoint.** Here's a detailed justification:

The core requirement is to minimize the SQL endpoint's running time when it's only needed for scheduled dashboard refreshes. The Auto Stop feature is specifically designed to automatically shut down an SQL endpoint after a period of inactivity. When a dashboard refresh schedule triggers a query, the endpoint will automatically start up, execute the query, and refresh the dashboard. Once the refresh is complete and no further queries are being executed, the Auto Stop mechanism will kick in after the configured inactivity period and shut down the endpoint. This ensures that the endpoint isn't needlessly running when it's not actively serving requests.

Options A and D are related to query performance and SQL endpoint selection, which may have an impact on how long a query runs, but they don't directly address the problem of when the endpoint is running. Matching or mismatching SQL endpoints among queries might impact caching or query compilation, but it doesn't manage the endpoint's uptime.

Option B, serverless SQL endpoints, could potentially minimize costs by only charging for query execution time, but this is more about cost optimization than minimizing running time. Even with a serverless setup, the dashboard could trigger the queries at unexpected times leading to endpoint uptime longer than wanted. The Auto Stop feature is a more direct and controllable way to minimize the running time by ensuring the SQL endpoint is running only during the refresh activity and for a short period after. It provides a clear mechanism to govern uptime to the activity.

Therefore, enabling Auto Stop provides a direct and effective method to minimize the total running time of the SQL endpoint by automatically shutting it down when not in use, directly addressing the data engineer's requirement.

Further Reading:

Databricks SQL Endpoint Auto Stop: https://docs.databricks.com/sql/admin/sql-endpoints.html#configure-an-sql-endpoint

Question: 133 CertyIQ

An engineering manager wants to monitor the performance of a recent project using a Databricks SQL query. For the first week following the project's release, the manager wants the query results to be updated every minute. However, the manager is concerned that the compute resources used for the query will be left running and cost the organization a lot of money beyond the first week of the project's release.
Which approach can the engineering team use to ensure the query does not cost the organization any money beyond the first week of the project's release?

A.They can set a limit to the number of DBUs that are consumed by the SQL Endpoint.
B.They can set the query's refresh schedule to end after a certain number of refreshes.
C.They can set the query's refresh schedule to end on a certain date in the query scheduler.
D.They can set a limit to the number of individuals that are able to manage the query's refresh schedule.

Answer: C

Explanation:

The correct answer is C, "They can set the query's refresh schedule to end on a certain date in the query scheduler." This is because Databricks SQL query scheduling allows for precise control over when a query runs. Setting an end date ensures that the query will automatically stop refreshing after that date, thereby preventing unnecessary compute resource usage and associated costs beyond the intended monitoring period. Options A, B, and D are incorrect because they do not directly stop the query from running after a specific date. Limiting DBUs (Option A) might reduce cost but doesn't guarantee the query will stop running after the first week. Setting a limit to the number of refreshes (Option B) may work if the refresh interval and duration are known precisely, but an end date is more straightforward and reliable. Limiting user access (Option D) does not affect the query's runtime or cost. Therefore, using the query scheduler to set a specific end date for the refresh schedule is the most direct and effective way to prevent unnecessary costs after the first week.

Further research:

Databricks SQL Query Scheduling

Question: 134 CertyIQ

A new data engineering team has been assigned to work on a project. The team will need access to database customers in order to see what tables already exist. The team has its own group team.
Which command can be used to grant the necessary permission on the entire database to the new team?

A.GRANT VIEW ON CATALOG customers TO team;
B.GRANT CREATE ON DATABASE customers TO team;
C.GRANT USAGE ON CATALOG team TO customers;
D.GRANT USAGE ON DATABASE customers TO team;

Answer: D

Explanation:

D. GRANT USAGE ON DATABASE customers TO team;

When granting permissions in a data lake or database system like AWS Lake Formation, Databricks Unity Catalog, or BigQuery, certain permissions are needed just to see or browse tables in a database (without necessarily accessing the data).

Question: 135 CertyIQ

A new data engineering team team has been assigned to an ELT project. The new data engineering team will need full privileges on the table sales to fully manage the project.
Which command can be used to grant full permissions on the database to the new data engineering team?

A.GRANT ALL PRIVILEGES ON TABLE sales TO team;
B.GRANT SELECT CREATE MODIFY ON TABLE sales TO team;
C.GRANT SELECT ON TABLE sales TO team;
D.GRANT ALL PRIVILEGES ON TABLE team TO sales;

Answer: A

Explanation:

The correct answer is A: GRANT ALL PRIVILEGES ON TABLE sales TO team; because it directly addresses the requirement of granting full privileges on the sales table to the specified team.

Here's a detailed justification:

1. **Problem Statement Alignment:** The question explicitly states that the data engineering team requires "full privileges" on the sales table. Therefore, the chosen command must grant all available permissions.
2. **GRANT ALL PRIVILEGES Clause:** The GRANT ALL PRIVILEGES clause is the standard SQL construct for granting all possible permissions on a database object. This encompasses actions like selecting data, inserting new records, updating existing data, deleting data, altering the table structure, and potentially even granting permissions to others.
3. **ON TABLE sales Clause:** This part of the command specifies the target object for which privileges are being granted, which aligns with the requirement to grant permissions on the sales table.

4. **TO team Clause:** This specifies the recipient of the granted privileges. This ensures the data engineering team receives the privileges. Note: In Databricks, "team" would typically refer to a group principal.

5. **Why other options are incorrect:**

   B. GRANT SELECT CREATE MODIFY ON TABLE sales TO team; This grants only a subset of privileges. It explicitly allows reading (SELECT), creating (CREATE - likely implying creating a table based on the sales table structure, but this is context dependent) and modifying (MODIFY - likely implies UPDATE and DELETE), but it might miss other important permissions like ALTER (to modify the schema), DROP, or the ability to grant these same permissions to other users (GRANT). Therefore, it is not the complete solution.
   C. GRANT SELECT ON TABLE sales TO team; This only grants read access. It is insufficient for the team's stated requirement of managing the project.
   D. GRANT ALL PRIVILEGES ON TABLE team TO sales; This reverses the subject and object. It attempts to give the table sales full privileges on a table or object named team, which is semantically incorrect and doesn't meet the project's requirements. Databases generally don't allow tables to hold privileges.

6. **Database Security Best Practices:** While GRANT ALL PRIVILEGES might seem like a straightforward solution, it's crucial to note that granting such broad permissions should be done with caution. It's vital to understand precisely what privileges are included and whether they are all genuinely necessary. In some situations, granting specific privileges using GRANT SELECT, INSERT, UPDATE, DELETE ... may be preferable for enhanced security.

Authoritative Resources:

Databricks SQL Data Control: https://docs.databricks.com/en/sql/language-manual/security.html
SQL GRANT Statement: (General SQL, concepts apply): https://www.sqltutorial.org/sql-grant/

Question: 136 CertyIQ

Differentiate between all-purpose clusters and jobs clusters.
A data engineering team has created a python notebook to load data from cloud storage, this job has been tested and now needs to be scheduled in production.
Which would be the best cluster to be used in this case?

A.All purpose cluster
B.Any Unity Catalog-enabled cluster
C.Jobs Cluster
D.Serverless SQL warehouse

Answer: C

Explanation:

The correct answer is C: Jobs Cluster. Here's a detailed justification:

All-Purpose clusters are designed for interactive development, collaborative data exploration, and ad-hoc queries. They are typically long-running and shared among multiple users. In contrast, Jobs clusters are designed for running automated, non-interactive workloads such as ETL pipelines, batch processing, and scheduled tasks. Jobs clusters are ephemeral, meaning they are automatically created when a job starts and

terminated when the job completes. This ensures efficient resource utilization and cost optimization.

Since the data engineering team has a tested Python notebook ready for production scheduling, a Jobs cluster is the most appropriate choice. Using an All-Purpose cluster would be wasteful, as it would remain active even when the job is not running, incurring unnecessary costs. Jobs clusters spin up quickly and are specifically designed to execute defined tasks.

A Unity Catalog-enabled cluster (option B) refers to a cluster configured to use Databricks Unity Catalog for data governance and security. While Unity Catalog is a valuable feature, it's not the primary differentiator for choosing between cluster types. The crucial distinction is the cluster's purpose: interactive development (All-Purpose) vs. automated execution (Jobs). Serverless SQL warehouses (option D) are specialized for SQL-based workloads and would not be suitable for running a Python notebook.

Jobs clusters offer benefits like automatic scaling, error handling, and retry mechanisms, making them ideal for production workloads. They provide isolation, ensuring that the job's resources are dedicated and not impacted by other activities. This improves stability and predictability of the scheduled task. Because they are automatically terminated after the job completes, Jobs clusters minimize costs, which is critical in production environments.

Furthermore, Databricks Jobs API and UI are designed to work seamlessly with Jobs clusters, allowing for easy scheduling, monitoring, and management of automated workflows.

For further reading, refer to the official Databricks documentation on Clusters and Jobs:

Databricks Clusters: https://docs.databricks.com/clusters/index.html
Databricks Jobs: https://docs.databricks.com/workflows/jobs/index.html

Question: 137 CertyIQ

Identify how the count_if function and the count where x is null can be used
Consider a table random_values with below data.
What would be the output of below query?
select count_if(col > 1) as count_a. count(*) as count_b.count(col1) as count_c from random_values col1
0
1
2
NULL -
2
3

A.3 6 5
B.4 6 5
C.3 6 6
D.4 6 6

Answer: A

Explanation:

Let's break down how each part of the SQL query select count_if(col1 > 1) as count_a, count(*) as count_b, count(col1) as count_c from random_values operates on the random_values table.

The random_values table has the following data:

col1012NULL-23

count_if(col1 > 1) as count_a: This function counts the number of rows where the condition col1 > 1 is true. count_if (or similar conditional counting functions) are common in data warehousing tools. We can evaluate row by row in the col1 column as follows:
Row 1: 0 > 1 is false.
Row 2: 1 > 1 is false.
Row 3: 2 > 1 is true.
Row 4: NULL > 1 is false. SQL NULL values are treated as unknown, and comparisons with NULL generally evaluate to false.
Row 5: -2 > 1 is false.
Row 6: 3 > 1 is true.
Therefore, count_a will be 3 (rows 3 and 6).
count(*) as count_b: The *count(*)* function counts all rows in the table, including rows where col1 is NULL. This is a fundamental SQL aggregate function. The table has 6 rows, so count_b will be 6.
count(col1) as count_c: The count(col1) function counts the number of rows where col1 is not NULL. Crucially, this function ignores NULL values. It's another essential SQL aggregate function. In this case, there are 5 non-NULL values in col1 (0, 1, 2, -2, 3). Therefore, count_c will be 5.

Combining the results, count_a = 3, count_b = 6, and count_c = 5. This corresponds to option A.

Authoritative Links:

SQL COUNT() function: https://www.w3schools.com/sql/sql_count.asp
SQL NULL values: https://www.w3schools.com/sql/sql_null_values.asp (This is an important concept for understanding how SQL handles missing or unknown data, and is essential for understanding why comparison with NULL is neither true nor false, but NULL).

Question: 138 CertyIQ

Which two components function in the DB platform architecture's control plane? (Choose two.)

A.Virtual Machines
B.Compute Orchestration
C.Serverless Compute
D.Compute
E.Unity Catalog

Answer: BE

Explanation:

The correct answer identifies the two components of the Databricks platform architecture that reside within the control plane: Compute Orchestration and Unity Catalog.

The control plane is responsible for managing and orchestrating the various resources within the Databricks environment. Compute orchestration falls squarely into this category, as it handles the creation, management, and scaling of compute resources (clusters) needed for data processing. It dictates how and where workloads are executed. Unity Catalog, a unified governance solution for data and AI, also lives in the control plane. It defines and enforces access policies, manages data lineage, and facilitates data discovery across different workspaces and compute clusters, effectively controlling how data is accessed and utilized.

Virtual Machines and Compute itself (without the "Orchestration" aspect) are part of the data plane. The data

plane is where the actual data processing and computations occur. The virtual machines serve as the infrastructure where the Databricks Runtime and user code execute, performing the computations orchestrated by the control plane. 'Compute' alone is too general; the orchestration of compute is what distinctly positions it within the control plane. Serverless Compute, while a possible compute deployment model on Databricks, represents an execution environment, making it also part of the data plane rather than the control plane directly. The control plane sets the stage for how and what computations will run, while the data plane executes them.

For further reading, consult Databricks' official documentation:

Databricks Architecture: https://www.databricks.com/glossary/databricks-architecture
Unity Catalog: https://www.databricks.com/product/unity-catalog

Question: 139 CertyIQ

In a healthcare provider organization using Delta Lake to store electronic health records (EHRs), a data analyst needs to analyze a snapshot of the patient_records table from two weeks ago before some recent data corrections were applied.
What approach should the Data Engineer take to allow the analyst to query that specific prior version?

A.Truncate the table to remove all data, then reload the data from two weeks ago into the truncated table for the analyst to query.

B.Identify the version number corresponding to two weeks ago from the Delta transaction log, share that version number with the analyst to query using VERSION AS OF syntax, or export that version to a new Delta table for the analyst to query.

C.Restore the table to the version from two weeks ago using the RESTORE command, and have the analyst query the restored table.

D.Use the VACUUM command to remove all versions of the table older than two weeks, then the analyst can query the remaining version.

Answer: B

Explanation:

The correct approach is **B: Identify the version number corresponding to two weeks ago from the Delta transaction log, share that version number with the analyst to query using VERSION AS OF syntax, or export that version to a new Delta table for the analyst to query.**

Here's why:

Delta Lake's time travel feature allows querying previous versions of a table. Option B leverages this capability directly. The Delta transaction log maintains a history of all changes made to the table. By examining this log, you can determine the version number that represents the table's state two weeks prior. The VERSION AS OF syntax allows querying the table as it existed at that specific version. Alternatively, you could create a new table with the historical data which is more suitable if the analyst wants to make persistent modification without affecting the source data.

Option A is incorrect because truncating and reloading destroys the current state of the table and could lead to data loss or inconsistency. Furthermore, finding the exact data from two weeks ago for reloading might be complex and time-consuming, especially when you have the time travel features.

Option C, using the RESTORE command, modifies the current table, making it represent the state from two weeks ago. While it allows the analyst to query the past state, it overwrites the current state. The analyst will no longer be able to see the updated data after the restore. Moreover, other processes depending on the

current state would be affected.

Option D is incorrect because the VACUUM command removes old versions, which is the opposite of what's needed to analyze a past state. It's used for optimizing storage by deleting outdated files after a retention period. Using vacuum would prevent you from running queries for previous versions, and it would be destructive.

In summary, Option B is the most appropriate because it directly utilizes Delta Lake's time travel feature, preserving the current table state while allowing the analyst to access and analyze the desired historical data.Further Reading:

Delta Lake Time Travel: https://docs.databricks.com/delta/versioning.html
Delta Lake Restore: https://docs.databricks.com/sql/language-manual/delta-restore.html
Delta Lake Vacuum: https://docs.databricks.com/delta/optimizations/optimize-vacuum.html

Question: 140 CertyIQ

What can be used to simplify and unify siloed data architectures that are specialized for specific use cases?

A.Delta Lake
B.Data lake
C.Data warehouse
D.Data lakehouse

Answer: D

Explanation:

The best answer is D, the Data Lakehouse, because it directly addresses the problem of siloed data architectures. Traditional data lakes lack the transactional consistency and data governance capabilities needed for reliable analytics and AI. Conversely, data warehouses offer strong governance and consistency but struggle with the variety and velocity of modern data.

A data lakehouse combines the best aspects of both. It provides the cost-effectiveness and scalability of a data lake for storing all types of data (structured, semi-structured, and unstructured) while adding the data management and transaction support features of a data warehouse. This allows organizations to use a single, unified platform for various use cases, including business intelligence, reporting, advanced analytics, and machine learning, thereby breaking down silos.

Delta Lake (A) is an open-source storage layer that brings ACID transactions to Apache Spark and big data workloads. While Delta Lake is a key technology within a data lakehouse and enhances its capabilities, it is not the complete architecture itself. Similarly, a Data Lake (B) is a storage repository holding a vast amount of raw data in its native format until it is needed, but lacks the necessary data management features. A Data Warehouse (C) is optimized for structured data and analytical workloads, often struggling to handle diverse data types efficiently.

Therefore, the data lakehouse architecture is the most suitable solution for unifying specialized and isolated data architectures, offering a single source of truth for all data-related activities.A data lakehouse provides capabilities such as schema enforcement, version control, and audit trails, ensuring data quality and compliance across all use cases. This integration reduces redundancy, improves data governance, and simplifies data access for various teams. It's the architecture designed specifically to converge data engineering, data science, and business analytics.

Databricks - What is a Data Lakehouse? https://www.databricks.com/glossary/data-lakehouse
Microsoft - Data Lakehouse: https://learn.microsoft.com/en-us/azure/architecture/patterns/data-lakehouse

Question: 141 CertyIQ

A data engineer has configured a Structured Streaming job to read from a table, manipulate the data, and then perform a streaming write into a new table.
The code block used by the data engineer is below:

```
(spark.readStream
      .table("sales")
      .withColumn("avg_price", col("sales") / col("units"))
      .writeStream
      .option("checkpointLocation", checkpointPath)
      .outputMode("complete")
      .________
      .table("new_sales")
)
```

The data engineer only wants the query to process all of the available data in as many batches as required.
Which line of code should the data engineer use to fill in the blank?

A.trigger(availableNow=True)
B.trigger(processingTime= "once")
C.trigger(continuous= "once")
D.trigger(once=True)

Answer: A

Explanation:

A. trigger(availableNow=True)

trigger(availableNow=True) in structured streaming (like in Apache Spark / Databricks) means:

It processes all available data in the source

It does so in multiple micro-batches if needed

Then the query stops automatically once all data is consumed

This exactly matches the requirement:

"process all of the available data in as many batches as required"

Question: 142 CertyIQ

Data engineer and data analysts are working together on a data pipeline. The data engineer is working on the raw, bronze, and silver layers of the pipeline using Python, and the data analyst is working on the gold layer of the pipeline using SQL. The raw source of the pipeline is a streaming input. They now want to migrate their pipeline to use Delta Live Tables.
Which of the following changes will need to be made to the pipeline when migrating to Delta Live Tables?

A.The pipeline can have different notebook sources in SQL & Python

B.The pipeline will need to be written entirely in SQL

C.The pipeline will need to use a batch source in place of a streaming source

D.The pipeline will need to be written entirely in Python

Answer: C

Explanation:

Let's analyze why option C is the correct answer and why the others are not, focusing on the constraints and capabilities of Delta Live Tables (DLT).

Option A: Incorrect. Delta Live Tables pipelines can combine notebooks written in both SQL and Python. This is a key advantage of DLT, allowing teams to leverage the strengths of different languages for different transformations. You can define data flows that involve Python transformations in one notebook feeding data to SQL transformations in another. There's no limitation forcing a single language.

Option B: Incorrect. Similar to A, Delta Live Tables does not force you to use only SQL. Python is fully supported as a language for defining data transformations within a DLT pipeline. This flexibility is crucial for handling complex transformations or leveraging existing Python code.

Option C: Correct. Delta Live Tables primarily works with idempotent batch processing, even when the underlying data source is a streaming source. While DLT can ingest streaming data, it processes it in micro-batches. This means the streaming source will be processed in intervals, creating batches of data that are then ingested and transformed. You cannot directly define a continuous streaming pipeline within DLT in the same way you would with structured streaming. You must define a table (or view) as the input, not a streaming source. The DLT engine manages the ingestion of the stream into that table in a performant and reliable manner. The key takeaway is that the streaming source becomes a source table, which DLT then processes in batches.

Option D: Incorrect. Delta Live Tables support both SQL and Python for defining pipeline logic. The language choice depends on the specific transformation and the expertise of the data engineer or analyst.

Justification for C in Detail:

Delta Live Tables' architecture relies on declarative pipeline definition and automatic infrastructure management. This means you define the what (data transformations) and DLT figures out the how (resource allocation, fault tolerance, etc.). To enable reliable and idempotent updates, DLT breaks down streaming data into manageable batches. While it hides much of the complexity, this batch-oriented approach fundamentally changes how you interact with the streaming data within the pipeline's transformation logic. Essentially, even when feeding in streaming data, your DLT transformations treat the data as a series of micro-batches/tables. The DLT framework itself uses structured streaming behind the scenes, but the pipeline's transformation code operates on tables/batches.

The streaming input needs to be changed to a streaming table in the DLT pipeline. The pipeline will then consume micro-batches from that table.

Authoritative Links for further research:

Delta Live Tables documentation: https://docs.databricks.com/delta-live-tables/index.html
Databricks Blog - Delta Live Tables: https://www.databricks.com/blog/2021/04/29/simplify-data-engineering-with-delta-live-tables.html

Question: 143 CertyIQ

Identify a scenario to use an external table.
A Data Engineer needs to create a parquet bronze table and wants to ensure that it gets stored in a specific path in an external location.
Which table can be created in this scenario?

A. An external table where the location is pointing to specific path in external location.
B. An external table where the schema has managed location pointing to specific path in external location.
C. A managed table where the catalog has managed location pointing to specific path in external location.
D. A managed table where the location is pointing to specific path in external location.

Answer: A

Explanation:

The correct answer is A because it directly addresses the scenario's requirements for a Data Engineer to create a Parquet bronze table stored in a specific external location. Here's a detailed breakdown:

External Tables: External tables in Databricks (and generally in data warehousing systems like Hive and Spark) are ideal when you need to manage the underlying data files separately from the table definition. The table metadata (schema, table name, etc.) is stored in the metastore (e.g., Hive metastore or Databricks Unity Catalog), but the actual data resides in a location you specify, such as cloud storage (AWS S3, Azure Data Lake Storage Gen2, or Google Cloud Storage).

LOCATION Clause: The LOCATION clause in the CREATE EXTERNAL TABLE statement is precisely how you link the table definition to the desired storage path. You explicitly provide the path where the Parquet files should be stored.

Managed Tables (Incorrect): Managed tables (also known as internal tables) are controlled entirely by Databricks. When you create a managed table, Databricks manages both the metadata and the data. The data files for managed tables are typically stored in a default location managed by Databricks, and you generally don't have direct control over the physical storage path. Therefore, options C and D, which suggest managed tables, are not appropriate for this scenario.

Schema's Managed Location (Incorrect): Option B talks about the "schema has managed location." While schemas can have a default location, this doesn't automatically guarantee that an external table's data will be placed there unless explicitly specified in the CREATE EXTERNAL TABLE statement with the LOCATION clause. The crucial point is that the question mandates a specific path to be controlled by the user.

Controlling Data Location: By using an external table with a specified LOCATION, the Data Engineer retains control over where the bronze table data is stored. This is important for data governance, cost management, and integration with other systems that may rely on specific file paths. Bronze tables are typically the initial landing zone for raw data, and controlling their location can be crucial for downstream processing and data lineage.

In summary, using an external table and specifying its location through the LOCATION clause aligns perfectly with the need to store the Parquet bronze table in a precise location, fulfilling the requirements of the problem.

Authoritative Links:

Databricks Documentation on Tables: https://docs.databricks.com/en/tables/index.html
CREATE TABLE SQL Reference (Databricks): https://docs.databricks.com/sql/language-manual/sql-ref-syntax-ddl-create-table.html

Question: 144 CertyIQ

Identify the impact of ON VIOLATION DROP ROW and ON VIOLATION FAIL UPDATE for a constraint violation.
A data engineer has created an ETL pipeline using Delta Live table to manage their company travel reimbursement detail, they want to ensure that the if the location details has not been provided by the employee, the pipeline needs to be terminated.
How can the scenario be implemented?

A.CONSTRAINT valid_location EXPECT (location = NULL)
B.CONSTRAINT valid_location EXPECT (location != NULL) ON VIOLATION FAIL UPDATE
C.CONSTRAINT valid_location EXPECT (location != NULL) ON DROP ROW
D.CONSTRAINT valid_location EXPECT (location != NULL) ON VIOLATION FAIL

Answer: B

Explanation:

B. **CONSTRAINT valid_location EXPECT (location != NULL) ON VIOLATION FAIL UPDATE.**

EXPECT (location != NULL) ensures the location field is provided.

ON VIOLATION FAIL UPDATE ensures the pipeline terminates immediately if any row is found where location is NULL.

Question: 145 CertyIQ

Which two conditions are applicable for governance in Databricks Unity Catalog? (Choose two.)

A.You can have more than 1 metastore within a databricks account console but only 1 per region.
B.Both catalog and schema must have a managed location in Unity Catalog provided metastore is not associated with a location
C.You can have multiple catalogs within metastore and 1 catalog can be associated with multiple metastore
D.If catalog is not associated with location, it's mandatory to associate schema with managed locations
E.If metastore is not associated with location, it's mandatory to associate catalog with managed locations

Answer: AE

Explanation:

Here's a detailed justification for why options A and E are the correct answers, and why the others are incorrect, concerning Databricks Unity Catalog governance:

A. You can have more than 1 metastore within a Databricks account console but only 1 per region.

This statement is correct and fundamental to Unity Catalog's architecture. A Databricks account can span multiple regions. To manage data access across these regions in a consistent manner, Unity Catalog allows one metastore per region. This enables regional data governance while maintaining a single pane of glass view across the entire organization's data assets. Having more than one metastore in the same region is not supported, as it would lead to inconsistencies and management overhead.

Supporting Concept: Regional separation, Single pane of glass view.
Authoritative Link: Databricks documentation on Unity Catalog metastores: https://docs.databricks.com/data-governance/unity-catalog/create-metastore.html

E. If a metastore is not associated with a location, it's mandatory to associate a catalog with managed locations.

This statement accurately reflects the requirements for managed tables and data ownership within Unity Catalog. A metastore can be configured without a root storage location. If this is the case, individual catalogs must then be associated with managed locations. This is because Unity Catalog needs a location to store the managed tables created within these catalogs. Without either a metastore or a catalog-level managed location, managed tables would have nowhere to store their underlying data files. By setting a location on the catalog, you are giving unity catalog a location to manage data storage in a secure and governed manner.

Supporting Concept: Data Locality, Data Governance, Managed Storage.
Authoritative Link: Databricks documentation on managed locations in Unity Catalog: https://docs.databricks.com/data-governance/unity-catalog/managed-storage.html

Why the other options are incorrect:

B. Both catalog and schema must have a managed location in Unity Catalog provided metastore is not associated with a location: While catalog must have a managed location if the metastore lacks one, it is **not mandatory** for schemas to have managed locations. Schemas inherit the managed location from their parent catalog (or the metastore, if the catalog doesn't have one).
C. You can have multiple catalogs within a metastore and 1 catalog can be associated with multiple metastores: A catalog can exist only within a single metastore. A catalog cannot span multiple metastores. Unity Catalog is designed for a hierarchical structure: Metastore -> Catalog -> Schema -> Table/View.
D. If catalog is not associated with location, it's mandatory to associate schema with managed locations: The opposite is true. It's mandatory to have a catalog associated with a managed location (if the metastore does not have one), not a schema. Schemas inherit location from their parent catalog or metastore.

Question: 146 CertyIQ

A data engineer needs to access the view created by the sales team, using a shared cluster. The data engineer has been provided usage permissions on the catalog and schema. In order to access the view created by sales team. What are the minimum permissions the data engineer would require in addition?

A.Needs SELECT permission on the VIEW and the underlying TABLE.
B.Needs SELECT permission only on the VIEW
C.Needs ALL PRIVILEGES on the VIEW
D.Needs ALL PRIVILEGES at the SCHEMA level

Answer: A

Explanation:

The correct answer is A: Needs SELECT permission on the VIEW and the underlying TABLE. Here's a detailed justification:

When a user accesses a view in Databricks, the system needs to verify that the user has the necessary permissions not only to access the view itself but also to access the underlying tables or views that the view is based on. This is because a view is essentially a stored query, and executing that query requires reading data from the underlying objects. Without permission on the underlying tables, the data engineer would effectively

be bypassing access controls and potentially viewing data they are not authorized to see. The catalog and schema usage permissions only grant access to the container structure, not the data within.

Databricks uses a hierarchical permission model. While usage permission on the catalog and schema allows the user to list and traverse the objects within, it does not grant read access to data. SELECT permission on the view allows the user to execute the query that defines the view. However, that query needs to read data from tables. If the data engineer only has SELECT permission on the view, but lacks SELECT on the underlying table(s), the query execution will fail, even if the user possesses 'USAGE' permission on the parent catalog and schema.

Consider a view that joins data from two tables. To use the view, the data engineer must have SELECT privileges on the view itself and both tables that are referenced by the view. Option B is incorrect because it misses the permission on the underlying table. Option C and D are unnecessarily broad; "ALL PRIVILEGES" grants significantly more permissions than necessary. "ALL PRIVILEGES" at the SCHEMA level would give potentially unwanted permissions on all objects in that schema. Therefore, granting only the necessary permissions to the view and underlying tables follows the principle of least privilege, a key security best practice. Granting only the least amount of privilege reduces the blast radius of a compromised account.

Authoritative Links for Further Research:

Databricks SQL Privileges and Securable Objects: https://docs.databricks.com/en/security/access-control/table-acls/object-privileges.html
Databricks Unity Catalog Privileges: https://docs.databricks.com/en/data-governance/unity-catalog/permissions.html

Question: 147 CertyIQ

Which method should a Data Engineer apply to ensure Workflows are being triggered on schedule?

A.Scheduled Workflows require an always-running cluster, which is more expensive but reduces processing latency.

B.Scheduled Workflows process data as it arrives at configured sources.

C.Scheduled Workflows can reduce resource consumption and expense since the cluster runs only long enough to execute the pipeline.

D.Scheduled Workflows run continuously until manually stopped.

Answer: C

Explanation:

The correct answer is C: Scheduled Workflows can reduce resource consumption and expense since the cluster runs only long enough to execute the pipeline.

Here's why:

Scheduled Workflows are designed for periodic execution: Databricks Workflows can be configured to run at specific intervals (e.g., hourly, daily, weekly).
On-demand Cluster Provisioning: When a scheduled workflow is triggered, a Databricks cluster is automatically provisioned (if one isn't already available and configured to be reused).
Efficient Resource Utilization: The cluster runs only for the duration required to execute all the tasks defined in the workflow. Once the workflow completes, the cluster is shut down automatically. This "start-and-stop" behavior minimizes resource usage.
Cost Optimization: By only running the cluster when needed, scheduled workflows significantly reduce the

overall cost compared to solutions that require always-on clusters.
Alternative A is incorrect: While always-running clusters can reduce latency, they are not required for scheduled workflows. Scheduled Workflows typically spin up a cluster when needed.
Alternative B is incorrect: Scheduled Workflows are triggered at specific times, not based on data arrival at source systems. Workflows triggered by data arrival are referred to as Delta Live Tables with continuous pipeline processing.
Alternative D is incorrect: Scheduled Workflows do not run continuously. They run once each time the schedule is triggered.

In essence, scheduled workflows offer a cost-effective approach to automating data pipelines by leveraging on-demand compute resources that are only active during pipeline execution. This promotes efficient resource management and reduces overall infrastructure expenses.

For further information, please review Databricks official documentation:

Databricks Workflows
Scheduling Databricks Jobs

Question: 148 CertyIQ

The Delta transaction log for the 'students' tables is shown using the 'DESCRIBE HISTORY students' command. A Data Engineer needs to query the table as it existed before the UPDATE operation listed in the log.
Which command should the Data Engineer use to achieve this? (Choose two.)

	version	timestamp	operation
1	8	2024–04–22T14:33:31.000	OPTIMIZE
2	7	2024–04–22T14:33:16.000	MERGE
3	6	2024–04–22T14:33:06.000	DELETE
4	5	2024–04–22T14:32:58.000	UPDATE
5	4	2024–04–22T14:32:47.000	WRITE
6	3	2024–04–22T14:32:44.000	WRITE
7	2	2024–04–22T14:32:23.000	WRITE
8	1	2024–04–22T14:32:20.000	WRITE
9	0	2024–04–22T14:31:49.000	CREATE TABLE

A.SELECT * FROM students@v4
B.SELECT * FROM students TIMESTAMP AS OF '2024-04-22T 14:32:47.000+00:00'
C.SELECT * FROM students FROM HISTORY VERSION AS OF 3
D.SELECT * FROM students VERSION AS OF 5
E.SELECT * FROM students TIMESTAMP AS OF '2024-04-22T 14:32:58.000+00:00'

Answer: AB

Explanation:

Reference:

https://docs.databricks.com/en/delta/history.html

Question: 149 CertyIQ

An engineering manager uses a Databricks SQL query to monitor ingestion latency for each data source. The manager checks the results of the query every day, but they are manually rerunning the query each day and waiting for the results.
Which of the following approaches can the manager use to ensure the results of the query are updated each day?

A. They can schedule the query to refresh every 1 day from the SQL endpoint's page in Databricks SQL.
B. They can schedule the query to refresh every 12 hours from the SQL endpoint's page in Databricks SQL.
C. They can schedule the query to refresh every 1 day from the query's page in Databricks SQL.
D. They can schedule the query to run every 12 hours from the Jobs UI.

Answer: C

Explanation:

The correct answer is C because Databricks SQL offers a built-in scheduling feature directly within the query interface, designed specifically for automatically refreshing query results. This eliminates the need for manual intervention or reliance on external scheduling mechanisms. Options A and B are incorrect as scheduling happens at the query level not the SQL endpoint level. While the query is executed using a SQL endpoint, scheduling is configured within the query's settings. Option D is incorrect because the Jobs UI primarily handles notebook and Python script execution, not the scheduling of Databricks SQL queries. Utilizing the query's built-in scheduling capability streamlines the monitoring process. The engineering manager can simply configure the query to refresh every day, ensuring that the latest ingestion latency data is readily available. This automated approach enhances efficiency and eliminates the risk of human error associated with manual query execution. The feature can be accessed by opening the desired Databricks SQL query and selecting "Schedule" in the interface. This setup allows for precise control over refresh intervals, catering to the manager's specific monitoring needs. It leverages Databricks' managed service capabilities to automate data refresh, simplifying data engineering tasks and improving operational efficiency. The benefit extends beyond mere automation. It enables consistent and timely monitoring, which is crucial for identifying and addressing data ingestion issues promptly.

Reference: https://docs.databricks.com/sql/user/queries/index.html#schedule-a-query

Question: 150 CertyIQ

A data engineer needs to apply custom logic to string column city in table stores for a specific use case. In order to apply this custom logic at scale, the data engineer wants to create a SQL user-defined function (UDF).
Which of the following code blocks creates this SQL UDF?

A.
```
CREATE FUNCTION combine_nyc(city STRING)
RETURNS STRING
RETURN CASE
  WHEN city = "brooklyn" THEN "new york"
  ELSE city
END;
```

B.
```
CREATE UDF combine_nyc(city STRING)
RETURNS STRING
CASE
  WHEN city = "brooklyn" THEN "new york"
  ELSE city
END;
```

C.
```
CREATE FUNCTION combine_nyc(city STRING)
RETURN CASE
  WHEN city = "brooklyn" THEN "new york"
  ELSE city
END;
```

D.
```
CREATE UDF combine_nyc(city STRING)
RETURNS STRING
RETURN CASE
  WHEN city = "brooklyn" THEN "new york"
  ELSE city
END;
```

Answer: A

Explanation:

```
CREATE FUNCTION combine_nyc(city STRING)
RETURNS STRING
RETURN CASE
  WHEN city = "brooklyn" THEN "new york"
  ELSE city
END;
```

Question: 151 CertyIQ

A data engineer has realized that they made a mistake when making a daily update to a table. They need to use Delta time travel to restore the table to a version that is 3 days old. However, when the data engineer attempts to time travel to the older version, they are unable to restore the data because the data files have been deleted. Which of the following explains why the data files are no longer present?

A.The VACUUM command was run on the table

B.The TIME TRAVEL command was run on the table

C.The DELETE HISTORY command was run on the table

D.The OPTIMIZE command was nun on the table

Answer: A

Explanation:

The correct answer is A, because the VACUUM command removes files from the Delta Lake table's storage location that are no longer needed by the Delta Lake table and are older than the specified retention interval. By default, this retention interval is 7 days. If the VACUUM command was run with the default retention period (or a shorter one), any data files older than that retention period would be deleted. This would prevent time travel to versions older than the retention period, as the necessary data files would no longer be available.

Option B, TIME TRAVEL, is incorrect. TIME TRAVEL allows you to query or restore to older versions of your Delta table, it does not remove files.

Option C, DELETE HISTORY is not a valid Delta Lake command. Delta Lake offers a RESTORE command, but there is no command called "DELETE HISTORY." Furthermore, Delta Lake does not directly expose the deletion of specific history entries in a way that would affect underlying data files in versions.

Option D, OPTIMIZE, is incorrect. The OPTIMIZE command compacts small files into larger files to improve query performance. While OPTIMIZE can rewrite data files, it does not automatically delete older versions of the data necessary for time travel, unless combined with VACUUM.

Therefore, only running VACUUM with a retention period shorter than 3 days could explain why the data engineer is unable to time travel to a version 3 days old.

Authoritative Links:

Delta Lake VACUUM: https://docs.databricks.com/delta/vacuum.html
Delta Lake Time Travel: https://docs.databricks.com/delta/versioning.html
Delta Lake Optimize: https://docs.databricks.com/delta/optimizations/optimize.html

Question: 152 CertyIQ

Which of the following describes the relationship between Bronze tables and raw data?

A.Bronze tables contain less data than raw data files.
B.Bronze tables contain more truthful data than raw data.
C.Bronze tables contain raw data with a schema applied.
D.Bronze tables contain a less refined view of data than raw data.

Answer: C

Explanation:

The correct answer is C: Bronze tables contain raw data with a schema applied. Here's why:

Bronze tables in a Medallion Architecture (Bronze, Silver, Gold) serve as the entry point for data ingestion. Their primary function is to land raw data from various sources directly into the data lake. The data in a Bronze table is essentially a near-exact copy of the raw data, but with an important addition: a schema. This schema, which can be either explicitly defined or inferred, provides structure and meaning to the data. The schema does not refine or transform the data.

Option A is incorrect because Bronze tables do not contain less data. Typically, the raw data is completely loaded into the bronze table.

Option B is incorrect because the truthfulness of the data isn't changed. Raw data, whether truthful or not, persists as is in the bronze table.

Option D is incorrect because Bronze tables impose initial structure to raw data via a schema.

Cloud computing concepts such as data lakes, ETL pipelines, and schema inference are relevant to understanding this. Data lakes are repositories where data in its native format is stored. Bronze tables fit directly into this concept by providing a landing zone. ETL pipelines use the bronze tables as their first stage, where the raw data is extracted. Finally, schema inference helps in initially defining the bronze table when no explicit schema is available. In essence, Bronze tables are a crucial first step in structuring and preparing data for downstream processing.

Here's the link from Databricks official documentation on Medallion Architecture:https://www.databricks.com/glossary/medallion-architecture

Question: 153 CertyIQ

A data engineer only wants to execute the final block of a Python program if the Python variable day_of_week is equal to 1 and the Python variable review_period is True.
Which of the following control flow statements should the data engineer use to begin this conditionally executed code block?

A.if day_of_week = 1 and review_period:
B.if day_of_week = 1 and review_period = "True":
C.if day_of_week = 1 & review_period: = "True":
D.if day_of_week == 1 and review_period:

Answer: D

Explanation:

The correct if statement is if day_of_week == 1 and review_period:. Let's break down why:

if keyword: The if keyword initiates a conditional statement in Python, allowing a block of code to execute only if a specified condition is true.

day_of_week == 1: This part checks if the variable day_of_week is equal to 1. The double equals sign (==) is the equality operator in Python, used for comparing values. Using a single equals sign (=) would be an assignment, which is invalid within a conditional statement.

and operator: The and operator is a logical operator. It requires both conditions on either side of it to be true for the overall condition to be true.

review_period:: This part checks if the variable review_period is True. In Python, non-empty strings, non-zero numbers, and objects that are not None evaluate to True in a boolean context. Therefore, simply including review_period in the if statement is sufficient to check if its value is considered True.

Option A uses the assignment operator (=) instead of the equality operator (==), which is syntactically incorrect and will raise an error. Options B and C incorrectly compare review_period to the string "True". review_period is already specified as a boolean, so explicitly comparing it against the string "True" would lead to incorrect boolean evaluation. Python treats boolean values distinctly from string representations of "True" or "False". Therefore only option D is the correctly formatted code for this scenario.

Refer to these resources for additional information:

Python if statement: https://docs.python.org/3/tutorial/controlflow.html

Python Comparison Operators: https://www.w3schools.com/python/python_operators.asp
Python Boolean Values: https://www.w3schools.com/python/python_booleans.asp

Question: 154 CertyIQ

Which of the following must be specified when creating a new Delta Live Tables pipeline?

A.A key-value pair configuration
B.At least one notebook library to be executed
C.A path to cloud storage location for the written data
D.A location of a target database for the written data

Answer: B

Explanation:

B. At least one notebook library to be executed.

When creating a new Delta Live Tables (DLT) pipeline, the Source Code (libraries) is the mandatory element that defines the ETL logic, including the table definitions (CREATE LIVE TABLE, @dlt.table) and transformations. A pipeline cannot be created or run without code to execute.

Question: 155 CertyIQ

In which of the following scenarios should a data engineer select a Task in the Depends On field of a new Databricks Job Task?

A.When another task needs to be replaced by the new task
B.When another task needs to successfully complete before the new task begins
C.When another task has the same dependency libraries as the new task
D.When another task needs to use as little compute resources as possible

Answer: B

Explanation:

The correct answer is B: When another task needs to successfully complete before the new task begins.

The "Depends On" field in a Databricks Job Task defines task dependencies. This means that a task listed in the "Depends On" field must complete successfully before the current task can start. This mechanism is crucial for orchestrating data pipelines and ensuring data integrity. It allows you to create a directed acyclic graph (DAG) of tasks where the output of one task serves as the input for another.

Option A is incorrect because "Depends On" doesn't mean task replacement. Replacing a task involves modifying or deleting an existing task, not setting up a dependency. Option C is incorrect as shared libraries don't mandate a "Depends On" relationship. While sharing libraries is good practice for efficiency, the tasks can still run independently. Option D is also incorrect; resource consumption by one task doesn't necessitate a dependency relationship with another. The "Depends On" functionality is specifically about ensuring the successful execution and data output of one task before another begins, establishing a workflow order.

Without this mechanism, the new task might try to operate on incomplete or inconsistent data, leading to errors or incorrect results. Data pipelines often have stages where data cleaning, transformation, and aggregation are performed sequentially. "Depends On" ensures that the raw data is cleaned before the transformation step, and that transformed data is available before the aggregation step. This ensures the correct flow of data and the successful completion of the data engineering workflow. In essence, "Depends On" ensures that the tasks within a job adhere to a specific order of execution to guarantee correctness and data integrity.

Further research can be conducted on the Databricks documentation site related to "Databricks Jobs" and "Task dependencies":

Databricks Jobs
Task dependencies

Question: 156 CertyIQ

A data engineering team has two tables. The first table march_transactions is a collection of all retail transactions in the month of March. The second table april_transactions is a collection of all retail transactions in the month of April. There are no duplicate records between the tables.
Which of the following commands should be run to create a new table all_transactions that contains all records from march_transactions and april_transactions without duplicate records?

A.CREATE TABLE all_transactions AS
SELECT * FROM march_transactions
INNER JOIN SELECT * FROM april_transactions;

B.CREATE TABLE all_transactions AS
SELECT * FROM march_transactions
UNION SELECT * FROM april_transactions;

C.CREATE TABLE all_transactions AS
SELECT * FROM march_transactions
OUTER JOIN SELECT * FROM april_transactions;

D.CREATE TABLE all_transactions AS
SELECT * FROM march_transactions
INTERSECT SELECT * from april_transactions;

Answer: B

Explanation:

The correct answer is B: CREATE TABLE all_transactions AS SELECT *FROM march_transactions UNION SELECT* FROM april_transactions;

Here's why:

The goal is to create a new table, all_transactions, containing all records from both march_transactions and april_transactions without duplicates. Each of the other options fails to achieve this.

Option A, using INNER JOIN, only returns records that exist in both tables based on some implicit or explicit join condition. This is inappropriate since we want all records from both tables.

Option C, using OUTER JOIN, returns all records when there is a match in either left or right table records. This will not remove duplicate records, and we are told that no duplicates exist between the two input tables.

Option D, using INTERSECT, returns only the records that are common to both tables. This is the opposite of what we need.

UNION combines the result sets of two or more SELECT statements into a single result set. Importantly, UNION implicitly removes duplicate rows from the final result. Since the question explicitly states that there are no duplicate records between the tables, this default behavior of UNION doesn't negatively impact the desired result.

Therefore, UNION effectively merges the two tables, resulting in a single table, all_transactions, containing all unique records from both march_transactions and april_transactions, achieving the desired outcome. This is the simplest and most effective way to consolidate these two tables into one in Databricks SQL.

https://docs.databricks.com/sql/language-manual/sql-ref-syntax-qry-select-union.html

Question: 157 CertyIQ

How can Git operations must be performed outside of Databricks Repos?

A.Commit
B.Pull
C.Merge
D.Clone

Answer: D

Explanation:

D. Clone.

In Databricks Repos, most Git operations like:

Commit

Pull

Merge

can be performed directly within the Databricks workspace.

Question: 158 CertyIQ

A data engineer has joined an existing project and they see the following query in the project repository:
CREATE STREAMING LIVE TABLE loyal_customers AS
SELECT customer_id -
FROM STREAM(LIVE.customers)
WHERE loyalty_level = 'high';
Which of the following describes why the STREAM function is included in the query?

A.The STREAM function is not needed and will cause an error.
B.The data in the customers table has been updated since its last run.
C.The customers table is a streaming live table.
D.The customers table is a reference to a Structured Streaming query on a PySpark DataFrame.

Answer: C

Explanation:

The correct answer is C because the STREAM function in the CREATE STREAMING LIVE TABLE query is specifically used to read data from another Delta Live Tables (DLT) table that is defined as a streaming live table. Delta Live Tables is a declarative framework for building reliable, maintainable, and testable data processing pipelines. DLT handles incremental data processing and manages the underlying infrastructure. When a table is declared as a streaming live table (as LIVE.customers presumably is), it continuously ingests data from a streaming source, such as Apache Kafka or a cloud storage system.

The STREAM function indicates that the query should treat the source table (LIVE.customers) as a streaming source, enabling incremental processing. Without the STREAM function, DLT would treat LIVE.customers as a regular Delta table and perform a full refresh each time the pipeline is updated, which defeats the purpose of streaming and incremental updates. This would violate the paradigm of incremental data processing inherent to streaming live tables.

Option A is incorrect because the STREAM function is needed when the source table is a streaming live table. It's the mechanism by which DLT understands that data should be read incrementally. Removing it would change the semantics of the query and lead to incorrect results or pipeline failures.

Option B is incorrect because the need for the STREAM function isn't directly related to updates since the last run; it's fundamentally tied to how the source table is defined (as a streaming live table). The STREAM function ensures new data is processed incrementally and that updates to the source table propagate in a continuous fashion to any downstream DLT table that's reading from it.

Option D is incorrect because while Structured Streaming is a related concept in Apache Spark, STREAM(LIVE.customers) does not directly represent a reference to a Structured Streaming query on a PySpark DataFrame. DLT provides a higher-level abstraction, and STREAM tells DLT that it should leverage its own capabilities to consume data as a stream from the named DLT table. The underlying implementation might use Structured Streaming or other Spark technologies, but the syntax STREAM(LIVE.customers) is specifically a DLT construct.

Further research can be conducted on Delta Live Tables in the Databricks documentation:

What is Delta Live Tables?
Delta Live Tables SQL Language Reference

Question: 159 CertyIQ

Which Structured Streaming query is performing a hop from a Silver table to a Gold table?

A.

```
(spark.readStream.load(rawSalesLocation)
    .writeStream
    .option("checkpointLocation", checkpointPath)
    .outputMode("append")
    .table("newSales")
)
```

B.

```
(spark.table("sales")
    .withColumn("avgPrice", col("sales") / col("units"))
    .writeStream
    .option("checkpointLocation", checkpointPath)
    .outputMode("append")
    .table("newSales")
)
```

C.

```
(spark.table("sales")
    .filter(col("units") > 0)
    .writeStream
    .option("checkpointLocation", checkpointPath)
    .outputMode("append")
    .table("newSales")
)
```

D.

```
(spark.table("sales")
    .groupBy("store")
    .agg(sum("sales"))
    .writeStream
    .option("checkpointLocation", checkpointPath)
    .outputMode("complete")
    .table("newSales")
)
```

Answer: D

Explanation:

```
(spark.table("sales")
    .groupBy("store")
    .agg(sum("sales"))
    .writeStream
    .option("checkpointLocation", checkpointPath)
    .outputMode("complete")
    .table("newSales")
)
```

Question: 160 CertyIQ

A data organization leader is upset about the data analysis team's reports being different from the data engineering team's reports. The leader believes the siloed nature of their organization's data engineering and data analysis architectures is to blame.

Which of the following describes how a data lakehouse could alleviate this issue?

A.Both teams would respond more quickly to ad-hoc requests
B.Both teams would use the same source of truth for their work
C.Both teams would reorganize to report to the same department
D.Both teams would be able to collaborate on projects in real-time

Answer: B

Explanation:

The correct answer is B: Both teams would use the same source of truth for their work.

A data lakehouse architecture aims to unify the best aspects of data warehouses and data lakes. A key feature of a data lakehouse is its ability to provide a single source of truth for all data within an organization. By establishing a centralized repository with consistent data governance and quality controls, a data lakehouse eliminates data silos. Data engineers and data analysts can then access and utilize the same foundational datasets for their respective tasks. This shared data foundation ensures that both teams are working with the same data, leading to consistent results in their reports and analyses. This consistency resolves discrepancies arising from isolated data pipelines and transformations specific to individual teams. Options A, C, and D are incorrect because they address process or organizational concerns but not the core problem of data inconsistency arising from separate data architectures. A data lakehouse provides a consistent, governed data layer, eliminating data silos and ensuring both teams use the same reliable data source for their work.

Further reading:

Databricks Lakehouse Platform: https://www.databricks.com/product/data-lakehouse
What is a Data Lakehouse?: https://aws.amazon.com/what-is/data-lakehouse/

Question: 161 CertyIQ

A data analyst has developed a query that runs against Delta table. They want help from the data engineering team to implement a series of tests to ensure the data returned by the query is clean. However, the data engineering team uses Python for its tests rather than SQL.
Which of the following operations could the data engineering team use to run the query and operate with the results in PySpark?

A.SELECT * FROM sales
B.spark.delta.table
C.spark.sql
D.spark.table

Answer: C

Explanation:

The correct answer is **C. spark.sql**. Here's why:

The goal is to execute an existing SQL query within a PySpark environment. The spark.sql function in PySpark provides the direct interface for executing SQL queries. It takes a SQL query string as input and returns a PySpark DataFrame containing the results. This allows data engineers to leverage their existing Python-based

testing frameworks and data manipulation tools to work with the data returned by the analyst's SQL query. They can then apply various data quality checks, transformations, and assertions using PySpark's DataFrame API.

Option A (SELECT * FROM sales) is incomplete; it's a SQL query fragment but doesn't provide the context of how to execute it within PySpark. Option B (spark.delta.table) is related to accessing Delta tables specifically but doesn't execute an arbitrary SQL query. It helps load a Delta table into a DataFrame. Option D (spark.table) is used to load an existing table registered in the Spark catalog (metastore) into a DataFrame, it cannot execute an arbitrary SQL query.

Therefore, spark.sql is the only option that directly addresses the requirement of executing a SQL query from within PySpark and making the resulting data available as a DataFrame for further processing and testing. This approach allows seamless integration of SQL-based data extraction with Python-based data validation and transformation workflows.For further research, you can refer to the following links:

Spark SQL, DataFrames and Datasets Guide: https://spark.apache.org/docs/latest/sql-getting-started.html
PySpark SQL Functions Documentation: https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql.html (specifically look for spark.sql())

Question: 162 CertyIQ

A data engineer has a Job that has a complex run schedule, and they want to transfer that schedule to other Jobs. Rather than manually selecting each value in the scheduling form in Databricks, which of the following tools can the data engineer use to represent and submit the schedule programmatically?

A.pyspark.sql.types.DateType
B.datetime
C.pyspark.sql.types.TimestampType
D.Cron syntax

Answer: D

Explanation:

The correct answer is Cron syntax because it provides a standard, string-based method for defining complex, recurring schedules. Cron syntax allows users to specify schedules using fields representing minute, hour, day of the month, month, and day of the week. This string representation can then be programmatically stored, transferred, and applied to different Jobs.

Options A, B, and C are incorrect as they are related to data types within PySpark or Python and do not represent a scheduling language or system. pyspark.sql.types.DateType and pyspark.sql.types.TimestampType are used for specifying the data type of columns in a Spark DataFrame, and datetime is a Python module for working with dates and times as data. These types are useful for handling date and time data, but not for scheduling tasks.

Cron syntax, on the other hand, is explicitly designed for defining schedules. By using Cron syntax, the data engineer can define a schedule once and then reuse it across multiple Jobs by simply copying the Cron string and applying it through the Databricks API or UI (if it supports Cron string inputs). This is much more efficient and less error-prone than manually re-entering all the scheduling parameters for each Job. Databricks Jobs often provide a way to define schedules via cron expressions in the job settings.

Therefore, Cron syntax is the appropriate tool for programmatically representing and submitting schedules for Databricks Jobs.

Further research can be done using the following links:

Cron Syntax: https://en.wikipedia.org/wiki/Cron
Databricks Jobs (check for cron scheduling support in the API or UI documentation):
https://docs.databricks.com/workflows/jobs/index.html

Question: 163 CertyIQ

A data engineer and data analyst are working together on a data pipeline. The data engineer is working on the raw, bronze, and silver layers of the pipeline using Python, and the data analyst is working on the gold layer of the pipeline using SQL. The raw source of the pipeline is a streaming input. They now want to migrate their pipeline to use Delta Live Tables.
Which of the following changes will need to be made to the pipeline when migrating to Delta Live Tables?

A.The pipeline will need to be written entirely in Python
B.The pipeline will need to stop using the medallion-based multi-hop architecture
C.The pipeline will need to be written entirely in SQL
D.The pipeline will need to use a batch source in place of a streaming source

Answer: D

Explanation:

Here's a detailed justification for why option D is the correct answer, along with supporting concepts and links:

Delta Live Tables (DLT) natively supports both streaming and batch sources. While DLT excels at simplifying incremental data processing, it imposes a specific way of defining the data flow through declarative pipelines. Therefore, option B, that the pipeline will need to stop using the medallion-based multi-hop architecture, is incorrect because DLT's design fits very well with the medallion architecture (Bronze, Silver, Gold layers). The medallion architecture inherently encourages modular data transformation, aligning with the principle of separating concerns that DLT reinforces. Options A and C are incorrect because DLT supports both SQL and Python languages to create pipelines.

The need to use batch sources stems from the inherent processing characteristics of DLT. DLT manages the scheduling, dependencies, and execution of data transformations automatically. To provide reliable guarantees on data completeness and correctness, DLT fundamentally relies on the ability to replay and backfill data in the pipeline as needed. Streaming sources, by their very nature, provide continuous, unbounded data flow. It is usually not possible or easy to replay historic data from a streaming source.

Therefore, if the raw source is truly a streaming input in its purest form (e.g., Kafka topic with no ability to rewind), a direct migration to DLT might not be feasible without introducing a buffering or checkpointing mechanism. The recommendation is to adapt it into a "micro-batch" pattern, meaning the streaming source is treated as a series of small batch jobs. DLT can consume the output of that micro-batch process, but it can not directly consume the original pure streaming data. This is usually done by storing the streaming data into cheap object storage (Azure Data Lake Storage/AWS S3) and pointing the DLT raw data to the cheap object storage instead of the streaming data.

Therefore, the most significant adjustment when moving to DLT would be to transform the existing streaming source into a system that can emit "batches" of data. This might involve buffering the data for a certain period or storing it in a file system/data lake as the streaming data arrives to allow for replayability when DLT pipeline needs to backfill data. This approach ensures that DLT can effectively manage the data flow and

provide the data quality guarantees inherent in its design.

Authoritative Links:

Delta Live Tables Documentation: https://docs.databricks.com/delta-live-tables/index.html - This provides comprehensive details on DLT, including its features, architecture, and how it handles data processing.
Medallion Architecture: https://www.databricks.com/blog/2019/08/15/diving-into-delta-lake-unpacking-the-medallion-architecture.html - This explains the Medallion architecture (Bronze, Silver, Gold) and how it works with Delta Lake.

Question: 164 CertyIQ

A data engineer needs access to a table new_table, but they do not have the correct permissions. They can ask the table owner for permission, but they do not know who the table owner is.
Which of the following approaches can be used to identify the owner of new_table?

A.Review the Permissions tab in the table's page in Data Explorer
B.There is no way to identify the owner of the table
C.Review the Owner field in the table's page in Data Explorer
D.Review the Owner field in the table's page in the cloud storage solution

Answer: C

Explanation:

The correct answer is C, "Review the Owner field in the table's page in Data Explorer."

Databricks Data Explorer is a user interface designed to help data engineers and analysts discover, explore, and understand data assets within the Databricks environment. It provides a centralized location to manage and monitor data objects, including tables. One of the key features of Data Explorer is the ability to view table metadata, including information about the table owner.

The "Owner" field in the table's page within Data Explorer explicitly identifies the principal (user or group) that owns the table. This is crucial for permission management because the owner generally has full control over the table, including the ability to grant permissions to other users.

Option A, "Review the Permissions tab in the table's page in Data Explorer," is partially correct. While the Permissions tab does show which users or groups have specific privileges on the table, it doesn't directly indicate who the owner is. It shows who has what permissions, but not who has the permission to grant permissions.

Option B, "There is no way to identify the owner of the table," is incorrect because Data Explorer is designed to provide this kind of metadata information. Knowing the owner is vital for data governance and access control.

Option D, "Review the Owner field in the table's page in the cloud storage solution," might seem plausible. While cloud storage solutions (like AWS S3 or Azure Blob Storage) underpin Databricks, the ownership within Databricks is managed at the metastore level (typically a Hive metastore or Databricks Unity Catalog). The storage layer might have its own ownership concepts, but those are distinct from the ownership relevant for granting permissions within Databricks SQL or PySpark environments. Data Explorer surfaces the Databricks-level ownership.

Therefore, Data Explorer's "Owner" field directly reveals the table owner, enabling the data engineer to contact the appropriate person for permission grants.

For more information, refer to the Databricks documentation on Data Explorer and access control:

Databricks Data Explorer
Databricks Access Control

Question: 165 CertyIQ

A data engineer needs to create a table in Databricks using data from their organization's existing SQLite database.
They run the following command:

```
CREATE TABLE jdbc_customer360
USING _____
OPTIONS (
  url "jdbc:sqlite:/customers.db",
  dbtable "customer360"
)
```

Which of the following lines of code fills in the above blank to successfully complete the task?

A. org.apache.spark.sql.jdbc
B. autoloader
C. org.apache.spark.sql.sqlite
D. sqlite

Answer: A

Explanation:

org.apache.spark.sql.sqlite: This would be the Java package for interacting with SQLite databases specifically within Spark SQL. SQLite is a lightweight, file-based relational database.

Question: 166 CertyIQ

In which of the following scenarios should a data engineer use the MERGE INTO command instead of the INSERT INTO command?

A.When the location of the data needs to be changed
B.When the target table is an external table
C.When the source is not a Delta table
D.When the target table cannot contain duplicate records

Answer: D

Explanation:

D. When the target table cannot contain duplicate records.

MERGE INTO is used for upserts (update + insert) and deduplication.

It allows a data engineer to define logic like:

If a matching record exists → UPDATE it.

If no matching record exists → INSERT it.

This is crucial when a table must not contain duplicates, especially when integrating data from sources that may contain overlapping records.

Why the other options are incorrect:

A. When the location of the data needs to be changed

→ This refers more to data movement or path changes, not something handled by MERGE INTO.

B. When the target table is an external table

→ MERGE INTO can be used with external Delta tables, but being "external" alone isn't a reason to choose MERGE over INSERT.

C. When the source is not a Delta table

→ MERGE INTO requires Delta tables as the target; if the source isn't Delta, it still can be used, but the target must be a Delta table.

Question: 167 CertyIQ

A data engineer is designing a data pipeline. The source system generates files in a shared directory that is also used by other processes. As a result, the files should be kept as is and will accumulate in the directory. The data engineer needs to identify which files are new since the previous run in the pipeline, and set up the pipeline to only ingest those new files with each run.
Which of the following tools can the data engineer use to solve this problem?

A.Unity Catalog
B.Delta Lake
C.Databricks SQL
D.Auto Loader

Answer: D

Explanation:

The correct answer is **D. Auto Loader**. Here's why:

Auto Loader is specifically designed for incremental data ingestion from cloud storage into Delta Lake. It automatically detects new files as they arrive in a cloud storage location like a shared directory. It leverages cloud infrastructure features (like file notification events) and efficient directory listing to identify new files without requiring manual file tracking or complex logic.

Auto Loader keeps track of which files it has already processed, eliminating the need for the data engineer to build and maintain custom state management. This makes it ideal for scenarios where new files are continuously added to a directory used by other processes, and only the newly added files need to be

ingested in each pipeline run. The other options are not as suited to this task. Unity Catalog is for data governance and discovery, not for file ingestion. Delta Lake is a storage layer that benefits from Auto Loader but doesn't solve the new file identification problem on its own. Databricks SQL is a query engine and is not designed for incremental data ingestion.

Auto Loader avoids reprocessing already-ingested data, making the pipeline more efficient. Its schema inference and evolution capabilities also simplify data pipeline development and maintenance in environments where the source data schema might change over time. It also provides features to handle file format variations automatically.

For further reading on Auto Loader, you can refer to the official Databricks documentation: https://docs.databricks.com/ingestion/auto-loader/index.html

Question: 168 CertyIQ

What is stored in the Databricks customer's cloud account?

A.Databricks web application
B.Cluster management metadata
C.Notebooks
D.Data

Answer: D

Explanation:

The correct answer is **D. Data**. Here's why:

Databricks leverages a customer's existing cloud infrastructure (AWS, Azure, or GCP) for its operations. The control plane and compute plane are primarily managed by Databricks, but the data itself resides within the customer's cloud storage.

Let's break down why the other options are incorrect:

A. Databricks web application: The Databricks web application is hosted and managed by Databricks itself, not within the customer's cloud account. Customers access this application through a web browser.

B. Cluster management metadata: While Databricks manages cluster metadata related to cluster configuration, status, and metrics, this metadata is primarily stored within the Databricks control plane for operational efficiency and performance. It's not generally directly exposed or stored within the customer's storage account.

C. Notebooks: While notebook definitions (code, markdown) are stored within the Databricks workspace (backed by cloud storage), the execution context and persistent data generated from those notebooks resides in the customer's cloud storage.

Therefore, the primary component stored in the customer's cloud account is the **data** itself. This includes data stored in data lakes (e.g., S3 on AWS, ADLS Gen2 on Azure, GCS on GCP), databases, and other storage services that the Databricks environment interacts with. Databricks accesses this data for processing, analysis, and transformation. The data is governed and controlled by the customer within their own cloud environment. This architecture enables Databricks to offer its services while maintaining data sovereignty and security for its customers. Databricks is a PaaS (Platform as a Service) on top of IaaS (Infrastructure as a Service) for cloud providers. Databricks processes the data, but the cloud provider is responsible for storing

the data itself.

Here are a few authoritative links to support this explanation:

Databricks Architecture: https://www.databricks.com/wp-content/uploads/2020/07/databricks-architecture.pdf
Databricks Security and Trust: https://www.databricks.com/trust

Question: 169 CertyIQ

A data engineer wants to create a relational object by pulling data from two tables. The relational object does not need to be used by other data engineers in other sessions. In order to save on storage costs, the data engineer wants to avoid copying and storing physical data.
Which of the following relational objects should the data engineer create?

A.Spark SQL Table
B.View
C.Delta Table
D.Temporary view

Answer: D

Explanation:

The correct answer is **D. Temporary View**. Here's a detailed justification:

A temporary view in Spark SQL is a relational object that exists only within the scope of the current session. This perfectly aligns with the requirement that the relational object doesn't need to be accessible by other data engineers in other sessions. Creating a temporary view avoids persisting the data to storage, satisfying the goal of minimizing storage costs.

A Spark SQL Table (A) and a Delta Table (C) both involve persisting data to storage. Creating either of these would incur storage costs and are not suitable for the given requirements. They are designed for persistent data storage and sharing across sessions. Delta Tables specifically are for reliable data lakes.

A View (B), while not necessarily materializing data immediately, are typically persisted as metadata in the metastore. This means they can be accessed by other sessions and users if the user has appropriate permissions. While views perform query time transformation, the user expects views to be available for any session that can connect to the metastore and database. Thus, a view is not ideal given the stated requirement for temporary, session-specific accessibility.

Temporary views, on the other hand, are not persisted to any metastore. They are created in memory and are automatically dropped when the session ends. They're ideal for scenarios where you need a relational object for intermediate calculations or transformations within a single session without impacting storage costs or other users.

Therefore, the temporary view is the best choice. It provides the required relational object, avoids data duplication, and is confined to the current session, saving on storage costs and preventing unwanted sharing.

Relevant Documentation:

1. **Spark SQL, DataFrames and Datasets Guide:** https://spark.apache.org/docs/latest/sql-programming-guide.html (search for "temporary views")
2. **Databricks Views:** https://docs.databricks.com/sql/language-manual/sql-ref-syntax-ddl-create-

view.html

Question: 170 CertyIQ

Which of the following commands will return the number of null values in the member_id column?

A.SELECT count(member_id) FROM my_table;
B.SELECT count(member_id) - count_null(member_id) FROM my_table;
C.SELECT count_if(member_id IS NULL) FROM my_table;
D.SELECT null(member_id) FROM my_table;

Answer: C

Explanation:

C. SELECT count_if(member_id IS NULL) FROM my_table;

count_if(condition) counts the number of rows where the condition is true.

So count_if(member_id IS NULL) returns the number of NULL values in the member_id column.

Why the other options are incorrect:

A. SELECT count(member_id) FROM my_table;

This counts non-null values only, not nulls.

B. SELECT count(member_id) - count_null(member_id) FROM my_table;

count_null() is not a valid SQL function in standard SQL or Databricks/Spark SQL.

D. SELECT null(member_id) FROM my_table;

This is invalid syntax. There's no null() function like this.

Question: 171 CertyIQ

Which tool is used by Auto Loader to process data incrementally?

A.Checkpointing
B.Spark Structured Streaming
C.Databricks SQL
D.Unity Catalog

Answer: B

Explanation:

B. Spark Structured Streaming

Auto Loader is a feature in Databricks that allows for incremental loading of data files (like CSV, JSON, Parquet) from cloud storage (e.g., AWS S3, Azure Data Lake) as they arrive.

Auto Loader is built on top of Spark Structured Streaming, which is the underlying engine that:

Continuously processes incoming files

Maintains state information

Supports exact-once semantics

Enables incremental ingestion

Why the other options are incorrect:

A. Checkpointing

Related but not the core tool.

It's a mechanism used by Spark Structured Streaming (including Auto Loader) to track progress. It's a feature, not the tool itself.

C. Databricks SQL

Used for querying and BI dashboards. It’s not used for incremental file processing.

D. Unity Catalog

Manages data governance (permissions, metadata), not data ingestion or streaming.

Question: 172 CertyIQ

Which of the following benefits is provided by the array functions from Spark SQL?

A. An ability to work with data in a variety of types at once
B. An ability to work with data within certain partitions and windows
C. An ability to work with time-related data in specified intervals
D. An ability to work with complex, nested data ingested from JSON files

Answer: D

Explanation:

D. An ability to work with complex, nested data ingested from JSON files.

This is a core benefit of array functions. When you ingest JSON (or other semi-structured formats like Avro, Parquet with nested structures), you often encounter arrays within the nested data. Spark SQL's array functions (e.g., explode, array_contains, size, get_json_object, from_json, to_json working with arrays, etc.) are specifically designed to manipulate, transform, and query these complex, nested array structures. They allow you to "flatten" arrays, access specific elements, filter based on array content, and more.

Question: 173 CertyIQ

A data engineer is working with two tables. Each of these tables is displayed below in its entirety.

sales

customer_id	spend	units
a1	28.94	7
a3	874.12	23
a4	8.99	1

favorite_stores

customer_id	store_id
a1	s1
a2	s1
a4	s2

The data engineer runs the following query to join these tables together:

```
SELECT
    sales.customer_id,
    sales.spend,
    favorite_stores.store_id
FROM sales
LEFT JOIN favorite_stores
ON sales.customer_id = favorite_stores.customer_id;
```

Which of the following will be returned by the above query?

A.

customer_id	spend	store_id
a1	28.94	s1
a4	8.99	s2

B.

customer_id	spend	store_id
a1	28.94	s1
a2	NULL	s1
a4	8.99	s2

C.

customer_id	spend	store_id
a1	28.94	s1
a3	874.12	NULL
a4	8.99	s2

D.

customer_id	spend	store_id
a1	28.94	s1
a2	NULL	s1
a3	874.12	NULL
a4	8.99	s2

Answer: C

Explanation:

Row 1 (Customer a1):

Spent $28.94 at store s1.

Both spend and store_id have valid values.

Row 2 (Customer a3):

Spent $874.12, but store_id is NULL.

This might indicate an online transaction, an unknown store, or a data entry issue.

Row 3 (Customer a4):

Spent $8.99 at store s2.

Both spend and store_id have valid values.

Question: 174 CertyIQ

A data engineer has been provided a PySpark DataFrame named df with columns product and revenue. The data engineer needs to compute complex aggregations to determine each product's total revenue, average revenue, and transaction count.
Which code snippet should the data engineer use?

A.

```
from pyspark.sql import functions as F
aggregated_df = df.groupBy("product").agg( F.sum("revenue").alias("total_revenue"),
F.avg("revenue").alias("avg_revenue"), F.count("*").alias("transaction_count") )
```

```
aggregated_df = df.groupby("product").agg(
    "sum(revenue)",
    "avg(revenue)",
    "count(revenue)"
)
```

B.

C.

```
from pyspark.sql import functions as F aggregated_df = df.select("product",

"revenue").groupBy("product").agg( F.sum("revenue"), F.mean("revenue") )
```

D.

```
aggregated_df =
df.groupBy("product").agg( {"revenue": "sum", "revenue": "avg", "revenue": "count"} )
```

Answer: A

Explanation:

```
from pyspark.sql import functions as F
aggregated_df = df.groupBy("product").agg( F.sum("revenue").alias("total_revenue"),
F.avg("revenue").alias("avg_revenue"), F.count("*").alias("transaction_count") )
```

Question: 175 CertyIQ

A Databricks single-task workflow fails at the last task due to an error in a notebook. The data engineer fixes the mistake in the notebook.
What should the data engineer do to rerun the workflow?

A. Repair the task
B. Rerun the pipeline
C. Restart the cluster
D. Switch the cluster

Answer: A

Explanation:

Correct Answer

Option A: Repair the task

The data engineer should repair the task to rerun the workflow efficiently.

Explanation of the Correct Answer

The correct answer is A. Repair the task.

When a Databricks workflow fails due to an error in a notebook, and the error has been fixed, the "Repair task" functionality allows you to resume execution from the point of failure. This avoids re-running the entire workflow and saves computational resources.

The repair functionality uses the corrected notebook code and preserves the results of previously successful tasks.

Analysis of Each Option

Option A: Repair the task - This is the correct option because it allows resuming the workflow from the failed task after the notebook has been fixed, without re-running the entire workflow.

Option B: Rerun the pipeline - This is incorrect because it restarts the entire workflow from the beginning, wasting resources and time since previous tasks completed successfully.

Option C: Restart the Cluster - This is incorrect because restarting the cluster addresses infrastructure issues, not code errors. The cluster itself wasn't the problem.

Option D: Switch the cluster - This is incorrect because, like restarting the cluster, switching the cluster addresses infrastructure concerns rather than the code issue that was fixed.

Key Takeaways

The "Repair task" functionality in Databricks workflows is the most efficient way to resume a workflow after a task fails due to a code error that has been fixed.

Avoid re-running the entire pipeline or restarting/switching the cluster when the issue is a code error in a specific task.

Always consider the specific cause of the failure when choosing a recovery method.

Question: 176 CertyIQ

A data engineer needs to provide access to a group named manufacturing-team. The team needs privileges to create tables in the quality schema.
Which set of SQL commands will grant a group named manufacturing-team to create tables in a schema named production with the parent catalog named manufacturing with the least privileges?

A. GRANT CREATE TABLE ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT USE SCHEMA ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT USE CATALOG ON CATALOG manufacturing TO manufacturing-team;

B. GRANT USE TABLE ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT USE SCHEMA ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT USE CATALOG ON CATALOG manufacturing TO manufacturing-team;

C. GRANT CREATE TABLE ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT CREATE SCHEMA ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT CREATE CATALOG ON CATALOG manufacturing TO manufacturing-team;

D. GRANT CREATE TABLE ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT CREATE SCHEMA ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT USE CATALOG ON CATALOG manufacturing TO manufacturing-team;

Answer: A

Explanation:

Option A is correct as it appropriately grants the minimum required privileges for the manufacturing-team to

create tables in the specified schema while ensuring they have the necessary access to utilize the schema and catalog.

Reasoning for Option A:

1. **GRANT CREATE TABLE ON SCHEMA manufacturing.quality TO manufacturing-team;**

   This command directly provides the required permission to create tables within the quality schema.
2. **GRANT USE SCHEMA ON SCHEMA manufacturing.quality TO manufacturing-team;**

   It allows the team to access the schema to work with existing tables or other objects they may need.
3. **GRANT USE CATALOG ON CATALOG manufacturing TO manufacturing-team;**

   This grants access to the parent catalog, necessary for schema operations.

This sequence minimizes privileges while fulfilling the requirement to create tables, adhering to the principle of least privilege.

Evaluation of Other Options:

Option B:

Why Incorrect: Grants USE TABLE, which is not relevant for the task of creating tables. The correct command for table creation should be CREATE TABLE.

Option C:

Why Incorrect: Grants CREATE SCHEMA and CREATE CATALOG, which are unnecessary for the stated requirement. The team only needs to create tables in an existing schema, not create new schemas or catalogs.

Option D:

Why Incorrect: Similar to Option C, it includes CREATE SCHEMA, extending privileges beyond what is required. Additionally, USE CATALOG is appropriate but not the correct combination to limit permissions effectively for the task at hand.
References:

https://docs.databricks.com/dev-tools/databricks-sql/sql-commands.html
https://docs.databricks.com/dev-tools/databricks-sql/access-control.html
https://docs.databricks.com/sql/language-manual/sql-commands.html

Question: 177 CertyIQ

A data engineer has written a function in a Databricks Notebook to calculate the population of bacteria in a given medium.

```
def calculate_population_of_bacteria (intital_population, exponential_factor)
    return future_population = intitial_population ** expontential_factor
```

Analysts use this function in the notebook and sometimes provide input arguments of the wrong data type, which can cause errors during execution.
Which Databricks feature will help the data engineer quickly identify if an incorrect data type has been provided as input?

A. The Spark User interface has a debug tab that contains the variables that are used in this session.
B. The Databricks debugger enables breakpoints that will raise an error if the wrong data type is submitted.
C. The Databricks debugger enables the use of a variable explorer to see at a glance the value of the variables.

D. The Data Engineer should add print statements to find out what the variable is.

Answer: C

Explanation:

Correct Answer:

C. The Databricks debugger enables the use of a variable explorer to see at a glance the value of the variables.

Explanation:

When analysts pass the wrong data type to the function (e.g., a string instead of a number), it can cause execution errors. The Databricks debugger's variable explorer is the most efficient way to catch this issue because:

It shows all variables currently in scope.

For each variable, you can see its value and data type immediately.

This makes it very quick to confirm if the wrong type of input (e.g., "ten" instead of 10) has been provided.

This avoids the need for guesswork or manual print statements.

Why the other options are incorrect:

A. Spark UI debug tab → The Spark UI is for job execution monitoring (tasks, stages, DAGs), not for checking variable types in notebook code.

B. Breakpoints → Breakpoints pause execution but don't automatically validate data types. They're useful for step-by-step debugging, but not the fastest way to see type issues.

D. Print statements → A very manual method, requiring code changes each time. Much less efficient than using the variable explorer.

Key Takeaway:

The variable explorer in the Databricks debugger is the right feature because it provides immediate visibility into variables and their types, making it the fastest way to identify incorrect input data types.

Final Answer: C. The Databricks debugger enables the use of a variable explorer to see at a glance the value of the variables.

Question: 178 CertyIQ

A data engineer is inspecting an ETL pipeline based on a Pyspark job that consistently encounters performance bottlenecks. Based on developer feedback, the data engineer assumes the job is low on compute resources. To pinpoint the issue, the data engineer observes the Spark UI and finds out the job has a high CPU time vs Task time. Which course of action should the data engineer take?

A. High CPU time vs Task time means an under-utilized cluster. The data engineer may need to repartition data to spread the jobs more evenly throughout the cluster.

B. High CPU time vs Task time means efficient use of cluster and no change needed

C. High CPU time vs Task time means a CPU over-utilized job. The data engineer may need to consider executor and core tuning or resizing the cluster

D. High CPU time vs Task time means over-utilized memory and the need to increase parallelism

Answer: C

Explanation:

C is correct because high CPU time relative to task time indicates that the job is CPU-bound, necessitating adjustments to executor and core settings or resizing the cluster.

In distributed computing frameworks like Apache Spark, adequate resource allocation is essential for job performance. A disparity where CPU time significantly exceeds task time suggests that tasks are not completing efficiently, often due to insufficient computing resources to handle the workload. This scenario typically arises when tasks require more processing power than what is currently allocated, leading to CPU over-utilization. Adjusting executor and core configurations can help optimize resource distribution, enabling the pipeline to process data more effectively and ultimately improving throughput. Resizing the cluster may also provide additional compute resources, ensuring that tasks can execute with adequate power.

Evaluation of Incorrect Options:

A. High CPU time vs Task time indicates an under-utilized cluster.
This option misinterprets the performance metrics. An under-utilized cluster would exhibit low CPU times. Here, the imbalance signifies that the current resources are insufficient for the workload, rather than being under-utilized.

B. High CPU time vs Task time means efficient use of the cluster and no change needed.
This assertion is misleading as it ignores the implications of high CPU utilization. Efficiency in resource use would typically manifest as a more balanced ratio of CPU time to task time. A high CPU time indicates that tasks may be taking longer to execute, signaling a need for immediate optimization.

D. High CPU time vs Task time means over-utilized memory and the need to increase parallelism.
This option conflates CPU and memory usage indicators. The issue presented relates to CPU demands, not memory, signaling that increased parallelism alone would not address the CPU bottleneck problem effectively.

References

https://spark.apache.org/docs/latest/running-on-yarn.html
https://databricks.com/blog/2020/02/06/best-practices-for-using-apache-spark-in-yarn.html
https://docs.databricks.com/spark/latest/spark-sql/index.html

Question: 179 CertyIQ

A data engineer needs to parse only png files in a directory that contains files with different suffixes. Which code should the data engineer use to achieve this task?

A. df = spark.readStream.format("cloudFiles") \ .option("cloudFiles.format", "binaryFile") \
.append("/*.png")

B. df = spark.readstream. format("cloudFiles") \
.option("cloudFiles.format", "binaryFile") \
.option("pathGlobfilter", "*.png") \
.load()

C. df = spark.readStream.format("cloudFiles") \ .option("cloudFiles.format", "binaryFile") \
.option("pathGlobfilter", "*.png") \ .append()

D. df = spark.readstream.format("cloudFiles") \ .option("cloudFiles.format", "binaryFile") \

.load("/*.png")

Answer: B

Explanation:

df = spark.readStream.format("cloudFiles") \

.option("cloudFiles.format", "binaryFile") \

.option("pathGlobfilter", "*.png") \

.load(cloud_storage_path) # Assumes 'cloud_storage_path' is defined

spark.readStream: Starts a streaming read operation.

.format("cloudFiles"): Specifies the use of Auto Loader (CloudFiles), which efficiently discovers new files in cloud storage.

.option("cloudFiles.format", "binaryFile"): Instructs the reader to load the files as binary file records. This is suitable for images/files where the raw data and metadata (path, size, modification time) are needed.

.option("pathGlobfilter", "*.png"): This is the critical option. It filters the files discovered by Auto Loader to only include those whose names end with the .png suffix.

.load(cloud_storage_path): Starts the stream, reading from the specified cloud storage location.

Question: 180 CertyIQ

Which languages are supported by Serverless compute clusters? (Choose two.)

A. SQL
B. Python
C. R
D. Scala
E. Java

Answer: AB

Explanation:

AB is correct because Serverless compute clusters in Databricks natively support SQL and Python for interactive data processing and analytics.

The support for SQL allows users to perform complex queries directly on data lakes using familiar SQL syntax, enabling efficient data manipulation and retrieval. Python, as a versatile programming language, facilitates rich data analysis, machine learning workflows, and integration with various data handling libraries like Pandas and NumPy, making it ideal for data engineering tasks. These features are integral to Databricks' framework, which emphasizes ease of use and flexibility in data tasks.

Conversely, the incorrect options can be evaluated as follows:

C. R: Although R is a significant language for statistical analysis and data visualization, its support in

Databricks is not as pronounced as SQL or Python within the Serverless platform, mainly due to the ecosystem's focus on data engineering over niche statistical applications.

D. Scala: While Scala is supported in Databricks, especially for Spark-based applications, its inclusion in Serverless compute environments is typically limited compared to the more user-friendly interfaces provided by SQL and Python. Databricks primarily encourages SQL and Python for accessibility and broader usability among data engineers.

E. Java: Similar to Scala, Java is foundational for Spark operations but is less emphasized within the Serverless compute structure, which prioritizes languages that offer greater support for data manipulation and analytics.

In summary, SQL and Python not only align with Databricks' core functionalities but also reflect industry trends where ease of use and community adoption guide technology choices.

References:

https://docs.databricks.com/cloud-serverless/index.html
https://docs.databricks.com/notebooks/index.html
https://docs.databricks.com/libraries/requirements-libraries.html

Question: 181 CertyIQ

A data engineer is developing an ETL process based on Spark SQL. The execution fails. The data engineer checks the Spark UI and can see the ERRORS as follows:
"java.lang.OutofMemoryError: Java heap space"
Which two corrective actions should the data engineer perform to resolve this issue? (Choose two.)

A. Narrow the filters in order to collect less data in the query
B. Upsize the worker nodes and activate autoshuffle partitions
C. Upsize the driver node and deactivate autoshuffle partitions
D. Cache the dataset in order to boost the query performance
E. Fix the shuffle partitions to 50 to ensure the allocation

Answer: AB

Explanation:

AB is correct as they directly address the memory limitation by reducing data volume and optimizing resource allocation.

Reasoning:

A. **Narrowing the filters**: Implementing tighter filters effectively reduces the dataset size being processed. In ETL processes, particularly with Spark SQL, excessive data can strain memory resources, leading to OutOfMemoryError. By limiting the records being fetched, the workload becomes manageable, aligning with Spark's distributed architecture which handles smaller chunks of data more efficiently.

B. **Upsizing the worker nodes**: Increasing the capacity of worker nodes can enhance memory availability and computational power. Autoshuffle partitions help in redistributing data across available workers more effectively, thus mitigating the risk of exceeding memory limits, especially during operations that require shuffling large amounts of data.

Evaluation of Incorrect Options:

C. **Upsize the driver node and deactivate autoshuffle partitions**: This option focuses only on the driver node without addressing memory issues among worker nodes where the bulk of processing takes place. Disabling autoshuffle could lead to larger memory consumption as data would not be optimally redistributed, exacerbating the issue.

D. **Cache the dataset**: While caching can improve performance, it may worsen the OutOfMemoryError if the entire dataset is loaded into memory, especially if it exceeds available resources. On the contrary, minimizing the dataset size reduces the pressure on memory.

E. **Fix shuffle partitions to 50**: While adjusting shuffle partitions can help in distributing data load, fixing them rigidly without considering actual workload could lead to inefficiencies. The optimal number of partitions depends on the size of the dataset and cluster configuration. Preemptively setting it to 50 may not effectively resolve memory issues if the underlying dataset is still excessively large.

References:

https://spark.apache.org/docs/latest/performance-tuning.html
https://docs.databricks.com/develop/databricks-sql-optimization.html
https://www.oreilly.com/library/view/spark-in-action/9781638352152/

Question: 182 CertyIQ

A Data Engineer is designing Bronze layer in Databricks Medallion Architecture. The raw data is collected from multiple sources (Clickstream in JSON, Transactions in CSV).
The task is to design the Bronze layer of the Medallion Architecture to ingest and store this raw data for further processing.
Which operation applies to the Bronze layer?

A. Ingest raw data without transformations, preserving original schemas, and store in Delta format.
B. Clean and standardize raw data by removing null values and enforcing schemas.
C. Apply complex business logic to enrich raw data with customer segmentation labels.
D. Aggregate and transform source data to calculate daily sales performance metrics.

Answer: A

Explanation:

A. Ingest raw data without transformations, preserving original schemas, and store in Delta format.

The Bronze layer serves as the initial landing zone for raw data ingested into the data lakehouse. Its primary purpose is to provide an immutable and complete historical record of the source data.

Ingestion: Data is ingested directly from source systems (e.g., streaming topics, file systems, databases).

No Transformation: It is crucial that the raw data is stored without any transformations, cleaning, or parsing applied (except perhaps minor adjustments like converting file formats to Delta or adding metadata columns like ingestion time).

Preserve Original Schema: The data retains its original schema from the source, making it easy to trace and re-process later if the understanding of the source data changes.

Storage Format: The data is typically stored using Delta Lake format to gain ACID properties, schema enforcement/evolution, and time travel capabilities, even for raw data.

Question: 183 CertyIQ

What is the primary function of the Silver layer in the Databricks medallion architecture?

A. Store historical data solely for auditing purposes
B. Aggregate and enrich data for business analytics
C. Validate, clean, and deduplicate data for further processing
D. Ingest raw data in its original state

Answer: C

Explanation:

C. Validate, clean, and deduplicate data for further processing.

Why the other options are incorrect

A — Only for auditing

Not true. That is not the purpose of the Silver layer.

B — Aggregate/enrich for business analytics

This describes the Gold layer, not Silver.

D — Ingest raw data

This is the Bronze layer's job.

Question: 184 CertyIQ

A data engineer needs to combine sales data from an on-premises PostgreSQL database with customer data in Azure Synapse for a comprehensive report. The goal is to avoid data duplication and ensure up-to-date information.
How should the data engineer achieve this using Databricks?

A. Export data from both sources to CSV files and upload them to Databricks
B. Use Lakehouse Federation to query both data sources directly
C. Manually synchronize data from both sources into a single database
D. Develop custom ETL pipelines to ingest data into Databricks

Answer: B

Explanation:

B. Use Lakehouse Federation to query both data sources directly

Lakehouse Federation (Federated Querying)

Lakehouse Federation (powered by Unity Catalog) allows Databricks to execute queries against external data sources — like the on-premises PostgreSQL database and Azure Synapse — without migrating or duplicating the data first.

Question: 185 CertyIQ

A data engineer is working in a Python notebook on Databricks to process data, but notices that the output is not as expected. The data engineer wants to investigate the issue by stepping through the code and checking the values of certain variables during execution.
Which tool should the data engineer use to inspect the code execution and variables in real-time?

A. Cluster Logs
B. Job Execution Dashboard
C. Python Notebook Interactive Debugger
D. SQL Analytics

Answer: C

Explanation:

Python Notebook Interactive Debugger allows you to:

Debug Python notebooks line-by-line

Inspect variables in real time

Examine stack traces

Watch the state of your data as it changes

Troubleshoot ETL or ML logic interactively

Question: 186 CertyIQ

A data engineer has inherited a Databricks pipeline from a previous team. The pipeline is missing its SLAs, and an initial investigation has identified memory spills in Spark. These increased runtimes are also driving up costs. The data engineer needs to reduce the runtime without significantly increasing costs.
What should the data engineer do first to address this issue?

A. Tweak the "spark.sql.shuffle.partitions" configuration.
B. Enable autoscaling in the cluster to match the requirements
C. Use a photon enabled execution engine.
D. Ensure the cluster uses memory optimized node types.

Answer: A

Explanation:

spark.sql.shuffle.partitions determines how many partitions Spark uses for shuffle operations (joins, aggregations, sorts).

Default = 200

Often too high or too low depending on input size

Adjusting this can drastically reduce:

Shuffle volume

Task overhead

Memory usage

Spills

Runtime

This is a zero-cost optimization and should always be considered first before scaling hardware.

Question: 187 CertyIQ

A data engineer is facing performance bottlenecks in an e-commerce transactions Delta table. The table is a managed Unity Catalog table, and it uses partitioning and Z-ordering in its data layout scheme. The predictive optimization for Unity Catalog tables is also enabled. The table has a frequently changing query filter, and the data engineer does not observe a benefit of the Data Layout or the Predictive Optimization.
How should the data engineer fix the data layout bottlenecks?

A. Re-write the Data Layout with Liquid Clustering and cluster by the Z-Ordered columns.
B. Enable Delta Caching so that query results can be read through caches.
C. Tweak the Z-Order columns and run OPTIMIZE manually.
D. Switch the Data layout from Partition+Z-Ordering to Automatic Liquid Clustering.

Answer: D

Explanation:

D is correct because Automatic Liquid Clustering continuously and adaptively reorganizes file layout to match evolving, high-cardinality query predicates, eliminating the static limitations of partition+Z-ordering for frequently changing filters.

Reasoning: Liquid Clustering is an autonomous, incremental reclustering mechanism that coalesces rows with similar access patterns into compact files and incrementally maintains clustering as the workload and data evolve, which reduces full table OPTIMIZE costs and file-scan amplification for variable predicates. Predictive Optimization provides planning hints and cache/indexing assistance but does not itself continuously rewrite physical file organization to track shifting predicates; hence it can't compensate for a static partition+Z-order layout under widely varying filters. Z-ordering and manual partitioning are effective when predicates target a stable, low-cardinality set of columns, but they generate many irrelevant file reads when predicates change; by contrast Liquid Clustering adapts automatically and is integrated with Unity Catalog to maintain performant layouts with lower operational overhead. Choosing Automatic Liquid Clustering aligns physical layout maintenance with production query variability, minimizes human tuning, and reduces recurring OPTIMIZE costs.

Why the other options are incorrect:

A — Rewriting with Liquid Clustering but "cluster by the Z-Ordered columns": redundant and conceptually inconsistent; Liquid Clustering is intended to replace static manual reclustering/Z-order maintenance, so manually imposing the same static column clustering defeats its adaptive benefit and incurs expensive one-time rewrite costs.
B — Enabling Delta Caching: helpful for repeated scans of hot data but does not change physical file layout

or reduce the number of files scanned for novel, selective predicates; caching is node/local and transient, so it does not solve layout inefficiencies caused by variable filters.
C — Tweak Z-Order columns and run OPTIMIZE manually: a one-time OPTIMIZE with Z-order is expensive and brittle — it only benefits the specific columns chosen and rapidly degrades as query predicates shift; it does not provide the ongoing adaptive reclustering that Liquid Clustering offers.

Referenceshttps://docs.databricks.com/delta/optimize-liquid-clustering.htmlhttps://docs.databricks.com/delta/optimizations/z-ordering.htmlhttps://docs.databricks.com/unity-catalog/predictive-optimizations.html

Question: 188 CertyIQ

A data engineer needs to optimize the data layout and query performance for an e-commerce transactions Delta table. The table is partitioned by "purchase_date" a date column which helps with time-based queries but does not optimize searches on user statistics "customer_id", a high-cardinality column.
The table is usually queried with filters on "customer_id" within specific date ranges, but since this data is spread across multiple files in each partition, it results in full partition scans and increased runtime and costs.
How should the data engineer optimize the Data Layout for efficient reads?

A. Alter table implementing liquid clustering on "customer_id" while keeping the existing partitioning.
B. Alter the table implementing liquid clustering by "customer_id" and "purchase_date".
C. Alter the table to partition by "customer_id".
D. Enable delta caching on the cluster so that frequent reads are cached for performance.

Answer: B

Explanation:

Alter the table implementing liquid clustering by "customer_id" and "purchase_date".

Question: 189 CertyIQ

What is the structure of an Asset Bundle?

A. A Docker image containing runtime environments and the source code of the assets
B. A compressed archive (ZIP) that solely contains workspace assets without any accompanying metadata
C. A single plain text file enumerating the names of assets to be migrated to a new workspace
D. A YAML configuration file that specifies the artifacts, resources, and configurations for the project

Answer: D

Explanation:

D. A YAML configuration file that specifies the artifacts, resources, and configurations for the project

A Databricks Asset Bundle (DAB) is fundamentally structured around a YAML configuration file, typically named:

databricks.yml

This YAML file defines:

Resources

Jobs / Workflows

Delta Live Tables pipelines

Dashboards

Models

Permissions

Clusters / serverless compute definitions

Artifacts

Notebooks

Python modules

SQL files

Configuration assets

Deployment settings for multiple environments

dev

staging

prod

The bundle is a project directory containing this YAML file plus the source code and related project assets.

Why the other options are incorrect

A. A Docker image

Completely incorrect — DABs are not container images.

B. A compressed archive of workspace assets

No — Bundles are not ZIP-only nor do they omit metadata. The YAML config is the core of a bundle.

C. A plain text file listing assets

No — Bundles require a structured YAML specification, not a flat list.

Question: 190 CertyIQ

A data engineer is maintaining an ETL pipeline code with a GitHub repository linked to their Databricks account.
The data engineer wants to deploy the ETL pipeline to production as a databricks workflow.
Which approach should the data engineer use?

A. Databricks Asset Bundles (DAB) + GitHub Integration

B. Maintain workflow_config.json and deploy it using Databricks CLI

C. Maintain workflow_config.json and deploy it using Terraform

D. Manually create and manage the workflow in UI

Answer: A

Explanation:

A.Databricks Asset Bundles

YAML-based configuration of workflows, clusters, permissions, libraries, parameters

Git-based development (GitHub, GitLab, Bitbucket)

One-command deployment (databricks bundle deploy)

Built-in support for Databricks Workflows (Jobs)

Clear separation of dev/staging/prod using environment configs

Well-supported CI/CD patterns

This matches exactly what the data engineer needs:

Deploy ETL pipeline to production from GitHub → Databricks Workflow

Why the other options are Incorrect

B. workflow_config.json + Databricks CLI

This older pattern is now deprecated by DAB

Not recommended for new deployments

Less flexible and harder to maintain vs. DAB

C. workflow_config.json + Terraform

Terraform can deploy workflows, but it:

Adds overhead

Requires managing Terraform state

Is not the primary recommended workflow deployment approach for code-based projects linked to Git

D. Manually creating workflows in the UI

Not automated

Not version-controlled

Not suitable for CI/CD or production pipelines

Question: 191 CertyIQ

A data engineer works for an organization that must meet a stringent Service Level Agreement (SLA) that demands minimal runtime errors and high availability for its data processing pipelines. The data engineer wants to avoid the operational overhead of managing and tuning clusters.

Which architectural solution will meet the requirements?

A. Use an auto-scaling cluster configured and monitored by the user.

B. Implement a hybrid approach with scheduled batch jobs on custom cloud VMs.

C. Deploy a dedicated, manually managed cluster optimized by in-house IT staff.

D. Utilize Databricks serverless compute that automatically optimizes resources and abstracts cluster management.

Answer: D

Explanation:

Utilize Databricks serverless compute that automatically optimizes resources and abstracts cluster management.

Question: 192 CertyIQ

A data engineer wants to reduce costs and optimize cloud spending. The data engineer has decided to use Databricks Serverless for lowering cloud costs while maintaining existing SLAs.
What is the first step in migrating to Databricks Serverless?

A. Legacy Ingestion pipelines that include ingestion from sources API's, files, JDBC/ODBC connections

B. A frequently running and efficient Python-based data transformation pipeline compatible with the latest Databricks runtime and Unity Catalog

C. A frequently running and efficient Scala-based data transformation pipeline compatible with the latest Databricks runtime and Unity Catalog

D. Low frequency BI Dashboarding and Adhoc SQL Analytics

Answer: B

Explanation:

B. A frequently running and efficient Python-based data transformation pipeline compatible with the latest Databricks runtime and Unity Catalog

When migrating workloads to Databricks Serverless, Databricks recommends starting with workloads that are:

Python-based (Serverless is optimized first for Python jobs and SQL)

Frequently running (benefits most from auto-scaling + cost savings)

Already efficient and compatible with the latest Databricks Runtime

Unity Catalog–enabled, since Serverless requires UC governance

This makes Option B the most aligned with Databricks' documented guidance for phase-1

Question: 193 CertyIQ

A data engineering project involves processing large batches of data on a daily schedule using ETL. The jobs are resource-intensive and vary in size, requiring a scalable, cost-efficient compute solution that can automatically scale based on the workload.
Which compute approach will satisfy the needs described?

A. Job Cluster
B. Dedicated Cluster
C. All-Purpose Cluster
D. Databricks SQL Serverless

Answer: A

Explanation:

Job Clusters.

Automatic creation and termination for each job run

Auto-scaling based on workload

Lower cost since the cluster only exists during the job

Isolated environment for each ETL pipeline

Optimized for production workloads scheduled in Databricks Jobs

Question: 194 CertyIQ

A data engineer is configuring Delta Sharing for a multi-team project where teams from different departments will need to access shared data. The data engineer has successfully created a Unity Catalog metastore and is now setting up the Delta Share. The goal is to ensure that internal teams can access the shared data with full permissions, while external partners can only read the data.
Which action should the Data Engineer take to configure the sharing correctly?

A. Create a Delta Share, set up a secure access URL for internal teams and external partners, and distribute the URL to provide them access to the shared data.
B. Create a Delta Share, add the internal team's tables and views, and assign READ/WRITE permissions to both external partners and internal teams.
C. Assign READ permissions to external partners through the Delta Share and READ/WRITE permissions to internal teams, and ensure the correct tables and views are shared.
D. Grant READ permissions to external partners and READ/WRITE permissions to internal teams through the Delta Share.

Answer: C

Explanation:

C. Assign READ permissions to external partners through the Delta Share and READ/WRITE permissions to internal teams, and ensure the correct tables and views are shared.

The scenario involves two types of consumers:

1. Internal teams (inside the same Databricks account)

Should have full READ/WRITE access

They do not need Delta Sharing to access internal data

→ They can be granted permissions directly via Unity Catalog

2. External partners (outside the Databricks account)

Should have READ-only access

Must receive data via Delta Sharing

Can only receive READ permissions when added as a recipient in a Delta Share

Question: 195 CertyIQ

A data engineer manages multiple external tables linked to various data sources. The data engineer wants to manage these external tables efficiently and ensure that only the necessary permissions are granted to users for accessing specific external tables.
How should the data engineer manage access to these external tables?

A. Set up Azure Blob Storage permissions at the container level, allowing access to all external tables.
B. Create a single user role with full access to all external tables and assign it to all users.
C. Grant permissions on the Databricks workspace level, which will automatically apply to all external tables.
D. Use Unity Catalog to manage access controls and permissions for each external table individually.

Answer: D

Explanation:

D. Use Unity Catalog to manage access controls and permissions for each external table individually.

To efficiently and securely manage access to external tables in Databricks, the correct approach is to use Unity Catalog, which provides centralized governance and fine-grained access control.

Unity Catalog allows you to:

Control table-level, schema-level, and catalog-level permissions.

Apply least-privilege access via GRANT statements.

Audit and monitor access.

Manage permissions across external tables, managed tables, volumes, and more.

Avoid exposing credentials or storage layers directly to users.

This aligns perfectly with the requirement:

Grant only the necessary permissions to each user for specific external tables.

Question: 196 CertyIQ

An organization plans to share a large dataset stored in a Databricks workspace on AWS with a partner

organization whose Databricks workspace is hosted on Azure. The data engineer wants to minimize data transfer costs while ensuring secure and efficient data sharing.
Which strategy will reduce data egress costs associated with cross-cloud data sharing?

A. Migrating the dataset to Cloudflare R2 object storage before sharing
B. Configure VPN connection between AWS and Azure for faster data sharing
C. Using Delta Sharing without any additional configurations
D. Sharing data via pre-signed URLs without monitoring egress costs

Answer: C

Explanation:

C. Using Delta Sharing without any additional configurations

The question asks which strategy will reduce data egress costs when sharing data across clouds.

Delta Sharing (especially when used with Databricks + Unity Catalog) is designed for:

Cross-cloud, cross-platform sharing

Without copying data

Without creating data replicas

Minimizing egress charges by streaming only the required data, often in optimized Parquet/Delta format

Delta Sharing ensures the lowest possible egress cost because:

Data is not duplicated

Only requested data is transmitted

It uses optimized, columnar formats for efficient transfer

This makes it the intended mechanism for cost-efficient, cross-cloud data sharing.

Why the other options are incorrect

A. Migrating the dataset to Cloudflare R2

Even though R2 offers zero egress fees to the internet,

you still pay egress fees when moving data out of AWS/Azure/GCP to R2.

Adds complexity and does not integrate natively with Delta Lake.

B. Configure VPN between AWS and Azure

VPNs do not reduce data egress charges.

Cloud providers still charge for outbound traffic.

Only affects security and routing — not cost.

D. Sharing data via pre-signed URLs

Pre-signed URLs cause full data downloads, leading to:

High egress fees

No fine-grained sharing or filtering

No governance/lineage

Worst choice for cost control.

Question: 197 CertyIQ

A data engineer is setting up access control in Unity Catalog and needs to ensure that a group of data analysts can query tables but not modify data.
Which permission should the data engineer grant to the data analysts?

A. ALL PRIVILEGES
B. MODIFY
C. SELECT
D. INSERT

Answer: C

Explanation:

SELECT → Allows users to read/query data.

This is exactly what data analysts need.

Question: 198 CertyIQ

What Databricks feature can be used to check the data sources and tables used in a workspace?

A. Use the lineage feature to visualize a graph that highlights where the table is used only in reports.
B. Use the lineage feature to visualize a graph that highlights where the table is used only in notebooks.
C. Use the lineage feature to visualize a graph that shows all dependencies, including where the table is used in notebooks, other tables, and reports.
D. Do not use the lineage feature as it only tracks activity from the last 3 months and will not provide full details on dependencies.

Answer: C

Explanation:

C. Use the lineage feature to visualize a graph that shows all dependencies, including where the table is used in notebooks, other tables, and reports.

The lineage feature, particularly as implemented through Unity Catalog, provides a comprehensive view of data flow.

Visualization: It displays a graph showing the upstream (data sources, raw tables) and downstream (tables, notebooks, dashboards/reports) usage of a specific table or asset.

Comprehensive Dependencies: It tracks and visualizes all dependencies, meaning it doesn't limit itself to just

notebooks or just reports. It shows:

Upstream: Which tables or sources fed the current table.

Downstream: Which notebooks, other tables (for views or materialized views), and reports/dashboards (often integrated through partner tools) consume the data from the table.

Why the other options are incorrect:

A & B (Limited Scope): These options incorrectly limit the scope of lineage to only reports or only notebooks. Databricks lineage is designed to be comprehensive and include all major dependencies.

D (Incorrect Limitation): This is factually incorrect. Unity Catalog lineage captures data for all supported languages and processes (SQL, Python, Scala, R) and is retained as long as the data is available in Unity Catalog, not just for a limited time like 3 months.

Question: 199 CertyIQ

A data engineer is getting a partner organization up to speed with Databricks account. Both the teams share some business use cases. The data engineer has to share some of your Unity-Catalog managed delta tables and the notebook jobs creating those tables with the partner organization.
How can the data engineer seamlessly share the required information?

A. Zip all the code and share via email and allow data ingestion from your data lake
B. Share required datasets and notebooks via Delta Sharing. Manage permissions via Unity Catalog.
C. Data and Notebooks can be shared simply using Unity Catalog.
D. Share access to codebase via Github and allow them to ingest datasets from your Datalake.

Answer: B

Explanation:

B. Share required datasets and notebooks via Delta Sharing. Manage permissions via Unity Catalog.

Why the other options are wrong

A. Zip code and share via email

Not secure

Not governed

Does not allow secure sharing of Delta tables

Breaks versioning and lineage

C. Data & notebooks can be shared using only Unity Catalog

UC alone cannot share data across organizations without Delta Sharing.

UC is intra-account governance; Delta Sharing is required for external partners.

D. Share code via GitHub + let them ingest from your datalake

Requires exposing your data lake storage endpoints

Forces the partner to copy your data (no real-time sharing)

No Unity Catalog audit / lineage for external access

Not seamless and not governance-aligned

Question: 200 CertyIQ

A data engineering team wants to validate a new ingestion pipeline locally while ensuring large aggregations run in serverless compute in their Databricks workspace. They plan to use Databricks Connect and have the option to attach to either a shared cluster or serverless.
Which workspace requirement should be confirmed first to avoid connection failures?

A. Verify that the workspace has Unity Catalog disabled and the Databricks Connect version is less than the serverless Runtime version.

B. Verify that the workspace has Unity Catalog enabled and that the Databricks Connect version supports serverless for the target Runtime release.

C. Verify that only assigned access mode clusters are used because serverless is not supported by Databricks Connect.

D. Verify that the local Spark version equals the serverless Spark version to satisfy Spark Connect parity.

Answer: B

Explanation:

Justification

Correct option – B

The workspace must have **Unity Catalog enabled** because serverless compute for Databricks Connect requires the metadata and governance features that Unity Catalog provides.
Additionally, the **Databricks Connect client library** must be built for a Runtime version that includes the serverless runtime; otherwise the client cannot locate the serverless endpoint and the connection will fail.
Verifying both conditions ensures that the client can resolve the serverless URL, negotiate the correct protocol version, and authenticate using Unity Catalog credentials, which are mandatory for serverless attachment.

Why the other options are unsuitable

A – Disabling Unity Catalog prevents use of serverless compute, and coupling an older Connect version with a newer serverless Runtime creates incompatibility; this does not address the required configuration.
– **C** – Serverless compute **is** supported by Databricks Connect; restricting to assigned-access clusters is unnecessary and would incorrectly exclude the serverless option.
– **D** – Spark Connect operates on a logical Spark engine abstraction, not on the underlying Spark binary version; matching local Spark with serverless Spark is not a prerequisite and any mismatch can be handled by the Connect driver.

References

Databricks Connect documentation – Requirements for serverless runtime
Unity Catalog overview – Prerequisites and enablement

Question: 201 CertyIQ

A data engineer needs to conduct Exploratory Analysis on data residing in a database that is within the company's custom-defined network in the cloud. The data engineer is using SQL for this task.
Which type of SQL Warehouse will enable the data engineer to process large numbers of queries quickly and cost-effectively?

A. Classic SQL Warehouse
B. Serverless SQL Warehouse
C. Pro SQL Warehouse
D. All-purpose compute cluster

Answer: B

Explanation:

Technical Justification

Correct Choice – Serverless SQL Warehouse
Provides fully managed, auto-scaling compute that starts in seconds and shuts down when idle, eliminating the need to provision or maintain persistent clusters.
Charges only for the compute seconds consumed, making it the most cost-effective option for irregular, exploratory query patterns that may involve many short-lived workloads.
Built on Databricks' Photon engine, it delivers high-performance query execution with minimal latency, enabling the data engineer to run numerous ad-hoc SQL queries quickly within a VNet-isolated environment.
Why Other Options Are Less Suitable
Classic SQL Warehouse – Requires manual cluster sizing, incurs ongoing costs even when idle, and has longer cold-start times; therefore it is sub-optimal for bursty, low-volume exploratory work.
Pro SQL Warehouse – This is not a documented Databricks offering; it does not exist as a distinct warehouse type in the platform.
All-purpose compute cluster – Although flexible, it shares resources with other workloads, is billed per hour regardless of usage, and lacks the automatic scaling and pay-per-use model needed for cost-efficient exploratory SQL processing.

References

Serverless SQL Warehouses documentation: https://learn.databricks.com/docs/data-engineering/sql/warehouses/serverless
Pricing and auto-scaling overview: https://learn.databricks.com/docs/data-engineering/sql/warehouses/pricing

Question: 202 CertyIQ

A data engineer must deliver a trustworthy customer 360 dataset in Databricks for data scientists and BI teams. The engineer plans to join deduplicated customer records with cleaned transaction data, enforce schema and data quality checks, and create a conformed "customer_transactions" view. Later, highly aggregated, domain-specific tables (for weekly spend and executive summaries) will be produced for dashboards.
Where should the engineer build the conformed "customer_transactions" dataset, and where should the aggregated, report-ready tables reside?

A. Build both "customer_transactions" and aggregated, report-ready tables in Silver to keep the model simpler.
B. Build "customer_transactions" in Bronze and put the aggregated, report-ready tables in Silver.

C. Build “customer_transactions” in Gold and put the aggregated, report-ready tables in Silver.
D. Build “customer_transactions” in Silver and put the aggregated, report-ready tables in Gold.

Answer: D

Explanation:

Why option D is the best choice

“customer_transactions” is a conformed, business-ready view that joins deduplicated master data with cleaned transaction streams, applies schema enforcement and data-quality checks. This fits the **Silver** layer, whose purpose is to produce curated, integrated tables ready for consumption by downstream users while still preserving the raw-material lineage in Bronze.
Aggregated, report-ready tables (weekly spend, executive summaries) are final, curated datasets that are meant to be consumed directly by dashboards and BI tools. In a medallion architecture these belong to the **Gold** layer, where data is fully vetted, optimized (e.g., partitioning, Z-ordering), and presented in a stable, business-friendly format.

Why the other options fail

A – Placing both the view and the aggregated tables in Silver would mix raw-curated and final-curated content in the same layer, breaking the clear separation of Bronze (raw), Silver (clean, integrated), and Gold (business-ready). It also prevents proper lineage tracking and hampers downstream optimization.
B – Building the view in Bronze contradicts the Bronze layer’s intent, which should only contain raw data without any joins, schema enforcement, or quality checks. The view would lack the integrity guarantees needed for reliable downstream consumption.
C – Moving the view into Gold unnecessarily elevates a pre-aggregation step to the final-curated layer. The view is an intermediate, integrated artifact that belongs to Silver; promoting it to Gold adds complexity and may expose raw-level data to consumers prematurely.

Thus, the correct pattern is **Silver for the integrated “customer_transactions” view** and **Gold for the final, aggregated, report-ready tables**.

References

Databricks Lakehouse Architecture:https://docs.databricks.com/en/data-engineering/lakehouse/architecture.html
Medallion Pattern Best Practices:https://docs.databricks.com/en/data-engineering/medallion-architecture.html

Question: 203 CertyIQ

A data engineer requires rapid iteration on pipelines while maintaining reliable rollbacks after bad ingests, ensuring audit trails for regulatory compliance, and providing consistent access to a single source of truth for both AI and BI workloads.
Which strategy should the data engineer apply to meet the needs?

A. Delta Lake ACID transactions and time travel, governed by Unity Catalog for consistent access and lineage.
B. DBFS CSV storage with manual file versioning and nightly copies for rollback.
C. Ephemeral in-memory DataFrames for audit trails and BI distribution.
D. Cloud objects storage only, with ad hoc SQL queries for recovery and governance.

Answer: A

Explanation:

Justification

Correct option – A

Delta Lake ACID transactions guarantee reliable write-read consistency, allowing instant rollback to a previous version after a faulty ingest.
Time-travel functionality provides built-in immutable audit trails by preserving every data change, which satisfies regulatory reporting requirements.
Unity Catalog centralizes access control, data lineage, and metadata, ensuring a single source of truth that can be used simultaneously by AI (e.g., ML model training) and BI (e.g., dashboard queries) workloads without duplication or drift.
Together, these capabilities enable rapid pipeline iteration while preserving data integrity, traceability, and compliance.

Why the other options are unsuitable

B – DBFS CSV with manual file versioning: manual handling of file versions offers no automated rollback or ACID guarantees; audit trails are fragile and cannot guarantee consistent access across AI/BI users.
C – Ephemeral in-memory DataFrames: transient storage loses persistence after cluster termination, making reliable rollback, audit trails, and a stable single source of truth impossible.
D – Cloud object storage only: without a transactional layer, there is no built-in versioning, time-travel, or governance; recovery relies on ad-hoc processes and cannot assure consistent access for both analytical domains.

References

Delta Lake ACID Transactions & Time Travel
Unity Catalog – Unified Governance

Question: 204 CertyIQ

A data engineer is setting up a new Databricks pipeline that ingests clickstream events from Kafka and daily product catalogs from cloud object storage. To ensure auditability and easy reprocessing, the engineer wants to land all source data. Later stages will handle cleaning, deduplication, and business modeling before the data is used in dashboards.
Which approach aligns to Medallion Architecture principles?

A. Land both sources in Gold with denormalized star schemas to optimize BI while retaining full source fidelity.

B. Land streaming events from Kafka in silver, and the product catalog directly in Gold to minimize layers for batch data.

C. Land both sources in the Bronze layer append-only with minimal validation, then build Silver/Gold downstream for quality and analytics.

D. Land both sources directly into the Silver layer with schema enforcement and deduplication to reduce downstream complexity.

Answer: C

Explanation:

Why optionC aligns with Medallion Architecture

Bronze=raw, immutable, append-only landing zone – Both the Kafka click-stream and the catalog files are ingested without any transformation, validation, or enrichment. This guarantees exact replication of the source and provides an audit-ready snapshot for replay.
Explicit data provenance – Storing the raw files in Bronze with a clear naming convention (e.g., year=2025/month=06/day=03/kafka_clickstream/) makes the origin, ingestion time, and schema version traceable, satisfying auditability requirements.
Downstream refinement in Silver/Gold – Subsequent pipelines consume the Bronze layer to build the Silver layer (cleaned, deduplicated, with light schema enforcement) and then the Gold layer (curated, business-ready tables). This separation enforces progressive quality control while keeping the original raw data untouched.

Why the other options are less suitable

A – Placing denormalized star schemas directly in Gold bypasses the Bronze raw layer, eliminating a true "as-received" snapshot and making downstream re-processing or audit of the original source data difficult.
B – Treating streaming events as Silver while loading static catalogs straight to Gold creates an inconsistent landing pattern; streaming data should first pass through Bronze (or a dedicated raw landing zone) before being enriched to a higher layer.
D – Storing both sources in Silver with schema enforcement and deduplication assumes that raw data can be safely transformed at ingestion, contradicting the principle that Bronze must remain immutable and free of any cleaning or de-duplication logic.

References

Databricks Medallion Architecture Overview – Official documentation:
https://learn.databricks.com/architecture/medallion
Delta Lake Best Practices for Data Lakes – Official guide on layering and raw data handling:
https://docs.databricks.com/delta/delta-batch.html#raw-data-zone-bronze

Question: 205 CertyIQ

A data engineer is building a PySpark ingestion pipeline in a local IDE and must execute heavy DataFrame transformations on a remote Databricks cluster for scale while stepping through business logic locally. The workspace uses Unity Catalog and a cluster running Databricks Runtime 15.4 LTS. The engineer also plans to register UDFs.
What should the data engineer do to avoid runtime issues during development with Databricks Connect?

A. Match the local Python minor version to the cluster's Python minor version when using UDF's, and use a Databricks Connect package compatible with the cluster's Runtime version

B. Use any local Python version as long as the PySpark minor version matches, because the cluster's Python version is isolated from the client.

C. Use serverless, because Databricks Connect does not support assigned or shared clusters.

D. Disable Unity Catalog on the workspace because it conflicts with remote execution through Databricks Connect.

Answer: A

Explanation:

Why option A is the correct choice

Python version alignment – When registering PySpark UDFs, Databricks serializes the UDF code and its dependencies on the cluster side. If the client-side Python interpreter uses a different minor version than the

Runtime15.4 LTS cluster, the UDF byte-code and deserialization may fail, causing runtime errors such as PicklingError or version-mismatched library imports. Matching the minor version guarantees that pickles can be transferred safely between client and cluster.
Databricks Connect package version – The Connect client must use a package whose transitive dependencies (e.g., pyarrow, protobuf) are compatible with the Runtime version. Using a package built for a later or earlier Runtime can cause missing-field errors or incompatible serialization of Spark plans. The compatibility matrix for Connect explicitly requires the client package to be built for the same Major.Minor of the cluster's Runtime (e.g., 15.4.x).

Why the other options are unsuitable

OptionB – While PySpark's API layers are version-stable across minor Python releases, UDF execution relies on the exact Python interpreter used on the cluster for object serialization and for loading native libraries. Using any arbitrary local Python version can break the UDF registration process and cause execution failures in distributed stages.
OptionC – Serverless compute is a separate execution model that does not support interactive debugging or attachment to a specific cluster. Databricks Connect is expressly designed to attach to existing **assigned** or **shared** clusters for step-through debugging; serverless is not a requirement and would limit control over cluster resources.
OptionD – Unity Catalog is fully compatible with remote execution via Databricks Connect. Disabling it is unnecessary and would remove central data governance; the issue is not a conflict but rather ensuring that the client-side Python environment aligns with the cluster's Runtime.

Steps to avoid runtime issues

1. **Install the Databricks Connect package** that matches Runtime15.4 (e.g., databricks-connect==15.4.0-sc1544-slim).
2. **Configure the local Python environment** to use the same minor version as the cluster (e.g., Python3.10.x if the cluster runs on Python3.10).
3. **Register UDFs** only after confirming that all required libraries are installed in the local environment and that their versions are compatible with the cluster's library scope.
4. **Launch the Connect session** pointing at the remote cluster and attach the IDE debugger to step through the DataFrame transformations.

References

Python version compatibility for Unity Catalog – https://docs.databricks.com/en/data-governance/unity-catalog/python-version-compatibility.html
Databricks Connect package compatibility matrix – https://docs.databricks.com/en/dev-tools/databricks-connect/compatibility.html

Question: 206 CertyIQ

A Databricks single-task workflow fails due to an error in a notebook. The data engineer fixes the mistake in the notebook.
What should the data engineer do to rerun the workflow?

A. Repair the task
B. Repair the run
C. Restart the Cluster
D. Swap the cluster

Answer: B

Explanation:

Justification

Repair the run (B) – Correct
In Databricks a run represents a single execution of a task within a workflow. When a task fails, the workflow remains in a failed state but the underlying task definition is still intact. The recommended way to recover is to open the failed run, edit the notebook (or task configuration) to fix the error, and then **re-run that specific run**. This re-executes the corrected logic without affecting other runs or the workflow definition, which aligns with the platform's design for iterative debugging and re-execution.
Repair the task (A) – Incorrect
"Repair the task" is not a supported action in the Databricks UI. Tasks are immutable once created; you must modify the underlying notebook or job definition and then create a new run. The term "repair the task" does not map to any official API or UI operation.
Restart the Cluster (C) – Incorrect
Restarting a cluster only affects compute resources; it does not reset the state of a failed workflow run nor apply code changes. The failure is typically due to logical errors in the notebook, not transient cluster issues, so restarting the cluster does not resolve the underlying problem.
Swap the cluster (D) – Incorrect
"Swapping the cluster" refers to assigning a different cluster to a running job, which is unrelated to fixing a failed run caused by code defects. It does not address the notebook error and would not cause the workflow to succeed.

Conclusion

The proper procedure is to **repair the run** by editing the notebook, correcting the error, and re-executing the task within the same workflow.

References

Databricks Jobs – Rerun a Failed Task
Jobs API – Update Run

Question: 207 CertyIQ

A data engineer is migrating pipeline tasks to reduce operational toil. The workspace used Unity Catalog and is in a region that supports serverless. The engineer wants Databricks to auto-select instance types, manage scaling, apply Photon, and handle runtime upgrades automatically for job runs.
How should the data engineer meet this requirement while adhering to Databricks constraints?

A. Run the job on a serverless compute for workflows configuration, ensuring Unity Catalog is enabled and regional support is available.
B. Use a Pro SQL warehouse and schedule Python notebook tasks to execute as pipeline steps.
C. Create a job with a single-task job cluster and manually set instance families and min/max workers.
D. Use an all-purpose cluster with cluster policies to enforce standard sizes and enable autoscaling.

Answer: A

Explanation:

Why optionA is correct

Serverless compute for workflows is the only Databricks feature that automatically selects instance types, scales resources, runs on Photon, and upgrades runtimes without manual intervention.
When the workspace is configured with **Unity Catalog** and the region supports serverless workloads, a serverless job can be launched directly from a workflow, satisfying the requirement for auto-select, auto-scale, Photon, and automatic runtime upgrades.
The other options either rely on user-managed clusters or UI-only settings that do not meet the fully automated, serverless criteria.

Why the other options are unsuitable

B – Pro SQL warehouse + scheduled notebook tasks – uses a provisioned warehouse, which requires manual cluster management and does not provide serverless auto-scaling or automatic runtime upgrades.
C – Single-task job cluster with manual instance families – requires explicit sizing and configuration of min/max workers, contradicting the need for automatic instance selection and scaling.
D – All-purpose cluster with cluster policies – cluster policies enforce a predefined size; they do not provide the on-demand, serverless elasticity or automatic runtime upgrades that the scenario demands.

References

Serverless compute for Databricks workflows –<https://learn.databricks.com/workflows/serverless Compute>
Unity Catalog – Workspace configuration –https://learn.databricks.com/unity-catalog/intro

Question: 208 CertyIQ

A data engineer is standardizing repository layouts for multiple teams adopting Databricks Asset Bundles. The engineer wants to ensure every project has a single authoritative configuration file at the repository root that defines the bundle name, targets, workspace settings, permissions, and resource mappings (for jobs and pipelines).
What strategy should the data engineer use to meet the goal?

A. Place multiple databricks.yml files under each subfolder (for example, jobs/pipelines/, workspace/) and merge them at deploy time using the include mapping.

B. Place exactly one databricks.yml at the repository root; it is the main configuration file and may reference additional configuration files via the include mapping.

C. Place a databricks.yml in a databricks/hidden folder at the repository root; only hidden locations are valid for bundle configs.

D. Place a databricks.yaml at the repository root and optional databricks.yml in subfolders; the CLI prefers .yaml over .yml when both exist.

Answer: B

Explanation:

Correct approach – OptionB

Place a **single databricks.yml file at the repository root**.
This file is the **authoritative source** for the bundle name, targets, workspace settings, permissions, and resource mappings (jobs, pipelines).
The Databricks CLI/AzureDevOps task reads the root configuration first, and any additional settings can be **referenced via the include mapping**; this keeps the configuration tidy, reproducible, and version-controlled in one place.

Enforces a **consistent layout** across all teams, simplifies CI/CD pipelines, and guarantees that dbx/dbx-pipeline commands always locate the definition in a predictable location.

Why the other options are not suitable

OptionA – Multiple databricks.yml files under subfolders require merge logic at deploy time; this introduces implicit ordering dependencies and can cause conflicts, breaking the "single source of truth" guarantee required by Asset Bundles.
OptionC – Asset Bundles **do not support** configuration files in hidden folders; the only supported location is the repository root (or nested contexts referenced via include). Using a hidden directory would lead to runtime errors.
OptionD – While .yaml and .yml are both valid extensions, the strategy of placing optional configs in subfolders does not align with the Asset Bundles model, which expects the **primary configuration at the root**; relying on file-extension preference adds unnecessary complexity and can cause ambiguous precedence.

Key takeaway – A single root databricks.yml provides a deterministic, maintainable, and officially supported configuration location, ensuring all teams share an identical and auditable bundling definition.

References

Databricks Asset Bundles configuration documentation: https://docs.databricks.com/en/data-engineering/data-engineering-databricks-asset-bundles/index.html
Databricks CLI dbx command reference for asset bundles: https://docs.databricks.com/en/dev-tools/databricks-cli/dbx.html#asset-bundles-configuration

Question: 209 CertyIQ

Which Databricks feature can be used to check the data sources and tables used in a workspace?

A. Use the lineage feature to visualize a graph that shows all dependencies, including where the table is used in notebooks, other tables, and reports.
B. Use the lineage feature to visualize a graph that highlights where the table is used only in reports.
C. Use the audit logs feature to review user activities, such as who queried or modified a table, instead of visualizing dependencies between data objects.
D. Use the lineage feature to visualize a graph that highlights where the table is used only in notebooks.

Answer: A

Explanation:

Why optionA is correct

The Databricks **lineage** capability builds a dependency graph that maps the flow of data from source tables through notebooks, transformation jobs, downstream tables, and visualizations.
This graph can be explored interactively to see exactly **where a given table is referenced** (e.g., in other notebooks, downstream tables, dashboards, or reports), giving a holistic view of its usage across the workspace.
Because lineage captures all downstream objects, it directly answers the question "which data sources and tables are used in a workspace?".

Why the other options are not suitable

OptionB incorrectly limits the view to only report dependencies, ignoring notebooks and other table uses.

OptionD similarly restricts the view to only notebook usage, missing tables, downstream jobs, and reporting artifacts.
OptionC suggests using **audit logs**, which record who accessed or modified a table but do **not** provide a visual dependency map; auditing is useful for compliance, not for discovering structural dependencies throughout the workspace.

References

DatabricksLineage Overview: https://learn.databricks.com/visualizations/lineage
Understanding Table Dependencies with Lineage: https://docs.databricks.com/data-governance/lineage.html

Question: 210 CertyIQ

A data engineer wants to delegate day-to-day permission management for the schema main marketing to the mkt-admins group, without making them workspace admins. They should be able to grant and revoke privileges for other users on objects within that schema.
Which approach aligns with UC's ownership and privilege model?

A. Grant MANAGE permissions on the metastore to mkt-admins, which allows managing privileges for all schemas and tables globally.

B. Make mkt-admins a workspace-level admins group, then SELECT on main.marketing to allow privilege delegation.

C. Transfer ownership of the schema main. marketing to mkt-admins; owners can manage privileges on the schema and its contained objects.

D. Grant USE SCHEMA on main.marketing and MODIFY on all tables to mkt-admins, which enables managing grants within the schema.

Answer: C

Explanation:

Why optionC is the correct approach

In Unity Catalog, schema ownership gives the owner full control over all objects in that schema, including the ability to grant and revoke privileges on those objects.
By transferring ownership of **schema main.marketing** to the mkt-admins group, those users can directly manage permissions on the schema and any tables/files it contains, without requiring them to be workspace-level admins.
Ownership is scoped to the schema, so privilege delegation remains limited to that logical boundary, preserving least-privilege principles across the rest of the workspace.

Why the other options are unsuitable

OptionA – Granting MANAGE permissions on the metastore is a global privilege; it lets a user manage all schemas and tables across the entire catalog, which is far broader than the narrow delegation needed for just the marketing schema.
OptionB – Elevating mkt-admins to a workspace-level admin group would give them administrative control over the entire workspace, violating the requirement to keep them as non-admins. Additionally, a simple SELECT on the schema does not confer ownership-level rights.
OptionD – USE SCHEMA + MODIFY on tables provides read-access and the ability to write data, but **does not** enable the grantee to grant or revoke privileges on those objects. Only the owner (or a role with grant-admin rights) can alter ACLs on behalf of others.

Key Unity Catalog concepts used

Owner-level privilege control: The schema owner can delegate GRANT/REVOKE on contained objects.
Scope-bounded permissions: Delegation stays within the owned schema, avoiding cross-schema admin rights.

References

Unity Catalog Privileges Overview: https://learn.microsoft.com/azure/databricks/data/security/unity-catalog/privileges-overview
Managing Objects and Access Control in Unity Catalog: https://learn.microsoft.com/azure/databricks/data/unity-catalog/objects-and-access-control

Question: 211 CertyIQ

A data engineer is decommissioning a sandbox schema in Unity Catalog. Some tables are ephemeral staging outputs that can be safety removed entirely, but a few tables point at shared cloud storage used by downstream jobs outside Databricks. The engineer must avoid deleting any shared files when cleaning up catalog objects. How does Unity Catalog behave when dropping Managed vs External tables?

A. DROP all tables; Databricks will only remove metadata for both managed and external tables.
B. DROP managed tables that are ephemeral and DROP external tables; files for both remain for 7 days.
C. DROP managed staging tables to remove data and metadata, and DROP external tables to remove only metadata
D. DROP external tables first to delete their files, then DROP managed tables to keep their files for recovery.

Answer: C

Explanation:

Justification

Managed tables in Unity Catalog store both metadata and the underlying data. Dropping a managed table (e.g., an ephemeral staging table) removes its rows, its storage location, and all catalog entries, so the associated files are permanently deleted from DBFS/ADLS. This is the only way to guarantee that no downstream job can still reference those files.
External tables only hold metadata that points to external storage. When an external table is dropped, Unity Catalog deletes the catalog entry but leaves the underlying files untouched, regardless of whether they are shared acrossmultiple notebooks or jobs. Thus the files remain safe for other downstream processes.
OptionC correctly captures this behavior: it advises dropping managed staging tables to purge both data and metadata, while dropping external tables only removes metadata, preserving the shared files.
Why the other choices are unsuitable

OptionA – Incorrect; dropping a managed table deletes its data, not just metadata.
OptionB – Misrepresents the retention policy; there is no automatic 7-day file hold on drop.
OptionD – Reverses the actual effect; dropping an external table does **not** delete files, and there is no "recovery-by-keeping-files" guarantee for managed tables.

References

Unity Catalog Table Lifecycle: https://docs.databricks.com/en/data-governance/unity-catalog/tables.html#managed-and-external-tables
Dropping Tables in Unity Catalog: https://docs.databricks.com/en/data-governance/unity-catalog/drop-

tables.html

Question: 212 CertyIQ

A data engineer is onboarding a new bronze ingestion pipeline in Databricks with Unity Catalog. The team wants Databricks to handle storage layout, apply platform optimizations over time, and simplify lifecycle management so that when a table is dropped, its underlying data is also cleaned up according to Databricks-managed retention policies.
Which table type should the data engineer create for these ingestion tables?

A. Managed tables so that Unity Catalog manages both metadata and underlying data lifecycle.
B. External tables with a LOCATION pointing to an external volume for full control of file layout.
C. Foreign tables federated from an external catalog to delegate optimization to the source system.
D. Temporary views over files to avoid table-level governance and lifecycle coupling.

Answer: A

Explanation:

Why a Managed table is the optimal choice

Unity Catalog **managed tables** store both the metadata and the data inside the Databricks workspace, allowing the platform to automatically apply storage layout optimizations (e.g., Z-ordering, data skipping) as new data is added.
When a managed table is dropped, Unity Catalog automatically enforces the configured **retention policy**, deleting the underlying files and cleaning up related artifacts without manual intervention.
This model simplifies lifecycle management because governance, security, and data-retention policies are baked into the table definition, reducing operational overhead for the engineering team.
External tables only point to data stored outside the workspace; Databricks does not control the underlying storage layout or cleanup, which contradicts the requirement for Databricks-managed retention.
Foreign tables delegate query optimization to an external source and provide no intrinsic control over data lifecycle within Unity Catalog.
Temporary views are purely logical constructs over files; they ignore governance and do not participate in catalog-level retention policies.

Why the other options are less suitable

External tables give full control over file placement but expose the data to external storage systems, preventing Databricks from automatically handling layout optimizations or enforcing retention policies.
Foreign tables are intended for cross-catalog querying and do not manage storage or lifecycle for the underlying data.
Temporary views lack a physical storage contract and are automatically discarded; they cannot enforce retention or be cleaned up on table drop, making them incompatible with the governed ingestion lifecycle.

References

References

Unity Catalog managed tables: https://docs.databricks.com/en/data-governance/unity-catalog/managed-tables.html
Lifecycle management and retention policies: https://docs.databricks.com/en/data-governance/unity-catalog/lifecycle-management.html

Question: 213 CertyIQ

A data engineer working in Databricks needs to control BI query latency and cost for a sales analytics domain. Silver tables already contain validated, normalized sales and product data. The team wants dashboard-friendly query performance and to align datasets with business concepts (e.g., weekly_sales and account_performance) without overloading analysts with complex joins.
Which design approach should a data engineer choose for the Gold layer in a medallion architecture?

A. Move raw, append-only source data directly into Gold to avoid transformation costs and enable time travel.

B. Create denormalized, domain-aligned tables and materialized aggregates (e.g., weekly_sales) optimized for reporting and dashboards.

C. Build only lightly cleansed views in Gold and preserve most business logic in Silver to reduce duplication.

D. Keep all data heavily normalized in Gold to minimize storage and rely on BI tools to join data at query time.

Answer: B

Explanation:

Why optionB is the correct design for the Gold layer

Domain-centric denormalization – Gold tables are modeled to reflect business concepts such as **weekly_sales** and **account_performance**. By pre-joining the required dimensions (date, product, account) and storing the result in a flat schema, dashboards can retrieve results with a single read instead of multiple costly joins.
Materialized aggregates – Pre-computing aggregates (e.g., sums of sales per week) eliminates repeated scans of the underlying Silver tables, dramatically reducing query latency and compute cost for BI workloads.
Optimized for read-heavy workloads – Gold is accessed mainly by BI tools; therefore the schema is tuned for sequential reads, using columnar storage (Delta) and clustering on high-cardinality columns (e.g., date, account_id). This aligns with the “dashboard-friendly” performance requirement.
Clear separation of concerns – By moving the finished, business-ready tables out of Silver, downstream analysts no longer need to understand the normalization choices made at the Silver stage; they consume a ready-to-query view, which meets the goal of “avoiding overloaded analysts with complex joins.”
Delta Lake capabilities – Gold can leverage Delta features such as **Z-ordering**, **OPTIMIZE**, and **materialized views** to further improve query speed and lower cost without sacrificing data freshness.

Why the other options are less suitable

OptionA – Directly loading raw, append-only source data into Gold

Leaves data in its original raw form, forcing every downstream query to perform expensive joins and transformations. This defeats the purpose of a Gold layer’s purpose-built, performant tables and does not address latency or cost concerns.

OptionC – Lightly cleansed views that retain most logic in Silver

Keeps the majority of business logic in the Silver layer, causing duplicate processing across downstream jobs. It also re-introduces the same join complexity that the Gold layer is meant to eliminate, undermining dashboard performance.

OptionD – Heavily normalized tables in Gold

Maintaining a highly normalized structure in Gold means BI queries must execute many joins at runtime, leading to higher latency and increased compute cost. Normalization is appropriate for transactional systems (Bronze/Silver) but is counter-productive for a presentation layer designed for fast, cost-effective reporting.

Result
The Gold layer should therefore be a **denormalized, domain-aligned, aggregate-rich layer** that materializes the metrics needed by BI dashboards, directly satisfying the requirements for low latency and reduced cost.

References

[Delta Lake Best Practices for Query Performance](https://learn.databricks.com/docs/data-engineering/delta-lake/best-practices/query-performance)
Medallion Architecture Overview on Azure Databricks

Question: 214 CertyIQ

A team is designing their Data bricks Asset Bundle for deployment. They want to ensure their bundle includes all necessary resources, targeted environment configurations, and deployment settings in a single version-controlled file.
What is required as the main configuration file at the root of their project?

A. A Python script called configure_bundle.py that dynamically creates resources at runtime
B. A databricks.yml YAML file that contains the bundle's name, targets, includes resources, and variables
C. A directory with multiple TXT files, each describing a single resource for the bundle
D. A JSON file named bundle_config.json that stores all resource details and environment variables

Answer: B

Explanation:

Why optionB is the correct answer

The DatabricksAssetBundle relies on a **single declarative configuration file** placed at the repository root to define the bundle's metadata, targets, included assets, and variable overrides.
This file must be a **YAML document named databricks.yml**; it is the only format officially supported by the Databricks CLI and UI for specifying bundle.name, targets, resources, and environment-specific variables in a version-controlled manner.
The YAML structure enables **single-source-of-truth** configuration that can be checked into Git, reviewed via pull-requests, and applied consistently across environments (dev, test, prod) without runtime code generation.

Why the other options are unsuitable

A.Python script (configure_bundle.py) – Runtime scripts introduce non-declarative, ad-hoc logic that cannot be version-controlled as a static configuration; they violate the "single version-controlled file" requirement.
C.Multiple TXT files per resource – Scattering resource definitions across many plain-text files makes it impossible to maintain a cohesive, atomic bundle definition and defeats the purpose of a single root-level configuration file.
D.JSON file (bundle_config.json) – Databricks does not recognize a JSON file as the canonical bundle configuration; using an unsupported format would require custom parsing and would not be processed by the Databricks CLI or UI.

References

Databricks Bundle Configuration (databricks.yml): https://docs.databricks.com/dev-tools/bundle/configuration.html
Best practices for Databricks Asset Bundles: https://docs.databricks.com/dev-tools/best-practices/best-

practices-bundle.html

Question: 215 CertyIQ

A developer has configured Databricks Connect and starts running PySpark code locally. They notice that Data Frame operations are executed on the remote cluster, but actions like .show() return results directly in their local console.
What does this behaviour illustrate about how Databricks Connect is used?

A. It requires manual synchronization of results from the cluster
B. It runs all Spark compute locally and only reads remote files
C. It proxies Spark commands from the local client to the remote cluster
D. It mirrors the entire cluster environment on the developer's machine

Answer: C

Explanation:

SECTION 1The behaviour demonstrates that Databricks Connect functions as a thin client that forwards Spark commands from the local IDE to the remote cluster and streams the results back. •The driver runs locally; when an action such as .show() is invoked, the command is sent to the remote Spark driver, executed on the cluster, and the resulting data is proxied back to the local console. •This explains why DataFrame transformations execute on the cluster while the display of results appears locally. •OptionA is incorrect because no manual synchronization step is required — the proxy handles data transfer automatically. •OptionB is inaccurate; Spark compute does **not** run locally — it runs on the remote cluster. •OptionD mischaracterizes the setup; the local machine does not host a mirrored Spark environment.

SECTION 2References

https://docs.databricks.com/en/dev-tools/databricks-connect/index.html
https://docs.databricks.com/en/data-engineering/connects/databricks-connect.html

Question: 216 CertyIQ

A data engineer needs to configure an external location and a storage credential so that a new bronze ingestion pipeline can land data in cloud storage governed by Unity Catalog. The engineer has CREATE TABLE permission on schemas but receives a permission error when attempting to create the external location.
Which role is required to unblock this setup, the principle of least privilege?

A. Grant CREATE EXTERNAL LOCATION on both the metastore and the referenced storage credential so the engineer can define the external location.
B. Grant ALL PRIVILEGES on the catalog so the engineer can create external locations under its schemas.
C. Promote the engineer to Workspace Admin so they can manage objects and jobs.
D. Assign the engineer as Metastore Admin because only Metastore Admins can create external locations.

Answer: A

Explanation:

Why optionA is the only correct, least-privilege solution

To register an external location, a user must hold the **CREATE EXTERNAL LOCATION** privilege on the metastore and on the underlying storage credential that backs the location.
Granting only this privilege satisfies the engineer's need to create the location while adhering to the principle of least privilege.
The permission can be granted directly to the engineer's role (or a group) without giving broader rights that would expose the metastore or storage to unnecessary risk.

Why the other choices are unsuitable

OptionB (ALL PRIVILEGES on the catalog) grants an excessive set of rights—including DROP, ALTER, and CONTROL—far beyond what is required for an external location, violating least-privilege guidelines.
OptionC (promoting to Workspace Admin) gives full workspace-wide administration capabilities, which is unnecessary for a single external-location creation and introduces unnecessary authority.
OptionD (assigning Metastore Admin) provides administration over the entire metastore, including the ability to manage schemas, tables, and other objects, which is broader than the specific CREATE EXTERNAL LOCATION right needed.

Result: Only granting CREATE EXTERNAL LOCATION on the metastore and the referenced storage credential unblocks the pipeline with the minimal set of privileges.

References

External locations – Databricks: https://docs.databricks.com/data/data-lo-auth/external-location.html
Manage storage credentials – Databricks: https://docs.databricks.com/data/data-lo-auth/credential.html#storage-credentials

Question: 217 CertyIQ

A data engineer needs to compute metrics such as:
•rolling 7-day sales
•percent of total revenue
•top-N most profitable categories
Which two approaches are appropriate for implementing these metrics with PySpark DataFrames? (Choose two.)

A. Using RDD aggregations for better performance.
B. Using mapPartitions to manually implement rolling windows.
C. Using window functions for rolling and ranking calculations.
D. Collecting the DataFrame to the driver for faster iterative processing.
E. Using joins and groupBy to compute percent-of-total metrics.

Answer: CE

Explanation:

Why C and E are the recommended approaches

C – Window functions – Spark's distributed window API lets you express rolling 7-day aggregates, rankings, and percentile-type calculations in a single pass while preserving row-level detail. Functions such as row_number(), rank(), dense_rank(), ntile(), sum(...) over (order by … rows between ...) handle rolling windows and top-N selections efficiently without manual looping or shuffling of data.

E – Joins and groupBy for percent-of-total – To obtain a metric like "percentage of total revenue", you can

aggregate the revenue at the required granularity with groupBy and then divide by the overall total (computed either with a separate agg(sum(...)) or by using a window with sum(...) over ()). This keeps the computation distributed and avoids pulling the full dataset into the driver.

Why the other options are unsuitable

A – RDD aggregations – While RDDs can be used, Spark's higher-level DataFrame / SQL APIs are optimized for the same operations. Using RDDs for these tasks generally yields poorer performance and requires more boiler-plate code, which is contrary to best practices for a Certified Data Engineer.

B – mapPartitions with manual rolling windows – Implementing windows by hand inside mapPartitions forces each partition to maintain state and perform large intermediate shuffles, leading to correctness issues and degraded scalability. Spark already provides a robust windowing layer that should be preferred.

D – Collecting to the driver – Bringing the entire DataFrame to the driver destroys the distributed nature of Spark and does not scale with larger inputs. It also introduces a single-point bottleneck and is not a viable strategy for production-level metrics.

References

Window Functions in Spark SQL: https://spark.apache.org/docs/latest/sql-ref-window-functions.html
GroupBy and Aggregation with DataFrames: https://spark.apache.org/docs/latest/sql-ref-groupby-function.html

Question: 218 CertyIQ

After a failed multi-task job run, a data engineer changes the cluster size and updates a notebook path for one task. The data engineer wants to re-run just the failed/skipped tasks with the updated settings and keep the run lineage intact.
Which strategy should the data engineer use to preserve the run history?

A. Use Jobs REST API repairRun to re-run failed tasks; the repair uses the current job and task settings and appears in the run history.

B. Increase the task retry count and click 'Retry' on the failed task to automatically apply new cluster settings within the same run.

C. Export the job as a Databricks Asset Bundle, redeploy it, and re-run only the modified tasks from the bundle.

D. Use 'Run now' to start a new run, so that the updated settings apply; the new run replaces the prior run history.

Answer: A

Explanation:

Justification

Option A – Using the **Jobs REST API repairRun** re-executes only the failed or skipped tasks with the **current job definition** (including the updated cluster size and notebook path). Because the API call references the original job ID, the new attempts are recorded as part of the **same run lineage**, preserving the original run history and metadata. This is the only method that directly modifies the existing run while respecting the changed settings.

Option B – Simply clicking **Retry** after raising the retry count will re-attempt the task, but it does **not apply new cluster or notebook settings** that were edited outside the retry flow; the retry uses the original task

configuration. Moreover, retries are limited to the job's retry policy and do not provide a controlled way to re-run just the impacted tasks with bespoke updates.

Option C – Exporting to a **Databricks Asset Bundle** and redeploying is a sound deployment practice, yet it requires **rebuilding the entire bundle and launching a fresh job run**. This breaks the existing run lineage and is overkill when only a single task needs adjustment.

Option D – Starting a **new "Run now"** creates an entirely new run ID, completely discarding the original run history. Although the new run reflects the updated settings, the lineage is lost, which contradicts the requirement to keep the prior history intact.

Therefore, **Option A** is the appropriate strategy to re-run only the failed/skipped tasks with updated parameters while preserving the original run lineage.

References

Jobs REST API – repairRun: https://docs.databricks.com/en/jobs/api-2.0/jobs.html#repairrun
Jobs UI – Retry a run: https://docs.databricks.com/en/jobs/run-jobs.html#retry-a-run

Question: 219 CertyIQ

When using MERGE INTO in Delta Lake on Databricks, which clause allows you to atomically delete target rows that no longer match any source rows?

A. WHEN MATCHED BY SOURCE THEN UPDATE SET NULL
B. WHEN MATCHED AND THEN DELETE
C. WHEN NOT MATCHED BY SOURCE THEN DELETE
D. WHEN NOT MATCHED THEN DELETE

Answer: C

Explanation:

Justification

Correct option – C:WHEN NOT MATCHED BY SOURCE THEN DELETE
This clause fires only for rows in the target Delta table that are matched (i.e., exist) but do not have a corresponding row in the source data. It therefore **atomically deletes** those target rows, ensuring that after the MERGE the target reflects exactly the rows present in the source.

Why the other options are unsuitable

A:WHEN MATCHED BY SOURCE THEN UPDATE SET NULL – This updates matched rows to NULL rather than deleting them; it does not remove rows that have no source counterpart.
B:WHEN MATCHED AND THEN DELETE – The syntax is incomplete and syntactically invalid; WHEN MATCHED cannot be followed directly by DELETE without specifying the matching condition (e.g., BY SOURCE).
D:WHEN NOT MATCHED THEN DELETE – This clause would delete rows that are **not matched at all** on the join condition, which does not correspond to the requirement of deleting rows that were matched but now lack a source match. It would incorrectly delete rows that never existed in the target.

Hence, option **C** precisely implements the needed "delete target rows that no longer match any source rows" in an atomic MERGE operation.

References

Databricks SQL Documentation – MERGE command syntax: https://docs.databricks.com/sql/language-manual/sql-data-definition-language/merge.html
Delta Lake MERGE clause reference: https://delta.io/docs/delta/merge/ (accessed 2025-09-25)

Question: 220 CertyIQ

A data engineer must support self-serve BI dashboards for hundreds of business users who run ad hoc, high-concurrency SQL queries throughout the day against governed Unity Catalog tables. The team needs a near-instant start, autoscaling without manual tuning, and the best performance for SQL while minimizing operational overhead.
Which Databricks compute should the data engineer use?

A. SQL Warehouse (Classic) with a fixed cluster size.
B. Jobs compute clusters triggered by dashboards on a schedule.
C. All-purpose classic cluster with manual autoscaling enabled.
D. SQL Warehouse (Serverless) with Photon enabled.

Answer: D

Explanation:

Why OptionD is the optimal choice

Instant start-up & serverless elasticity – A ServerlessSQL warehouse with Photon launches in seconds and automatically expands or contracts to match query load, eliminating the need for any manual scaling or capacity planning.
Optimal performance for ad-hoc SQL – Photon's vectorized engine and columnar execution are purpose-built for high-throughput SQL workloads, delivering the best latency and throughput among Databricks compute options.
Zero operational overhead – No cluster provisioning, version upgrades, or configuration tuning are required; the service is fully managed, which aligns with the requirement for "minimizing operational overhead."
Governance-ready – Serverless warehouses can be associated with Unity Catalog, enabling row-level security, data masking, and audit logging for governed access to the governed tables that business users query.

Why the other options are less suitable

A – SQL Warehouse (Classic) with a fixed cluster size – Requires a pre-sized, always-on cluster; scaling must be done manually, leading to either over-provisioned resources or performance bottlenecks under high concurrency.
B – Jobs compute clusters triggered by dashboards on a schedule – Jobs clusters are designed for batch ETL tasks, not for serving interactive, ad-hoc dashboard queries; they start on a schedule and lack the low-latency start-up needed for self-serve BI.
C – All-purpose classic cluster with manual autoscaling enabled – Although autoscaling is available, the cluster must be provisioned and managed by the engineer, introducing operational complexity and longer cold-start times compared to a serverless service.

References

Serverless SQL warehouses with Photon – https://learn.microsoft.com/azure/databricks/sql/warehouses-serverless

Governed data access with Unity Catalog – https://learn.databricks.com/data-governance/unity-catalog

Question: 221 CertyIQ

A team is using a Databricks workspace and needs to continuously ingest JSON files as they arrive.
Which code snippet shows a valid Auto Loader source configuration?

A.

```
spark.readStream.format("json")
    .option("cloudFiles.format", "json")
        .load("<path>")
```

B.

```
spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "json")
        .load("jdbc:sqlserver://;database=")
```

C.

```
spark.readStream.format("cloudFiles")
    .option("format", "json")
        .load("<path>")
```

D.

```
spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "json")
        .load("<path>")
```

Answer: D

Explanation:

1. spark.readStream

This tells Spark to read data as a stream, not as a static batch.

It continuously monitors a location for new incoming files.

2. .format("cloudFiles")

This specifies the use of Auto Loader (cloudFiles).

Auto Loader is a Databricks feature designed to:

Efficiently ingest new files incrementally

Handle large-scale file ingestion

Automatically track which files have already been processed

It is much more efficient than traditional file streaming.

3. .option("cloudFiles.format", "json")

Defines the format of incoming files.

In this case, files are:

JSON files

Other supported formats include:

CSV

Parquet

Avro

etc.

4. .load("<path>")

Specifies the source directory where files arrive.

Question: 222 CertyIQ

A data engineering team is analyzing customer transactions using a PySpark DataFrame named transactions with columns customer_id, amount, and category.
The team needs to compute, for each customer, the total transaction amount, the average transaction amount, and the maximum amount - all in a single operation for efficiency.
Which code fragment should be used to achieve this in PySpark?

A.

```
transactions.groupBy("customer_id")
    .agg(sum("amount"), avg("amount"), max("amount")))
```

B.

```
transactions.groupBy("customer id")
    .select(sum("amount"), avg("amount"), max("amount"))
```

C.

```
transactions.agg(groupBy("customer_id"),
                 sum("amount"),
                 avg("amount"),
                 max("amount"))
```

D.

```
transactions.groupBy("customer_id")
    .sum("amount")
        .avg("amount").max("amount")
```

Answer: A

Explanation:

```
transactions.groupBy("customer_id")
    .agg(sum("amount"), avg("amount"), max("amount")))
```

transactions: This is your initial DataFrame (a distributed collection of data).

.groupBy("customer_id"): This organizes the data into distinct groups based on the unique values in the customer_id column. Every row that shares the same customer_id is collected together into a single group.

.agg(...): This is the "aggregate" function. It tells Spark that you want to perform mathematical operations on the rows within each group.

(sum("amount"), avg("amount"), max("amount")): These are the specific metrics being calculated for each customer:

sum("amount"): Calculates the total spending for that customer.

avg("amount"): Calculates the average transaction value for that customer.

max("amount"): Identifies the single largest transaction amount for that customer

Question: 223 CertyIQ

A Lakeflow Jobs workflow fails at the last task because of an error in a notebook task. This workflow is scheduled to run daily. The data engineer fixes the notebook and wants to rerun the pipeline. This workflow is very compute-intensive and expensive to run.
Which action should the data engineer take in order to minimize downtime and compute cost?

A. Restart the cluster
B. Re-run the entire workflow
C. Switch to another cluster
D. Repair run

Answer: D

Explanation:

The notebook task is the only failing step in an otherwise successful Lakeflow pipeline; rerunning the entire workflow or recreating the cluster would waste significant compute resources because the upstream jobs have already produced their results.
Repair run allows the engineer to re-execute only the failed notebook task while preserving the previously materialized downstream outputs, keeping cluster reuse, job configuration, and resource allocations intact.

This minimizes both **downtime** (the pipeline can resume as soon as the single task completes) and **cost** (avoids paying for additional compute to reprocess data that is already available).
Restarting the cluster or switching to a different cluster adds overhead and does not address the root cause; merely re-submitting the whole workflow repeats all expensive steps unnecessarily.

References

Databricks Lakeflow Jobs – Rerun a failed task: https://learn.microsoft.com/en-us/databricks/jobs/runners/repair-run
Optimizing compute costs in Databricks Jobs: https://learn.microsoft.com/en-us/databricks/jobs/optimize-costs

Question: 224 CertyIQ

A data engineer is investigating a Lakehouse ETL job on Databricks that has suddenly become four times slower. They need to:
•Identify the specific Spark stages that regressed.
•Drill down to per-task metrics such as shuffle read/write size, spill, and GC time.
•Correlate these with the executor resource usage during that run.
Which Databricks-native capability is the starting point for this detailed job-level root-cause analysis?

A. Cluster Metrics (Metrics tab on the compute) to view historical Spark and hardware metrics, without using the Spark UI.

B. The Jobs run details page timeline and graph views, without opening the Spark UI, to analyze all stage and task-level Spark metrics.

C. Ganglia metrics for the cluster, which provide full visibility into Spark stages, tasks, and shuffle metrics for the job run.

D. Spark UI opened from the job run's compute, using the Jobs and Stages tabs to inspect task metrics.

Answer: D

Explanation:

Why optionD is the most appropriate choice

The question asks for a native Databricks capability that lets a data engineer start a **job-level root-cause analysis** and then **drill into per-task Spark metrics** such as shuffle read/write size, spill, GC time, and executor resource usage.
Databricks automatically embeds a fully-interactive **Spark UI** inside each job run. From the **Jobs and Stages tabs** of that UI you can:

1. View each Spark stage's aggregated metrics.
2. Click into any stage to see **task-level details** – the exact shuffle read/write bytes, spill metrics, GC duration, CPU/ memory usage per executor, and the underlying host/resource consumption.
3. Correlate these metrics with the **executor resource usage** displayed alongside each task (cores, memory, network I/O).

Because the UI runs directly from the cluster used by the job, it always reflects the exact resource configuration (driver/executor memory, cores, pools, etc.) of that run, ensuring the root-cause findings are accurate and reproducible.

Why the other options fall short

OptionA – Cluster Metrics on the Metrics tab

Provides high-level historical CPU, memory, and I/O trends for a cluster over time, but it does **not expose Spark-specific task metrics** (shuffle size, spill, GC) or let you drill into individual Spark jobs/stages. It is useful for capacity planning, not detailed Spark job debugging.

OptionB – Jobs run details page timeline/graph views (without Spark UI)

The timeline and graph visualisations give an overview of job durations and stage breakdowns, yet they are **high-level aggregates** and do not expose the deep per-task Spark metrics required for the analysis described.

OptionC – Ganglia metrics for the cluster

While Ganglia (or the underlying Cloud metrics) can show host-level resource usage, it is **not a Spark-native capability** and often lacks the fine-grained Spark-centric parameters (stage/task shuffle read/write, spill, GC time) needed for pinpointing Spark-specific regressions. Moreover, Databricks does not expose raw Ganglia dashboards in the UI; the built-in Spark UI does.

Hence, the **starting point for a detailed, job-level Spark root-cause analysis is to open the native Spark UI from the job's compute and use its Jobs and Stages tabs** – exactly what optionD describes.

References

Spark UI in Databricks Jobs: https://learn.microsoft.com/en-us/azure/databricks/jobs/jobs-structured-streaming#spark-ui

Analyzing Spark Jobs with the Spark UI: https://docs.databricks.com/spark/latest/monitoring/spark-metrics/index.html

Question: 225 CertyIQ

A data engineer is troubleshooting a Lakeflow Spark Declarative Pipelines (SDP) where several rows are dropped by expectations. The product owner requests details on which expectation failed most frequently during the last run and the corresponding affected table. The engineer wants to use Databricks' built-in observability data to answer this without adding new logging code.
How should the data engineer troubleshoot SDP to verify the expectation failure?

A. Export notebook run results to HTML and search for the expectations summary in the rendered output to compile the metrics.

B. Query the Lakeflow Spark Declarative Pipelines event log to retrieve expectation metrics and failure details for the pipeline's most recent update.

C. Use the Spark UI's SQL tab to identify failed rows and expectation names from physical query plans.

D. Increase the cluster's log level to DEBUG and parse the driver's Log4j logs for expectation metrics for the last run.

Answer: B

Explanation:

Why optionB is correct

Lakeflow Spark Declarative Pipelines automatically emit an **event log** that records every expectation evaluation, including the expectation name, outcome (pass/fail), and affected table.
By querying this event log directly (e.g., SELECT expectation_name, COUNT(*) AS failures FROM lakeflow_event_log WHERE event='expectation_failure' GROUP BY expectation_name ORDER BY failures DESC LIMIT 1;), the engineer can instantly see which expectation failed most often in the last pipeline run and map it to the corresponding table.
This approach uses Databricks' built-in observability data, requires no additional logging configuration, and works for any pipeline without modifying user code.

Why the other options are less suitable

A – Exporting notebook results to HTML adds an extra manual step and does not guarantee the underlying event metrics are captured; it relies on rendering rather than querying the source data.
C – The Spark UI SQL tab shows only the execution plan of the physical query; it does not surface high-level expectation statistics or failure frequencies across multiple runs.
D – Increasing the Log4j level to DEBUG generates raw driver logs, which can be noisy and require parsing complex log messages; this also necessitates code changes or cluster configuration changes and does not directly expose structured expectation metrics.

Result – Querying the Lakeflow event log (optionB) provides a precise, efficient way to retrieve the exact failure count per expectation and associate it with the affected table, fulfilling the product owner's request using native observability.

References

Monitoring expectations in Lakeflow pipelines
Lakeflow event log and its schema

Question: 226 CertyIQ

A data engineer uses the Databricks workspace UI with Unity Catalog enabled. In Data Explorer, they select catalog corp_marketing, then schema campaigns, and see table email_stats. The engineer must let the growth-analysts group read email_stats from their SQL warehouses, but not create, alter, or delete any objects in corp_marketing or campaigns.
Which action sequence meets the requirement?

A. 1. On schema campaigns, open Permissions and grant USE SCHEMA to growth-analysts
2. On the table email_stats, open Permissions and grant SELECT and MODIFY to growth-analysts

B. 1. On catalog corp_marketing, open Permissions and grant USE CATALOG to growth-analysts
2. On schema campaigns, open Permissions and grant USE SCHEMA to growth-analysts
3. On the table email_stats, open Permissions and grant SELECT to growth-analysts

C. 1. On catalog corp_marketing, open Permissions and grant USE CATALOG and CREATE SCHEMA to growth-analysts
2. On schema campaigns, open Permissions and grant USE SCHEMA to growth-analysts
3. On the table email_stats, open Permissions and grant SELECT to growth-analysts

D. 1. On schema campaigns, open Permissions and grant USE SCHEMA and SELECT to growth-analysts
2. On the table email_stats, open Permissions and grant SELECT to growth-analysts

Answer: B

Explanation:

Technical Justification for Correct Answer B

Requirement Recap: Grant the growth-analysts group **read-only access** to the email_stats table in the campaigns schema within the corp_marketing catalog, without permissions to create, alter, or delete any objects in the catalog or schema.

Why B is the Correct Option:

Step 1 (B): On catalog corp_marketing, grant USE CATALOG to growth-analysts

Necessity: USE CATALOG permission is required for the group to even see the catalog and its contents in the UI. This is a prerequisite for any further actions on the catalog's objects.

Sufficiency for Requirement: Does not grant object modification rights.
Step 2 (B): On schema campaigns, grant USE SCHEMA to growth-analysts

Necessity: Similar to the catalog, USE SCHEMA is needed for the group to access the schema's objects.
Sufficiency for Requirement: Still, no object modification permissions are granted.
Step 3 (B): On the table email_stats, grant SELECT to growth-analysts

Direct Alignment with Requirement: Explicitly grants **read-only** access to email_stats.
No Unnecessary Permissions: Unlike option A, it does not grant MODIFY, which would allow alterations, contradicting the requirements.

Why Other Options are Less Suitable:

Option A:

Insufficient for Requirement: Grants MODIFY on email_stats, allowing alterations.
Incomplete Access Path: Misses granting access at the catalog and schema levels explicitly for read operations.
Option C:

Excessive Permissions: Grants CREATE SCHEMA at the catalog level, enabling the creation of new schemas, which is not required and violates the "read-only" intent.
Option D:

Inconsistent and Redundant:

Grants SELECT at both the schema and table levels, which is redundant since table-level permissions override schema-level ones in this context.
Fails to address the necessity of USE CATALOG and appropriately managed schema access.

Correct Action Sequence (B) ensures the growth-analysts group can **read** email_stats without having the ability to **create, alter, or delete** objects within corp_marketing or campaigns, aligning perfectly with the requirement.

References

Databricks Unity Catalog Permissions Documentation
Databricks Access Control: Best Practices for Unity Catalog

Question: 227 CertyIQ

An organization is building a data lakehouse and needs to ingest data from multiple sources into Unity Catalog-managed tables:
•Source 1: Salesforce (50+ objects, frequent schema changes, OAuth authentication required)
•Source 2: On-premises SQL server database (100+ tables, CDC enabled, requires network connectivity)
•Source 3: Daily JSON files landing in Azure Data Lake Storage Gen2
The organization wants all ingested data to be governed by Unity Catalog, to minimize data engineering effort for schema changes, and to use serverless wherever possible to reduce operational overhead.
Which ingestion strategy meets these requirements?

A. Use partner connectors (Fivetran) for all three sources via Partner Connect, as they provide the most comprehensive integration with Unity Catalog and automated schema evolution.

B. Use Lakeflow Connect managed connectors for all three sources, as they provide consistent serverless execution and Unity Catalog integration across all data source types.

C. Use Lakeflow Connect managed connectors for Salesforce and SQL Server, and Auto Loader for ADLS files, all within Lakeflow Spark Declarative Pipelines.

D. Use Auto Loader for all three sources with custom Python code to handle Salesforce API calls and SQL server CDC extraction, then write to Unity Catalog tables.

Answer: C

Explanation:

Technical Justification for Correct Answer: C

The correct ingestion strategy that meets the organization's requirements is **Option C: Use Lakeflow Connect managed connectors for Salesforce and SQL Server, and Auto Loader for ADLS files, all within Lakeflow Spark Declarative Pipelines**. Here's why:

Why Option C is the Best Choice:

Salesforce (Source 1): Lakeflow Connect managed connectors are ideal for Salesforce due to its ability to handle **OAuth authentication** and **frequent schema changes** with minimal engineering effort, ensuring Unity Catalog stays up-to-date.
On-premises SQL Server (Source 2): Lakeflow Connect can efficiently manage **CDC (Change Data Capture)** and the required **network connectivity** through properly configured tunnels or connectors, integrating seamlessly with Unity Catalog. While Lakeflow Connect is preferred for its managed features, ensuring network connectivity might require additional setup (e.g., securely exposing the SQL Server to the cloud or using a gateway).
Daily JSON Files in ADLS (Source 3): **Auto Loader** is the most suitable for ingesting files from ADLS, especially for **serverless** and **schema evolution** requirements, directly writing to Unity Catalog-managed tables. Auto Loader's event-triggered (or scheduled) nature efficiently handles the daily file uploads, optimizing compute usage.
Why Other Options are Less Suitable:

Option A (Fivetran): While Fivetran provides comprehensive integrations, it **doesn't inherently leverage Unity Catalog's governance** natively as managed connectors or Databricks-native solutions would. Additionally, it **adds operational overhead** of managing another external service and may not fully support **serverless execution** as seamlessly as native Databricks solutions.

Option B (Lakeflow Connect for all): Currently, Lakeflow Connect's managed experience is optimized for certain cloud and SaaS sources but **might not fully support or be the best fit for file-based ingestion from ADLS** in the same streamlined manner as Auto Loader, which is specifically designed for such scenarios.

Option D (Auto Loader for all with custom code):

Salesforce & SQL Server: Requiring **custom Python code** for these sources **increases data engineering effort**, especially for handling OAuth and frequent schema changes for Salesforce, and CDC for SQL Server.
Less Serverless: The custom approach might not fully leverage **serverless benefits** as effectively as using managed connectors for these dynamic sources.
Key Requirements Fulfillment by Option C:

Governed by Unity Catalog: Checked for all sources.
Minimize Data Engineering Effort for Schema Changes: Checked for Salesforce and implicitly for ADLS files via Auto Loader's schema evolution capabilities.
Serverless Wherever Possible: Checked, with managed connectors and Auto Loader optimized for serverless execution.
References

Databricks Lakeflow Connect Documentation
Databricks Auto Loader Documentation

Question: 228 CertyIQ

A data engineer has manually created several Databricks jobs and dashboards using the workspace UI. The team now wants to manage these resources as code using Declarative Automation Bundles (formerly Databricks Asset Bundles), store the configuration in a Git repository, and deploy changes through CI/CD.
Which approach converts the existing resources into a bundle project?

A. Run databricks bundle init to create a new bundle project, then use databricks bundle deploy pointed at the workspace to automatically detect and import all existing resources into the bundle configuration.

B. Use the Databricks REST API to export each job's JSON definition, save the JSON files into a Git repository, and configure CI/CD to import them using the REST API on each deployment.

C. Run databricks bundle generate for each existing resource, use --bind to link the generated configuration to the existing workspace resources, and commit the bundle project to Git for CI/CD deployment.

D. Clone an empty Git repository into a Git Folder, recreate each job and dashboard configuration in YAML by referencing the settings using workspace UI, then commit and deploy with databricks bundle deploy.

Answer: C

Explanation:

Technical Justification for Correct Answer (C)

The correct approach to convert existing Databricks resources into a Declarative Automation Bundle (formerly Databricks Asset Bundle) for managed deployment via CI/CD from a Git repository is **Option C**. Here's why:

Why Option C is Correct:

databricks bundle generate is the appropriate command for initializing the process of creating a bundle from existing resources. This command generates the bundle configuration based on the specified resources.
--bind ensures that the generated configuration is linked to the existing workspace resources, maintaining a direct relationship between the code (in Git) and the live workspace configuration. This linkage is crucial for accurate and targeted deployments.
Committing the Bundle Project to Git aligns with best practices for Infrastructure as Code (IaC) and application configuration management, enabling version control and CI/CD pipeline integration.
Deployment via CI/CD (implied after committing to Git) allows for automated, managed deployment of changes, fulfilling the requirement for automated deployment through Continuous Integration/Continuous Deployment processes.
Why Other Options are Less Suitable:

Option A - Incorrect:

databricks bundle init solely initializes a new, empty bundle project. It does not automatically detect and import existing resources from a workspace. The --detect flag might seem relevant, but as of my last update, automatic detection of all existing resources for import isn't fully supported in the manner implied by this option for bundling existing resources into a new project.

Option B - Less Suitable:

While feasible, using the REST API for export/import is more manual and error-prone compared to the declarative approach offered by Declarative Automation Bundles. It lacks the direct resource linking and ease of management provided by bundle projects, making it less ideal for managed, automated deployments.

Option D - Incorrect:

Recreating each configuration manually in YAML without automated generation introduces significant potential for human error. It also misses the direct binding to existing resources, which databricks bundle generate with --bind provides, making the process inefficient and prone to discrepancies.

Key Points in Bullet Form for Quick Reference:

Correct Approach (C):

Utilizes databricks bundle generate for automated configuration creation.
Employs --bind for direct resource linking.
Supports version control and CI/CD through Git.

Incorrect/ Less Suitable Approaches:

A: Incorrect assumption about automatic resource detection and import.
B: More manual, prone to errors, less suitable for automated managed deployments.
D: Error-prone manual recreation, lacks direct resource binding.

References

Databricks Documentation - Declarative Automation Bundles
Databricks CLI Reference - databricks bundle Commands

Question: 229 CertyIQ

A developer is building a data pipeline that processes records from a bronze table into a silver table. The bronze table, bronze_events, contains duplicate records due to at-least-once delivery guarantees from the upstream ingestion system. The developer writes the following PySpark code to prepare the data:

```
deduped_df = df.dropDuplicates()
deduped_df.summary("count", "mean", "stddev").show()
from pyspark.sql.functions import approx_count_distinct
deduped_df.select(approx_count_distinct("user_id")).show()
```

The developer notices that after running dropDuplicates() without arguments, some rows that differ only in the event_timestamp column remain. The developer wants to de-duplicate based only on user_id and event_type, keeping one row per unique combination of these two columns.
Which code change achieves this deduplication requirement?

A. Replace df.dropDuplicates() with df.groupBy("user_id", "event_type").count()
B. Replace df.dropDuplicates() with df.distinct().select("user_id", "event_type")
C. Replace df.dropDuplicates() with df.dropDuplicates(["user_id", "event_type"])
D. Replace df.dropDuplicates() with df.drop("event_timestamp").dropDuplicates()

Answer: C

Question: 230 CertyIQ

A data engineer needs to enforce row-level security on main.secure.events(region STRING, event_id STRING). Members of the account group all_regions need to see all rows. All other users need to see only rows where region = 'EU'.
Which SQL sequence satisfies the requirement?

A.

```
CREATE FUNCTION main.secure.region_filter(region STRING)
RETURN IF(is_account_group_member('all_regions'), true, region = 'EU');
```

B.

```
ALTER TABLE main.secure.events
SET ROW FILTER main.secure.region_filter ON (region);
CREATE FUNCTION main.secure.region_filter(region STRING)
RETURN IF(is_account_group_member('all_regions'), true, region = 'EU');
```

C.

```
ALTER VIEW main.secure.events
SET ROW FILTER main.secure.region_filter ON (region);
CREATE FUNCTION main.secure.region_filter(region STRING)
RETURN IF(is_account_group_member('all_regions'), true, region = 'EU');
```

D.

```
ALTER TABLE main.secure.events
ALTER COLUMN region
SET MASK main.secure.region_filter;
CREATE FUNCTION main.secure.region_filter(region STRING)
RETURN IF(is_account_group_member('all_regions'), true, region = 'EU');

ALTER TABLE main.secure.events
SET ROW FILTER main.secure.region_filter USING COLUMNS (region);
```

Answer: A

Question: 231 CertyIQ

A governance team is evaluating whether to use Unity Catalog attribute-based access control (ABAC) policies or manually applied row filters and column masks to protect sensitive data across their catalog.
Why should the team use ABAC policies instead of manually applied row filters and column masks?

A. ABAC policies use governed tags to dynamically match tables and columns, so a single policy defined on a catalog automatically applies to all current and future tables within that catalog without per-table configuration.

B. ABAC policies allow row filters and column masks to be written in Python and Scala directly, whereas manually applied filters and masks can only use SQL user-defined functions.

C. ABAC policies can be applied to views and materialized views in addition to tables, whereas manually applied row filters and column masks can only be applied to tables.

D. ABAC policies support applying multiple distinct row filters to the same table for a single user at query time, whereas manually applied row filters are limited to one filter per table.

Answer: A

Explanation:

Technical Justification

The governance team should opt for Unity Catalog attribute-based access control (ABAC) policies over manually applied row filters and column masks for the following reason, with an explanation of why the correct option stands out and why others are less suitable:

Why Option A is Correct:

Scalability and Efficiency: ABAC policies utilize governed tags to dynamically match tables and columns. This functionality enables a single policy, defined at the catalog level, to automatically apply to all existing and future tables within that catalog without requiring per-table configuration. This approach significantly reduces administrative overhead and ensures consistent security enforcement across the catalog as it evolves.

Why Other Options are Less Suitable:

Option B:

Misleading on Flexibility: While it's true that ABAC policies can leverage various programming languages for complex logic, the statement about manually applied filters and masks being limited to SQL user-defined functions is not the primary advantage of ABAC over manual methods. The core benefit of ABAC lies in its dynamic application based on governed tags, not just the programming languages supported.
Irrelevant to the Core Benefit: The language support, although a plus, does not address the scalability and automation benefits that make ABAC superior for governance across a large, dynamic catalog.
Option C:

Partial Truth but Not the Best Reason: Although ABAC policies can indeed be applied to views and materialized views, this is a specific use case benefit rather than the overarching reason to choose ABAC over manual filters for protecting sensitive data across the entire catalog.
Narrow Scope: This option focuses on a particular data structure (views and materialized views) rather than the broad catalog management efficiency provided by ABAC.
Option D:

Complexity Without the Primary Benefit: Supporting multiple distinct row filters for a single user at query time introduces complexity. However, this does not address the primary advantage of ABAC, which is the automatic application of policies across all tables in a catalog based on governed tags.
Overly Specific Use Case: This highlights a specific capability rather than the foundational scalability and efficiency benefit of ABAC policies.

ConclusionOption A is the correct choice because it highlights the scalable and efficient nature of ABAC policies in automatically applying to all current and future tables within a catalog, based on governed tags, without needing per-table configuration. This directly addresses the challenge of protecting sensitive data across a potentially large and evolving catalog with minimal administrative overhead.

References

1. **Databricks Documentation - Unity Catalog ABAC Policies**: https://docs.databricks.com/security/access-control/abac.html
2. **Databricks Documentation - Governed Tags in Unity Catalog**: https://docs.databricks.com/security/access-control/governed-tags.html

Question: 232 CertyIQ

A data engineering team needs to ingest historical CSV files from a cloud storage location that already contains 50,000 existing files. They also expect new files to arrive continuously going forward. The team wants to use Auto Loader to incrementally process both existing files and new arrivals efficiently.
Which Auto Loader mode should the team configure for this use case?

A. Use file notification mode because it scales efficiently for both large numbers of existing files and continuous new arrivals by using cloud storage events rather than repeatedly listing directories.

B. Use directory listing mode because it provides better performance for the initial processing of 50,000 existing files through optimized incremental listing capabilities.

C. Use directory listing mode initially to process the 50,000 existing files, then reconfigure the stream to use file notification mode for ongoing ingestion of new arrivals.

D. Use file notification mode only after all existing files have been processed, as it cannot efficiently handle large volumes of pre-existing files.

Answer: A

Explanation:

Justification for Correct Answer A and Analysis of Other Options

Correct Answer A: Use file notification mode

Efficiency for Existing Files: Although file notification mode is often associated with handling new arrivals, modern cloud storage systems (e.g., AWS S3, GCP Cloud Storage, Azure Data Lake Storage) can generate notifications for existing files when a one-time "backfill" notification is triggered, making it efficient for processing the initial 50,000 files without the need for explicit directory listing.
Scalability for Continuous New Arrivals: It inherently scales for continuous new file arrivals by leveraging cloud storage events, avoiding the overhead of periodic directory listing, which can be resource-intensive and less scalable.
Unified Approach: Allows for a unified configuration for both historical and newly arriving data, simplifying the ingestion pipeline.

Why Other Options are Less Suitable

B. Directory Listing Mode

Inefficient for Large Existing Files: While optimized for incremental listing, processing 50,000 existing files via directory listing can still be less efficient compared to event-driven approaches, especially in very large datasets.
Ongoing Overhead: Continues to incur the overhead of periodic directory listing for new files, which is less scalable and efficient than event-driven notifications.
C. Hybrid Approach (Directory Listing then File Notification)

Added Complexity: Requires reconfiguration of the stream, introducing operational complexity and potential downtime or gaps in ingestion.
No Significant Gain: Given file notification can handle the backfill efficiently, the hybrid approach does not offer substantial benefits to justify the added complexity.
D. File Notification Mode Only After Processing Existing Files

Misunderstanding of File Notification Capabilities: Incorrectly assumes file notification cannot handle existing files efficiently, overlooking the backfill notification capability.
Operational Gap: Leaves a gap in how the existing files are efficiently processed before switching to file notification mode.

References

Databricks Documentation: Auto Loader Modes
Databricks Documentation: Backfilling Existing Files with Auto Loader

Question: 233 CertyIQ

A data engineer has multiple Unity Catalog-managed Delta tables that require regular maintenance. Currently, the data engineer manually schedules OPTIMIZE and VACUUM jobs for each table, adjusting the frequency based on how often each table is queried and updated. The engineer needs to eliminate this manual maintenance overhead and have the platform automatically determine when and how to run these operations.
Which action should the engineer take to meet the requirements?

A. Enable Liquid Clustering with CLUSTER BY on each table to automatically compact and remove stale data files on a scheduled basis.

B. Set the delta.autoOptimize.autoCompact and delta.autoOptimize.optimizeWrite table properties to trigger automatic VACUUM and file compaction.

C. Configure Auto Loader with schema evolution enabled to automatically trigger OPTIMIZE and VACUUM after each data ingestion batch completes.

D. Enable predictive optimization at the account, catalog, or schema level so the platform automatically identifies and runs OPTIMIZE and VACUUM on managed tables.

Answer: D

Explanation:

Technical Justification for Correct Answer: D

Why D is the Best Option: Enabling predictive optimization at the account, catalog, or schema level is the most suitable approach for automatic OPTIMIZE and VACUUM operations on Unity Catalog-managed Delta tables. This feature leverages Databricks' advanced analytics capabilities to monitor table usage patterns, query frequencies, update rates, and storage efficiency. By analyzing these metrics, the platform can dynamically schedule OPTIMIZE (for compaction and file reorganization) and VACUUM (for removing stale data files) operations at optimal times, minimizing manual intervention and ensuring tables are always query-efficient.

Key Advantages of D:

Autonomous Decision Making: The platform determines the best schedule based on actual usage and storage needs.
Scalability: Easily applies to multiple tables without proportional increase in management complexity.
Adaptability: Automatically adjusts to changes in table usage patterns over time.

Why Other Options are Less Suitable:

A. Enable Liquid Clustering with CLUSTER BY:

Limited Scope: Primarily designed for data partitioning and compaction, not for automatic stale data removal (VACUUM).
Manual Configuration: Requires specifying CLUSTER BY columns, which may not directly address the optimization needs based on query patterns.
Schedule Not Dynamic: Does not automatically adjust the schedule based on table activity.

B. Set delta.autoOptimize.autoCompact and delta.autoOptimize.optimizeWrite:

Table Property Overhead: Needs to be set on each table individually, leading to management overhead for multiple tables.
Lack of Global Optimization: Does not leverage global usage patterns across tables for scheduling decisions.
VACUUM Not Addressed: These properties primarily focus on compaction, not on automatically triggering VACUUM for stale data removal.

C. Configure Auto Loader with Schema Evolution:

Trigger Limitation: Ties OPTIMIZE and VACUUM to ingestion batches, which may not align with optimal maintenance windows based on query activity.
Inefficient for Infrequently Updated Tables: Could lead to unnecessary operations if triggered solely by ingestion frequency.
No Global Optimization Insight: Lacks the broader context of table usage to inform maintenance scheduling.

Question: 234 CertyIQ

A data engineering team needs to incrementally ingest customer transactions from a SaaS app into Data Intelligence Platform with:
•Built-in change data capture (including updates and deletes)
•Automatic schema evolution
•Serverless execution with retries and minimal maintenance
•OAuth support and basic monitoring
Which solution meets all of these requirements?

A. Use a Lakeflow Connect managed connector for the SaaS app with built-in CDC, schema evolution, OAuth, and serverless compute with retries, then process downstream with Lakeflow Spark Declarative Pipelines.

B. Use Auto Loader on exported SaaS files and an SDP that calls the SaaS APIs, handling CDC, schema changes, OAuth, and retries in your own code.

C. Schedule batch jobs per region that call SaaS REST APIs and write directly to silver tables with custom CDC and schema evolution, and use system tables for ad hoc basic monitoring.

D. Use a Lakeflow Connect standard connector, implement MERGE-based CDC, schema evolution, and retries in workflows, and set up a dedicated monitoring dashboard.

Answer: A

Explanation:

Technical Justification for Correct Answer: A

The correct solution is **A** because it comprehensively meets all the specified requirements with minimal manual intervention and maximum leverage of managed services, ensuring efficiency and reliability.

Why A is the Best Choice:

Built-in Change Data Capture (CDC): Lakeflow Connect managed connectors often come with built-in CDC capabilities, including support for updates and deletes, reducing the need for custom implementation.
Automatic Schema Evolution: Managed connectors typically support automatic schema evolution, adapting to changes in the data source without manual intervention.
Serverless Execution with Retries: The use of Lakeflow Spark Declarative Pipelines for downstream processing leverages serverless compute, providing automatic scaling and built-in retry mechanisms for robustness.
OAuth Support: Managed connectors usually include support for OAuth, facilitating secure authentication with the SaaS app.
Basic Monitoring: While not explicitly detailed, the integration with Lakeflow's ecosystem likely provides basic monitoring capabilities out of the box or with minimal setup.

Why Other Options are Less Suitable:

B. Auto Loader with Custom Implementation

More Maintenance: Requires handling CDC, schema changes, OAuth, and retries manually, increasing

maintenance burden.
Less Reliable for CDC: Custom CDC implementations can be error-prone and less efficient than built-in solutions.

C. Scheduled Batch Jobs

No Built-in CDC or Schema Evolution: Completely custom implementation required for these critical features.
Not Serverless: Scheduled batch jobs imply a managed or self-managed compute resource, lacking the auto-scaling benefits of serverless execution.
Basic Monitoring via System Tables: Less integrated and more manual than monitoring solutions tied to the data pipeline platform.

D. Lakeflow Connect Standard Connector with Custom Workflow

Missing Built-in CDC and Schema Evolution: Unlike managed connectors, standard connectors may not offer these features out of the box, requiring custom implementation (MERGE-based CDC, manual schema evolution).
More Custom Work: Setting up a dedicated monitoring dashboard adds to the overall custom work required compared to a more integrated solution.

References:(For the purpose of this exercise, links are hypothetical due to the proprietary nature of "Lakeflow" and its components, which seem to be fictional representations possibly based on real technologies like Databricks. Real references would depend on the actual platform's documentation.)

[Reference 1]: https://lakeflow-docs.fictional/lakeflow-connect/managed-connectors (Hypothetical - For Lakeflow Connect Managed Connectors Documentation)
[Reference 2]: https://lakeflow-docs.fictional/declarative-pipelines/serverless-compute (Hypothetical - For Lakeflow Spark Declarative Pipelines Serverless Compute Documentation)

Real-World Equivalents for References (Assuming "Lakeflow" is inspired by Databricks):

[Real Reference 1]: https://docs.databricks.com/data/data-sources/connectors.html (Databricks Connectors Documentation)
[Real Reference 2]: https://docs.databricks.com/pipelines.html (Databricks Pipelines Documentation)

Question: 235 CertyIQ

A data engineer is designing a cost-optimized, event-driven pipeline. They configure a Lakeflow Job with a File Arrival trigger to watch an S3 bucket. The job runs a notebook that uses Auto Loader with trigger(availableNow=True) to ingest data into a Bronze table.
What is the technical relationship between the File Arrival trigger and Auto Loader in this integration pattern?

A. Auto Loader must be configured in File Notification mode to work with a Lakeflow Job File Arrival trigger.

B. The File Arrival trigger starts the job run, while Auto Loader uses its internal checkpoint to independently identify and process only the new files that have arrived since the last successful commit.

C. The File Arrival trigger automatically passes the specific file path of the new arrival to Auto Loader, allowing the engineer to skip the source path configuration in the code.

D. Using a File Arrival trigger requires the engineer to disable Auto Loader's checkpointing to prevent the job from processing the same file multiple times.

Answer: B

Explanation:

Technical Justification for Correct Answer B

The correct answer is **B** due to the distinct yet complementary roles of the Lakeflow Job's File Arrival trigger and Auto Loader's functionality within the described event-driven pipeline. Here's a breakdown of why B is the best choice and why the other options are less suitable:

Why B is Correct

File Arrival Trigger's Role: The primary function of the File Arrival trigger in a Lakeflow Job is to **detect the arrival of new files** in the specified S3 bucket and **initiate the job run** in response. This trigger is responsible for starting the pipeline execution based on the file arrival event.
Auto Loader's Role with trigger(availableNow=True): Auto Loader is configured to ingest data into a Bronze table. The trigger(availableNow=True) setting indicates that Auto Loader should **immediately process all available files** upon the job's initiation. **Critically**, Auto Loader maintains its own **internal checkpointing mechanism**. This means it **independently tracks and processes only the new files** that have been added since the last successful commit, ensuring **idempotence** and preventing reprocessing of already handled files.

Technical Relationship: The File Arrival trigger **starts the job** in response to new file detections, while Auto Loader **handles the ingestion**, using its checkpoint to **independently identify and process only the new files** since the last run. This relationship ensures efficient, cost-optimized, and event-driven data ingestion without redundant processing.

Why Other Options are Less Suitable

A. Incorrect because Auto Loader's mode (File Notification or not) isn't directly dictated by the use of a Lakeflow File Arrival trigger. The described setup doesn't necessitate Auto Loader to be in File Notification mode; its internal checkpointing suffices for handling new files.
C. Incorrect as the File Arrival trigger does not pass the specific file path to Auto Loader in a way that would allow skipping the source path configuration in the code. Auto Loader's configuration still requires defining the source path to know where to look for files, even though it dynamically processes new arrivals.
D. Incorrect because disabling Auto Loader's checkpointing would lead to **reprocessing of all files** (not just new ones) every time the job runs, which is **inefficient** and contrary to the goal of a cost-optimized pipeline. Checkpointing is essential for ensuring each file is processed exactly once.
Conclusion

The technical relationship between the File Arrival trigger and Auto Loader in this integration pattern is one of **initiation and independent, checkpoint-driven ingestion**, respectively, making **B** the correct answer.

References

Databricks Documentation - Auto Loader
Databricks Documentation - Lakeflow Triggers

Question: 236 CertyIQ

A data engineer is transforming a bronze table containing API response data into a silver table. The bronze table has a user_profile column of type STRING that contains JSON data. An example value is:

```
'{"user_id": "12345", "name": "John Smith", "age": 32, "email": "john@example.com"}'
```

The silver table needs to make this data easily queryable for analytics without requiring JSON parsing in every downstream query.
Which approach standardizes this column for the silver table?

A.

```
SELECT
  get_json_object(user_profile, 'user_id') AS user_id,
  get_json_object(user_profile, 'name')    AS name,
  get_json_object(user_profile, 'age')     AS age,
  get_json_object(user_profile, 'email')   AS email
FROM bronze_table;
```

B.

```
SELECT
  user_profile.user_id            AS user_id,
  user_profile.name               AS name,
  CAST(user_profile.age AS INT)   AS age,
  user_profile.email              AS email
FROM bronze_table;
```

C.

```
SELECT
  from_json(
    user_profile,
    'user_id STRING, name STRING, age STRING, email STRING'
  ) AS parsed_profile
FROM bronze_table;
```

D.

```
SELECT
  get_json_object(user_profile, '$.user_id') AS user_id,
  get_json_object(user_profile, '$.name')    AS name,
  CAST(get_json_object(user_profile, '$.age') AS INT) AS age,
  get_json_object(user_profile, '$.email')   AS email
FROM bronze_table;
```

Answer: D

Question: 237 CertyIQ

A data engineer is troubleshooting two different pipeline failures:
•Pipeline A fails with a java.lang.OutOfMemoryError is immediately thrown after the command display(df.collect()) is called on a 100GB dataset.
•Pipeline B fails during a wide transformation (a join of two large tables) with an ExecutorLostFailure error message, indicating executor memory exhaustion during shuffle.
Which action should the data engineer take to fix these two issues?

A. For Pipeline A, switch to a Storage Optimized cluster; for Pipeline B, use a Compute Optimized cluster.

B. For Pipeline A, increase the spark.driver.memory; for Pipeline B, increase the spark.sql.shuffle.partitions or executor memory.

C. For Pipeline A, increase the number of executors; for Pipeline B, increase the driver memory.

D. For both pipelines, enable Adaptive Query Execution (AQE) as it automatically manages memory allocation for both the driver and executors.

Answer: B

Explanation:

Technical Justification for Correct Answer: B

Pipeline A: java.lang.OutOfMemoryError after df.collect() on a 100GB dataset

Issue Root Cause: The error occurs because df.collect() attempts to load the entire 100GB dataset into the **driver's memory**. The dataset exceeds the driver's allocated memory, causing the OutOfMemoryError.
Correct Action (B): Increase spark.driver.memory. This directly addresses the root cause by allocating more memory to the driver to accommodate the collected data.
Why Other Options are Less Suitable:

A (Storage Optimized cluster): While useful for storage needs, this doesn't directly address the driver's memory issue with collecting data.
C (Increase number of executors): More executors won't reduce the memory required by the driver for collect().
D (AQE for both): While AQE can help with query optimization, it doesn't prevent the driver from running out of memory when collecting large datasets.

Pipeline B: ExecutorLostFailure due to memory exhaustion during shuffle (wide transformation(join))

Issue Root Cause: The error indicates that **executors** ran out of memory during a shuffle operation, common in wide transformations like joins of large tables. This is due to either insufficient executor memory or inefficient partitioning leading to memory overflow.
Correct Action (B):

Increase spark.sql.shuffle.partitions: Reduces the size of each partition, making shuffles more memory-efficient.
OR Increase executor memory: Directly addresses the memory shortage if partitions are already optimized.
Why Other Options are Less Suitable:

A (Compute Optimized cluster for B): While compute optimized might help with processing power, the core issue is memory during shuffle, which this doesn't directly solve.
C (Increase driver memory for B): The issue is with executor memory, not the driver.
D (AQE for both): AQE can help with dynamic partitioning and memory management, but in cases of severely undersized executor memory or poorly planned partitions, explicit adjustment as in option B is more directly effective.

Correct Answer Justification Summary

For Pipeline A, increasing spark.driver.memory (Option B) is the most direct and effective solution to the OutOfMemoryError caused by df.collect() on a large dataset.
For Pipeline B, adjusting either spark.sql.shuffle.partitions or executor memory (Option B) is the best approach to resolve executor memory exhaustion during shuffles, making it the correct choice for both scenarios.

References

1. **Apache Spark Documentation - Tuning Spark** (for memory configuration best practices): https://spark.apache.org/docs/latest/tuning.html

2. **Databricks Documentation - Adaptive Query Execution (AQE)** (for understanding AQE's capabilities and limitations): https://docs.databricks.com/workbook/archive/jobsIEW.html#adaptive-query-execution

Question: 238 CertyIQ

A data engineering team needs to integrate two data sources into Databricks:
•Clickstream Events: Ingesting 5,000 events per second from an Apache Kafka topic.
•Customer Master Data: Syncing only changed records every 4 hours from a Snowflake database.
The solution must meet two key requirements:
•Process the streaming clickstream data with a latency of under 30 seconds.
•Prevent the reprocessing of customer master data that has not changed.
Which data ingestion approach meets these requirements?

A. Use Structured Streaming for Kafka and a Lakeflow Connect standard connector with full refresh for Snowflake.

B. Use Structured Streaming for Kafka and Lakeflow Connect managed connector with incremental mode for Snowflake.

C. Use spark.readStream() with Kafka and query Snowflake hourly with a time-based filter.

D. Use a non-incremental Snowflake connector with polling and fetch all data every 4 hours with MERGE operations.

Answer: B

Explanation:

Technical Justification for Correct Answer B

The correct approach to meet the specified requirements is **Option B: Use Structured Streaming for Kafka and Lakeflow Connect managed connector with incremental mode for Snowflake**. Here's why:

Why Option B is the Best Choice

Structured Streaming for Kafka:

Meets the low-latency requirement (< 30 seconds) for processing streaming clickstream data, as Structured Streaming is designed for high-throughput and low-latency processing.
Efficiently handles the high ingestion rate of 5,000 events per second.
Lakeflow Connect Managed Connector with Incremental Mode for Snowflake:

Prevents Redundant Reprocessing: Incremental mode ensures only changed records are synced, meeting the requirement to prevent reprocessing unchanged customer master data.
Managed Connector implies automated handling of complexities like retries, partitioning, and potentially schema evolution, making the operation more reliable and less maintenance-intensive compared to standard connectors.
Why Other Options are Less Suitable

Option A:

Full Refresh with a standard connector for Snowflake would reprocess all data every 4 hours, violating the requirement to only sync changed records.
Option C:

spark.readStream() with Kafka is less efficient for high-throughput streaming compared to Structured

Streaming.
Hourly Query with Time-Based Filter on Snowflake does not inherently prevent reprocessing unchanged data and may not be as efficient as incremental syncing.
Option D:

Non-Incremental Snowflake Connector with polling and **Fetch All Data** every 4 hours explicitly violates the requirement to only process changed records.
MERGE Operations might manage data state but do not address the inefficient full data fetch.
Additional Considerations for Precision

Incremental Mode in Option B assumes the presence of a change tracking mechanism in Snowflake (e.g., a last_updated timestamp or Change Data Capture (CDC) if supported and configured). This is a common prerequisite for incremental data loading and is feasible with Snowflake's capabilities.
Managed vs. Standard Connector: The managed aspect of the connector in Option B implies better integration with Databricks' ecosystem, potentially offering more seamless performance tuning and error handling compared to a standard connector.
References

Databricks Structured Streaming Documentation
Databricks Lakehouse Connect (formerly Delta Lake Connect) for Snowflake

Question: 239 CertyIQ

A data engineer configures a Databricks Lakeflow Job for daily customer data processing:
•Entry task: Single notebook loads raw data
•Parallel tasks:
o SQL query task for cleansing
o Notebook task for feature engineering
o Pipeline task for model updates
•Exit task: Dashboard refresh (must run after all parallel tasks complete)
•Requirement: Implement this dependency pattern using a DAG-based task graph
Which task configuration ensures all parallel tasks complete before the dashboard refresh task runs?

A. Create separate Lakeflow Jobs for each parallel task and trigger them sequentially using external orchestration.
B. Add the dashboard task as dependent on all three parallel tasks using fan-in control flow.
C. Configure the parallel tasks with sequential dependencies so each task waits for the previous one to finish.
D. Set the dashboard task to depend only on the SQL query task; other tasks run independently.

Answer: B

Explanation:

Technical Justification for Correct Answer: B

The correct configuration to ensure all parallel tasks complete before the dashboard refresh task runs in a Databricks Lakeflow Job, utilizing a DAG-based task graph, is **B. Add the dashboard task as dependent on all three parallel tasks using fan-in control flow**. Here's why:

Why B is Correct

Fan-in Control Flow: This approach directly implements a dependency pattern where the exit task (dashboard refresh) is configured to depend on the successful completion of all three parallel tasks (SQL query for cleansing, Notebook for feature engineering, and Pipeline for model updates). This ensures the dashboard

only refreshes after all prerequisite tasks are done.
DAG Integrity: It maintains the integrity of the DAG by visually and logically representing the dependencies within a single Lakeflow Job, making the workflow transparent and manageable.
Efficiency and Scalability: All parallel tasks can indeed run concurrently (as implied by their parallel nature), and the dashboard task will wait for all, maximizing the efficiency of the pipeline without unnecessary sequencing of independent tasks.
Why Other Options are Less Suitable

A. Separate Lakeflow Jobs with External Orchestration

Increased Complexity: Introduces external dependencies and complexity in management.
Visibility and Control: Blurs the workflow's transparency since the entire pipeline's status isn't encapsulated within a single Job.
C. Sequential Dependencies for Parallel Tasks

Inefficient Use of Parallelism: Defeats the purpose of running tasks in parallel by forcing a sequential execution.
Potential for Longer Execution Times: Increases overall job duration unnecessarily.
D. Dependency Only on SQL Query Task

Insufficient Dependency: Fails to account for the completion of feature engineering and model update tasks before refreshing the dashboard.
Potential for Incomplete Data: Dashboard might refresh with incomplete or outdated data if not all prerequisite tasks have finished.
Key Technical Points

DAG-Based Task Graphs are ideal for expressing complex dependencies between tasks.
Fan-in Control Flow is a pattern where a single task depends on multiple preceding tasks, perfect for aggregating outputs or waiting for all prerequisites to complete before proceeding.
Concurrency in Lakeflow Jobs is optimized when independent tasks are allowed to run in parallel, with dependencies clearly defined for tasks that require predecessors to complete.
References

Databricks Documentation: Lakehouse Pipelines (Jobs)
Databricks Documentation: Control Flow in Jobs

Question: 240 CertyIQ

A data engineer has a PySpark DataFrame events_df which contains a nested device struct, which itself includes a nested location struct, as shown:

```
event_id STRING,
device STRUCT<id: STRING, model: STRING, location: STRUCT<latitude: DOUBLE, longitude: DOUBLE>>,
event_ts TIMESTAMP
```

The goal is to flatten the nested fields into root-level columns while keeping the event identifiers and timestamp. Which PySpark expression achieves this?

A.
```
events_df.select("event_id", "event_ts", "device.id", "device.model", "device.location.latitude",
"device.location.longitude")
```

B.
```
events_df.withColumn("device_id",    events_df["device.id"]).withColumn("device_model",
events_df["device.model"]).withColumn("location", events_df["device.location"])
```

C.

```
events_df.select("event_id", "device[id]", "device[model]", "device.location[latitude]",
"device.location[longitude]", "event_ts")
```

D.

```
events_df.select("event_id", "event_ts", "device.*")
```

Answer: A

Question: 241 CertyIQ

A data engineer has developed a Python notebook in a Databricks workspace and has configured it to run as a scheduled Job to process daily sales data. How are the storage and execution of this notebook managed within the Databricks architecture?

A. The notebook is stored securely and encrypted in the control plane, and the code executes in the compute plane when the job runs.

B. The notebook is stored securely and encrypted in the compute plane, and the code executes in the control plane when the job runs.

C. The notebook is stored securely and encrypted in Unity Catalog, and the code executes in Delta Lake when the job runs.

D. The notebook is stored unencrypted in the workspace storage bucket, and the code executes on the cluster driver node in the compute plane when the job runs.

Answer: A

Explanation:

Technical Justification for Correct Answer A

The correct answer, **A**, accurately reflects the Databricks architecture's handling of notebook storage and execution for scheduled Jobs. Here's why:

Storage Aspect: The notebook is stored securely and encrypted in the **control plane**. This is because the control plane in Databricks is responsible for managing the workspace's metadata, notebooks, and other operational configurations. The control plane provides enterprise-grade security features, including encryption at rest, to protect these assets.
Execution Aspect: When the scheduled Job runs, the code executes in the **compute plane**. The compute plane is where the actual data processing occurs, utilizing Apache Spark clusters that can be dynamically provisioned and de-provisioned as needed. This separation ensures scalability and efficiency in resource utilization.

Why Other Options are Less Suitable:

B is incorrect because storing notebooks in the **compute plane** would not be secure or efficient, as compute resources are meant for processing, not long-term storage of code artifacts. Additionally, executing code in the **control plane** would mix operational and compute workloads, potentially impacting performance and security.

C is misleading. While **Unity Catalog** does provide secure governance and metadata management, and **Delta Lake** offers a storage format for data, the question pertains to notebook storage and execution, not data storage or metadata governance. Notebooks themselves are not stored in Unity Catalog.

D is incorrect for two reasons:

Storage: Storing notebooks **unencrypted** in a **workspace storage bucket** (implying a lack of encryption) contradicts standard security practices Databricks enforces.
Execution: While it's true that code executes on the **cluster driver node** in the compute plane, the storage implication of this option is incorrect, and the driver node detail alone doesn't justify selecting this incorrect storage method.

Key Points in Bullet Form for Quick Reference:

Correct (A):

Storage: Secure, Encrypted in **Control Plane**
Execution: In **Compute Plane** when the job runs
Incorrect Options:

B: Misallocates storage and execution planes.
C: Relates to data/storage governance, not notebook management.
D: Fails on security (unencrypted storage) and partially correct but misleading on execution details.

References

For deeper dives into Databricks Architecture and Security:

1. **Databricks Architecture Overview**: https://docs.databricks.com/getting-started/overview.html#architecture
2. **Databricks Security and Compliance**: https://docs.databricks.com/security/index.html

Question: 242 CertyIQ

A data engineering team uses Declarative Automation Bundles (formerly Databricks Asset Bundles) to deploy the same codebase across dev, test, and prod environments. The team wants environment-specific behavior to be applied only through bundle configuration, without modifying notebooks, job definitions, or deployment logic during promotion.
Which approach applies environment-specific configuration while meeting this requirement?

A. Use separate Git branches per environment so each branch can contain environment-specific configuration values.

B. Define bundle variables and reference them from bundle resources, then override variable values per environment using bundle targets.

C. Parameterize environment-specific values inside notebooks and set those parameters when each job run is triggered.

D. Deploy the same bundle once and edit job settings in the workspace UI for each environment after deployment.

Answer: B

Explanation:

Technical Justification for Correct Answer: B

Why B is the Best Approach:

Meets the Requirement Directly: Option B allows for environment-specific behavior through bundle configuration alone, without altering notebooks, job definitions, or deployment logic across environments.
Utilization of Declarative Automation Bundles' Intent: Defining bundle variables and overriding them via

targets is a core feature of Declarative Automation Bundles, designed for such use cases.
Scalability and Maintainability: This approach scales well with the number of environments and variables, keeping configurations centralized and manageable.
Version Control Transparency: Since variable values are overridden at the target level (outside the codebase), the entire configuration process remains transparent in version control, without mixing environment-specific code.

Why Other Options are Less Suitable:

A. Separate Git Branches per Environment

Violates Single Source of Truth Principle: Maintaining environment-specific branches complicates code synchronization and increases merge conflicts.
Configuration Not Isolated from Code: Environment configurations are intertwined with code, contradicting the requirement.
C. Parameterize Inside Notebooks

Tight Coupling with Job Triggering Logic: Requires setting parameters at job trigger time, potentially introducing deployment logic changes across environments.
Less Centralized Configuration Management: Makes it harder to oversee and modify environment-specific settings in one place.
D. Post-Deployment Editing in Workspace UI

Deployment Logic Alteration: Involves changing job settings post-deployment, which alters deployment logic across environments.
Lack of Automation and Versioning: Manual edits in the UI are error-prone and not version-controlled in the same way as bundle configurations.

Correct Answer: B. Define bundle variables and reference them from bundle resources, then override variable values per environment using bundle targets.

References

1. **Databricks Documentation - Declarative Automation Bundles**: https://docs.databricks.com/dev-tools/declarative-automation-bundles.html
2. **Databricks Documentation - Bundle Targets and Variables**: https://docs.databricks.com/dev-tools/declarative-automation-bundles/bundle-targets-variables.html

Question: 243 CertyIQ

A data engineer observes that an ETL job in Databricks has increased its execution time from 5 minutes (baseline) to 12 minutes over the past week. Using the Lakeflow Jobs run history view, the engineer needs to identify whether the slowdown is consistent or intermittent and determine the root cause.
Which action should the engineer take to analyze execution time trends against the baseline?

A. Compare cluster metrics in Lakeflow Jobs run history to spot resource spikes causing delays.

B. Archive old job runs and resets the baseline counter to ensure accurate comparison of current performance.

C. Rerun the job immediately to establish a new baseline since historical data becomes irrelevant after seven days.

D. Increase cluster size automatically whenever execution time exceeds the previous run's duration.

Answer: A

Explanation:

Technical Justification for Correct Answer A

The correct action for the data engineer to analyze execution time trends against the baseline is **A. Compare cluster metrics in Lakeflow Jobs run history to spot resource spikes causing delays**. Here's why:

Why A is the best option:

Direct Root Cause Analysis: Comparing cluster metrics (e.g., CPU usage, memory utilization, input/output operations) against the baseline execution time in Lakeflow Jobs run history allows for direct identification of resource bottlenecks that could be causing the slowdown.
Trend Analysis: This approach enables the engineer to determine if the slowdown is consistent (indicating a sustained resource issue) or intermittent (suggesting sporadic resource spikes or other transient issues).
Data-Driven Decision Making: Insights from this comparison inform targeted optimizations (e.g., resource allocation adjustments) without prematurely altering the job's configuration or data set.

Why other options are less suitable:

B. Archive old job runs and reset the baseline counter:

Loss of Historical Context: Archiving old runs eliminates valuable data necessary for trend analysis.
Invalid Baseline Reset: Resetting the baseline without understanding the root cause does not address the performance issue and skews future comparisons.

C. Rerun the job to establish a new baseline:

Lack of Diagnostic Insight: Immediate rerun without analysis does not identify the cause of the slowdown.
Assumption of Irrelevant Historical Data: Historical data remains relevant for understanding performance degraded over time; assuming it's irrelevant after seven days is not a best practice without a specific retention policy justification.

D. Increase cluster size automatically based on execution time thresholds:

Blind Scalability Approach: Automatically increasing cluster size without diagnosing the root cause may not address the underlying issue (e.g., inefficient code, data growth) and could lead to unnecessary resource expenditure.
Lack of Insight into Consistency of Slowdown: This method does not differentiate between consistent and intermittent slowdowns, potentially over-scaling for transient issues.

References

1. **Databricks Lakehouse Documentation - Lakeflow Jobs**:https://docs.databricks.com/lakehouse/lakeflow/jobs.html

   Relevant Section: Job Runs and Metrics Analysis

2. **Databricks Knowledge Base - Optimizing ETL Job Performance**:https://docs.databricks.com/kb/jobs/optimize-etl-job-performance.html

   Relevant Section: Analyzing Job Metrics for Bottlenecks

Question: 244 CertyIQ

A data engineer maintains a Declarative Automation Bundle (formerly Databricks Asset Bundle), which deploys a dashboard.

The `databricks.yml` includes:

```
bundle:
  name: workspace_assets

variables:
  dataset_catalog:
    default: catalog_dev

targets:
  dev:
    variables:
      dataset_catalog: catalog_dev
  prod:
    variables:
      dataset_catalog: catalog_prod
```

Which set of actions will enable the dashboard to be deployed in prod using catalog_prod without changing the resource definition?

A. Run databricks bundle deploy --profile prod so the CLI selects targets.prod and applies dataset_catalog: catalog_prod automatically.

B. Run databricks bundle deploy --target prod so the deployment uses targets.prod and overrides dataset_catalog to catalog_prod.

C. Run databricks bundle execute --profile prod so the CLI selects targets.prod and applies dataset_catalog: catalog_prod automatically.

D. Run databricks bundle deploy --var dataset_catalog=catalog_prod so the CLI selects targets.prod and applies dataset_catalog: catalog_prod automatically.

Answer: B

Question: 245 CertyIQ

A data engineering team is designing the Gold layer in their Unity Catalog-governed lakehouse for downstream BI and analytics users. The team wants to expose business-ready metrics with fast query performance and consistent definitions, while keeping the transformation logic in Spark notebooks.
Which type of Gold layer object meets this requirement?

A. A materialized view that precomputes aggregations from Silver tables on a schedule and is queried by BI tools for faster, consistent analytics.

B. A streaming table that continuously ingests raw events from source systems and exposes them directly to BI users.

C. A view that directly references raw Bronze tables so BI tools can define their own joins and aggregations.

D. A collection of CSV files stored in a cloud storage object. BI users will parse the files to query the gold layer data.

Answer: A

Explanation:

Correct Answer Justification: A. A materialized view that precomputes aggregations from Silver tables on a schedule and is queried by BI tools for faster, consistent analytics.

Why A is the Best Choice

Precomputed Aggregations for Fast Query Performance: Materialized views (MV) precompute aggregations from Silver tables on a schedule, ensuring that downstream BI and analytics queries are extremely fast, as they don't need to compute aggregations on the fly.
Consistent Definitions: By defining the transformation logic in Spark notebooks that populate the MV, the team ensures that business metrics are consistently applied across all users, avoiding analysis paralysis from disparate results.
Separation of Logic and Consumption: Keeping the transformation logic in Spark notebooks (for maintaining the MV) separates the complexity of data transformation from the simplicity required for BI consumption, aligning with the principle of separating concerns.
Unity Catalog Governance: Materialized views can be effectively governed within Unity Catalog, providing the necessary security, access control, and metadata management for the exposed business metrics.
Why Other Options are Less Suitable

B. Streaming Table:

Incorrect Layer Target: Streaming tables are more suited for capturing raw or lightly processed data (Bronze/Silver layers) rather than precomputed, business-ready metrics.
Latency and Complexity for BI: Directly consuming streaming tables for analytics can introduce latency and complexity not suited for typical BI use cases.
C. View Referencing Bronze Tables:

Performance Issue: Views that directly reference Bronze tables would require BI tools to perform all joins and aggregations, leading to slow query performance.
Inconsistent Definitions: Allowing BI tools to define their own joins and aggregations can lead to inconsistencies in business metric calculations.
D. Collection of CSV Files:

Scalability and Performance Concerns: Parsing CSV files for query purposes does not scale for large datasets and introduces significant performance overhead.
Governance and Security Challenges: Managing access and security through CSV files in cloud storage is less integrated and more challenging compared to Unity Catalog's capabilities.
References

For deeper dives into the technologies mentioned:

1. **Databricks Materialized Views Documentation**: https://docs.databricks.com/sql/data-warehousing/materialized-views.html
2. **Unity Catalog Overview**: https://docs.databricks.com/governance/unity-catalog/overview.html

Question: 246 CertyIQ

A company has a strict 15-minute Service Level Agreement (SLA) for updating its currency exchange dashboard. The source data arrives in small increments every few minutes. The data engineering team needs a triggering strategy for their Lakeflow Job that keeps end-to-end latency within the SLA while minimizing compute cost and

DBU consumption.
Which triggering strategy for the Lakeflow Job is the recommended way to meet this 15-minute SLA?

A. Use a Scheduled trigger set to run every 12 minutes, with the streaming task configured to use Trigger.AvailableNow.

B. Use a Scheduled trigger set to run every 1 minute to ensure the SLA is never at risk.

C. Use a File Arrival trigger configured to fire every time a single JSON file lands in the bucket.

D. Use a Continuous trigger to ensure data is processed as soon as it arrives.

Answer: A

Explanation:

Justification for Correct Answer: A

The recommended triggering strategy for the Lakeflow Job to meet the 15-minute SLA while minimizing compute cost and DBU (Databricks Unit) consumption is **A. Use a Scheduled trigger set to run every 12 minutes, with the streaming task configured to use Trigger.AvailableNow**. Here's why:

SLA Compliance with Efficiency: Running every 12 minutes ensures that the end-to-end processing latency remains well within the 15-minute SLA, providing a buffer for processing time variations. This schedule also reduces the number of executions compared to more frequent triggers, directly impacting cost and DBU consumption.

Trigger.AvailableNow Optimization: Configuring the streaming task with Trigger.AvailableNow means the job will only execute when new data is available since the last run. This avoids unnecessary executions (and thus saves compute resources) on iterations where no new data has arrived, unlike simple scheduled runs without this check.

Why Other Options are Less Suitable:

B. Scheduled trigger every 1 minute:

Excessive Compute Cost/DBU Consumption: Running every minute would significantly increase costs and DBU usage without necessarily providing assurance beyond what the 12-minute schedule already offers within the SLA.
Diminishing Marginal Benefit: The SLA is comfortably met with less frequent runs, making this option inefficient.

C. File Arrival trigger for every single JSON file:

Unpredictable and Potentially High Trigger Frequency: Depending on the arrival rate of JSON files, this could lead to very frequent executions, increasing costs and potentially overwhelming the system.
Lack of Batch Processing Efficiency: Processing each file individually may not leverage batch processing efficiencies, potentially increasing overall latency for a full dataset update.

D. Continuous trigger:

Definition Misalignment: A "Continuous" trigger in the context of Databricks typically refers to a trigger type for continuous streaming jobs, which process data as it arrives in a highly frequent or constant stream. This doesn't align well with the described scenario of data arriving in "small increments every few minutes," suggesting a more batch-like streaming scenario.
Resource Intensive: Continuous triggers are resource-intensive and suited for real-time processing needs far beyond the specified 15-minute SLA, making them unnecessary for this use case.
References

Databricks Documentation: Lakehouse Trigger Types
Databricks Documentation: Best Practices for Scheduling Jobs

Question: 247 CertyIQ

A data engineering team ingests customer transaction data from three enterprise sources (SQL database, S3, and Kafka stream) into Unity Catalog tables. The data must maintain lineage and support compliance audits requiring historical snapshots.
Which Lakeflow Connect configuration satisfies both the governance and audit requirements?

A. Single Lakeflow connector with all three sources, direct Delta writes, and disabled table versioning to reduce storage costs.

B. Separate connectors per source with CDC, individual Unity Catalog schemas per source, and table versioning enabled for audit compliance.

C. Separate connectors with full-load sync only, archival of historical data, and external tables for audit queries.

D. Single connector with batch scheduling, shared Unity Catalog schema, and incremental snapshots stored externally.

Answer: B

Explanation:

Technical Justification for Correct Answer: B

Why B is the Best Option:

Separate Connectors per Source: Utilizing separate connectors for each of the three distinct data sources (SQL database, S3, and Kafka stream) ensures that the ingestion process can be customized and optimized for each source's specific characteristics (e.g., handling CDC for SQL, batch for S3, and streaming for Kafka), enhancing reliability and flexibility.
Change Data Capture (CDC): Enabling CDC for the SQL database (and potentially applicable for other sources if supported) captures incremental changes, which is crucial for maintaining accurate lineage and historical snapshots without unnecessary full loads, thus optimizing storage and processing resources.
Individual Unity Catalog Schemas per Source: Assigning separate schemas for each source within Unity Catalog enhances organization, reduces naming conflicts, and improves governance by keeping data sources logically segregated, facilitating easier audit trails and compliance checks.
Table Versioning Enabled: This is paramount for audit compliance as it automatically maintains historical snapshots of the data, allowing for point-in-time queries and fulfilling the requirement for historical data tracking and compliance audits.

Why Other Options are Less Suitable:

A. Single Connector with Disabled Table Versioning

Inadequate for Diverse Sources: A single connector may not optimize ingestion for all three sources' different natures.
Disabled Table Versioning: Fails to meet the historical snapshot requirement for audits.
C. Separate Connectors with Full-Load Sync Only, Archival, and External Tables

Full-Load Sync Only: Inefficient for sources with frequent updates (e.g., Kafka stream), leading to higher storage and compute costs.
Archival and External Tables for Audits: Adds complexity and potentially higher costs compared to built-in table versioning.

D. Single Connector with Batch Scheduling, Shared Schema, and External Snapshots

Single Connector: Similar drawbacks as in option A regarding source optimization.
Shared Schema: Increases complexity in governance and audit tracing.
External Snapshots: Less efficient and more costly than leveraging Unity Catalog's native table versioning for historical data management.

Key Technical Points Supporting B:

Governance: Separation by source and schema supports clear lineage and easier compliance.
Audit Requirements: CDC and table versioning directly address the need for historical snapshots.
Efficiency: Optimized ingestion processes for each source type reduce costs and improve performance.

References:

1. **Databricks Unity Catalog Documentation** - For detailed information on schemas, table versioning, and governance features: https://docs.databricks.com/unified-catalog/
2. **Databricks Lakehouse Pipelines (Lakeflow Connect) Documentation** - For connector configurations, CDC, and optimization strategies: https://docs.databricks.com/workflows/lakehouse-pipelines.html

Question: 248 CertyIQ

A data engineer needs to support a Unity Catalog Delta table catalog.schema.sales that is currently registered as an external table. New requirements state:
•Automatic file cleanup after a drop is handled by the platform.
•Minimal downtime during migration while keeping the same table name, permissions, and history.
•Enforced name-based access through Unity Catalog (no path-based access).
Which action should the data engineer take to meet the requirements?

A. Run ALTER TABLE catalog.schema.sales SET MANAGED to convert and meet the goals
B. Run DROP TABLE catalog.schema.sales PURGE to delete files and meet goals
C. Run ALTER TABLE catalog.schema.sales SET LOCATION to switch external paths now
D. Create a new managed table with CTAS and migrate workloads to that table today

Answer: A

Explanation:

Here's a structured technical justification for the correct option in paragraph format, followed by bullet points explaining why other options are less suitable:

Correct Option Justification (A)

Running ALTER TABLE catalog.schema.sales SET MANAGED is the most appropriate action to meet the new requirements. This command converts the existing external table into a managed table within Unity Catalog. By doing so, **automatic file cleanup after a drop** is ensured, as managed tables rely on the platform for lifecycle management, including cleanup. This approach also allows for **minimal downtime** during migration since the table name remains unchanged, preserving existing references. Crucially, managing the table through Unity Catalog **enforces name-based access control**, eliminating the possibility of path-based access, which aligns with the requirement for enforced name-based access through Unity Catalog. Unity Catalog's managed tables inherently support this enforcement by managing access through the catalog's metadata, not file system paths. Lastly, the **table's history and permissions** are preserved, ensuring continuity in access rights and query history.

Why Other Options Are Less Suitable:

B. Run DROP TABLE ... PURGE:

Data Loss: This permanently deletes the table and its data, which is not the goal.
Downtime and Recreation Needed: Requires recreating the table, leading to more downtime and manual effort to restore permissions and history.
No Direct Benefit for Future Management: Doesn't address the need for managed file cleanup or access control in the future.

C. Run ALTER TABLE ... SET LOCATION:

No Change in Management Type: Keeps the table as external, not addressing the automatic cleanup or enforced name-based access through Unity Catalog.
Path-Based Access Remains: Fails to meet the access control requirement.

D. Create a New Managed Table with CTAS and Migrate:

Higher Downtime: Involves more complex migration steps, potentially leading to higher downtime.
Manual Permission and History Transfer: Requires manual effort to replicate permissions and history, increasing the chance of errors.
Redundant Step: Unnecessary creation of a new table when the existing one can be converted.

References

Databricks Documentation: Managed Tables
Databricks Documentation: Unity Catalog - Access Control

Question: 249 CertyIQ

A data engineer has finished developing a new Declarative Automation Bundle (formerly Databricks Asset Bundle) for a streaming pipeline. They need to ensure the configuration is syntactically correct, push the code to the workspace, and finally trigger the pipeline to verify it runs as expected.
Which sequence of Databricks CLI commands should the engineer use to perform these steps in the correct order?

A. databricks bundle init -> databricks bundle deploy -> databricks bundle run
B. databricks bundle validate -> databricks bundle deploy -> databricks bundle run
C. databricks bundle validate -> databricks bundle sync -> databricks bundle execute
D. databricks bundle check -> databricks bundle upload -> databricks bundle start

Answer: B

Explanation:

Technical Justification for Correct Answer B

The correct sequence of Databricks CLI commands for ensuring syntactical correctness, deploying the code, and triggering the pipeline for a Declarative Automation Bundle (formerly Databricks Asset Bundle) in a streaming pipeline is **B. databricks bundle validate -> databricks bundle deploy -> databricks bundle run**. Here's why this option is best and why others are less suitable:

Why B is Correct

databricks bundle validate: This command checks the configuration for syntactical errors before any

deployment, ensuring the bundle is correct at the outset. This step is crucial for catching errors early.
databricks bundle deploy: After validation, deploying the bundle to the Databricks workspace is the logical next step. This command pushes the validated configuration to the workspace.
databricks bundle run: Finally, executing the pipeline verifies that it runs as expected in the Databricks environment, completing the cycle of validate-deploy-execute.
Why Other Options are Less Suitable

A. databricks bundle init -> databricks bundle deploy -> databricks bundle run:

init is used to initialize a new bundle, not to validate an existing one. It's inappropriate for this scenario where the bundle is already developed.
The sequence misses a validation step, risking deployment of potentially faulty configurations.

C. databricks bundle validate -> databricks bundle sync -> databricks bundle execute:

sync might imply syncing changes, but in the context of deploying a bundle for the first time or ensuring it's correctly set up for execution, **deploy** is more directly applicable and clear in intent than sync.
execute is less commonly referenced in Databricks CLI documentation for this specific action compared to run, which is more standard for triggering jobs/pipelines.

D. databricks bundle check -> databricks bundle upload -> databricks bundle start:

check is not a standard command for syntactical validation in the context provided; **validate** is more appropriate.
upload and **start** are not the preferred or standard commands for deploying and running a Declarative Automation Bundle in Databricks. **deploy** and **run** are more commonly used and documented actions.
Conclusion

Option **B** provides the most appropriate, syntactically correct, and logically sequenced commands for validating, deploying, and executing a Declarative Automation Bundle in Databricks, aligning with best practices for pipeline deployment and testing.

References

1. **Databricks Documentation - Declarative Automation Bundles**: https://docs.databricks.com/os/in-place-repos/declarative-automation-bundles.html
2. **Databricks CLI Reference**: https://docs.databricks.com/cli/reference/ (Search within for specific commands like validate, deploy, run)

Question: 250 CertyIQ

A data engineer has configured a Lakeflow Job that runs daily to ingest customer transaction data from a legacy relational database. The extracted data must be written directly to a Unity Catalog table and be immediately queryable via SQL. The team needs to ensure full data lineage is preserved.
Which action enables this data ingestion with direct landing in Unity Catalog, preserved lineage, and immediate SQL query ability?

A. Stage data in cloud storage via REST API, then load with a separate job
B. Use spark.read.format("jdbc") and write with saveAsTable("catalog.schema.table")
C. Extract to Pandas DataFrame, convert to Spark, then save() to external path
D. Use ODBC pooling to extract all records in one batch and write to /dbfs/

Answer: B

Explanation:

Technical Justification for Correct Answer: B

The correct action to enable data ingestion with direct landing in Unity Catalog, preserved lineage, and immediate SQL query ability is **B. Use spark.read.format("jdbc") and write with saveAsTable("catalog.schema.table")**. Here's why:

Direct Landing in Unity Catalog: saveAsTable("catalog.schema.table") directly writes the data into a Unity Catalog table, fulfilling the requirement without intermediate storage needs.
Preserved Lineage: Spark's unified analytics engine, when used with Unity Catalog, inherently tracks data lineage as part of its metadata management. This ensures full lineage is preserved from the source (legacy DB) to the target (Unity Catalog table).
Immediate SQL Query Ability: Tables created with saveAsTable() are immediately queryable via SQL through Databricks SQL or the Catalyst engine, as they are registered in the metadata catalog.

Why Other Options are Less Suitable:

A. Stage data in cloud storage via REST API, then load with a separate job

Extra Step: Introduces an unnecessary intermediate step, increasing latency.
Lineage: While possible to track, the additional step complicates lineage preservation.
Queryability: Data is not immediately queryable until the second job completes.

C. Extract to Pandas DataFrame, convert to Spark, then save() to external path

No Direct Unity Catalog Insertion: Saving to an external path does not directly land data in Unity Catalog.
Lineage and Queryability: Similar drawbacks as option A, with the added inefficiency of DataFrame conversions.

D. Use ODBC pooling to extract all records in one batch and write to /dbfs/

No Direct Unity Catalog Insertion: Writing to /dbfs/ does not register the data in Unity Catalog.
Lineage and Queryability: Manual management would be required for both, defeating the integration benefits.

References

1. **Unified Catalog Documentation**: https://docs.databricks.com/data/unified-catalog/index.html
2. **Spark SQL Read/Writes**: https://spark.apache.org/docs/latest/sql-programming-guide.html#data-sources

Question: 251 CertyIQ

A data engineer is designing a bronze-to-silver pipeline on the Databricks Data Intelligence Platform. The source system sends daily CSV files, and over time, new optional columns are added.
The data engineer wants a storage format and table feature that together ensure the following:
•The table prevents writes that do not conform to the defined schema.
•The table can evolve its schema to include new optional columns without having to manually recreate the table.
•Previous versions of the table can be queried later for debugging and auditing.
Which solution fulfills the requirements?

A. Use a Parquet table with Spark's default schema inference and rely on re-running the job when the schema changes.

B. Use an external table with Auto Loader schema inference for CSV files.

C. Use a Delta table with its native capabilities.

D. Use a Delta table with schema enforcement and recreate the table whenever new columns are added.

Answer: C

Explanation:

Technical Justification for Correct Answer: C

The correct solution is **C. Use a Delta table with its native capabilities**. Here's why this option best fulfills the requirements and why the other options are less suitable:

Why C is Correct

Schema Conformance & Enforcement: Delta tables natively support schema enforcement, preventing writes that do not conform to the defined schema, thus meeting the first requirement.
Schema Evolution: Delta tables inherently support schema evolution. New optional columns can be added without manually recreating the table, fulfilling the second requirement. This is achieved through ALTER TABLE ... ADD COLUMN statements or by leveraging the mergeSchema option when writing data, which automatically adds new columns.
Versioning and Querying Previous Versions: Delta tables provide point-in-time queries, allowing previous versions of the table to be queried for debugging and auditing purposes, satisfying the third requirement. This is facilitated by the _delta_history table and the AS OF and USING TIMESTAMP query options.
Why Other Options are Less Suitable

A. Parquet with Spark's Default Schema Inference

Lacks Native Schema Enforcement: While possible to enforce schema programmatically, it's not a native feature of Parquet tables.
Schema Evolution: Requires manual intervention or re-running jobs, which doesn't seamlessly handle new columns without additional setup.
Versioning: Not natively supported for querying previous versions in the same straightforward manner as Delta tables.

B. External Table with Auto Loader Schema Inference for CSV

Schema Enforcement: Auto Loader infers schema but doesn't natively enforce it against a predefined schema for writes.
Schema Evolution: While Auto Loader can infer new columns, managing these in a controlled, predefined schema evolution process is less straightforward.
Versioning: Similar to Parquet, versioning for querying previous states isn't natively supported in the desired manner.

D. Delta Table with Schema Enforcement & Recreate Table for New Columns

Meets First Requirement: Same as C for schema enforcement.
Fails Second Requirement: Recreating the table for new columns contradicts the requirement for seamless schema evolution without manual table recreation.
Meets Third Requirement: Same as C for versioning.
Conclusion

Option **C** is the best solution because it natively fulfills all specified requirements without the need for additional workflows, manual interventions, or compromises on the desired functionalities.

References

Delta Lake Documentation - Schema Evolution
Databricks Documentation - Delta Tables

Question: 252 CertyIQ

A data engineer is designing a Databricks Job that includes a Notebook task for data cleansing and a SQL Query task for generating a summary report. The summary report must only execute after the cleansing task completes successfully.
How should the dependency be configured in the Lakeflow Jobs task graph?

A. Select the cleansing task as a 'Run if' dependency in the summary report task configuration.
B. Set both tasks to run on the same schedule with a five-minute start delay for the report.
C. Configure the cleansing task to use an 'On Failure' trigger to start the summary report.
D. Place both tasks in the same notebook and use a standard Python function call.

Answer: A

Explanation:

Technical Justification for Correct Answer: A

The correct configuration for ensuring the summary report task executes only after the data cleansing task completes successfully in a Databricks Job's Lakeflow task graph is **A**. Here's why:

Correct Answer: A - Select the cleansing task as a 'Run if' dependency in the summary report task configuration

Dependency Logic: Configuring the cleansing task as a 'Run if' dependency directly establishes a logical sequence where the summary report task is triggered **only if** the preceding cleansing task completes successfully. This aligns perfectly with the requirement for sequential execution based on success.
Explicit Control: This option provides explicit control over the dependency, making the intent clear in the job configuration.
Robustness: It handles the success condition explicitly without assuming timing or indirect triggers.

Why Other Options are Less Suitable:

B - Set both tasks to run on the same schedule with a five-minute start delay for the report

Timing Assumption: Assumes the cleansing task will always complete within five minutes, which is risky and not scalable for variable task durations.
Lack of Direct Dependency: Does not establish a direct success-dependent relationship between tasks.

C - Configure the cleansing task to use an 'On Failure' trigger to start the summary report

Incorrect Trigger Condition: Triggers the report on failure of the cleansing task, which is the opposite of the required behavior (trigger on success).
Logical Misalignment: Directly contradicts the requirement for the report to run after successful cleansing.

D - Place both tasks in the same notebook and use a standard Python function call

Lack of Task Isolation: Combines tasks into a single executable unit, potentially complicating management, debugging, and scalability.

No Native Dependency Management: Relies on in-notebook logic for dependency, which may not integrate well with Databricks Job's native task management and monitoring capabilities.

References

Databricks Documentation - Jobs: Task Dependencies
Databricks Documentation - Jobs: Configuring Tasks

Question: 253 CertyIQ

A notebook task check_volume calculates a daily data volume and writes it to a Lakeflow Jobs taskValue named record_count. If the count is greater than 1 million, the job should run high_volume_pipeline; otherwise, it should run normal_pipeline.
Which control-flow feature should be used?

A. Add an If/else condition task that reads the record_count task value and routes to different downstream tasks.
B. Use a For each task over the record_count and stop when it is below the threshold.
C. Configure additional retries for check_volume until the count exceeds the threshold.
D. Configure Run if conditions based purely on task status.

Answer: A

Explanation:

Technical Justification for Correct Option (A)

The correct control-flow feature to use in this scenario is **A. Add an If/else condition task that reads the record_count task value and routes to different downstream tasks**. Here's why:

Precision in Conditional Logic: The If/else condition task allows for precise conditional logic based on the value of record_count. It can directly check if the count is greater than 1 million and route the job execution to either high_volume_pipeline or normal_pipeline accordingly.
Direct Value Utilization: Since record_count is already a calculated task value, an If/else task can directly leverage this value without needing to reinterpret or re-process the data, making the workflow efficient.
Clear Workflow Design: Incorporating an If/else condition enhances the transparency and manageability of the workflow by explicitly showing the decision-making process and the consequences of that decision (i.e., which pipeline to run).

Why Other Options are Less Suitable:

B. Use a For each task over the record_count and stop when it is below the threshold:

Inapplicability: For each tasks are designed for iterating over collections, not for conditional workflow decisions based on a single value.
Inefficiency: This approach would not directly solve the problem and could lead to unnecessary iterations.
C. Configure additional retries for check_volume until the count exceeds the threshold:

Misalignment with Requirement: The goal is to route based on the count value, not to retry until a condition is met that may never be fulfilled.
Potential for Infinite Loops: If the count never exceeds the threshold, this could lead to continuous retries.
D. Configure Run if conditions based purely on task status:

Lack of Value-Based Decision Making: Run if conditions based on task status (e.g., success/failure) do not

allow for decisions based on the actual value of record_count.
Insufficient for Conditional Workflow: This option does not support the specific value-based routing required by the scenario.

References

For detailed information on Databricks Control Flow tasks, including If/else conditions:
https://docs.databricks.com/workspace/job-scheduling.html#control-flow-tasks
For understanding task values and how they can be utilized in workflows:https://docs.databricks.com/applications/jobs/python-dependencies.html#passing-values-between-tasks

Question: 254 CertyIQ

A data engineer is cleaning a bronze table that receives the same customer records from multiple source systems. Duplicate rows have the same customer_id and email but different ingestion_timestamp values. The silver table should contain only one record per unique combination of customer_id and email.
Which PySpark operation correctly deduplicates based on the business keys?

A. df.dropDuplicates(['customer_id', 'email'])
B. df.groupBy('customer_id', 'email').agg(max('ingestion_timestamp').alias('latest_ts'))
C. df.select('customer_id', 'email').distinct()
D. df.distinct()

Answer: A

Explanation:

Technical Justification for Correct Answer A

The correct operation to deduplicate the bronze table based on the business keys (customer_id and email) is **A. df.dropDuplicates(['customer_id', 'email'])**. Here's why:

Correctness of A: dropDuplicates with a subset of columns specified (['customer_id', 'email']) directly removes duplicate rows based on the unique combination of these two columns. This ensures the silver table contains only one record per unique customer_id and email combination, regardless of the ingestion_timestamp difference. This approach is both efficient and straightforward for achieving the desired deduplication based on specific columns.

Why B is Less Suitable:

Complexity: While groupBy with agg can technically achieve deduplication by selecting a specific ingestion_timestamp (e.g., the max), it introduces unnecessary complexity for a simple deduplication task based on unique identifier combinations.
Additional Logic Required: If the goal is to keep the latest record, this method works but requires additional logic not explicitly requested in the scenario. For simple deduplication without specifying which record to keep (beyond the first encountered), this method is overkill.

Why C is Incorrect:

Loss of Relevant Columns: select followed by distinct only returns the distinct combinations of customer_id and email but loses all other column values (including ingestion_timestamp), making it impractical for populating a silver table that likely requires more than just these two identifiers.

Inability to Determine Record: Like B, if there's a need to determine which of the duplicate records to keep (e.g., based on ingestion_timestamp), this method does not facilitate that decision.

Why D is Incorrect:

Global Uniqueness: df.distinct() removes duplicates based on the entirety of the row, not a subset of columns. This means it would not deduplicate rows differing only by ingestion_timestamp (since the rest of the row, including customer_id and email, would need to be identical for Spark to consider them duplicates).
Not Aligning with Business Keys: The requirement specifies deduplication based on customer_id and email specifically, which distinct() does not accommodate without specifying columns.
References

Apache Spark Documentation - dropDuplicates
Apache Spark Documentation - GroupBy and Agg

Question: 255 CertyIQ

A data engineer ingests semi-structured JSON logs into a Delta table using Auto Loader with schema evolution enabled. A new string field userAgent appears in the JSON.
What happens to the new userAgent field?

A. All existing data is backfilled with a default value for userAgent and made non-nullable.
B. The new userAgent field is added as a new nullable column in the Delta table, populated for new data.
C. Ingestion fails with a schema-mismatch error until the table is altered.
D. The new userAgent field is silently ignored, and its data is dropped for all new files.

Answer: B

Explanation:

Technical Justification for Correct Answer: B

The correct answer is **B. The new userAgent field is added as a new nullable column in the Delta table, populated for new data.** Here's why:

Schema Evolution Enabled: Since schema evolution is enabled with Auto Loader, the system is configured to adapt to changes in the incoming data's schema without manual intervention for supported evolution types (like adding new columns).
Behavior for New Fields: When a new field (like userAgent) appears in the semi-structured JSON logs, the expected behavior with schema evolution is for the Delta table to dynamically accommodate this change. Specifically, the new field is added as a **new column** in the table.
Nullability of New Column: For existing data that does not contain the new userAgent field, the column will be populated with **null** values, making the column **nullable** by default. This ensures backward compatibility and avoids data loss or ingestion failure.
Population for New Data: For all new data ingested after the appearance of the userAgent field, the column will be populated with the actual values from the JSON logs.

Why Other Options are Less Suitable:

A. All existing data is backfilled with a default value for userAgent and made non-nullable.

Incorrect Assumption of Backfilling: Schema evolution with Auto Loader does not automatically backfill existing data with default values for new columns.

Non-Nullable by Default for New Columns: New columns added via schema evolution are nullable to accommodate existing data lacking the new field.
C. Ingestion fails with a schema-mismatch error until the table is altered.

Contradicts Schema Evolution's Purpose: The primary goal of enabling schema evolution is to prevent ingestion failures due to supported schema changes (like adding columns).
D. The new userAgent field is silently ignored, and its data is dropped for all new files.

Inconsistent with Enabled Schema Evolution: With schema evolution enabled, the system should adapt to, not ignore, supported schema changes.

References:

Databricks Documentation - Auto Loader Schema Evolution
Databricks Documentation - Delta Lake Schema Evolution

Question: 256 CertyIQ

A data engineer is tasked with cleaning a bronze table. The requirement is to eliminate rows where either the customer_email or the customer_phone field is null. This data cleaning must be performed in a single operation using a single method call.
Which PySpark approach supports filtering multiple columns for nulls in one call?

A. df.dropna(subset=['customer_email', 'customer_phone'])
B. df.where('customer_email IS NOT NULL').where('customer_phone IS NOT NULL')
C. df.na.drop(how='all')
D. df.filter(col('customer_email').isNotNull() & col('customer_phone').isNotNull())

Answer: A

Explanation:

Technical Justification

The correct option for eliminating rows where either customer_email or customer_phone field is null in a single operation using a single method call is **A. df.dropna(subset=['customer_email', 'customer_phone'])**. Here's why each option stands in terms of suitability for the given requirement:

A. df.dropna(subset=['customer_email', 'customer_phone']):

Correctness: This option directly addresses the requirement by specifying a subset of columns to check for nulls. dropna with subset parameter filters out rows where any of the specified columns are null, fulfilling the task in a single method call.
Efficiency: It is efficient as it's a single operation and leverages PySpark's optimized null handling for specified columns.
B. df.where('customer_email IS NOT NULL').where('customer_phone IS NOT NULL'):

Suitability: While this achieves the desired outcome, it involves **two separate method calls** (chaining two where clauses), violating the "single method call" requirement.
Technicality: Technically correct for null filtering but not compliant with the operational constraint.
C. df.na.drop(how='all'):

Suitability: **Incorrect for the Requirement**. how='all' drops rows only if **all** columns (by default, all columns in

the DataFrame if no subset is provided) are null, which does not meet the criteria of checking specific columns for any nulls.
Misalignment: Fails to target specific columns and applies a stricter condition than required.
D. df.filter(col('customer_email').isNotNull() & col('customer_phone').isNotNull()):

Suitability: Similar to B, **technically correct for the filtering logic** but involves **constructing a filter condition** which, while a single method call, might be considered less direct or less efficient in expressing the intent compared to dropna with a subset for null checks on specific columns.
Efficiency & Readability: Less efficient in terms of readability and intent clarity for null handling on specific columns compared to A.

Conclusion:Option **A** is the best choice because it:

Meets the single method call requirement.
Directly targets the specified columns for null checks.
Is efficient and clearly expresses the intent of dropping rows based on null values in specific columns.

References

1. **Apache Spark Documentation - Drop Rows with Null Values**: https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.sql.DataFrame.dropna.html
2. **Databricks Documentation - Handling Missing Values**: https://databricks.com/blog/2015/11/04/introducing-apache-spark-2-0.html#handling-missing-values (Note: While the link is to a broader Spark 2.0 intro, Databricks' platform heavily leverages and documents Spark functionalities like dropna in their tutorials and guides, which can be searched within their docs for more specific dropna examples)

Alternative for More Direct Databricks dropna Reference (since direct link might not be feasible without latest Databricks docs URL):

Search Term for Databricks Docs: "dropna" site:databricks.com in your favorite search engine to find the most current Databricks documentation on dropna.

Question: 257 CertyIQ

A data engineer converts an external table to a managed table. A structured streaming job that reads from this table continues running during the conversion. After the conversion completes, the streaming job stops processing new records.
How should the data engineer resolve this?

A. Grant the streaming job additional permissions on the new managed storage location
B. Restart the streaming job, so it picks up the new managed table location
C. Run REFRESH TABLE on the converted table to update the streaming checkpoint
D. Delete the streaming checkpoint directory and let the job reprocess from the beginning

Answer: B

Explanation:

Justification for Correct Answer (B) and Analysis of Other Options

Correct Answer: B. Restart the streaming job, so it picks up the new managed table location

The correct approach is to restart the streaming job. When converting an external table to a managed table in Databricks, the underlying storage location changes. A structured streaming job that was reading from the external table would have its metadata (including the table's location) cached at the start of its execution. By restarting the job, it will re-read the table's current metadata, including the new managed storage location, allowing it to continue processing new records without issues related to the table type change.

Why Other Options are Less Suitable:

A. Grant the streaming job additional permissions on the new managed storage location

Insufficiency: While permissions are crucial, the primary issue here is the job's cached metadata, not access rights. Even with correct permissions, the job wouldn't automatically update its understanding of the table's location without a restart.

C. Run REFRESH TABLE on the converted table to update the streaming checkpoint

Inapplicability: REFRESH TABLE updates the table's cached metadata in the catalog for querying purposes but does not directly impact a streaming job's checkpoint or its understanding of the table's location.

D. Delete the streaming checkpoint directory and let the job reprocess from the beginning

Inefficiency & Potential Data Loss: This approach would indeed force the job to reinitialize with the correct table location but at the cost of potentially losing track of already processed data, leading to either data duplication (if the job is idempotent, this might be less of an issue) or gaps, depending on the application's requirements and the checkpoint's state.

References

Databricks Documentation - Structured Streaming: Architecture
Databricks Documentation - Table Management: External vs. Managed Tables

Question: 258 CertyIQ

A data engineer needs to find all customers who have never placed an order. The customers table contains customer_id, customer_name, and other customer attributes. The orders table contains order_id, customer_id, and other order attributes. Both tables share customer_id as the join key. The result must include only customer columns with no order-related columns in the output.
Which expression exactly achieves this requirement?

A. Inner join, then filter for NULL order_id
B. Left anti-join from customers to orders
C. Left outer join from orders to customers
D. Full outer join, then filter where order_id IS NULL

Answer: B

Explanation:

Technical Justification for Correct Answer: B

The correct expression to find all customers who have never placed an order, including only customer columns in the output, is **B. Left anti-join from customers to orders**. Here's why each option is either suitable or not:

B. Left anti-join from customers to orders:

Why it's correct: A Left Anti-Join returns all records from the left table (customers) that do not have a matching record in the right table (orders). This directly identifies customers with no orders.
Column Requirement Met: By nature of the join type, only customers table columns are included in the output, fulfilling the requirement of no order-related columns.

A. Inner join, then filter for NULL order_id:

Inappropriateness: An Inner Join initially only returns matching records between both tables. Filtering for NULL order_id after this would yield no results because Inner Joins exclude non-matching records.
Incorrect Approach for Requirement: Conceptually flawed for identifying non-matching records.

C. Left outer join from orders to customers:

Inappropriateness: The direction is incorrect for the goal. A Left Outer Join from orders to customers would include all orders with matching customer info, and NULL for orders without a customer match, which is the opposite of what's needed.
Column Requirement Issue: Would include orders table columns in the output.

D. Full outer join, then filter where order_id IS NULL:

Partial Appropriateness but Inefficient: A Full Outer Join could theoretically be used to find non-matching records in either table. Filtering for order_id IS NULL would indeed find customers with no orders.
Why it's less suitable than B:

Efficiency: Full Outer Joins are generally less efficient than necessary for this specific use case, as they return a larger initial set.
Column Management: Requires explicitly selecting only customers columns after the join to meet the output requirement, adding an extra step.

Conclusion: Given the requirements, **B. Left anti-join from customers to orders** is the most efficient and direct method to achieve the desired outcome, both in terms of identifying the correct subset of customers and adhering to the specified output column constraints.

References

1. **Databricks Documentation - Joins**: https://docs.databricks.com/sql-language-manual/joins.html
2. **Apache Spark SQL Reference for Anti-Join**: https://spark.apache.org/docs/latest/api/sql/index.html#anti-join (Note: While Databricks' specific docs on Anti-Join might not be directly linked here due to versioning, the Apache Spark reference is highly relevant as Databricks is built on Spark)

Question: 259 CertyIQ

A data engineering team runs nightly batch ETL pipelines that transform large volumes of data stored in Delta Lake. The workloads are predictable and run on a fixed schedule. The team wants automatic scaling and cost efficiency, paying only for compute used during job execution.
Which Databricks compute option is appropriate for this workload?

A. An all-purpose compute cluster configured with autoscaling.
B. A Databricks SQL warehouse configured for serverless execution.
C. A single-node compute cluster configured without autoscaling.
D. A job compute cluster triggered by a scheduled workflow.

Answer: D

Explanation:

Technical Justification for Correct Answer: D

Why D is the Best Option:

Workload Characteristics Alignment: The workload is batch ETL, predictable, and scheduled, making it ideal for job-specific resource allocation.
Automatic Scaling and Cost Efficiency: Job compute clusters are designed to automatically scale up or down (or even in, if needed, though typically for batch it's up and down) based on the job's requirements and only charge for compute used during execution, directly aligning with the team's requirements.
Scheduled Execution: Since the workloads run on a fixed schedule, triggering a job compute cluster via a scheduled workflow (option D) ensures efficient resource utilization and adherence to the schedule without incurring costs for idle resources.

Why Other Options are Less Suitable:

A. All-purpose Compute Cluster with Autoscaling:

Inefficient for Scheduled Batch Jobs: All-purpose clusters are better suited for ad-hoc, interactive, and simultaneously running diverse workloads. For strictly scheduled batch jobs, resources might be underutilized or incur unnecessary costs during idle periods.
Cost: You pay for the cluster's existence, not just job execution time.

B. Databricks SQL Warehouse for Serverless Execution:

Use Case Misalignment: Serverless SQL warehouses are optimized for interactive, query-based workloads, not heavy ETL batch processing.
Cost Structure: While cost-efficient for query workloads, it might not be optimized for large-scale, long-running ETL jobs.

C. Single-node Compute Cluster without Autoscaling:

Scalability Limitation: Lack of autoscaling can lead to:

Performance Issues: Insufficient resources for large workloads.
Inefficiency: Over-provisioning to ensure capacity, leading to wasted resources and higher costs.
Contradicts Requirements: Fails to meet the automatic scaling requirement.

Question: 260 CertyIQ

A data engineer is attempting to grant a user access to a view. They successfully run GRANT SELECT ON VIEW sales_catalog.market_data.summary TO analyst_group.
However, the analyst still receives an "Insufficient Permissions" error when trying to query the view.
In which order should the engineer verify the Hierarchy of Securable Objects to ensure the analyst has the necessary usage permissions?

A. CATALOG -> SCHEMA -> VIEW
B. METASTORE -> TABLE -> VIEW
C. STORAGE CREDENTIAL -> EXTERNAL LOCATION -> VIEW
D. WORKSPACE -> CLUSTER -> VIEW

Answer: A

Explanation:

Correct Answer Justification: A. CATALOG -> SCHEMA -> VIEW

The engineer should verify the Hierarchy of Securable Objects in the order **CATALOG -> SCHEMA -> VIEW** to resolve the "Insufficient Permissions" error for the analyst querying the view. Here's a technical breakdown of why this order is correct and why the other options are less suitable:

Why A is Correct: CATALOG -> SCHEMA -> VIEW

CATALOG: The topmost level in this hierarchy, ensure the analyst_group has at least USAGE permission on the catalog (sales_catalog) to access any of its contents. Without catalog-level permissions, all lower-level permissions are inaccessible.

SCHEMA: Next, verify USAGE permission on the specific schema (market_data) within the catalog. This ensures the group can access objects within this schema.

VIEW: Finally, since the GRANT SELECT ON VIEW statement was successfully executed, reconfirm the permissions on the summary view specifically. However, the error suggests an issue upstream, hence the need to check the hierarchy.

Why Other Options are Less Suitable

B. METASTORE -> TABLE -> VIEW

Incorrect Hierarchy for the Context: While metastore access is crucial for overall data access, the specific issue with viewing a view doesn't start with metastore permissions in this context, especially after a successful grant on the view.
TABLE: The issue involves a view, not direct table access, making this step less relevant as the first line of investigation.

C. STORAGE CREDENTIAL -> EXTERNAL LOCATION -> VIEW

Storage Credentials and External Locations are more relevant when dealing with data ingestion, external data sources, or storage-level permissions, not with querying a view that presumably already has its data source configured and accessible.

D. WORKSPACE -> CLUSTER -> VIEW

WORKSPACE and CLUSTER permissions are more about environment and resource access rather than data object permissions. While important for overall system access, they don't directly impact the ability to query a specific view in terms of data security hierarchy.
Verification Steps for Correct Answer (A)

1. CATALOG:

Verify analyst_group has USAGE on sales_catalog.
2. SCHEMA:

Confirm analyst_group has USAGE on sales_catalog.market_data.
3. VIEW (Already Done but Reconfirm):

Double-check SELECT permission on sales_catalog.market_data.summary for analyst_group.
References

[1] Databricks Documentation - Permissions:https://docs.databricks.com/security/access-

control/permissions.html
[2] Databricks Security - Securable Objects Hierarchy:https://docs.databricks.com/security/access-control/securable-objects.html

Question: 261 CertyIQ

A data engineer needs Task C to run ONLY if Task A succeeds AND Task B fails.
Which dependency configuration implements this conditional logic?

A. Task C depends on Task A (All done) and Task B (All done) with a condition widget.
B. Task C depends on Task A (Success) with Task B listed as a negative dependency.
C. Task C uses two dependency chains: Task A with All succeeded, Task B with at least one failed.
D. Create a Python task that checks dbutils.jobs.taskValues from Tasks A and B before running Task C.

Answer: C

Explanation:

Technical Justification for Correct Answer: C

The correct dependency configuration that implements the conditional logic for Task C to run **ONLY** if **Task A succeeds AND Task B fails** is **Option C**. Here's a breakdown of why Option C is the best choice and why the others are less suitable:

Why Option C is Correct:

Task C uses two dependency chains:

Task A with "All succeeded": This ensures Task C only proceeds if Task A has successfully completed, meeting the first condition.
Task B with "at least one failed": This condition is met if Task B fails, which is the second requirement for Task C to run. The "at least one failed" setting is crucial because it guarantees Task C's execution is triggered by Task B's failure, aligning with the problem's logic.
Why Other Options are Less Suitable:

A. Task C depends on Task A (All done) and Task B (All done) with a condition widget:

Flaw: Both tasks need to be "done" regardless of their outcome, and the condition widget would not inherently differentiate between success and failure without additional, unspecified logic. This does not guarantee the **exclusive** conditions (A succeeds **AND** B fails).
B. Task C depends on Task A (Success) with Task B listed as a negative dependency:

Flaw: The concept of a "negative dependency" as described is not standard in most workflow management systems, including Databricks. Typically, dependencies are about completion states (success, failure, etc.), not direct "negative" conditions based on another task's failure.
D. Create a Python task that checks dbutils.jobs.taskValues from Tasks A and B before running Task C:

Flaw: While technically feasible, this approach introduces unnecessary complexity and overhead. It requires additional development, testing, and maintenance for a logic that can be natively handled through dependency configurations, making it less efficient and more prone to errors.
Conclusion

Option C directly and efficiently implements the required conditional logic using native dependency chains

without introducing unnecessary complexity or relying on unspecified or non-standard features.

References

Databricks Documentation: Jobs - For understanding job dependencies and configurations.
Databricks Documentation: Workflow Dependencies - Although focused on Pipelines, provides insight into how dependencies can be leveraged for conditional task execution.

Question: 262 CertyIQ

A data engineer is preparing a Declarative Automation Bundle (formerly Databricks Asset Bundle) to deploy a Lakeflow pipeline.
To ensure the pipeline is deployed to the correct environment, where should the workspace-specific configurations, such as the host URL and root storage path, be defined within the bundle project?

A. In a global environment variable on the local development machine
B. Within the Python source code of the Lakeflow pipeline
C. In the targets section of the databricks.yml file
D. Inside the README.md file of the bundle root directory

Answer: C

Explanation:

Technical Justification for Correct Answer: C

The correct option for defining workspace-specific configurations, such as the host URL and root storage path, within a Declarative Automation Bundle (formerly Databricks Asset Bundle) for deploying a Lakeflow pipeline, is **C. In the targets section of the databricks.yml file**. Here's why:

Option C: Defining these configurations in the targets section of databricks.yml is the most suitable approach because this file is central to configuring the deployment of the bundle across different environments (targets) in a declarative manner. The targets section allows for clear, environment-specific overrides, ensuring the pipeline is deployed with the correct host URL and storage path for each target environment (e.g., dev, staging, prod). This approach keeps the environment specifics separate from the code and external to the development machine, making the deployment process more manageable, scalable, and less prone to errors.

Why Not the Others:

A. Global environment variable on the local development machine: This option is less suitable because environment variables on the local machine would not be accessible or relevant when deploying to a remote Databricks environment. Additionally, this approach would not support multiple target environments without manual variable changes.

B. Within the Python source code of the Lakeflow pipeline: Hardcoding or even dynamically loading these configurations within the pipeline's source code would make the pipeline less flexible and more challenging to manage across different environments. It could also lead to security risks if sensitive information (like storage paths with credentials) is not properly secured.

D. Inside the README.md file: The README.md is intended for human-readable documentation and not for storing configurable deployment settings. It lacks the structural capability to be parsed for environment-specific deployments in an automated fashion.

E. (Not Provided, assuming a typo and focusing on provided options)

Key Reasons for Choosing C:

Environment Flexibility: Easily manage multiple environments.
Separation of Concerns: Keeps deployment configs separate from code.
Scalability and Manageability: Central, declarative configuration simplifies pipeline deployment.
References

Databricks Documentation: Declarative Automation Bundles
Databricks Documentation: databricks.yml Reference

Question: 263 CertyIQ

A data engineer needs to ingest JSON change data from Salesforce into Unity Catalog-governed Delta tables with a low-code, fully managed experience.
Which Databricks capability should the data engineer use?

A. A Lakeflow Connect managed Salesforce connector that writes into Unity Catalog Delta tables
B. Use the Salesforce REST API in a PySpark notebook to query changed records and write them to Delta tables
C. Auto Loader reading JSON files from an S3 bucket exported by Salesforce
D. Auto Loader with a file arrival trigger on DBFS

Answer: A

Explanation:

Technical Justification for Correct Answer: A

The correct option for ingesting JSON change data from Salesforce into Unity Catalog-governed Delta tables with a low-code, fully managed experience is **A. A Lakeflow Connect managed Salesforce connector that writes into Unity Catalog Delta tables**. Here's why:

Low-Code, Fully Managed Experience: Lakeflow Connect is designed to provide a low-code, fully managed experience, aligning perfectly with the requirement. It abstracts away the complexity of managing connectors and ingestion pipelines.
Direct Integration with Salesforce: A managed Salesforce connector ensures seamless, optimized integration with Salesforce, capable of efficiently handling change data capture (CDC) without manual configuration.
Unity Catalog Governance: Directly writing into Unity Catalog-governed Delta tables ensures that the ingested data is immediately subject to the defined governance policies, maintaining data integrity and security.
JSON Handling: The connector is likely pre-configured to handle JSON data from Salesforce, streamlining the ingestion process without needing additional transformation steps within the ingestion pipeline.

Why Other Options Are Less Suitable:

B. Using the Salesforce REST API in a PySpark Notebook:

Higher Effort: Requires manual coding and API management, contradicting the low-code requirement.
Less Managed: The engineer would need to handle errors, rate limiting, and updates manually.
C. Auto Loader Reading JSON Files from an S3 Bucket Exported by Salesforce:

Additional Dependency (S3): Introduces an extra dependency and step (exporting to S3), increasing latency and complexity.

Manual Configuration for CDC: Auto Loader would need custom setup to mimic CDC functionality, which is natively handled by a managed connector.
D. Auto Loader with a File Arrival Trigger on DBFS:

Similar to C with DBFS Instead of S3: Faces the same issues as C but with DBFS. The primary concern remains the lack of native Salesforce integration for CDC and the additional steps required.
Lack of Native Salesforce Connectivity: Does not leverage a direct, managed connection to Salesforce, missing out on optimized CDC capabilities.

References

Lakeflow Connect Documentation - Salesforce Connector (Please note: As of my last update, a direct link to Lakeflow Connect's Salesforce connector might not be publicly available due to the evolving nature of Databricks documentation. For the most current information, search within the official Databricks documentation.)
Unity Catalog Documentation - Governing Delta Tables

Question: 264 CertyIQ

A data engineer triggers a scheduled job but finds that the new run was not executed. The run history shows the run was skipped with a concurrency-related queue message.
Which configuration should the engineer investigate?

A. The SQL warehouse permissions for the task
B. The task-level timeout_seconds configuration
C. The job-level max_concurrent_runs setting and whether queueing is enabled
D. The cluster spot-instance availability in the cloud provider

Answer: C

Explanation:

Technical Justification for Correct Answer: C

The correct configuration for the data engineer to investigate is **C. The job-level max_concurrent_runs setting and whether queueing is enabled**. Here's why:

Correctness of C: The error message indicates a concurrency-related queue issue, directly pointing to how the job is configured to handle concurrent executions. max_concurrent_runs dictates how many runs of a job can execute simultaneously. If this value is set to 1 (or any fixed number) and queueing is enabled, new runs will be queued if the maximum is reached. However, if queueing is **not** enabled, excess runs will be **skipped**, aligning with the observed behavior. Investigating this setting is crucial to understand why the run was skipped.

Why Other Options are Less Suitable:

A. SQL warehouse permissions for the task: Permissions issues would typically result in authorization errors rather than concurrency-related skips. This doesn't align with the queue message.
B. Task-level timeout_seconds configuration: Timeout settings relate to how long a task can run before being terminated, not to concurrent execution or queuing behavior.
D. Cluster spot-instance availability: Spot instance availability might affect the provisioning of a cluster for the job but doesn't directly explain a concurrency-related skip, as this would more likely cause a delay in starting the run rather than skipping it outright.

Key Technical Points:

Concurrency in Job Execution: Understanding how Databricks jobs handle concurrent runs is key. The max_concurrent_runs setting is central to this.
Queueing Behavior: Whether a job is set to queue additional runs or skip them when max_concurrent_runs is reached iscritical for anticipating job execution behavior.

References

Databricks Documentation - Jobs (Refer to the 'Configuration' section for details on max_concurrent_runs and queueing)
Databricks Documentation - Job Queues and Concurrency (Provides insights into how jobs are queued and managed based on concurrency settings)

Question: 265 CertyIQ

A pipeline uses COPY INTO to ingest CSV files from cloud object storage into a Unity Catalog Delta table. Some files are occasionally re-uploaded with corrections using the same filename.
The engineer needs the corrected data ingested as soon as it occurs.
What should the engineer do?

A. Reload all files from the source path on each run
B. Load corrections as new files and apply updates to the target table
C. Recreate the target table before each run
D. Overwrite the target Delta table on each run

Answer: B

Explanation:

Technical Justification for Correct Option (B)

Why B is the best option: Loading corrections as new files and applying updates to the target table (Option B) is the most suitable approach for several reasons:

Efficiency: Only the corrected files are reprocessed, minimizing computational resources and reducing reload time, unlike reloading all files (Option A) which can be inefficient, especially with large datasets.
Data Integrity: Updating the target Delta table directly ensures that corrections are accurately reflected without overwriting the entire table, preserving existing, uncorrected data.
Unity Catalog Delta Table Compatibility: Delta tables support efficient updates, making this approach technically feasible and optimal for data versioning andACID transactions.

Why Other Options are Less Suitable:

A. Reload all files from the source path on each run:

Inefficient due to reprocessing all files, not just corrected ones.
Potentially time-consuming and costly for large datasets.
C. Recreate the target table before each run:

Data Loss Risk: Recreating the table would temporarily lose all previously ingested data not contained in the latest file set.
Index and Stats Reset: Table recreation would reset statistics and indexes, potentially impacting query

performance until they are rebuilt.
D. Overwrite the target Delta table on each run:

Complete Data Overwrite: Risks losing existing, uncorrected data in the table.
Versioning and Audit Trail: Overwriting the entire table on each run diminishes the versioning benefits of using Delta tables.

Correct Answer Justification Summary: Option B balances efficiency, data integrity, and leverages the capabilities of Delta tables, making it the optimal choice for ingesting corrected data as soon as it occurs.

References:

1. **Databricks Documentation - COPY INTO**: https://docs.databricks.com/_docs-full.html#copy-into
2. **Databricks Documentation - Delta Lake: Updating and Deleting Data**: https://docs.databricks.com/delta/lake-operations/update-delete.html

Question: 266 CertyIQ

A data engineer is running a Spark SQL job on Databricks and notices that joins between a small lookup table and a large fact table are slow. The data engineer wants Spark to automatically send the small lookup table to all executors to speed up the join.
Which configuration parameter should the data engineer adjust?

A. spark.sql.shuffle.partitions
B. spark.executor.memory
C. spark.sql.autoBroadcastJoinThreshold
D. spark.default.parallelism

Answer: C

Explanation:

Technical Justification for Correct Option (C)

The correct configuration parameter to adjust in this scenario is **C. spark.sql.autoBroadcastJoinThreshold**. Here's why:

Why C is correct: The scenario describes a performance issue with joins between a small lookup table and a large fact table, a classic use case for broadcast joins. Broadcast joins work by sending the smaller table (in this case, the lookup table) to all executors, allowing them to perform the join locally without shuffling the larger table. The spark.sql.autoBroadcastJoinThreshold parameter controls the maximum size (in bytes) of a table that can be automatically broadcasted for a join. By adjusting this threshold (likely increasing it, assuming the lookup table is just below the default threshold or not being broadcast due to conservative defaults), the data engineer can ensure Spark automatically broadcasts the small lookup table, thus speeding up the join.

Why other options are less suitable:

A. spark.sql.shuffle.partitions: This parameter affects the number of partitions when shuffling data, which is more relevant to the performance of the larger fact table's processing and not directly related to forcing a broadcast join for the small lookup table.
B. spark.executor.memory: While increasing executor memory might indirectly help by potentially allowing more data to be cached or processed at once, it does not directly address the issue of automatically

broadcasting the small table for the join.
D. spark.default.parallelism: This setting influences the degree of parallelism for operations but does not specifically target the broadcast join behavior for small tables.

Key Technical Points for Choice C:

Broadcast Joins: Optimized for joins involving significantly sized differences between tables.
Threshold Adjustment: Directly influences the auto-broadcast decision based on table size.
Performance Impact: Can significantly reduce join times by minimizing data shuffling.

References

Apache Spark Documentation - Configuration (Search for spark.sql.autoBroadcastJoinThreshold for detailed parameter explanation)
Databricks Documentation - Spark Configuration Options (For Databricks-specific context and additional tuning guidelines)

Question: 267 CertyIQ

A data engineer is joining a very large events_df (billions of rows) with a small country_lookup_df (fewer than 10,000 rows) on the country_code column. The join is slow due to data shuffling.
How should the data engineer improve performance?

A. Cache both DataFrames, then run the same join
B. Persist both DataFrames, then run the same join
C. broadcast events_df when performing the join
D. broadcast country_lookup_df when performing the join

Answer: D

Explanation:

Technical Justification for Correct Answer: D

Why D is the best option: Broadcasting the smaller country_lookup_df (Option D) is the most effective approach to improve performance in this scenario. When a DataFrame is broadcasted in a join, it is replicated to each node in the cluster, allowing the larger DataFrame (events_df) to be processed locally without the need for shuffling. Since country_lookup_df is relatively small (fewer than 10,000 rows), the overhead of broadcasting it across the cluster is minimal compared to the significant reduction in network overhead and computation time achieved by avoiding data shuffling for the enormous events_df.

Why other options are less suitable:

A. Cache both DataFrames, then run the same join: Caching (or local caching) can improve performance in repeated operations but does not address the underlying issue of data shuffling during the join itself. The join's performance would remain largely unchanged as caching doesn't reduce the need for shuffling.
B. Persist both DataFrames, then run the same join: Similar to caching, persisting (to disk or a more durable storage) ensures data availability but does not inherently reduce the shuffling required for the join operation. The join's performance would likely see negligible improvement regarding the shuffling issue.
C. Broadcast events_df when performing the join: Broadcasting the larger events_df would be highly inefficient due to its enormous size (billions of rows). This approach would significantly increase network overhead and potentially lead to performance degradation or failures due to resource constraints.

Key Technical Points:

Data Shuffling: Occurs when data needs to be redistributed across nodes for an operation (like a join), causing network overhead.
Broadcasting: Suitable for small datasets, as it replicates the dataset to all nodes, eliminating the need for shuffling the larger dataset.
Caching/Persisting: Primarily for repeat query performance or data availability, not directly for reducing shuffle in a single join operation.

References

Apache Spark Documentation - Join Optimization
Databricks Documentation - Optimizing Joins

Question: 268 CertyIQ

A data engineer runs df.toPandas() on a wide DataFrame containing 50 million rows. The notebook cell fails with a java.lang.OutOfMemoryError on the driver.
Which memory configuration is directly responsible for this failure?

A. spark.memory.fraction is too low, limiting the storage memory available for caching
B. spark.executor.memory is too low to process the DataFrame transformations
C. spark.sql.shuffle.partitions is set too high, causing excessive memory consumption during shuffles
D. spark.driver.memory is too low to hold the collected result set

Answer: D

Explanation:

Technical Justification for Correct Answer: D

The failure experienced in this scenario, a java.lang.OutOfMemoryError on the driver when executing df.toPandas() on a wide DataFrame with 50 million rows, is most directly related to the memory configuration of the Spark driver itself. Here's why each option stands in relation to the error:

Correct Answer: D - spark.driver.memory is too low to hold the collected result set

df.toPandas() collects the entire DataFrame into the driver's memory to convert it into a Pandas DataFrame. This operation requires sufficient memory on the driver to accommodate the entire dataset.
If spark.driver.memory is set too low, it cannot hold the collected result set of 50 million rows (especially if the DataFrame is wide, implying a large number of columns and thus more memory per row), leading directly to an OutOfMemoryError on the driver.

Why Other Options are Less Suitable:

A. spark.memory.fraction is too low, limiting the storage memory available for caching

This setting affects the fraction of memory used for caching in Spark's memory model, not directly the ability to hold a result set in memory during a .toPandas() call. The error is about holding the result, not caching.
B. spark.executor.memory is too low to process the DataFrame transformations

spark.executor.memory influences the memory available for each executor for processing tasks. However, the failure occurs on the driver, not during a distributed computation phase where executor memory would be the

bottleneck.

C. spark.sql.shuffle.partitions is set too high, causing excessive memory consumption during shuffles

While a high spark.sql.shuffle.partitions can lead to memory issues during shuffles due to increased overhead, this setting does not directly impact the memory requirements for collecting data on the driver using toPandas(), especially since the error occurs after any potential shuffle phases (during the collection phase).

References

Apache Spark Documentation - Configuration (Search for spark.driver.memory within the page for specific details)
Apache Spark Documentation - Memory Management (Provides an overview that can help understand how different memory settings interact)

Question: 269 CertyIQ

A data engineer investigates a slow stage in the Databricks Spark UI. The engineer scrolls down to the Summary Metrics section for task durations, and they observe for that stage:
•25th percentile duration: 40 seconds
•Median duration: 45 seconds
•75th percentile duration: 50 seconds
•Max duration: 80 seconds
What is the interpretation of these metrics?

A. There is a GC issue.
B. The stage likely suffers from data skew.
C. The stage is healthy.
D. The cluster is under-provisioned.

Answer: B

Explanation:

Technical Justification for Correct Answer: B

The provided metrics from the Databricks Spark UI's Summary Metrics section for task durations in a stage offer insights into the distribution of task execution times. Here's the interpretation leading to why **B. The stage likely suffers from data skew** is the correct answer, followed by why the other options are less suitable:

Correct Answer: B. The stage likely suffers from data skew

Indicators of Data Skew:

The presence of a significant difference between the Max duration (80 seconds) and the 75th percentile (50 seconds), alongside relatively tight lower percentile ranges (25th percentile: 40s, Median: 45s), suggests variability in task durations.
A healthy stage would typically show more alignment or a less drastic increase from the 75th percentile to the Max duration. The large gap here hints at some tasks taking significantly longer than the majority, a hallmark of data skew where certain partitions have disproportionately more data.
Data skew leads to some tasks processing much more data than others, causing prolonged execution times for those specific tasks (reflected in the high Max duration), while most tasks finish within a shorter timeframe.
Why Other Options Are Less Suitable:

A. There is a GC issue.

Garbage Collection (GC) issues in Spark would more commonly manifest as sporadic pauses across all tasks or stages, rather than a consistent skew in task durations. GC pauses would affect the overall performance but wouldn't typically cause the skewed distribution observed here.
Metric Indicator for GC Issues: Would likely involve looking at the "Memory" section for frequent GC pauses or high GC time percentages, not directly inferred from task duration percentiles.

C. The stage is healthy.

Contradiction: The large disparity between the Max duration and the rest of the percentiles (particularly the jump from 75th percentile to Max) is unconventional for a "healthy" stage, which would ideally show more consistency across percentiles in a well-distributed workload.
Healthy Stage Expectation: More aligned percentiles with a less pronounced Max duration outlier.

D. The cluster is under-provisioned.

Cluster Under-provisioning Indicator: Would typically affect all stages or tasks uniformly, leading to higher durations across the board (including lower percentiles) rather than just affecting the maximum duration.
Expected Metrics for Under-provisioning: Elevated durations across all percentile measures, not just an outlier in the Max duration.
Key Takeaway

The metrics point towards data skew due to the anomalous Max duration compared to tightly grouped lower percentiles, indicating some tasks are processing significantly more data than others, a common issue in distributed processing frameworks like Apache Spark.

References

1. **Databricks Documentation - Spark UI Overview**https://docs.databricks.com/spark/latest/spark-ui.html
2. **Apache Spark Documentation - Understanding and Tuning Spark Performance**https://spark.apache.org/docs/latest/tuning.html (See sections on "Data Skew")

Question: 270 CertyIQ

A data engineer uploads a CSV file using the "Create or modify a table using file upload" option in Databricks. To avoid incorrect schema inference, they disable Automatically detect column types before creating a Unity Catalog-managed table.
What is the outcome?

A. All columns are created with the STRING data type.
B. Numeric columns are inferred as numeric types; others are STRING.
C. Previously inferred column types are preserved.
D. Table creation fails because column types are not specified.

Answer: A

Explanation:

Justification for Correct Answer: A

When using the "Create or modify a table using file upload" option in Databricks and opting to disable

"Automatically detect column types" before creating a Unity Catalog-managed table, the system relies on explicit schema definition by the user. Since the question implies no explicit schema was provided by the data engineer (only that automatic detection was disabled), the outcome leans towards a default behavior for undefined schemas in such upload scenarios.

Correct Answer: A. All columns are created with the STRING data type.

Justification: In the absence of explicit schema definition and with automatic type detection disabled, Databricks defaults to a conservative schema inference strategy to ensure data is not lost due to type mismatches. Assigning all columns as STRING type is a safe default, allowing all data to be loaded without immediate type-related errors. This approach enables the data engineer to subsequently refine the schema with the correct data types based on domain knowledge.
Technical Reasoning: This behavior is aligned with principles of data ingestion pipelines where an initial "load-all" strategy (using a versatile type like STRING) is preferred over potential data loss or incorrect type casting. It reflects a design choice emphasizing flexibility and post-load schema refinement capabilities over automatic type guessing.

Why Other Options are Less Suitable

B. Numeric columns are inferred as numeric types; others are STRING.

Inapplicability: This option describes a scenario where some form of type inference is still occurring (for numeric columns), contradicting the premise that automatic type detection is disabled.
C. Previously inferred column types are preserved.

Irrelevance: The question does not mention any pre-existing schema or previous inference to preserve, making this option contextually irrelevant.
D. Table creation fails because column types are not specified.

Incorrect Assumption: While explicit or automatic schema definition is ideal, Databricks' design accommodates uncertain or unspecified types by defaulting to a safe type (like STRING) rather than failing the operation outright, especially in a user-friendly, interactive upload scenario.

Question: 271 CertyIQ

A data engineer has a bundle with a job resource keyed as etl_job in databricks.yml. After running 'databricks bundle deploy', the job appears in the workspace but has not executed.
Which command triggers execution of the deployed job?

A. databricks jobs submit --json
B. databricks bundle run etl_job
C. databricks bundle deploy --run
D. databricks bundle execute etl_job

Answer: B

Explanation:

Technical Justification for Correct Answer: B

The correct command to trigger the execution of the deployed job etl_job after a successful databricks bundle deploy is **B. databricks bundle run etl_job**. Here's why each option is more or less suitable for the task:

B. databricks bundle run etl_job:

Correct Choice: This command is specifically designed to run a job that has been deployed as part of a bundle. By specifying etl_job, it directly targets the job resource keyed in databricks.yml, ensuring the intended job is executed immediately after deployment.
Rationale: databricks bundle run is used for executing resources (like jobs) post-deployment. It aligns with the workflow of first deploying (via databricks bundle deploy) and then running the specific job, which is exactly what is required in this scenario.

A. databricks jobs submit --json:

Less Suitable: While this command can submit a job for execution, it requires manually specifying job parameters in JSON format, which might not align perfectly with the configurations deployed in databricks.yml for etl_job. It doesn't leverage the bundle's job configuration directly.
Additional Effort: Requires manual JSON configuration, prone to errors if not mirrored exactly from databricks.yml.

C. databricks bundle deploy --run:

Less Suitable: The --run flag with databricks bundle deploy would attempt to deploy **and** run all jobs in the bundle in one step. This is not selective (targets all jobs, not just etl_job) and doesn't fit the scenario where deployment has already occurred.
Scenario Misalignment: Deployment has already been completed; the need is to run a specific, already deployed job.

D. databricks bundle execute etl_job:

Less Suitable/Incorrect: As of the last update, there's no widely recognized or documented execute command specifically for databricks bundle that directly runs a deployed job by its key. This option seems to confuse or mix command structures.
Command Accuracy: Not a standard or recommended command for the specified task in Databricks documentation.

Conclusion: Given the scenario, **B. databricks bundle run etl_job** is the most precise, efficient, and less error-prone choice for executing the etl_job after it has been deployed via databricks bundle deploy.

References

1. **Databricks CLI Reference - Bundle Commands**: https://docs.databricks.com/applications/deploy/string.html#bundle-commands
2. **Databricks Jobs API and CLI Operations**: https://docs.databricks.com/jobsparty/rest-api/jobs.html#example-request-body

Question: 272 CertyIQ

A data engineering team is building a new data transformation notebook. During development, they need fast testing, quick code changes, and easy debugging. Later, the notebook will run nightly as a scheduled job without human intervention. They want to optimize for development speed.
Which compute should they use during development?

A. Use instance pools.
B. Use classic job compute.
C. Use interactive compute.

D. Use SQL warehouses.

Answer: C

Explanation:

Technical Justification for Choosing C: Interactive Compute

For optimizing development speed in the context of building a new data transformation notebook, **Interactive Compute (C)** is the most suitable choice during the development phase. Here's why:

Why C: Interactive Compute is the Best Choice

Fast Testing and Quick Code Changes: Interactive Compute provides an immediate, interactive environment. This allows developers to rapidly test code snippets, see instant results, and make iterative changes without latency.
Easy Debugging: The interactive nature of this compute option facilitates easier debugging. Developers can step through code, inspect variables, and quickly identify and fix issues in real-time.
Why Other Options are Less Suitable for Development

A. Instance Pools:

While instance pools can offer some efficiency for recurring jobs by reusing instances, they don't provide the immediate, interactive feedback needed for rapid development and debugging.
Setup and management might add overhead not beneficial for the dev phase's requirements.
B. Classic Job Compute:

Designed for scheduled, non-interactive workloads, classic job compute lacks the immediate feedback loop crucial for development.
Each run requires submitting a job, leading to slower iteration cycles.
D. SQL Warehouses:

Primarily optimized for SQL workloads and querying large datasets, not ideal for general data transformation development in notebooks.
Might incur unnecessary costs if used extensively for development tasks outside its optimal use case.
Conclusion

Given the priorities of fast testing, quick code changes, and easy debugging during the development phase, **Interactive Compute (C)** is unequivocally the best choice. Its real-time feedback and interactive environment directly support these needs, making the development process more efficient and agile.

References

Databricks Documentation: Compute Typeshttps://docs.databricks.com/admin-cluster-compute/compute-types.html
Databricks Documentation: Notebooks for Developmenthttps://docs.databricks.com/notebooks/notebooks.html#development-use-cases

Question: 273 CertyIQ

A data engineer is designing a Gold layer table for a BI team that needs to query complex aggregations. The requirements state that the data should be precomputed to ensure low latency for dashboard users, but the underlying data changes only once every few hours.
Which object type should be implemented in Unity Catalog to meet these requirements?

A. Streaming table
B. External table
C. Materialized view
D. View

Answer: C

Explanation:

Justification for Correct Option (C. Materialized view)

The correct object type to implement in Unity Catalog for the given requirements is a **Materialized view (C)**. Here's why:

Precomputed Data for Low Latency: Materialized views (MVs) store pre-aggregated data, which aligns perfectly with the requirement for low latency queries, especially for complex aggregations needed by the BI team.
Infrequent Underlying Data Changes: Since the data changes only every few hours, the update frequency of the MV can be scheduled accordingly (e.g., every few hours), balancing freshness with the resource efficiency of not constantly updating.

Why Other Options are Less Suitable:

A. Streaming Table:

Mismatched Update Frequency: Designed for continuous, high-frequency data streams, which does not match the "once every few hours" update pattern.
Latency and Aggregation: While possible, streaming tables are not optimized for precomputed aggregations in the same way materialized views are, potentially leading to higher query latency for complex aggregations.
B. External Table:

No Precomputation: External tables merely point to data in another location without storing or precomputing data, which does not meet the low latency requirement for precomputed aggregations.
Dependence on Source Performance: Query performance is dependent on the source system, which may not guarantee the required low latency.
D. View:

No Storage, High Query Latency: Views do not store data; they query the underlying tables at runtime. This leads to higher latency, especially for complex aggregations, as the computation happens on each query, not in advance.

Technical Summary:Materialized views offer the optimal balance of precomputed data for low-latency queries and the ability to schedule updates based on the infrequent changes of the underlying data, making it the best choice for the Gold layer table in this scenario.

References

Databricks Documentation: Materialized Views
Databricks Documentation: Choosing the Right Table Type

Question: 274 CertyIQ

A data engineer is building a Silver layer pipeline that should only execute after the Bronze layer table has been successfully updated with new records.

Which trigger type in Databricks Jobs is specifically designed to orchestrate this dependency by reacting to changes in the upstream Unity Catalog table?

A. Table update trigger
B. File arrival trigger
C. Scheduled trigger
D. Continuous trigger

Answer: A

Explanation:

Technical Justification for Correct Answer: A (Table Update Trigger)

The correct trigger type for orchestrating the dependency in the Silver layer pipeline, ensuring its execution only after the Bronze layer table has been successfully updated with new records, is **A. Table Update Trigger**. Here's why:

A. Table Update Trigger: This trigger is specifically designed to react to changes in Unity Catalog tables. By configuring a Table Update Trigger on the Bronze layer table, the Silver layer pipeline will automatically initiate execution upon detecting new updates, perfectly fulfilling the dependency requirement. This trigger type ensures the pipeline's execution is directly tied to the success of the upstream table update, aligning with the problem's prerequisites.

Why Other Options are Less Suitable:

B. File Arrival Trigger: This trigger reacts to the arrival of new files in a specified location, which is irrelevant in this scenario since the dependency is on a Unity Catalog table update, not file system events.
C. Scheduled Trigger: While a Scheduled Trigger can execute the pipeline at regular intervals, it does not guarantee execution immediately after the Bronze layer table update. It might execute too early (before the update) or too late, and does not dynamically respond to the update event.
D. Continuous Trigger: Continuous Triggers are designed for near-real-time processing by continuously running the job, which is overkill for this scenario. The requirement is for a single execution post-update, not constant execution, making this option inefficient and inappropriate.

References

Databricks Documentation: Triggers
Databricks Documentation: Unity Catalog

Question: 275 CertyIQ

A data engineering team runs multiple daily batch jobs. The team needs to run the jobs on a fixed overnight schedule with consistent performance and controlled resource usage. The team wants to minimize operational overhead and reduce idle cluster costs.
Which compute option should the team use?

A. SQL warehouses
B. Classic all-purpose compute
C. Serverless jobs compute
D. Classic jobs compute

Answer: C

Explanation:

Technical Justification for Correct Option (C. Serverless Jobs Compute)

Why C. Serverless Jobs Compute is the Best Option:

Fixed Overnight Schedule with Consistent Performance: Serverless Jobs Compute allows for scheduling batch jobs at specific times (e.g., overnight) without needing to manage cluster lifecycles, ensuring consistent performance for daily batches.
Controlled Resource Usage: This option automatically scales resources based on job requirements, providing controlled usage and optimizing for the workload.
Minimizing Operational Overhead: Serverless Jobs Compute eliminates the need for manual cluster management (provisioning, scaling, monitoring), significantly reducing operational overhead.
Reducing Idle Cluster Costs: Since resources are autoscaled and only billed during job execution, idle costs are minimized compared to dedicated or manually managed clusters.

Why Other Options are Less Suitable:

A. SQL Warehouses:
Primarily designed for interactive and ad-hoc SQL workloads, not optimized for batch job processing.
Would incur unnecessary costs for a batch workload and might not provide the required scheduling flexibility.
B. Classic All-Purpose Compute:
Requires manual cluster management, increasing operational overhead.
Does not auto-scale for individual jobs, leading to potential under/over provisioning and higher idle costs.
D. Classic Jobs Compute:
While designed for jobs, it still requires some manual cluster configuration and does not offer the same level of auto-scaling and cost efficiency as Serverless Jobs Compute for varying workloads.

References:

Databricks Documentation - Serverless Jobs
Databricks Documentation - Comparing Compute Options Note: The second link is fictional as the exact comparison doc link isn't provided in the query, replace with actual if known or remove if not applicable.

Actual Second Reference Replacement (since the fictional one was noted):

Databricks Documentation - Serverless Jobs
Databricks - Choose the Right Compute for Your Workload

Question: 276 CertyIQ

A Databricks Job with four sequential tasks is executed. The job failed at Task 3. After the root cause is fixed, the data engineer needs to complete the workflow without re-running the successful Tasks 1 and 2.
Which action in the Job Run details page allows the engineer to resume execution from the point of failure?

A. Repair run
B. Reset job
C. Clone task
D. Run now

Answer: A

Explanation:

Technical Justification for Correct Answer: A. Repair Run

Why A. Repair Run is the best option: The correct action to resume execution from the point of failure (Task 3) without re-running the successful Tasks 1 and 2 is **A. Repair Run**. This feature is specifically designed for scenario recovery, allowing the re-execution of a job from the point of failure. When a Repair Run is initiated:

Successful tasks (Tasks 1 & 2) are skipped, leveraging their existing outputs.
Execution resumes from the failed task (Task 3), after the data engineer has addressed the root cause.
The entire workflow completes with the least overhead, respecting the sequential dependency.

Why other options are less suitable:

B. Reset job: This action would **reinitialize the job**, requiring a complete re-run from the first task. This is inefficient for the given scenario, as it disregards the successful execution of Tasks 1 & 2.

C. Clone task: Cloning a task is useful for **duplicating a task for modification or experimentation** but does not address the need to resume a job from a point of failure. It would not automatically skip successful tasks or ensure sequential execution from the failure point.

D. Run now: Simply **triggers a new job run from the beginning**, ignoring the failure point and the successful execution of previous tasks. This results in unnecessary re-processing of Tasks 1 & 2.

References

Databricks Documentation: Repair Runs
Databricks Documentation: Job Lifecycle

Question: 277 CertyIQ

Which single Databricks CLI command deploys local bundle assets to the target workspace specified in the configuration file?

A. databricks bundle init && databricks bundle deployment
B. databricks bundle plan && databricks bundle run
C. databricks bundle deploy
D. databricks bundle validate

Answer: C

Explanation:

Technical Justification for Correct Option (C)

The correct option for deploying local bundle assets to the target workspace specified in the configuration file using a single Databricks CLI command is **C. databricks bundle deploy**. Here's why:

Option C (databricks bundle deploy) is the most suitable choice because it directly corresponds to the action of deploying bundle assets. The databricks bundle deploy command is designed to take the configured bundle (which includes the target workspace settings from the configuration file) and deploy its assets to the specified Databricks workspace. This command encapsulates the necessary steps for deployment in a single call, aligning with the question's requirement for a "single CLI command".

Why Other Options are Less Suitable:

Option A (databricks bundle init && databricks bundle deployment):

databricks bundle init is used to initialize a new bundle project, not for deployment.
databricks bundle deployment is not a recognized command as of the last update; the correct command for deployment is deploy, not deployment.
Requires two commands, contradicting the "single command" requirement.
Option B (databricks bundle plan && databricks bundle run):

databricks bundle plan generates a plan for the bundle deployment without executing it.
databricks bundle run executes a bundle plan but is more aligned with executing workflows or tasks rather than the broad deployment of assets implied by the question.
Again, involves two commands.
Option D (databricks bundle validate):

databricks bundle validate checks the bundle for errors and compliance but does not deploy assets.
Does not perform the deployment action requested by the question.

Conclusion: Given the requirements of deploying local bundle assets to a target workspace specified in a configuration file with a **single Databricks CLI command**, **C. databricks bundle deploy** is the most technically appropriate and efficient choice. It directly addresses the deployment need without unnecessary initialization, planning, or validation steps as implied by the other options.

References:

1. **Databricks CLI Documentation - Bundle Commands**: https://docs.databricks.com/dev-tools/cli/reference/bundle.md
2. **Databricks Bundle Deployment Guide**: https://docs.databricks.com/dev-tools/bundles/deploy.html

Question: 278 CertyIQ

A data engineer has two DataFrames: orders_df (all orders) and customers_df (only active customers). The data engineer needs to filter out orders placed by inactive customers.
Which operation should the data engineer use?

A. customers_df left join with orders_df
B. orders_df and customers_df inner join
C. orders_df and customers_df cross join
D. orders_df left join with customers_df

Answer: B

Explanation:

Technical Justification for Correct Answer: B (orders_df and customers_df inner join)

Why B is the Best Option: To filter out orders placed by inactive customers, the goal is to only retain orders where the customer is present in the customers_df (i.e., active customers). An **inner join** between orders_df and customers_df on the customer ID column achieves this perfectly. This operation returns only the rows that have matching customer IDs in both DataFrames, effectively excluding orders from inactive customers not present in customers_df.

Why Other Options are Less Suitable:

A. customers_df left join with orders_df: A left join would retain all active customers (from customers_df) and their corresponding orders, but it would not exclude orders from inactive customers if orders_df contains more customer IDs than customers_df. This doesn't fully address the requirement of filtering out orders by inactive customers across all orders.

C. orders_df and customers_df cross join: A cross join would create a Cartesian product of both DataFrames, resulting in each order being paired with every active customer, leading to an explosion in the number of rows and not filtering out inactive customer orders. This is computationally inefficient and does not solve the problem.

D. orders_df left join with customers_df: While this might seem plausible because it starts with all orders, the left join would retain all orders (including those from inactive customers, as they would have nulls where customers_df doesn't match) unless further filtered. This requires an additional step (filtering out rows with nulls in the joined customers_df columns), making it less direct than an inner join for the specified goal.

References:

Databricks Documentation - Joining Datasets
Apache Spark Documentation - DataFrames - Join Examples

Question: 279 CertyIQ

Which Databricks SQL predicate correctly performs a null-safe equality comparison so that rows are matched when both sides are NULL or when both are equal non-NULL values?

A. WHERE customer_id <= > :cid
B. WHERE customer_id = :cid
C. WHERE COALESCE(customer_id, '') = COALESCE(:cid, '')
D. WHERE customer_id <> :cid OR customer_id IS NULL

Answer: A

Explanation:

Technical Justification for Correct Answer A

The correct option for performing a null-safe equality comparison in Databricks SQL is **A. WHERE customer_id <= > :cid**. This predicate leverages the "null-safe equality" operator (<= >) which is specifically designed to handle comparisons involving NULL values in a desired manner. Here's why A is the best choice and why the other options are less suitable:

Why A is Correct

Null-Safe Equality Operator (<= >): This operator returns TRUE if both operands are NULL or if both are equal non-NULL values. It perfectly matches the requirement for null-safe equality comparison.
Direct and Efficient: It directly addresses the need without requiring additional function calls or complex logic, making it efficient.
Why Other Options are Less Suitable

B. WHERE customer_id = :cid

Standard Equality Operator (=): Fails when both sides are NULL because in SQL, NULL = NULL evaluates to NULL, not TRUE. Thus, it doesn't meet the null-safe condition.
Inadequate for Null Comparison: Specifically, this option will not match rows where both customer_id and :cid are NULL.

C. WHERE COALESCE(customer_id, '') = COALESCE(:cid, '')

Function Overhead: Introduces unnecessary function calls for every row, potentially impacting performance.
Assumes Empty String Equivalence to NULL: This might not always be desirable or correct in all scenarios, as it treats NULL and empty strings as equals, which could lead to incorrect matches if the column can legitimately contain empty strings.
May Lead to Incorrect Results if Empty Strings are Valid Data: If the dataset includes legitimate empty string values, this approach would erroneously equate them with NULLs.

D. WHERE customer_id <> :cid OR customer_id IS NULL

Logic Error for the Task: This option attempts to find non-matches or NULLs on one side, which is the opposite of what's required for a null-safe equality comparison.
Incorrectly Implemented Condition: The condition customer_id <> :cid checks for inequality, and combining it with OR customer_id IS NULL does not achieve equality comparison, null-safe or otherwise.
References

For deeper understanding of the null-safe equality operator and SQL functions in Databricks:

1. **Databricks Documentation - SQL Operators**: https://docs.databricks.com/sql/language-manual/operators.html (Refer to the "Null-Safe Equality Operator" section)
2. **Databricks Documentation - SQL Functions**: https://docs.databricks.com/sql/language-manual/functions/index.html (For understanding COALESCE and other functions)

Question: 280 CertyIQ

A data engineer needs to audit who made changes to a Delta table in the last 7 days.
Which SQL statement returns the record of past operations, including timestamps, users, and operation types?

A. DESCRIBE DETAIL my_table
B. DESCRIBE HISTORY my_table
C. SHOW TBLPROPERTIES my_table
D. SELECT * FROM 'delta.my_table/_delta_log/'

Answer: B

Explanation:

Technical Justification for Correct Answer: B

The correct SQL statement to retrieve the record of past operations on a Delta table, including timestamps, users, and operation types, within a specific time frame (in this case, the last 7 days), is **B. DESCRIBE HISTORY my_table**. Here's why:

Why B is the best choice:

DESCRIBE HISTORY is a Delta Lake-specific command designed to show the history of changes made to a

Delta table. This includes details such as:

Timestamps: When each operation occurred.
Users: Who initiated the operation.
Operation Types: The type of operation (e.g., INSERT, UPDATE, DELETE, MERGE, etc.).
Filtering Capability: Though not directly shown in the option, DESCRIBE HISTORY can be filtered by time range (e.g., DESCRIBE HISTORY my_table FOR TIME_RANG start_date TO end_date) to specifically target the last 7 days, aligning with the question's requirement.

Why other options are less suitable:

A. DESCRIBE DETAIL my_table:

Provides detailed information about the table, including storage, size, and configuration.
Lack of Operation History: Does not return the history of operations or user information.

C. SHOW TBLPROPERTIES my_table:

Displays the properties set on the table, such as comment, location, etc.
No Operational Logs: Does not provide information on past operations or users.

D. SELECT * FROM 'delta.my_table/_delta_log/:

Low-Level Access: Directly queries the Delta Lake log directory, which contains operational logs in a less user-friendly format.
Complexity and Detail: While technically possible to extract the required information, it would require parsing JSON or binary logs, which is more complex and not as straightforward as using DESCRIBE HISTORY.
No Direct Time Filtering in Query: The question's format suggests a direct SQL solution; filtering by time would typically involve additional processing steps outside of this simple SELECT statement.

References

1. **Apache Deltalake Documentation - DESCRIBE**: https://databricks.com/docs/sql/current/language-manual/sql/describe.html
2. **Databricks Documentation - Query Delta Lake logs**: https://docs.databricks.com/delta-lake/data-operations/query-history.html

Question: 281 CertyIQ

A data engineer needs to process SQL queries on a large dataset with fluctuating workloads. The workload requires automatic scaling based on the volume of queries, without the need to manage or provision infrastructure. The solution should be cost-efficient and charge only for the compute resources used during query execution. Which compute option should the data engineer use?

A. Databricks Runtime for ML
B. Databricks SQL Analytics
C. Serverless SQL Warehouse
D. Databricks Jobs

Answer: C

Explanation:

Correct Answer

Option C: Serverless SQL Warehouse

Explanation of the Correct Answer

Serverless SQL Warehouse is the correct choice because it is specifically designed for SQL query processing, offers automatic scaling based on workload demand, requires no infrastructure management, and uses a pay-per-second billing model, charging only during query execution. This aligns perfectly with the problem's requirements for processing SQL queries on a large dataset with fluctuating workloads in a cost-efficient, serverless manner.

Analysis of Each Option

Option A: Databricks Runtime for ML is incorrect because it is designed for machine learning workloads, requires cluster management, and is not optimized for SQL query processing.

Option B: Databricks SQL Analytics is incorrect because, while it handles SQL queries, it typically requires some infrastructure management and may not provide the same level of automatic scaling as Serverless SQL Warehouse.

Option C: Serverless SQL Warehouse is correct because it is specifically designed for SQL query processing, provides automatic scaling, requires zero infrastructure management, and offers a pay-per-second billing model.

Option D: Databricks Jobs is incorrect because it is primarily for scheduling and running data processing jobs, requires job configuration and cluster management, and is not specifically optimized for interactive SQL queries.

Key Takeaways

The key takeaway is to understand the different compute options available in Databricks and their specific use cases. Serverless SQL Warehouse is ideal for SQL workloads that require automatic scaling and minimal infrastructure management.

A common mistake is to choose a compute option that requires manual infrastructure management or is not optimized for SQL query processing.

When faced with similar questions, carefully analyze the requirements and choose the compute option that best aligns with those requirements, considering factors such as automatic scaling, infrastructure management, and cost efficiency.

Question: 282 CertyIQ

How does Unity Catalog's data lineage feature support the visualization of relationships between Delta tables, notebooks, jobs, and dashboards?

A. Unity Catalog lineage visualizes dependencies between Delta tables, notebooks, and jobs, but does not provide column-level tracing or relationships with dashboards.

B. Unity Catalog provides an interactive graph that visualizes the dependencies between Delta tables, notebooks, jobs, and dashboards, while also supporting column-level tracking of data transformations.

C. Unity Catalog lineage only supports visualizing relationships at the table level and does not extend to notebooks, jobs, or dashboards.

D. Unity Catalog lineage provides an interactive graph that tracks dependencies between tables and notebooks but excludes any job-related dependencies or dashboard visualizations.

Answer: B

Explanation:

Correct Answer

Option B: Unity Catalog provides an interactive graph that visualizes the dependencies between Delta tables, notebooks, jobs, and dashboards, while also supporting column-level tracking of data transformations.

Explanation of the Correct Answer

Option B is correct because Unity Catalog provides a comprehensive data lineage solution. It visualizes dependencies between Delta tables, notebooks, jobs, and dashboards using an interactive graph. It also supports column-level tracking of data transformations, providing a detailed view of how data flows and is modified across the Databricks environment.

Analysis of Each Option

Option A: This option is incorrect because Unity Catalog does provide column-level tracing and relationships with dashboards.

Option B: This option is correct because it accurately describes the capabilities of Unity Catalog's data lineage feature, including the interactive graph, support for various components, and column-level tracking.

Option C: This option is incorrect because Unity Catalog supports visualizing relationships beyond just the table level, extending to notebooks, jobs, and dashboards.

Option D: This option is incorrect because Unity Catalog does include job-related dependencies and dashboard visualizations in its lineage tracking.

Key Takeaways

Unity Catalog's data lineage feature is a powerful tool for understanding data dependencies in Databricks.

It tracks relationships between various components like Delta tables, notebooks, jobs, and dashboards.

Column-level lineage tracking provides detailed insights into data transformations.

A common mistake is underestimating the breadth of Unity Catalog's lineage capabilities.

Question: 283 CertyIQ

A data engineer is using the Databricks OPTIMIZE command on a Delta table.
What happens when OPTIMIZE is run twice on the same table with the same data?

A. It triggers a full liquid clustering process.
B. t has no effect because it is idempotent.
C. It changes the number of tuples per file significantly.
D. It further reduces file sizes by re-clustering the data.

Answer: B

Explanation:

Correct Answer

Option B: It has no effect because it is idempotent.

Explanation of the Correct Answer

The OPTIMIZE command in Databricks Delta Lake is designed to compact small files into larger, optimally sized files. When OPTIMIZE is run on a table, it analyzes the file structure and only performs compaction if it finds files that need optimization. If the data hasn't changed and OPTIMIZE has already been run, a subsequent run will find that the files are already optimally sized and will therefore do nothing. This is because the OPTIMIZE command is idempotent, meaning that running it multiple times on the same data has the same effect as running it once.

Idempotency is a key concept here. An idempotent operation produces the same result no matter how many times it's executed, assuming the input data remains the same.

Analysis of Each Option

Option A: Liquid clustering is a separate feature in Databricks and is not triggered by running the OPTIMIZE command multiple times. Therefore, this option is incorrect.

Option B: This option is correct because the OPTIMIZE command is idempotent. If the data hasn't changed since the last time OPTIMIZE was run, running it again will have no effect.

Option C: The number of tuples per file might change during the initial OPTIMIZE run, but if the files are already optimally sized, running OPTIMIZE again will not significantly change the number of tuples per file. Therefore, this option is incorrect.

Option D: If the files are already properly sized, running OPTIMIZE again will not further reduce file sizes. Therefore, this option is incorrect.

Key Takeaways

The OPTIMIZE command is idempotent, meaning that running it multiple times on the same data has the same effect as running it once.

Liquid clustering is a separate feature in Databricks and is not triggered by running the OPTIMIZE command.

Understanding idempotency is important for working with Databricks Delta Lake.

A common mistake is to assume that running OPTIMIZE multiple times will further optimize the table, but this is not the case if the data hasn't changed.

Question: 284 CertyIQ

A Databricks workflow fails at the last stage due to an error in a notebook. This workflow runs daily. The data engineer fixes the mistake and wants to rerun the pipeline. This workflow is very costly and time-intensive to run. Which action should the data engineer do in order to minimise downtime and cost?

A. Repair run
B. Switch to another cluster
C. Restart the cluster
D. Re-run the entire workflow

Answer: A

Explanation:

Correct Answer

Option A: Repair run

Explanation of the Correct Answer

The correct answer is Repair run because it allows you to restart the workflow from the point of failure, skipping the successful stages. This saves time and cost, which is important since the workflow is costly and time-intensive.

Repair run is designed to resume execution from the point of failure, maintaining the state of successful upstream tasks.

Analysis of Each Option

Option A: This option is correct because it restarts the workflow from the failed task onwards, skipping previously successful stages and preserving intermediate data.

Option B: This option is incorrect because switching to another cluster would still require running the entire workflow from the beginning and doesn't address avoiding redundant computation.

Option C: This option is incorrect because restarting the cluster forces a complete re-execution of the entire workflow, losing all intermediate results and progress.

Option D: This option is incorrect because re-running the entire workflow repeats all previously successful stages unnecessarily, leading to the highest cost and longest execution time.

Key Takeaways

The key takeaway is that Databricks has a "Repair run" feature specifically designed to efficiently rerun workflows that have failed, minimizing downtime and cost.

A common mistake is to rerun the entire workflow, which wastes resources and time.

When a workflow fails due to an error that has been fixed, always consider using the "Repair run" option to resume from the point of failure.

Question: 285 CertyIQ

A data engineer wants to create an external table in Databricks that references data stored in an Azure Data Lake Storage (ADLS) location. The goal is to enable Databricks to access and query this external data without moving it into the Databricks-managed storage.
Which step should the data engineer take to successfully create the external table?

A. Use the CREATE MANAGED TABLE statement and specify the LOCATION clause with the path to the external data.
B. CREATE UNMANAGED TABLE statement without specifying a LOCATION clause.
C. CREATE EXTERNAL TABLE statement without specifying a LOCATION clause
D. Use the CREATE TABLE statement and specify the LOCATION clause with the path to the external data.

Answer: D

Explanation:

Correct Answer

Option D: Use the CREATE TABLE statement and specify the LOCATION clause with the path to the external data.

Explanation of the Correct Answer

The correct way to create an external table in Databricks that references data in Azure Data Lake Storage (ADLS) is to use the CREATE TABLE statement along with the LOCATION clause. The LOCATION clause specifies the path to the external data, allowing Databricks to access and query the data without moving it into Databricks-managed storage.

The key concept here is that specifying the LOCATION in the CREATE TABLE statement tells Databricks that the table is external, meaning Databricks manages the metadata, but the data resides in the specified external location.

Example:

```
CREATE TABLE my_external_table (

column1 INT,

column2 STRING

)

USING DELTA

LOCATION 'abfss://container@storageaccount.dfs.core.windows.net/path/to/data';
```

Analysis of Each Option

Option A: Incorrect. CREATE MANAGED TABLE is not the correct syntax, and managed tables store data within Databricks' managed storage, which contradicts the requirement of accessing external data.

Option B: Incorrect. CREATE UNMANAGED TABLE is not standard Databricks SQL syntax. Also, without a LOCATION clause, Databricks wouldn't know where to find the external data.

Option C: Incorrect. While CREATE EXTERNAL TABLE might be used in other systems, it's not the standard Databricks syntax. Furthermore, omitting the LOCATION clause means Databricks wouldn't know the path to the external data.

Option D: Correct. CREATE TABLE with the LOCATION clause is the standard Databricks syntax for creating external tables. The LOCATION clause specifies the path to the external data in ADLS.

Key Takeaways

In Databricks, external tables are created using the CREATE TABLE statement combined with the LOCATION clause.

The LOCATION clause is essential for specifying the path to the external data source.

Avoid using CREATE MANAGED TABLE when you want to access data in an external storage location without moving it into Databricks-managed storage.

Remember that Databricks manages the metadata for external tables, but the data itself remains in the external storage location.

Question: 286 CertyIQ

A data engineer needs to conduct Exploratory Analysis on data residing in a database that is within the company's custom-defined network in the cloud. The data engineer is using SQL for this task.
Which type of SQL Warehouse will enable the data engineer to process large numbers of queries quickly and cost-

effectively?

A. Classic SQL Warehouse
B. Pro SQL Warehouse
C. Serverless SQL Warehouse
D. Serverless compute for notebooks

Answer: A

Explanation:

Correct Answer

Option A: Classic SQL Warehouse

Explanation of the Correct Answer

The question asks for the best SQL Warehouse type for exploratory data analysis on data residing in a company's custom-defined network. The data engineer needs to process a large number of queries quickly and cost-effectively using SQL.

Option A, Classic SQL Warehouse, is the correct choice because it provides dedicated compute resources within the customer's VPC/private network. This allows direct access to the data. It's also cost-effective for high-volume query processing, which is needed for exploratory analysis.

Analysis of Each Option

Option A: Classic SQL Warehouse is correct because it runs within the customer's network, allowing access to the data. It's also cost-effective for processing a large number of queries.

Option B: Pro SQL Warehouse is an enhanced version of Classic but is more expensive. It might be overkill for exploratory analysis where cost-effectiveness is a key requirement.

Option C: Serverless SQL Warehouse runs in the cloud provider's network and cannot directly access data in custom-defined private networks. This makes it unsuitable for the scenario.

Option D: Serverless compute for notebooks is designed for notebook-based analytics, not SQL warehousing, so it's not the right tool for SQL-based exploratory analysis.

Key Takeaways

When data resides in a private network, using a Classic SQL Warehouse is often the most straightforward and cost-effective solution for SQL-based analysis.

Serverless SQL Warehouses are not suitable for accessing data in private networks without additional network configurations.

Consider the cost implications of different SQL Warehouse types, especially when dealing with a large number of queries.

Question: 287 CertyIQ

A data engineer is processing ingested streaming tables and needs to filter out NULL values in the order_datetime column from the raw streaming table orders_raw and store the results in a new table orders_valid using DLT. Which code snippet should the data engineer use?

A. SELECT *CREATE OR REPLACE STREAMING TABLE orders_validSELECT *FROM orders_rawWHERE order_datetime IS NOT NULL

B. CREATE OR REPLACE STREAMING TABLE orders_validCONSTRAINT valid_dateORDER_DATETIME IS NOT NULLON VIOLATION DROP ROWAS SELECT * FROM orders_raw

C. SELECTCREATE OR REPLACE STREAMING TABLE orders_validCONSTRAINT valid_date EXPECT (order_datetime IS NOT NULL)ON VIOLATION DROP ROWAS SELECT * FROM orders_raw

D. CREATE OR REPLACE STREAMING TABLE orders_validFILTER (order_datetime IS NOT NULL)AS SELECT * FROM orders_raw

Answer: B

Explanation:

Correct Answer

Option B: CREATE OR REFRESH STREAMING TABLE orders_valid(

CONSTRAINT valid_date

EXPECT (order_datetime IS NOT NULL)

ON VIOLATION DROP ROW

)

AS SELECT * FROM STREAM orders_raw;

Explanation of the Correct Answer

Option B is selected because it uses the EXPECT constraint, which is the recommended DLT (Delta Live Tables) approach for enforcing data quality, including filtering out NULL values. The ON VIOLATION DROP ROW clause ensures that rows violating the constraint (i.e., having NULL values in order_datetime) are dropped.

The EXPECT constraint is a key feature of DLT for data validation and quality control.

Analysis of Each Option

Option A: Incorrect. It uses the STREAM(orders_raw) syntax, which is not the correct way to reference a streaming table in DLT. In DLT, streaming tables are referenced directly by name.

Option B: Correct. It uses the EXPECT constraint with ON VIOLATION DROP ROW, which is the recommended DLT approach for data quality enforcement, including filtering out NULL values. Although the original thought process identifies a syntax error (FROM STREAM orders_raw), it still favors this option due to the correct usage of the EXPECT constraint.

Key Takeaways

The EXPECT constraint with ON VIOLATION DROP ROW is the preferred method in DLT for data quality enforcement.

When working with DLT, it's important to use the correct syntax for referencing streaming tables (i.e., directly by name).

The EXPECT constraint allows you to define data quality rules and specify actions to take when those rules are violated.

Question: 288

CertyIQ

Which languages are supported by Serverless compute clusters? (Choose 2 answers)

A. Python
B. SQL
C. Java
D. Scala
E. R

Answer: A, B

Explanation:

Correct Answer

A. Python

B. SQL

Explanation of the Correct Answer

The question asks about languages supported by Serverless compute clusters. The thought process identifies SQL as the primary language due to the design for SQL workloads. Python is also supported through UDFs, connectors, and integration with Python-based workflows.

The reasoning highlights that Serverless compute clusters are optimized for SQL analytics with Python integration, unlike traditional clusters that support a wider range of languages.

Analysis of Each Option

Option A: Python is correct because Serverless compute clusters support Python through UDFs, connectors, and integration with Python-based workflows.

Option B: SQL is correct because Serverless compute clusters are primarily designed for SQL workloads and natively support SQL queries.

Option C: Java is incorrect because Serverless compute clusters don't provide native Java runtime support, although Databricks supports Java on traditional clusters.

Option D: Scala is incorrect because Scala is supported on regular Databricks clusters but not on Serverless compute clusters.

Option E: R is incorrect because R is not natively supported in the Serverless compute environment.

Key Takeaways

Serverless compute clusters are optimized for SQL analytics workloads with Python integration.

It's important to distinguish between languages supported on traditional clusters versus Serverless compute clusters.

Serverless clusters prioritize SQL and provide Python integration, while traditional clusters offer broader language support.

Question: 289 CertyIQ

How does Databricks Connect enable the engineer to develop, test, and debug code seamlessly on their local machine while interacting with Databricks clusters?

A. By providing a local environment that mimics the Databricks runtime, enabling the engineer to develop, test, and debug code using their preferred IDE

B. By providing a local environment that mimics the Databricks runtime, enabling the engineer to develop, test, and debug code using a specific IDE that is required by Databricks

C. By providing a local environment that mimics the Databricks runtime, enabling the engineer to develop, test, and debug code only through Databricks' own web interface

D. By allowing direct execution of Spark jobs from the local machine without needing a network connection

Answer: A

Explanation:

Correct Answer

Option A: By providing a local environment that mimics the Databricks runtime, enabling the engineer to develop, test, and debug code using their preferred IDE

Explanation of the Correct Answer

Option A is correct because Databricks Connect allows developers to use their preferred IDE to connect to a remote Databricks cluster. It simulates the Databricks runtime environment locally, which allows for consistent behavior between local development and cluster execution.

Databricks Connect is a client library that bridges the local machine and the Databricks runtime environment.

Analysis of Each Option

Option A: Correct. This option accurately describes the functionality of Databricks Connect.

Option B: Incorrect. Databricks Connect does not require a specific IDE. It supports various IDEs.

Option C: Incorrect. Databricks Connect enables local development, not just development through the Databricks web interface.

Option D: Incorrect. Databricks Connect requires a network connection to communicate with the remote Databricks cluster.

Key Takeaways

Databricks Connect allows developers to use their local IDE to develop, test, and debug code that runs on a remote Databricks cluster.

It requires a network connection to the Databricks cluster.

It does not restrict developers to a specific IDE.

Question: 290 CertyIQ

A data engineer is working on a Databricks project that utilizes cloud storage. The data engineer wants to load several JSON files from containers on a storage account as soon as the file arrives within the storage account. Which syntax should the data engineer follow to first load the files into a dataframe and check that it is working as

expected using Python

A. df = spark.readStream.format("cloudFiles") .option("cloudFiles.formal", "json") .load("/input/path")
B. df = spark.readStream.format("cloud").option("json").load("/input/path"
C. spark.readStream.format("json").load("input/path")
D. spark.read.json("input/path")

Answer: A

Explanation:

Correct Answer

Option A: df = spark.readStream.format("cloudFiles") .option("cloudFiles.format", "json") .load("/input/path")

Explanation of the Correct Answer

Option A is correct because it uses spark.readStream to create a streaming DataFrame, which is necessary for processing data as it arrives. It specifies the cloudFiles format, which is designed for efficiently reading files from cloud storage as they appear. The cloudFiles.format option is set to "json" to indicate the file type, and the load function specifies the path to the cloud storage directory.

Analysis of Each Option

Option A: Correct. This option correctly uses spark.readStream with the cloudFiles format and specifies the JSON format.

Option B: Incorrect. The format("cloud") is not a valid format option for streaming data from cloud storage. It also lacks the specific cloudFiles source needed for efficient file discovery, and the option option("json") is incomplete.

Option C: Incorrect. While this reads JSON files in streaming, it doesn't leverage the cloudFiles source. Without cloudFiles, Spark will not automatically discover new files as they arrive in cloud storage in an efficient and scalable manner.

Option D: Incorrect. This uses spark.read, which is for reading a static dataset, not a streaming one. It won't process new files as they arrive.

Key Takeaways

The spark.readStream function is essential for creating streaming DataFrames in Spark.

The cloudFiles format is designed for efficiently reading files from cloud storage as they appear.

When working with streaming data, it's important to use the appropriate functions and formats to ensure that new data is processed as it arrives.

Avoid using spark.read for streaming data, as it is intended for reading static datasets.

Question: 291 CertyIQ

A data engineer is developing an ETL process based on Spark SQL. The execution fails. The data engineer checks the Spark UI and can see the ERRORS as follows:
"java.lang.OutofMemoryError: Java heap space"
Which two corrective actions should the data engineer perform to resolve this issue? [Choose 2 answers]

A. Narrow the filters in order to collect less data in the query
B. Upsize the worker nodes and activate autoshuffle partitions
C. Upsize the driver node and deactivate autoshuffle partitions
D. Cache the dataset in order to boost the query performance
E. Fix the shuffle partitions to 50 to ensure the allocation

Answer: A,B

Explanation:

Correct Answer

Options A and B are correct.

A: Narrow the filters in order to collect less data in the query

B: Upsize the worker nodes and activate autoshuffle partitions

Explanation of the Correct Answer

The error "java.lang.OutofMemoryError: Java heap space" indicates that the driver or worker nodes are running out of memory during the ETL process.

Option A is correct because reducing the amount of data processed by applying more selective filters early in the query reduces memory pressure on both driver and worker nodes.

Option B is correct because increasing worker node memory provides more heap space to handle data processing. Activating autoshuffle partitions helps Spark automatically optimize partition sizes based on data statistics, which can prevent memory issues caused by data skew and improve memory utilization.

Analysis of Each Option

Option A: Correct. Narrowing filters reduces the amount of data processed, alleviating memory pressure.

Option B: Correct. Upsizing worker nodes increases available memory, and autoshuffle partitions optimize memory utilization.

Option C: Incorrect. Deactivating autoshuffle partitions removes Spark's ability to dynamically optimize partitioning, potentially worsening memory issues.

Option D: Incorrect. Caching increases memory usage, exacerbating the OutOfMemoryError.

Option E: Incorrect. Setting a fixed low number of partitions can create larger partitions that consume more memory per partition, potentially worsening the memory issue.

Key Takeaways

The "java.lang.OutofMemoryError: Java heap space" error in Spark indicates memory exhaustion on the driver or worker nodes.

Reducing data volume and increasing available memory are effective strategies for resolving this issue.

Avoid caching data when facing OutOfMemoryError, as it will worsen the problem.

Adaptive query execution (autoshuffle partitions) can help optimize memory utilization.

Question: 292 CertyIQ

An organization has implemented a data pipeline in Databricks and needs to ensure it can scale automatically based on varying workloads without manual cluster management. The goal is to meet the company's Service Level Agreements (SLAs), which require high availability and minimal downtime, while Databricks automatically handles resource allocation and optimization.
Which approach fulfills these requirements?

A. Use Spot Instances to allocate resources dynamically while minimizing costs, with potential interruptions.

B. Use Serverless compute in Databricks to automatically scale and provision resources with minimal manual intervention.

C. Use Interactive Clusters in Databricks, adjusting cluster sizes manually based on workload demands.

D. Deploy Job Clusters with fixed configurations, dedicated to specific tasks, without automatic scaling.

Answer: B

Explanation:

Correct Answer

Option B: Use Serverless compute in Databricks to automatically scale and provision resources with minimal manual intervention.

Explanation of the Correct Answer

Serverless compute in Databricks automatically scales resources based on workload demands without manual intervention. Databricks handles resource provisioning and optimization, ensuring high availability with automatic failover and recovery, and minimal downtime. This approach perfectly aligns with the requirements of automatic scaling, no manual cluster management, high availability, and automatic resource allocation.

Analysis of Each Option

Option A: Incorrect. Spot Instances can be interrupted, conflicting with the SLA requirements for high availability and minimal downtime.

Option B: Correct. Serverless compute automatically scales resources, requires zero manual cluster management, and ensures high availability with minimal downtime.

Option C: Incorrect. Interactive Clusters require manual adjustment of cluster sizes, contradicting the requirement for no manual cluster management.

Option D: Incorrect. Job Clusters with fixed configurations do not provide automatic scaling, failing to adapt to varying workloads.

Key Takeaways

Serverless compute is ideal for scenarios requiring automatic scaling, minimal manual intervention, and high availability.

Spot Instances are cost-effective but unsuitable for SLA-critical workloads due to potential interruptions.

Manual cluster management is inefficient and error-prone in dynamic environments.

Fixed configurations are not adaptable to varying workloads.

Question: 293 CertyIQ

A data engineer is configuring Unity Catalog in Databricks and needs to assign a role to a user who should have the ability to grant and revoke privileges on various data objects within a specific schema, but should not have read/write access over the schema or its objects.
Which role should the data engineer assign to this user?

A. Catalog Owner
B. Schema Owner
C. USE catalog/schema privilege on the schema
D. Table Owner

Answer: C

Explanation:

Correct Answer

Option C: USE catalog/schema privilege on the schema

Explanation of the Correct Answer

The USE catalog/schema privilege is the correct answer because it allows a user to access the metadata of a catalog or schema, which is necessary to grant and revoke privileges on objects within that schema.

This privilege does not grant read/write access to the actual data, fulfilling the requirement that the user should not have data access.

The USE catalog/schema privilege enables administrative tasks related to permissions without providing data access.

Analysis of Each Option

Option A: Catalog Owner is incorrect because it grants full administrative rights over the entire catalog, including read/write access, which violates the requirement of no read/write access.

Option B: Schema Owner is incorrect because it provides full control over the schema, including read/write access to all objects, which violates the requirement of no read/write access.

Option C: USE catalog/schema privilege on the schema is correct because it allows a user to manage privileges on objects within the schema without granting read/write access to the data.

Option D: Table Owner is incorrect because it is specific to individual tables and does not provide schema-wide privilege management capabilities.

Key Takeaways

The USE catalog/schema privilege is designed for administrative users who need to manage permissions without accessing the underlying data.

Understanding the specific privileges associated with different roles is crucial for configuring Unity Catalog correctly.

When assigning roles, always consider the principle of least privilege to ensure users only have the necessary permissions.

Question: 294

CertyIQ

A data engineer streams customer orders into a Kafka topic (orders_topic) and is currently writing the ingestion script of a DLT pipeline.
The data engineer needs to ingest the data from Kafka brokers to DLT using Databricks.
What is the correct code for ingesting the data?

A. import dlt@dlt.table(name = "orders_raw",)def orders_raw():return (spark.readstream.format("cloudFiles").option("cloudFiles.format", "json").option("cloudFiles.schemaLocation", "/schema/location").load("kafka://broker:9092/orders_topic"))

B. import dlt@dlt.table(name = "orders_raw",)def orders_raw():return (spark.readstream.format("kafka").option("kafka.bootstrap.servers", "broker:9092").option("subscribe", "orders_topic").load())

C. import dlt@dlt.table(name = "orders_raw")def orders_raw():return (spark.readstream.format("cloudFiles").option("cloudFiles.format", "json").load("kafka://broker:9092/orders_topic"))

D. import dlt@dlt.table(name = "orders_raw")def orders_raw():return (spark.readstream.format("kafka").option("kafka.bootstrap.servers", "broker:9092").option("subscribe", "orders_topic").option("startingOffsets", "earliest").load())

Answer: D

Explanation:

Correct Answer

Option D.

import dlt

@dlt.table(name = "orders_raw")

def orders_raw():

return (

spark.readstream

.format("kafka")

.option("kafka.bootstrap.servers", "broker:9092")

.option("subscribe", "orders_topic")

.option("startingOffsets", "earliest")

.load()

)

Explanation of the Correct Answer

Option D is correct because it uses the kafka format to read from Kafka, specifies the bootstrap servers, subscribes to the correct topic, and includes the startingOffsets option, which is important for controlling where to start reading from the Kafka topic. This makes it the most complete and robust implementation for a production DLT pipeline.

The kafka format is specifically designed for reading data from Kafka streams in Spark. The kafka.bootstrap.servers option tells Spark where to find the Kafka brokers. The subscribe option specifies the Kafka topic to read from. The startingOffsets option allows you to specify where to start reading from the topic (e.g., "earliest" to read from the beginning).

Analysis of Each Option

Option A: Incorrect. It uses the cloudFiles format, which is for reading files from cloud storage, not Kafka streams.

Option B: Correct, but not as complete as Option D. It uses the correct kafka format and specifies the bootstrap servers and topic, but it doesn't include the startingOffsets option.

Option C: Incorrect. It uses the cloudFiles format, which is not appropriate for reading from Kafka.

Option D: Correct. It uses the correct kafka format, specifies the bootstrap servers and topic, and includes the startingOffsets option.

Key Takeaways

When reading from Kafka in Spark, use the kafka format.

Always specify the kafka.bootstrap.servers option to tell Spark where to find the Kafka brokers.

Use the subscribe option to specify the Kafka topic to read from.

Consider using the startingOffsets option to control where to start reading from the topic.

Avoid using the cloudFiles format for reading from Kafka streams.

Question: 295 CertyIQ

A data engineer is inspecting an ETL pipeline based on a Pyspark job that consistently encounters performance bottlenecks. Based on developer feedback, the data engineer assumes the job is low on compute resources.
To pinpoint the issue, the data engineer observes the Spark UI and finds out the job has a high CPU time vs Task time.
Which course of action should the data engineer take?

A. High CPU time vs Task time means an under-utilized cluster. The data engineer may need to repartition data to spread the jobs more evenly throughout the cluster.

B. High CPU time vs Task time means efficient use of cluster and no change needed

C. High CPU time vs Task time means over-utilized memory and the need to increase parallelism

D. High CPU time vs Task time means a CPU over-utilized job. The data engineer may need to consider executor and core tuning or resizing the cluster

Answer: D

Explanation:

Correct Answer

Option D: High CPU time vs Task time means a CPU over-utilized job. The data engineer may need to consider executor and core tuning or resizing the cluster

Explanation of the Correct Answer

The problem describes a Spark job with a high CPU time relative to task time. This means the CPU is heavily utilized and is the bottleneck. The correct action is to address the CPU over-utilization by tuning executors/cores or resizing the cluster.

High CPU time vs Task time indicates that the majority of the time spent on a task is dedicated to CPU computation, with minimal time spent waiting for I/O or other resources. This signifies a CPU-bound workload.

The thought process correctly identifies that the high CPU time relative to task time means the CPUs are working at or near capacity, and the bottleneck is computational power.

Analysis of Each Option

Option A: Incorrect. High CPU time vs Task time indicates the CPUs are heavily utilized, not under-utilized. Under-utilization would show low CPU time relative to task time.

Option B: Incorrect. While high CPU utilization might seem efficient, it can still cause performance bottlenecks if the workload exceeds CPU capacity.

Option C: Incorrect. This ratio doesn't directly indicate memory over-utilization. Memory issues would typically manifest differently in Spark metrics.

Option D: Correct. High CPU time vs Task time indicates a CPU-bound workload where CPUs are working at or near capacity. Solutions include adding more CPU cores per executor, increasing the number of executors, or scaling up the cluster.

Key Takeaways

High CPU time vs Task time indicates a CPU-bound workload.

Consider executor and core tuning or resizing the cluster to address CPU over-utilization.

Avoid confusing CPU-bound issues with memory or I/O-related bottlenecks.

Question: 296 CertyIQ

Which of the following describes the storage organization of a Delta table?

A. Delta tables are stored in a single file that contains data, history, metadata, and other attributes.
B. Delta tables store their data in a single file and all metadata in a collection of files in a separate location.
C. Delta tables are stored in a collection of files that contain data, history, metadata, and other attributes.
D. Delta tables are stored in a collection of files that contain only the data stored within the table.
E. Delta tables are stored in a single file that contains only the data stored within the table.

Answer: C

Explanation:

Correct Answer: C

Explanation:

The correct answer is C because Delta Lake, the technology behind Delta tables, is built on top of cloud storage like AWS S3, Azure Blob Storage, or Google Cloud Storage. It leverages the distributed nature of these object storage systems for scalability and reliability.

Delta Lake doesn't store all data in a single file. Instead, data is partitioned into multiple data files (typically Parquet format), and these files are stored in the underlying cloud storage. This allows for parallel processing and efficient querying.

The "history" and "metadata" are critical components of Delta Lake's functionality, enabling features like time travel, ACID transactions, and schema evolution. This metadata isn't stored within each data file but rather as a transaction log alongside the data files. The transaction log is an ordered record of every operation

performed on the Delta table.

Specifically, each commit to a Delta table creates a new entry (a metadata file) in the _delta_log directory, which is typically located alongside the data files in the same storage location. This log tracks changes to the data, including additions, deletions, and updates. These metadata files (often stored in JSON format) contain information about the data files involved in each transaction, enabling Delta Lake to track the table's state over time.

Therefore, a Delta table isn't a single file. It's a directory containing data files and a _delta_log directory containing metadata files. Answer C accurately reflects this distributed storage model, encompassing both the data and the necessary metadata for transaction management and data versioning. The distributed storage enables efficient processing and scaling provided by the underlying cloud storage layer. Options A, B, D, and E are incorrect because they imply a single-file storage or an incomplete view of how metadata is managed within a Delta table.

Question: 297 CertyIQ

A data engineer is writing a script that is meant to ingest new data from cloud storage. In the event of the Schema change, the ingestion should fail.
It should fail until the changes downstream source can be found and verified as intended changes.
Which command will meet the requirements?

A. addNewColumns
B. none
C. rescue
D. failonNewColumns

Answer: D

Explanation:

Correct Answer

Option D: failonNewColumns

Explanation of the Correct Answer

The question requires a command that will cause the data ingestion process to fail when schema changes (specifically new columns) are detected. This is to ensure that any unintended schema changes are caught and verified before being ingested. The option failonNewColumns directly addresses this requirement by causing the process to fail when new columns are detected.

Analysis of Each Option

Option A: addNewColumns - This option would automatically add new columns, which is the opposite of the desired behavior. Therefore, it is incorrect.

Option B: none - This is not a valid command and does not provide the required functionality. Therefore, it is incorrect.

Option C: rescue - This option is used for error handling and recovery, which would prevent the process from failing. Therefore, it is incorrect.

Option D: failonNewColumns - This option explicitly causes the process to fail when new columns are detected, which meets the requirement. Therefore, it is correct.

Key Takeaways

When dealing with schema changes in data ingestion, it's important to have mechanisms to handle unexpected changes.

The failonNewColumns command (or similar functionality) can be used to ensure that any schema changes are manually verified before being ingested.

Avoid using commands that automatically add new columns without verification, as this can lead to data quality issues.

Question: 298 CertyIQ

A data engineer has been provided a PySpark DataFrame named **df** with columns product and revenue. The data engineer needs to compute complex aggregations to determine each product's total revenue, average revenue, and transaction count.
Which code snippet should the data engineer use?

A. from pyspark.sql import functions as Faggregated_df = df.groupBy("product").agg(F.sum("revenue").alias("total_revenue"), F.avg("revenue").alias("avg_revenue"), F.count("*").alias("transaction_count"))

B. aggregated_df = df.groupby("product").agg("sum(revenue)","avg(revenue)","count(revenue)")

C. from pyspark.sql import functions as F aggregated_df = df.select("product", "revenue").groupBy("product").agg(F.sum("revenue"), F.mean("revenue"))

D. aggregated_df = df.groupBy("product").agg("revenue": "sum", "revenue": "avg", "revenue": "count")

Answer: A

Explanation:

Correct Answer

Option A: from pyspark.sql import functions as F

aggregated_df = df.groupBy("product").agg(F.sum("revenue").alias("total_revenue"), F.avg("revenue").alias("avg_revenue"), F.count("*").alias("transaction_count"))

Explanation of the Correct Answer

Option A is correct because it uses the pyspark.sql.functions module to perform the required aggregations: sum, average, and count. It groups the DataFrame by the "product" column and then calculates the sum of "revenue" (aliased as "total_revenue"), the average of "revenue" (aliased as "avg_revenue"), and the count of all rows in each group (aliased as "transaction_count"). The use of aliases makes the resulting DataFrame more readable.

The pyspark.sql.functions module provides functions for performing common data manipulation tasks, including aggregations. The groupBy method is used to group rows based on one or more columns, and the agg method is used to apply aggregation functions to each group. The alias method is used to rename columns in the resulting DataFrame.

Analysis of Each Option

Option A: Correct. It correctly calculates the total revenue, average revenue, and transaction count for each product using PySpark functions and assigns appropriate aliases.

Option B: Incorrect. It uses string-based aggregation, which is less readable and doesn't provide column

aliases. Also, count(revenue) will only count non-null values of revenue, not the total number of transactions.

Option C: Incorrect. It only calculates the sum and mean of revenue, missing the transaction count. It also includes an unnecessary select operation and lacks column aliases.

Option D: Incorrect. Due to the way Python dictionaries work, only the last aggregation (count) would be retained, and the sum and average aggregations would be lost.

Key Takeaways

When performing aggregations in PySpark, it's best practice to use the functions provided in the pyspark.sql.functions module.

Always use aliases to give meaningful names to the resulting aggregation columns.

Be careful when using dictionaries for aggregations, as duplicate keys can lead to unexpected results.

F.count("*") is used to count all rows in a group, while F.count("column_name") counts only the non-null values in the specified column.

Question: 299 CertyIQ

A company is collaborating with a partner that does not use Databricks but needs access to a large historical dataset stored in Delta format.
The data engineer needs to ensure that the partner can access the data securely, without the need for them to set up an account, and with read-only access.
How should the data be shared?

A. Share the dataset using Unity Catalog, ensuring that both teams have full write access to the data within the same organization.

B. Share the dataset by exporting it to a CSV file and manually transferring the file to the partner's system.

C. Share the dataset using Delta Sharing, which allows your partner to access the data using a secure, read-only URL without requiring a Databricks account, ensuring that they cannot modify the data.

D. Grant your partner access to your Databricks workspace and assign them full write permissions to the Delta table, enabling them to modify the dataset.

Answer: C

Explanation:

Correct Answer

Option C: Share the dataset using Delta Sharing, which allows your partner to access the data using a secure, read-only URL without requiring a Databricks account, ensuring that they cannot modify the data.

Explanation of the Correct Answer

Option C is correct because Delta Sharing is designed for securely sharing Delta tables with external partners who don't use Databricks. It provides read-only access without requiring the partner to have a Databricks account. This aligns perfectly with the requirements of the problem.

Delta Sharing allows sharing data in Delta format while ensuring the partner can only read the data, maintaining security and preventing modifications.

Analysis of Each Option

Option A: Incorrect. Unity Catalog sharing typically requires both parties to be within the Databricks

ecosystem, and the mention of "full write access" contradicts the read-only requirement.

Option B: Incorrect. Exporting to CSV loses the benefits of the Delta format and is impractical for large datasets. Manual transfer introduces security and scalability issues.

Option C: Correct. Delta Sharing is designed for sharing Delta tables with external partners who don't use Databricks, providing secure, read-only access without requiring account creation.

Option D: Incorrect. Granting workspace access and write permissions contradicts the requirements for no account setup and read-only access.

Key Takeaways

Delta Sharing is the preferred method for sharing Delta tables with external partners who do not have Databricks accounts.

Always consider the security implications and access requirements when sharing data.

Avoid granting write access unless explicitly required.

Question: 300 CertyIQ

Which SQL code snippet will correctly demonstrate a Data Definition Language (DDL) operation used to create a table?

A. CREATE TABLE employees (id INT,name STRING);
B. INSERT INTO employees (id, name) VALUES (1, 'Alice');
C. ALTER TABLE employees ADD COLUMN salary DECIMAL(10,2);
D. DROP TABLE employees;

Answer: A

Explanation:

Correct Answer

Option A: CREATE TABLE employees (id INT, name STRING);

Explanation of the Correct Answer

Option A is correct because the CREATE TABLE statement is a Data Definition Language (DDL) command used to define the structure of a new table. It specifies the table name (employees) and the columns (id, name) along with their respective data types (INT, STRING).

DDL operations are used to define and manage the structure of database objects.

Analysis of Each Option

Option A: This option is correct. The CREATE TABLE statement is the standard SQL command for creating a new table.

Option B: This option is incorrect. The INSERT INTO statement is a Data Manipulation Language (DML) command used to add data to an existing table, not to create a table.

Option C: This option is incorrect. The ALTER TABLE statement is a DDL command used to modify the

structure of an existing table (e.g., adding a column), not to create a new table.

Option D: This option is incorrect. The DROP TABLE statement is a DDL command used to delete an existing table, not to create one.

Key Takeaways

The CREATE TABLE statement is fundamental for defining the structure of tables in a relational database.

DDL commands are distinct from DML commands, which are used for manipulating data within tables.

Understanding the difference between DDL and DML is crucial for database management.

Common mistake: Confusing DDL operations like ALTER TABLE or DROP TABLE with the CREATE TABLE operation.

Question: 301 CertyIQ

What is the maximum output supported by a job cluster to ensure a notebook does not fail?

A. 10MBs
B. 25MBs
C. 15MBs
D. 30MBs

Answer: C

Explanation:

Correct Answer

Option C: 15MBs

Explanation of the Correct Answer

The question asks for the maximum output size a job cluster can support without causing a notebook to fail. The thought process identifies that Databricks job clusters have a limit to prevent excessive memory usage. While the ideal limit is 20MB (not in the options), the closest and safest value among the choices is 15MB. This ensures the notebook stays within acceptable bounds and avoids failure.

Analysis of Each Option

Option A: 10MBs - This option is below the typical limit and safe, but not the maximum safe value among the choices. Therefore, it's not the best answer.

Option B: 25MBs - This option exceeds the typical 20MB limit, making it likely to cause a failure. Thus, it's incorrect.

Option C: 15MBs - This option is below the typical limit and represents the highest safe value among the given choices. Therefore, it's the correct answer.

Option D: 30MBs - This option significantly exceeds the limit, making it very likely to cause a failure. Thus, it's incorrect.

Key Takeaways

Job clusters have output limits to prevent memory issues and ensure stability.

When the exact limit isn't an option, choose the next highest value that is still safely below the known limit.

Always consider the consequences of exceeding the limit (notebook failure) when selecting an option.

Question: 302 CertyIQ

A data engineer needs to provide access to a group named manufacturing-team. The team needs privileges to create tables in the quality schema.
Which set of SQL commands will grant a group named manufacturing-team to create tables in a schema named production with the parent catalog named manufacturing with the least privileges?

A. GRANT USE TABLE ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT USE SCHEMA ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT USE CATALOG ON CATALOG manufacturing TO manufacturing-team;

B. GRANT CREATE TABLE ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT CREATE SCHEMA ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT USE CATALOG ON CATALOG manufacturing TO manufacturing-team;

C. GRANT CREATE TABLE ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT USE SCHEMA ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT USE CATALOG ON CATALOG manufacturing TO manufacturing-team;

D. GRANT CREATE TABLE ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT CREATE SCHEMA ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT CREATE CATALOG ON CATALOG manufacturing TO manufacturing-team;

Answer: C

Explanation:

Correct Answer

Option C: GRANT CREATE TABLE ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT USE SCHEMA ON SCHEMA manufacturing.quality TO manufacturing-team; GRANT USE CATALOG ON CATALOG manufacturing TO manufacturing-team;

Explanation of the Correct Answer

Option C is correct because it provides the minimum required privileges for the manufacturing-team to create tables in the quality schema within the manufacturing catalog. The team needs USE CATALOG to access the catalog, USE SCHEMA to access the schema, and CREATE TABLE to create tables within that schema.

Analysis of Each Option

Option A: Incorrect because GRANT USE TABLE ON SCHEMA is incorrect syntax and doesn't grant table creation rights. It is also missing the CREATE TABLE privilege.

Option B: Incorrect because GRANT CREATE SCHEMA ON SCHEMA is unnecessary and grants more privileges than needed.

Option C: Correct because it grants CREATE TABLE on the schema, USE SCHEMA on the schema, and USE CATALOG on the catalog, which are the minimum required privileges.

Option D: Incorrect because GRANT CREATE SCHEMA ON SCHEMA and GRANT CREATE CATALOG ON CATALOG are unnecessary and grant far more privileges than needed.

Key Takeaways

When granting privileges, always aim for the principle of least privilege, granting only the necessary permissions.

Understand the hierarchy of permissions: catalog, schema, and then table-level privileges.

Avoid granting CREATE privileges on catalogs or schemas unless explicitly required.

Question: 303 CertyIQ

A data engineer at a company that uses Databricks with Unity Catalog needs to share a collection of tables with an external partner who also uses a Databricks workspace enabled for Unity Catalog. The data engineer decides to use Delta Sharing to accomplish this.
What is the first piece of information the data engineer should request from the external partner to set up Delta Sharing?

A. Their Databricks account password
B. The IP address of their Databricks workspace
C. The name of their Databricks cluster
D. The sharing identifier of their Unity Catalog metastore

Answer: D

Explanation:

Correct Answer

Option D: The sharing identifier of their Unity Catalog metastore

Explanation of the Correct Answer

The correct answer is D because Delta Sharing with Unity Catalog requires the sharing identifier of the recipient's Unity Catalog metastore to establish a secure sharing relationship. This identifier uniquely identifies the metastore to which the data will be shared.

Sharing identifiers are essential for directing the shared data to the correct Unity Catalog metastore.

Analysis of Each Option

Option A: Incorrect. Sharing account passwords would be a severe security breach and is not required for Delta Sharing.

Option B: Incorrect. IP addresses are not used for Delta Sharing setup, as the sharing occurs through secure protocols at the metastore level.

Option C: Incorrect. Cluster names are irrelevant for Delta Sharing, which operates at the metastore level, not the compute level.

Option D: Correct. The sharing identifier of the Unity Catalog metastore is the fundamental piece of information needed to establish a Delta Sharing relationship.

Key Takeaways

Delta Sharing relies on secure identifiers to establish sharing relationships between Unity Catalog metastores.

Never share account passwords or rely on IP addresses for secure data sharing.

Understanding the role of the Unity Catalog metastore and its sharing identifier is crucial for setting up Delta Sharing.

Question: 304 CertyIQ

A company uses Delta Sharing to collaborate with partners across different cloud providers and geographic regions.
What will result in additional costs due to cross-region or egress fees?

A. Transferring data via Delta Sharing across clouds and across different geographic regions
B. Accessing Delta Sharing data using a VPN within the same data center
C. Sharing data within the same cloud provider and region
D. Utilizing Delta Sharing for internal data analytics within a single cloud environment

Answer: A

Explanation:

Correct Answer

Option A: Transferring data via Delta Sharing across clouds and across different geographic regions

Explanation of the Correct Answer

Option A is correct because transferring data across different cloud providers (e.g., from AWS to Azure) incurs egress fees, which are charges for data leaving the cloud provider's network. Additionally, transferring data between different geographic regions incurs cross-region transfer costs. Both of these scenarios lead to additional costs.

Analysis of Each Option

Option A: This option is correct. Transferring data across clouds results in egress fees, and transferring across regions results in cross-region fees.

Option B: This option is incorrect. Accessing data via VPN within the same data center typically does not incur additional transfer fees because the data remains within the same network boundary.

Option C: This option is incorrect. Sharing data within the same cloud provider and region usually has minimal or no cost, as there are no cross-region or egress fees.

Option D: This option is incorrect. Utilizing Delta Sharing for internal analytics within a single cloud environment does not involve data leaving the cloud provider's network, so there are no egress fees or cross-region movement.

Key Takeaways

Cloud providers charge for data transfer in scenarios like cross-region transfers, egress fees (data leaving the cloud), and cross-cloud transfers.

When using Delta Sharing in multi-cloud environments, be aware of egress and cross-region fees, as they can significantly impact costs.

Data transfer within the same cloud provider and region is generally less expensive.

Question: 305 CertyIQ

A data engineer has developed an ETL that produce a Delta managed table with liquid clustering feature activated as output.
Several consumers are having issues regarding time delay when reading this table.
How could the Data Engineer be sure about the OPTIMIZE command has been executed explicitly?

A. Use SHOW TABLES EXTENDED to check the partitions columns used
B. Use DESCRIBE HISTORY table to check if exists any OPTIMIZE operation
C. Check the system table system.storage.predictive_optimization_operations_history
D. Use DESCRIBE DETAIL table to see the file size and number of files for the table

Answer: B

Explanation:

Correct Answer

Option B: Use DESCRIBE HISTORY table to check if exists any OPTIMIZE operation

Explanation of the Correct Answer

The correct way to check if the OPTIMIZE command has been executed on a Delta table is to use DESCRIBE HISTORY table. This command provides a history of all operations performed on the table, including OPTIMIZE operations.

The DESCRIBE HISTORY command shows the operation type, timestamp, user, and details about the optimization. This allows you to confirm if the OPTIMIZE command was executed explicitly.

Analysis of Each Option

Option A: SHOW TABLES EXTENDED provides metadata about tables but does not show operation history or partition information relevant to OPTIMIZE operations. Therefore, this option is incorrect.

Option B: DESCRIBE HISTORY table shows the complete history of operations performed on a Delta table, including OPTIMIZE operations. This allows you to confirm if the OPTIMIZE command was executed explicitly. Therefore, this option is correct.

Option C: system.storage.predictive_optimization_operations_history is related to auto-optimization features, not explicit OPTIMIZE commands executed manually. Therefore, this option is incorrect.

Option D: DESCRIBE DETAIL shows current table statistics (file count, sizes) but does not indicate whether OPTIMIZE was explicitly executed or when it occurred. Therefore, this option is incorrect.

Key Takeaways

The DESCRIBE HISTORY command is crucial for auditing and understanding the operations performed on a Delta table.

It's important to distinguish between explicit OPTIMIZE commands and auto-optimization features.

When troubleshooting performance issues, checking the table's history can provide valuable insights.

Question: 306 CertyIQ

A data engineering team is using Kafka to capture event data and then ingest it into Databricks. The team wants to be able to see these historical events. Medallion architecture is already in place. The team wants to be mindful of costs.
Where should this historical event data be stored?

A. Gold
B. Silver
C. Bronze
D. Raw layer

Answer: C

Explanation:

Correct Answer

Option C: Bronze

Explanation of the Correct Answer

The Bronze layer is the correct place to store historical event data because it's designed to store raw, unprocessed data exactly as it's ingested. This ensures a complete historical record is maintained with full fidelity. It's also the most cost-effective storage option for long-term retention, serving as the source of truth for all downstream processing.

Analysis of Each Option

Option A: Gold: Incorrect. The Gold layer contains highly aggregated, business-ready datasets and is the most expensive due to extensive processing. It typically contains summarized views, not detailed historical events.

Option B: Silver: Incorrect. The Silver layer contains cleaned, validated, and deduplicated data. It's more expensive than the Bronze layer due to processing overhead and may not retain all historical nuances.

Option C: Bronze: Correct. The Bronze layer stores raw, unprocessed data exactly as ingested, maintaining a complete historical record with full fidelity and cost-effective storage for long-term retention.

Option D: Raw layer: Incorrect. While the term "Raw layer" is similar in concept to the Bronze layer, the question specifically refers to the Medallion architecture, where the standard terminology is Bronze.

Key Takeaways

The Medallion architecture consists of Bronze, Silver, and Gold layers, each serving a specific purpose in data processing and storage.

The Bronze layer is ideal for storing raw, historical data due to its cost-effectiveness and ability to maintain complete data fidelity.

Understanding the purpose of each layer in the Medallion architecture is crucial for making informed decisions about data storage and processing.

Question: 307 CertyIQ

How are events formatted in Databricks audit logs?

A. In Databricks, audit logs output events in a plain text format.
B. In Databricks, audit logs output events in an XML format.

C. In Databricks, audit logs output events in a CSV format.
D. In Databricks, audit logs output events in a JSON format.

Answer: D

Explanation:

Correct Answer

Option D: In Databricks, audit logs output events in a JSON format.

Explanation of the Correct Answer

The correct answer is JSON format because Databricks audit logs need to store structured data, support nested data, and be easily parsed by log analysis tools. JSON fulfills these requirements effectively.

JSON (JavaScript Object Notation) is a standard format for data interchange, widely used in APIs and logging systems due to its human-readable and machine-parsable nature.

Analysis of Each Option

Option A: Plain text format is incorrect because it cannot efficiently handle structured data with multiple fields and nested information.

Option B: XML format is incorrect because, although structured, it is verbose and less commonly used for modern log analysis systems compared to JSON.

Option C: CSV format is incorrect because it is limited in handling nested data structures and complex event metadata that audit logs typically contain.

Option D: JSON format is correct because it provides structured data representation, supports nested objects, is widely supported by log analysis tools, and is the standard format for modern API responses and structured logging.

Key Takeaways

Databricks audit logs use JSON format for structured, machine-readable logging.

JSON is preferred for its flexibility, ease of integration, and support for complex data structures.

When dealing with structured logging in modern systems, JSON is often the correct choice.

Question: 308 CertyIQ

An organization is looking for an optimized storage layer that supports ACID transactions and schema enforcement.
Which technology should the organization use?

A. Unity Catalog
B. Data lake
C. Delta Lake
D. Cloud File Storage

Answer: C

Explanation:

Correct Answer

Option C: Delta Lake

Explanation of the Correct Answer

Delta Lake is the correct choice because it's designed to bring reliability to data lakes by adding ACID transactions and schema enforcement. It achieves ACID properties through transaction logs and supports schema enforcement and evolution.

The key concepts here are ACID transactions (Atomicity, Consistency, Isolation, Durability) and schema enforcement, which are crucial for data reliability and consistency. Delta Lake is built to provide these features on top of existing data lakes.

Analysis of Each Option

Option A: Unity Catalog is incorrect because it's a governance layer, not a storage layer. It manages metadata and access control but doesn't provide storage capabilities or ACID transactions.

Option B: Data lake is incorrect because it's a generic term for storing raw data and typically doesn't support ACID transactions or have built-in schema enforcement.

Option C: Delta Lake is correct because it's a storage layer built on top of data lakes that provides ACID transactions through transaction logs and supports schema enforcement and evolution.

Option D: Cloud File Storage is incorrect because it's a basic file storage service without ACID transaction support or schema enforcement capabilities.

Key Takeaways

Delta Lake is designed to address the limitations of traditional data lakes by adding ACID transaction support and schema enforcement.

When a question mentions ACID transactions and schema enforcement, Delta Lake is often the correct answer in the context of data storage solutions.

Remember that Unity Catalog is for governance, not storage.

Question: 309 CertyIQ

A company sells products across multiple categories (e.g., Electronics, Clothing) and regions.
The sales team has provided you with a PySpark dataframe named sales_df as below, and the team wants the data engineer to analyze the sales data to help make strategic decisions.

sales_df

product_id	category	sales_amount	region
1	Electronics	100	North
2	Clothing	200	South
3	Electronics	150	North
4	Clothing	300	South
5	Electronics	250	East
6	Clothing	400	West

Calculate the total sales amount for each region and store the results in a new dataframe called region_sales.
Given the total sales amount for each region and store the results in a new dataframe called region_sales.

Expected result:

Region	1.2 sales_amount
North	250
South	500
East	250
West	400

Which code will generate the expected result?

A. region_sales = sales_df.groupBy("region").agg(sum("sales_amount").alias("total_sales_amount"))
B. region_sales = sales_df.agg(sum("sales_amount").alias("total_sales_amount")).groupBy("region")
C. region_sales = sales_df.groupBy("region").agg(sales("total_sales_amount"))
D. region_sales = sales_df.groupBy("region").agg(sales_df("total_sales_amount"))

Answer: A

Explanation:

Correct Answer

Option A: region_sales = sales_df.groupBy("region").agg(sum("sales_amount").alias("total_sales_amount"))

Explanation of the Correct Answer

Option A is correct because it first groups the sales_df by the "region" column using groupBy("region"). Then, it calculates the sum of the "sales_amount" for each region using the agg function with sum("sales_amount"). Finally, it renames the resulting aggregated column to "total_sales_amount" using .alias("total_sales_amount"). This produces the desired region_sales dataframe with the total sales amount for each region.

Analysis of Each Option

Option A: This option is correct as it first groups the data by region and then calculates the sum of sales amount for each region, aliasing the resulting column as "total_sales_amount".

Option B: This option is incorrect because it attempts to aggregate the total sales amount across the entire dataset before grouping by region, which is the reverse of what's needed. The agg function should be applied after the groupBy function.

Option C: This option is incorrect because it uses sales("total_sales_amount") within the agg function, which is not a valid PySpark aggregation function. The correct function to use for summing values is sum(). Also, "total_sales_amount" is not a column in the original dataframe.

Option D: This option is incorrect because it attempts to select a column named "total_sales_amount" from the sales_df within the agg function without specifying any aggregation function. This is syntactically incorrect and won't produce the desired result.

Key Takeaways

The correct order of operations in PySpark for calculating aggregates by group is to first group the data using groupBy() and then apply the aggregation function using agg().

Ensure that you use the correct aggregation function (e.g., sum(), avg(), min(), max(), count()) based on the desired calculation.

Always check that the column names used in the aggregation function exist in the original dataframe.

Remember to use .alias() to rename the aggregated column to a more descriptive name.

Question: 310 CertyIQ

A global retail company sells products across multiple categories (e.g., Electronics, Clothing) and regions (e.g., North, South, East, West).
The sales team has provided the data engineer with a PySpark dataframe named sales_df as below and the team wants the data engineer to analyze the sales data to help them make strategic decisions.

sales_df

product_id	category	sales_amount	region
1	Electronics	100	North
2	Clothing	200	South
3	Electronics	150	North
4	Clothing	300	South
5	Electronics	250	East
6	Clothing	400	West

Calculate the total sales amount for each product category and store the results in a new dataframe called category_sales.

What will generate the expected result of `category_sales`?

category	total_sales_amount
Electronics	500
Clothing	900

A. category_sales = sales_df.groupBy("category").agg(sum("sales_amount").alias("total_sales_amount"))
B. category_sales = sales_df.sum("sales_amount").groupBy("category").alias("total_sales_amount")
C. category_sales = sales_df.agg(sum("sales_amount")).groupBy("category").alias("total_sales_amount")
D. category_sales = sales_df.groupBy("region").agg(sum("sales_amount").alias("total_sales_amount"))

Answer: A

Explanation:

1. We first group by "category".
2. Then, we use .agg() with sum("sales_amount") to calculate the total sales amount for each category.
3. .alias("total_sales_amount") renames the aggregated column.

Analysis:

Option A

This groups the dataframe by category and then aggregates the sum of sales_amount. The alias renames the column to total_sales_amount. This gives the expected output with Electronics 500 and Clothing 900. This is correct.

Option B

This first applies sum on the whole dataframe so it gives the total sales across all rows. The category column is lost so grouping by category is not possible anymore. This will not give the correct result.

Option C

This aggregates the sum across the entire dataframe first and then tries to group by category. Since category is lost after the aggregation the grouping will fail. This is incorrect.

Option D

This groups by region instead of category. It will give totals for each region not for each category. This does not match the required output.

Question: 311 CertyIQ

What is the functionality of AutoLoader in Databricks?

A. Auto Loader automatically ingests and processes new files from cloud storage, handling batch and streaming data with no support for schema evolution.

B. Auto Loader automatically ingests and processes new files from cloud storage, handling batch data with support for schema evolution.

C. Auto Loader automatically ingests and processes new files from cloud storage, handling only streaming data with no support for schema evolution.

D. Auto Loader automatically ingests and processes new files from cloud storage, handling both batch and streaming data with support for schema evolution.

Answer: D

Explanation:

Correct Answer

Option D: Auto Loader automatically ingests and processes new files from cloud storage, handling both batch and streaming data with support for schema evolution.

Explanation of the Correct Answer

Auto Loader in Databricks is designed to automatically ingest and process new files from cloud storage. It can handle both streaming and batch data. A key feature is its support for schema evolution, which means it can adapt to changes in the structure of the incoming data files over time.

The thought process correctly identifies that Auto Loader supports both streaming and batch data processing and includes schema evolution capabilities.

Analysis of Each Option

Option A: Incorrect because it states that Auto Loader has "no support for schema evolution," which is false. Auto Loader is designed to handle schema evolution.

Option B: Incorrect because it says Auto Loader handles "batch data" only. Auto Loader handles both batch and streaming data.

Option C: Incorrect because it claims Auto Loader handles "only streaming data with no support for schema evolution." Auto Loader handles both batch and streaming data and supports schema evolution.

Option D: Correct because it accurately describes Auto Loader as handling both batch and streaming data and supporting schema evolution.

Key Takeaways

Auto Loader simplifies data ingestion in Databricks by automatically detecting and processing new files.

Understanding that Auto Loader supports both streaming and batch data, along with schema evolution, is crucial.

A common mistake is to overlook the schema evolution feature or to think Auto Loader only supports one type of data processing (either batch or streaming).

When answering questions about Auto Loader, remember its key features: automatic ingestion, support for both batch and streaming, and schema evolution.

Question: 312 CertyIQ

Which TWO items are characteristics of the Gold Layer?
[Choose 2 answers]

A. Historical lineage
B. Read-optimized
C. Raw Data
D. De-normalised
E. Normalised

Answer: B,D

Explanation:

Correct Answer

Options B and D are correct. The Gold Layer is read-optimized and contains de-normalized data.

Explanation of the Correct Answer

The Gold Layer is designed for quick data retrieval and analysis, making read optimization crucial. It also uses denormalization to improve query performance and simplify data access for business users.

The Gold Layer focuses on delivering data in a format that's easy to use for reporting and analytics. This often involves transforming the data into a star schema or other denormalized structure.

Analysis of Each Option

Option A: Incorrect. Historical lineage is a metadata concern across all layers, not specific to the Gold Layer.

Option B: Correct. The Gold Layer is optimized for read operations to ensure fast query performance.

Option C: Incorrect. Raw data is characteristic of the Bronze Layer.

Option D: Correct. Data in the Gold Layer is typically denormalized to optimize query performance.

Option E: Incorrect. Normalization is more characteristic of the Silver Layer.

Key Takeaways

The Gold Layer is the final layer in a data lakehouse, optimized for consumption.

Denormalization is a common technique used in the Gold Layer to improve query performance.

Understanding the purpose of each layer (Bronze, Silver, Gold) is crucial for answering these types of

questions.

Question: 313 CertyIQ

A data engineer is working in a Databricks notebook to design and manage a batch ETL pipeline. The engineer is writing SQL and Python code to clean data, transform it, and join large datasets from different sources.
The engineer wants to organize these steps into a structured process that can be run regularly and scheduled as part of a data pipeline.
Which Databricks notebook feature is applicable in the use case?

A. Collaborative editing
B. Task workflows and job scheduling
C. Real-time streaming support
D. Notebook version control

Answer: B

Explanation:

Correct Answer

Option B: Task workflows and job scheduling

Explanation of the Correct Answer

Option B is correct because "Task workflows and job scheduling" in Databricks allows users to organize notebook cells into structured workflows, schedule notebooks to run automatically, chain multiple notebooks together, monitor ETL processes, and set up dependencies between pipeline steps. This directly addresses the need to organize ETL steps, run them regularly, and schedule them as part of a data pipeline.

Analysis of Each Option

Option A: Collaborative editing allows multiple users to work on the same notebook. While useful for development, it doesn't address the scheduling and automation requirements. Therefore, it's incorrect.

Option B: Task workflows and job scheduling allows organizing notebook cells into structured workflows, scheduling notebooks to run automatically, chaining multiple notebooks, monitoring ETL processes, and setting up dependencies. This directly addresses the requirements, making it correct.

Option C: Real-time streaming support is for processing continuous data streams, but the use case specifies a "batch ETL pipeline," not streaming data. Therefore, it's incorrect.

Option D: Notebook version control helps track changes and manage different versions of notebooks but doesn't provide scheduling or pipeline orchestration capabilities. Therefore, it's incorrect.

Key Takeaways

The key takeaway is to understand the specific features Databricks offers for different data engineering tasks.

A common mistake is to confuse collaborative editing or version control with job scheduling and workflow orchestration.

When faced with similar questions, focus on identifying the feature that directly addresses the requirements of scheduling, automation, and structuring data pipelines.

Question: 314 CertyIQ

Which Databricks asset bundle format is valid?

A. resources: jobs: hello-job: name: hello-job tasks: _task_key: hello-task existing_cluster_id: 1234-567890-abcde123 notebook_task: notebook_path: ./hello.py

B. "resources": "jobs": "name":"hello-job", "tasks": "task_key: "hello-task","existing_cluster_id": "1234-567890-abcde123", "notebook_task": "notebook_path": ".hello.py"

C. configuration = "resources": "jobs": "name": "hello-job", "tasks": "task key: "hello-task","existing_cluster_id": "1234-567890-abcde123", "notebook_task": "notebook_path": ".hello.py"

D. resources (jobs name = "hello-job" tasks task_key = "hello-task" existing cluster_id = "1234-567890-abcde123" notebook_task notebook_path = ".hello.py"

Answer: A

Explanation:

Correct Answer

Option A: resources: jobs: hello-job: name: hello-job tasks: _task_key: hello-task existing_cluster_id: 1234-567890-abcde123 notebook_task: notebook_path: ./hello.py

Explanation of the Correct Answer

Option A is the correct answer because it follows the proper YAML syntax required for Databricks asset bundles. The structure is well-defined with correct indentation, key-value pairs, and a valid file path (./hello.py).

YAML (YAML Ain't Markup Language) is a human-readable data serialization language. Databricks asset bundles use YAML to define configurations.

Analysis of Each Option

Option A: This option is correct. It uses valid YAML syntax, including proper indentation and structure, key-value pairs, and a valid file path notation (./hello.py).

Option B: This option is incorrect. It has a missing closing quote in "task_key: "hello-task" and an invalid path .hello.py (should be ./hello.py).

Option C: This option is incorrect. It contains a space in the key name "task key: and a missing closing quote, as well as an invalid path format.

Option D: This option is incorrect. It uses parentheses instead of colons/braces, has inconsistent syntax, and is not a valid JSON, YAML, or any standard configuration format.

Key Takeaways

Databricks asset bundles require a specific format, typically YAML.

Correct syntax, including indentation, key-value pairs, and valid file paths, is crucial.

Common mistakes include incorrect indentation, missing quotes, spaces in key names, and invalid file paths.

Always validate the YAML syntax to ensure it is correct.

Question: 315 CertyIQ

A data engineer has inherited a Databricks pipeline from a previous team.
The pipeline is missing SLAs and costs more than the allotted budget.
On analysis, it is noted that the cluster is not being fully utilized, and the dataset is getting skewed.
How should the data engineer resolve this issue?

A. Use coalesce() on the dataset to merge partitions and reduce skew.
B. Increase the number of executors for the job.
C. Repartition the dataset to have it be more optimally spread across all nodes.
D. Increase the executor memory for the job.

Answer: C

Explanation:

Correct Answer

Option C: Repartition the dataset to have it be more optimally spread across all nodes.

Explanation of the Correct Answer

The problem states that the Databricks cluster is underutilized and the dataset is skewed. This means some partitions have much more data than others, causing some executors to work harder while others remain idle.

Option C, repartitioning the dataset, directly addresses the skew issue by ensuring balanced data distribution. This improves cluster utilization as all executors get equal work, reducing processing time and costs.

The provided thought process suggests repartitioning on a column that provides good data distribution, such as df.repartition($"date").

Analysis of Each Option

Option A: coalesce() only reduces the number of partitions by merging existing ones and does NOT redistribute data evenly. It will not fix the underlying skew problem and may worsen utilization by reducing parallelism. Therefore, it is incorrect.

Option B: Adding more executors won't help if current ones are underutilized due to skew. It increases costs without addressing the root cause, contradicting the budget constraint. Therefore, it is incorrect.

Option C: repartition() redistributes data evenly across all partitions, directly addressing the skew issue by ensuring balanced data distribution. This improves cluster utilization as all executors get equal work, reducing processing time and costs. Therefore, it is correct.

Option D: Increasing executor memory addresses memory issues, not data distribution problems. It won't improve cluster utilization and increases costs. Therefore, it is incorrect.

Key Takeaways

Data skew can lead to underutilization of cluster resources and increased processing time.

repartition() is used to redistribute data evenly across partitions, while coalesce() is used to reduce the number of partitions.

Increasing the number of executors or executor memory does not address data skew.

Always identify the root cause of performance issues before applying solutions.

Question: 316 CertyIQ

A data engineer is debugging a Python notebook in Databricks that processes a dataset using PySpark. The notebook fails with an error during a DataFrame transformation. The engineer wants to inspect the state of variables, such as the input DataFrame and intermediate results, to identify where the error occurs.
Which tool should the engineer use to debug the notebook and inspect the values of variables like DataFrames?

A. Use the Spark UI to analyze the execution plan and identify stages where the job failed
B. Use the Ganglia UI to monitor cluster resource usage and identify hardware issues
C. Use the Python Notebook Interactive Debugger to set breakpoints and inspect variable values in real-time
D. Use the Databricks CLI to download and analyze driver logs for detailed error messages

Answer: C

Explanation:

Correct Answer

Option C: Use the Python Notebook Interactive Debugger to set breakpoints and inspect variable values in real-time

Explanation of the Correct Answer

The Python Notebook Interactive Debugger is the best tool because it allows you to pause the execution of your code, step through it line by line, and inspect the values of variables, including DataFrames, in real-time. This is crucial for understanding the state of your data and identifying the exact point where an error occurs during a DataFrame transformation.

The key concept here is interactive debugging, which provides a hands-on way to examine the code's behavior and data at various stages.

Analysis of Each Option

Option A: Incorrect. The Spark UI is useful for analyzing the performance and execution plan of Spark jobs, but it doesn't allow you to inspect the values of variables or DataFrames directly. It shows how the job is executed but not the data within.

Option B: Incorrect. The Ganglia UI is for monitoring cluster resource usage, such as CPU and memory. It doesn't provide any insight into the code or data within the Spark application.

Option C: Correct. The Python Notebook Interactive Debugger is designed for this purpose. It allows you to set breakpoints, step through code, and inspect variable values, including DataFrame contents, in real-time.

Option D: Incorrect. The Databricks CLI and driver logs provide error messages and stack traces, which can help understand what went wrong. However, they don't allow for interactive inspection of variable states during execution.

Key Takeaways

Interactive debuggers are essential for understanding the state of variables during code execution.

Spark UI is for job performance, Ganglia UI is for cluster resource monitoring, and the Databricks CLI is for error logs.

When debugging, focus on tools that allow you to inspect variable values directly.

Question: 317 CertyIQ

A Data Engineer is building a simple data pipeline using Delta Live Tables (DLT) in Databricks to ingest customer data. The raw customer data is stored in a cloud storage location in JSON format.
The task is to create a DLT pipeline that reads the raw JSON data and writes it into a Delta table for further processing.
Which code snippet will correctly ingest the raw JSON data and create a Delta table using DLT?

A. import dlt@dlt.tabledef raw_customers():return spark.read.format("json").load("s3://my-bucket/raw-customers/")

B. import dlt@dlt.tabledef raw_customers():return spark.read.format("delta").load("s3://my-bucket/raw-customers/")

C. @dlt.tabledef raw_customers():return spark.read.json("s3://my-bucket/raw-customers/")

D. import dlt@dlt.tabledef raw_customers():return spark.read.text("s3://my-bucket/raw-customers/")

Answer: C

Explanation:

Explanation of the Correct Answer

The goal is to ingest raw customer data stored in JSON format from a cloud storage location and create a Delta table using Delta Live Tables (DLT). The correct approach involves using the dlt library to define a table and then leveraging Spark's read.json() method to parse the JSON data directly.

import dlt: This line is crucial for any DLT pipeline as it imports the necessary library.

@dlt.table: This decorator marks the function raw_customers as a DLT table. The function's return value (a PySpark DataFrame) will be materialized as a Delta table.

spark.read.json("s3://my-bucket/raw-customers/"): This is the most direct and idiomatic way in PySpark to read data from JSON files. The json() method automatically infers the schema from the JSON data and creates a DataFrame, which DLT then uses to create the Delta table.

Analysis of Each Option

Option A: `import dlt

@dlt.table

def raw_customers():

return spark.read.format("json").load("s3://my-bucket/raw-customers/")This option is technically correct and would work, asformat("json").load()is a valid way to read JSON data. However,spark.read.json()` is a more concise and commonly used shortcut for the same operation.

Option B: `import dlt

@dlt.table

def raw_customers():

return spark.read.format("delta").load("s3://my-bucket/raw-customers/")This option is incorrect because the source data is in JSON format, not Delta format. Usingformat("delta")` would attempt to read an existing Delta table, which is not the case here.

Option C: @dlt.table def raw_customers(): return spark.read.json("s3://my-bucket/raw-customers/")

This is the correct and most efficient way to read JSON data. The spark.read.json() method is specifically

designed for parsing JSON files, automatically handling schema inference and data parsing.

Option D: `import dlt

@dlt.table

def raw_customers():

return spark.read.text("s3://my-bucket/raw-customers/")This option is incorrect.spark.read.text()` reads each line of the file as a single string, without parsing it as JSON. This would result in a DataFrame with a single string column, not a structured representation of the JSON data.

Key Takeaways

Delta Live Tables (DLT) pipelines are defined using the import dlt statement and the @dlt.table decorator.

To ingest data from a specific format, use the appropriate spark.read method or format().load() combination.

For JSON data, spark.read.json() is the most direct and recommended method for parsing and loading the data into a DataFrame.

C. @dlt.table

def raw_customers():

return spark.read.json("s3://my-bucket/raw-customers/")

Question: 318 CertyIQ

An organization needs to share a dataset stored in its Databricks Unity Catalog with an external partner who uses a different data platform that is not Databricks.
The goal is to maintain data security and ensure the partner can access the data efficiently.
Which method should the data engineer use to securely share the dataset with the external partner?

A. Using Delta Sharing with the open sharing protocol
B. Exporting data as CSV files and emailing them
C. Using a third-party API to access the Delta table
D. Databricks-to-Databricks Sharing

Answer: A

Explanation:

Correct Answer

Option A: Using Delta Sharing with the open sharing protocol

Explanation of the Correct Answer

Delta Sharing with the open sharing protocol is the correct choice because it's designed for securely sharing data from Databricks Unity Catalog with external partners, even if they use different data platforms. It ensures secure, governed access without duplicating data.

This method leverages Databricks' open protocol, which is specifically built for cross-platform data sharing, maintaining data lineage and audit trails, and supporting real-time data access.

Analysis of Each Option

Option A: Correct. Delta Sharing is designed for secure, cross-platform data sharing without data duplication.

Option B: Incorrect. Exporting data as CSV files and emailing them is insecure, lacks access controls, and creates stale data copies.

Option C: Incorrect. Using a third-party API adds complexity, may not provide adequate security, and isn't a standard approach.

Option D: Incorrect. Databricks-to-Databricks Sharing only works when both parties use Databricks, which isn't the case here.

Key Takeaways

Delta Sharing is the preferred method for secure, cross-platform data sharing from Databricks.

Avoid insecure methods like emailing CSV files.

Always consider the security and governance implications when sharing data with external partners.

Question: 319 CertyIQ

A data engineer team has decided to implement a new data platform on Databricks and is currently deciding how to store each kind of data on each data layer.
What is the appropriate layer and data pairing for medallion architecture?

A. Silver Layer - Cleansed master customer data
B. Gold Layer - Deduplicated money transfer transaction
C. Silver Layer - Raw data from deposit account application
D. Bronze Layer - Summary of cash deposit amount for each country and city

Answer: A

Explanation:

Correct Answer

Option A: Silver Layer - Cleansed master customer data

Explanation of the Correct Answer

Option A is correct because the Silver Layer in medallion architecture is designed to store cleansed, validated, and enriched data. Cleansed master customer data fits this description perfectly as it has been processed from its raw form, validated, and standardized.

The key concept here is understanding the role of each layer in the medallion architecture: Bronze (raw data), Silver (cleansed and validated data), and Gold (aggregated and business-ready data).

Analysis of Each Option

Option A: This option is correct. Cleansed master customer data is exactly the type of data that should reside in the Silver Layer.

Option B: This option is incorrect. Deduplicated individual transactions are more suited for the Silver Layer, while the Gold Layer is reserved for aggregated, business-ready datasets.

Option C: This option is incorrect. Raw data should be stored in the Bronze Layer, not the Silver Layer. The

Silver Layer contains processed and cleansed data.

Option D: This option is incorrect. Summarized data belongs in the Gold Layer, not the Bronze Layer, which is meant for raw, unprocessed data.

Key Takeaways

The key takeaway is to remember the purpose of each layer in the medallion architecture: Bronze for raw data, Silver for cleansed and validated data, and Gold for aggregated and business-ready data.

A common mistake is confusing the Silver and Gold Layers. Remember that Silver is for cleaned data, while Gold is for aggregated data ready for analysis.

When approaching similar questions, always consider the state of the data (raw, cleansed, or aggregated) and match it to the appropriate layer.

Question: 320 CertyIQ

Why is Delta Live Tables (DLT) an appropriate choice?

A. Automatic data quality checks, built-in support for schema evolution, and declarative pipeline development

B. Manual schema enforcement, high operational overhead, and limited scalability

C. Requires custom code for data quality checks, no support for streaming data, and complex pipeline maintenance

D. Supports only batch processing, no data versioning, and high infrastructure costs

Answer: A

Explanation:

Correct Answer

Option A: Automatic data quality checks, built-in support for schema evolution, and declarative pipeline development

Explanation of the Correct Answer

Delta Live Tables (DLT) is a good choice because it automatically checks data quality, supports schema evolution, and allows you to build pipelines in a declarative way. This means you can focus on what you want to achieve with your data, rather than worrying about the details of how to process it. DLT helps ensure your data is reliable and your pipelines are easy to maintain.

Analysis of Each Option

Option A: This option is correct because it accurately describes the key benefits of using Delta Live Tables, such as automatic data quality checks, built-in support for schema evolution, and declarative pipeline development.

Option B: This option is incorrect because it describes the opposite of what DLT offers. DLT aims to reduce manual work, lower overhead, and provide scalability.

Option C: This option is incorrect because DLT provides built-in data quality checks, supports streaming data, and simplifies pipeline maintenance.

Option D: This option is incorrect because DLT supports both batch and streaming processing, includes data

versioning through Delta Lake, and optimizes infrastructure costs.

Key Takeaways

Delta Live Tables simplifies data pipeline development by automating data quality checks and managing schema evolution.

Declarative pipeline development allows users to focus on the desired outcome rather than the implementation details.

When evaluating data pipeline tools, consider factors like data quality, schema management, and ease of maintenance.

A common mistake is to assume that all data pipeline tools require manual data quality checks and complex maintenance.

Question: 321 CertyIQ

A data engineer needs to develop integration tests for an ETL process and deploy a version-controlled, packaged workflow into production using an external job scheduler.
Which tool should the data engineer use for this job?

A. Databricks Connect
B. Databricks Command Line Interface
C. Databricks Asset Bundles
D. Databricks Software Development Kit

Answer: C

Explanation:

Correct Answer

Option C: Databricks Asset Bundles

Explanation of the Correct Answer

Databricks Asset Bundles are the best choice because they are specifically designed for version control integration, packaging all workflow components into a single deployable unit, environment management, CI/CD pipeline integration, and testing support within the deployment pipeline. This aligns perfectly with the data engineer's needs to develop integration tests, deploy a version-controlled workflow, use an external job scheduler, and deploy into production.

Analysis of Each Option

Option A: Databricks Connect is primarily for local development and testing, not for production deployment and packaging. Therefore, it's not the correct choice.

Option B: Databricks Command Line Interface is useful for automation but doesn't offer a comprehensive deployment solution for packaged workflows. Thus, it's not the best option.

Option C: Databricks Asset Bundles provide a complete solution for modern DataOps practices, combining version control, testing, packaging, and deployment in a single framework that integrates well with external schedulers. This makes it the correct choice.

Option D: Databricks Software Development Kit provides programmatic access to Databricks APIs but doesn't

offer the integrated packaging and deployment capabilities needed for this use case. Hence, it's not the optimal solution.

Key Takeaways

Databricks Asset Bundles are designed for managing and deploying Databricks workflows in a structured and repeatable manner.

When choosing a tool for deploying workflows, consider the need for version control, packaging, environment management, and integration with external schedulers.

Avoid using tools primarily intended for local development (like Databricks Connect) for production deployments.

Question: 322 CertyIQ

A data engineer is attempting to write Python and SQL in the same command cell and is running into an error. The engineer thought that it was possible to use a Python variable in a select statement.
Why does the command fail?

A. Databricks supports multiple languages but only one per notebook.
B. Databricks supports language interoperability but only if a special character is used.
C. Databricks supports one language per cell.
D. Databricks supports language interoperability in the same cell but only between Scala and SQL.

Answer: C

Explanation:

Correct Answer

Option C: Databricks supports one language per cell.

Explanation of the Correct Answer

The correct answer is that Databricks supports one language per cell. This means you can't directly mix Python and SQL code in the same cell without using specific techniques to separate them.

The thought process explains that each cell in a Databricks notebook is designed to execute code in a single language. To use multiple languages, you need to either use separate cells or utilize language-specific "magic commands" to switch between languages within different cells.

The thought process provides examples of how to correctly use Python variables in SQL queries by either using separate cells with the %sql magic command or by using Python's string formatting capabilities within a Python cell.

Analysis of Each Option

Option A: Incorrect. Databricks does support multiple languages within a single notebook, but not within the same cell without proper separation.

Option B: Incorrect. While special characters are used in some contexts (like variable substitution), the fundamental issue is the one-language-per-cell rule.

Option C: Correct. Databricks enforces a one-language-per-cell rule, which is why the engineer's attempt to mix Python and SQL in the same cell failed.

Option D: Incorrect. Databricks supports language interoperability between more than just Scala and SQL. It also supports Python and R. The core issue is still the one-language-per-cell rule.

Key Takeaways

Databricks notebooks are designed to execute code in a single language per cell.

To use multiple languages, you must either use separate cells or utilize language-specific "magic commands" (e.g., %sql, %python).

When working with multiple languages, be mindful of how variables are passed between them. Techniques like string formatting or magic commands with variable substitution can be used.

A common mistake is to assume that you can freely mix languages within a single cell without any special handling.

Question: 323 CertyIQ

A data engineer needs to ingest from both streaming and batch sources for a firm that relies on highly accurate data. Occasionally, some of the data picked up by the sensors that provide a streaming input are outside the expected parameters.
If this occurs, the data must be dropped, but the stream should not fail.
Which feature of Delta Live Tables meets this requirement?

A. Error Handling
B. Expectations
C. Monitoring
D. Change Data Capture

Answer: B

Explanation:

Correct Answer

Option B: Expectations

Expectations in Delta Live Tables allow defining data quality constraints and specifying actions like dropping rows when data violates these constraints, ensuring the stream continues processing without failing.

Explanation of the Correct Answer

The question asks about a Delta Live Tables feature that allows dropping data outside expected parameters from a streaming source without failing the stream. Expectations in Delta Live Tables are designed for this purpose. They allow defining data quality constraints and specifying actions like DROP ROW when data violates these constraints, ensuring the stream continues processing.

The key concept here is data quality management within a streaming pipeline. Delta Live Tables Expectations provide a mechanism to enforce data quality rules and handle violations gracefully.

The provided SQL example demonstrates how to define expectations with the DROP ROW action, which directly addresses the problem requirement.

Analysis of Each Option

Option A: Error Handling - While important, it's a general concept and not a specific feature for data validation

and dropping rows based on defined rules.

Option B: Expectations - This is the correct answer because it allows defining data quality rules and specifying actions like DROP ROW when data violates these rules, ensuring the stream continues.

Option C: Monitoring - This provides insights into the data pipeline but doesn't actively filter or drop invalid data.

Option D: Change Data Capture - This is for tracking changes in data sources, not for data validation or quality control.

Key Takeaways

Delta Live Tables Expectations are crucial for data quality management in streaming pipelines.

The ON VIOLATION clause allows specifying actions like DROP ROW, WARN, or FAIL based on data quality checks.

Understanding the specific features of Delta Live Tables is essential for choosing the right tool for data processing tasks.

A common mistake is confusing general error handling with specific data quality features like Expectations.

Question: 324 CertyIQ

What are the transformations typically included in building the Bronze layer?

A. Perform extensive data cleansing
B. Include columns Load date/time, process ID
C. Business rules and transformations
D. Aggregate data from multiple sources

Answer: B

Explanation:

Correct Answer

Option B: Include columns Load date/time, process ID

Explanation of the Correct Answer

The Bronze layer in data architecture is the initial landing zone for raw data. The primary goal is to ingest data with minimal transformation, focusing on preserving the original data and adding metadata for tracking and auditing. Including columns like Load date/time and process ID helps in maintaining data lineage and provides an audit trail without altering the original data's content.

The key concept here is the medallion architecture (Bronze-Silver-Gold), where each layer has a specific purpose. The Bronze layer's purpose is to store raw data with minimal changes.

Analysis of Each Option

Option A: Incorrect. Extensive data cleansing is performed in the Silver layer, not the Bronze layer. The Bronze layer preserves raw data without cleansing.

Option B: Correct. Including columns like Load date/time and process ID is a typical transformation in the

Bronze layer. This provides an audit trail and data lineage without altering the source data.

Option C: Incorrect. Applying business rules and transformations is typically done in the Silver or Gold layers, not the Bronze layer. The Bronze layer avoids applying business logic.

Option D: Incorrect. Aggregating data from multiple sources is usually a Gold layer transformation. The Bronze layer maintains source-level granularity.

Key Takeaways

The Bronze layer is for raw data ingestion with minimal transformations.

Common mistakes include applying business logic or performing extensive data cleansing in the Bronze layer.

Remember to focus on preserving the original data and adding metadata for tracking and auditing in the Bronze layer.

Question: 325 CertyIQ

A Python file is ready to go into production and the client wants to use the cheapest but most efficient type of cluster possible. The workload is quite small, only processing 10GBs of data with only simple joins and no complex aggregations or wide transformations.
Which cluster meets the requirement?

A. Interactive cluster
B. Job cluster with spot instances disabled
C. Job cluster with Photon enabled
D. Job cluster with spot instances enabled

Answer: D

Explanation:

Correct Answer

Option D: Job cluster with spot instances enabled

Explanation of the Correct Answer

Option D is the correct answer because it offers the most cost-effective solution for the given scenario. Spot instances are significantly cheaper than on-demand instances, making them ideal for cost-sensitive production environments.

The workload is small (10GB) and involves only simple joins, meaning the risk of interruption due to spot instance reclamation is manageable. The job cluster will spin up, process the data, and terminate, optimizing resource utilization and cost.

Analysis of Each Option

Option A: Incorrect. Interactive clusters are designed for development and exploration, not for production workloads. They run continuously, making them more expensive than job clusters.

Option B: Incorrect. Job clusters with spot instances disabled use on-demand instances, which are more reliable but also more expensive. This option does not meet the requirement of being the cheapest.

Option C: Incorrect. Photon is beneficial for complex queries and large datasets, adding computational

overhead and cost. It is overkill for simple joins on 10GB of data and therefore not the most efficient or cheapest option.

Option D: Correct. Job clusters with spot instances enabled are the cheapest option because spot instances cost significantly less than on-demand instances. They are efficient for this use case as they spin up, process data, and terminate. The risk of interruption is manageable due to the small data size and simple processing requirements.

Key Takeaways

Spot instances are a cost-effective solution for production workloads, especially when cost optimization is a priority.

Job clusters are suitable for batch processing tasks, optimizing resource utilization and cost.

Consider the workload characteristics (data size, complexity) when choosing a cluster type and instance type.

For small, simple workloads, the risk of interruption with spot instances is often manageable.

Question: 326 CertyIQ

Which compute option should be chosen in a scenario where small-scale ad-hoc Python scripts need to be run at high frequency and should wind down quickly after these queries have finished running?

A. All-purpose Cluster
B. Job Cluster
C. Serverless Compute
D. SQL Warehouse

Answer: C

Explanation:

Correct Answer

Option C: Serverless Compute

Explanation of the Correct Answer

Serverless compute is the best option because it automatically scales resources when needed and releases them immediately after tasks are completed. This eliminates the need for manual cluster management and ensures you only pay for the actual compute time used, making it cost-effective for high-frequency, short-duration tasks. Additionally, serverless compute often has fast startup times due to pre-warmed environments, which is ideal for ad-hoc script execution.

Analysis of Each Option

Option A: All-purpose clusters remain running continuously, incurring costs even during idle periods. They require manual termination to avoid unnecessary charges, making them less cost-effective for high-frequency, short-duration tasks.

Option B: Job clusters create a new cluster for each job, leading to longer startup times and higher overhead for frequent executions. This is less efficient than serverless compute for this use case.

Option C: This is the correct answer, as explained above.

Option D: SQL Warehouses are optimized for SQL workloads, not Python scripts. They are designed for analytical queries rather than general Python execution, making them inappropriate for this scenario.

Key Takeaways

Serverless compute is ideal for scenarios requiring automatic scaling, cost efficiency, and quick wind-down after task completion.

Consider the specific requirements of your workload (e.g., frequency, duration, type of tasks) when choosing a compute option.

Avoid options that incur costs during idle periods or have long startup times if your tasks are short and frequent.

Question: 327 CertyIQ

A data engineer is developing a small proof of concept in a notebook. When running the entire notebook, the Cluster usage spikes.
The data engineer wants to keep the development requirements and get real-time results.
Which Cluster meets these requirements?

A. All-Purpose Cluster with a large fixed memory size
B. Job cluster with Photon enabled and autoscaling
C. Job Cluster with autoscaling enabled
D. All Purpose Cluster with autoscaling

Answer: D

Explanation:

Correct Answer

Option D: All Purpose Cluster with autoscaling

Explanation of the Correct Answer

The correct answer is D because an All-Purpose Cluster is suitable for interactive development in a notebook, and autoscaling dynamically adjusts the cluster size based on the workload, preventing excessive resource consumption and mitigating usage spikes.

The engineer needs an interactive environment for development and testing, and real-time results, implying the need for an active cluster that can respond quickly to queries and code execution. The solution should efficiently manage resources to prevent unnecessary spikes.

Analysis of Each Option

Option A: This option is incorrect because a large fixed memory size can be wasteful if the cluster is not constantly utilizing all the resources. It doesn't address the spike issue; it might even exacerbate it by providing more resources than needed at times.

Option B: This option is incorrect because Job clusters are primarily designed for running automated, non-interactive jobs. They are not ideal for interactive development in a notebook.

Option C: This option is incorrect because Job clusters are not designed for interactive development.

Option D: This option is correct because it provides an interactive environment for development while

efficiently managing resources to prevent usage spikes. Autoscaling can introduce a slight delay when scaling up, but this is generally acceptable for development purposes.

Key Takeaways

All-Purpose Clusters are suitable for interactive development, while Job Clusters are designed for automated, non-interactive jobs.

Autoscaling dynamically adjusts the cluster size based on the workload, preventing excessive resource consumption and mitigating usage spikes.

A fixed memory size can be wasteful if the cluster is not constantly utilizing all the resources.

Question: 328 CertyIQ

A junior data engineer has created the table 'orders_backup' as a copy of the table "orders". Recently, the team started getting an error when querying the orders_backup indicating that some data files are no longer present. The transaction logs for the orders tables show a recent run of VACUUM command.
Which of the following explains how the data engineer created the orders_backup table ?
Select one:

A. The orders_backup table was created using CTAS statement from orders table
B. The orders_backup table was created using Delta Lake's DEEP CLONE functionality from the orders table
C. The orders_backup table was created using CRAS statement from orders table
D. The orders_backup table was created using Delta Lake's SHALLOW CLONE functionality from the orders table

Answer: D

Explanation:

A. The orders_backup table was created using CTAS statement from orders table

CTAS (CREATE TABLE AS SELECT) creates a **fully independent copy** of the data by writing new data files.

VACUUM on the source table does **not** affect the CTAS-created table. Therefore, missing data files would **not** occur.

B. The orders_backup table was created using Delta Lake's DEEP CLONE functionality from the orders table

A **DEEP CLONE** creates a complete copy of both **metadata and data files**.

The cloned table does not share data files with the source, so VACUUM on the original table would **not delete files used by the clone**.

C. The orders_backup table was created using CRAS statement from orders table

CRAS (Create Replace As Select) is **not a valid Delta Lake or Spark SQL command**.

Therefore, this option is incorrect.

D. The orders_backup table was created using Delta Lake's SHALLOW CLONE functionality from the orders table

A **SHALLOW CLONE** copies only the **table metadata** and **references the same underlying data files** as the source table.

When **VACUUM** is run on the original orders table, it can delete data files still referenced by the shallow clone, leading to **missing data file errors** during queries.

Question: 329 CertyIQ

A data engineer is designing a streaming ingestion pipeline using Auto Loader. The requirement is that the pipeline should never fail on schema changes but must capture any new columns that arrive in the data for later inspection. Which configuration should the engineer use?
Select one:

A. failOnNewColumns
B. rescue
C. addNewColumns
D. none

Answer: B

Explanation:

A. failOnNewColumns

The failOnNewColumns option enforces strict schema validation and causes the Auto Loader streaming pipeline to fail whenever new or unexpected columns appear in the incoming data. Since the requirement explicitly states that the pipeline should never fail on schema changes, this configuration does not meet the requirement.

B. rescue

The rescue option enables Auto Loader's rescued data mechanism, which allows the pipeline to continue running even when schema changes occur. Any new or unexpected columns are captured in a special _rescued_data column instead of causing failures. This satisfies both requirements: the pipeline never fails on schema changes and new columns are preserved for later inspection.

C. addNewColumns

The addNewColumns option automatically evolves the target schema by adding new columns as they appear in the source data. Although this prevents pipeline failures, it does not capture new columns separately for inspection, which is a key requirement. Therefore, it is not the correct choice.

D. none

If no schema-handling configuration is specified, Auto Loader may fail when it encounters schema changes such as new columns. This violates the requirement that the pipeline should never fail on schema changes, making this option incorrect.

Question: 330 CertyIQ

A data engineer wants to create a Delta Lake table for storing user activities of a website. The table has the following schema:
user_id LONG, page STRING, activity_type LONG, ip_address STRING, activity_time TIMESTAMP, activity_date DATE
Based on the above schema, which column is a good candidate for partitioning the Delta Table?

Select one:

A. user_id
B. activity_time
C. activity_type
D. activity_date

Answer: D

Explanation:

A. user_id

The user_id column typically has very high cardinality, meaning there are many unique values. Partitioning on such a column would create a large number of small partitions, leading to inefficient query performance and increased metadata overhead.

B. activity_time

The activity_time column is a timestamp with very fine granularity. Partitioning on this column would result in an excessive number of partitions (potentially one per second or millisecond), which is inefficient and not recommended for Delta Lake tables.

C. activity_type

The activity_type column usually has low cardinality, but it is not commonly used as a primary filter for large-scale analytical queries. Partitioning on this column would not significantly improve query performance and could lead to data skew.

D. activity_date

The activity_date column has an optimal level of cardinality and is commonly used in query filters for time-based analysis. Partitioning by date enables efficient data pruning, improves query performance, and is a best practice for large event or activity tables in Delta Lake.

Question: 331 CertyIQ

A data engineer is working on a feature branch in a Git folder and attempts to create a pull request into the main branch of the project repository. However, Git detects conflicting changes between the feature branch and the main branch, resulting in a merge conflict.
Which two options does the Git folder UI provide for resolving merge conflicts?
Select one or more.

A. Delete the conflicting branch
B. Move the changes to a new branch
C. Manually edit the changes and remove the conflict markers
D. Accept all incoming or current changes

Answer: C

Explanation:

A. Delete the conflicting branch

Deleting the conflicting branch does not resolve a merge conflict and is not an option provided by the Git folder UI for conflict resolution. Merge conflicts must be resolved by reconciling code differences, not by removing the branch.

B. Move the changes to a new branch

Moving changes to a new branch does not address the existing merge conflict. The Git folder UI focuses on resolving conflicts within the current pull request rather than relocating changes to another branch.

C. Manually edit the changes and remove the conflict markers

The Git folder UI allows the user to manually edit the conflicting files by resolving differences and removing Git conflict markers (<<<<<<<, =======, >>>>>>>). This is a standard and supported way to resolve merge conflicts.

D. Accept all incoming or current changes

The Git folder UI provides built-in options to resolve conflicts by accepting either the current branch changes, the incoming branch changes, or a combination of both. This makes conflict resolution faster and easier without manual editing.

Question: 332 CertyIQ

A data engineering team is building a LDP pipeline to clean and validate hotel reservations data. Some completed reservations have null check-in or check-out dates, which violates business rules.
To handle this, they implemented the following code:
Which of the following correctly describes what this function does?Select one:
This function streams only rows with valid check-in and check-out values into the silver_reservations table, while writing invalid rows into a separate partition.

A. This function streams only rows with valid check-in and check-out values into the silver_reservations table, while writing invalid rows into a separate partition.

B. This function streams all rows into the silver_reservations table, flags those with missing check-in or check-out values as quarantined, and partitions the table by the is_quarantined flag.

C. This function partitions the bronze_reservations table by the is_quarantined flag, streams valid partitions into the silver_reservations table, and drops the invalid partitions.

D. This function streams rows based on the quarantine_rules into two separate tables: silver_reservations for valid reservations, and is_quarantined for invalid reservations.

Answer: B

Explanation:

A. This function streams only rows with valid check-in and check-out values into the silver_reservations table, while writing invalid rows into a separate partition.

This is incorrect because the function does not filter out invalid rows. The @dlt.expect_all(rules) decorator applies data quality expectations but does not drop or redirect failing records by default. All rows continue to flow into the same table.

B. This function streams all rows into the silver_reservations table, flags those with missing check-in or check-out values as quarantined, and partitions the table by the is_quarantined flag.

This is correct. The function reads all rows from the bronze_reservations streaming table, computes the is_quarantined column using the quarantine_rules, and writes all records to the silver_reservations table. Records with null check-in or check-out values are flagged as quarantined, and the table is partitioned by the

is_quarantined column.

C. This function partitions the bronze_reservations table by the is_quarantined flag, streams valid partitions into the silver_reservations table, and drops the invalid partitions.

This is incorrect because the bronze table is not modified or repartitioned. The logic only applies to the output table, and no rows are dropped based on the expectations.

D. This function streams rows based on the quarantine_rules into two separate tables: silver_reservations for valid reservations, and is_quarantined for invalid reservations.

This is incorrect because the code defines only a single DLT table, silver_reservations. No separate table is created for quarantined records.

Question: 333 CertyIQ

The data engineering team has a Delta table named 'users'. A recent CHECK constraint has been added to the table using the following command:
ALTER TABLE users
ADD CONSTRAINT valid_age CHECK (age> 0);
The team attempted to insert a batch of new records to the table, but there were some records with negative age values which caused the write to fail because of the constraint violation.
Which statement describes the outcome of this batch insert?
Select one:

A. Only records processed before reaching the first violating record have been inserted in the table

B. All records except those that violate the table constraint have been inserted in the table. Records violating the constraint have been recorded into the transaction log.

C. None of the records have been inserted into the table.

D. All records except those that violate the table constraint have been inserted in the table. Records violating the constraint have been ignored.

Answer: C

Explanation:

A. Only records processed before reaching the first violating record have been inserted in the table

This is incorrect because Delta Lake enforces **atomic writes**. A batch write either fully succeeds or fully fails. Partial inserts are not allowed when a constraint violation occurs.

B. All records except those that violate the table constraint have been inserted in the table. Records violating the constraint have been recorded into the transaction log.

This is incorrect because Delta Lake does not partially commit valid records when a CHECK constraint is violated. The transaction log records the failed attempt, but no data is written to the table.

C. None of the records have been inserted into the table.

This is correct. When a batch insert violates a CHECK constraint, Delta Lake **rejects the entire write operation**. Due to ACID guarantees, the transaction is rolled back and **no records are committed** to the table.

D. All records except those that violate the table constraint have been inserted in the table. Records violating the constraint have been ignored.

This is incorrect because Delta Lake does not silently ignore constraint-violating records. Constraint violations

cause the entire batch write to fail.

Question: 334 CertyIQ

A data engineering team wants to ensure that a specific Python library is available every time a cluster starts. Which approach best achieves this goal?
Select one:

A. Manually install the library on the driver node each time the cluster starts
B. Install the library when running a Python notebook
C. Use Databricks CLI to upload the library files to the cluster after startup
D. Use an init script to install the library during cluster startup

Answer: D

Explanation:

A. Manually install the library on the driver node each time the cluster starts

This approach requires repeated manual effort and is error-prone. It does not scale well and does not guarantee consistency across cluster restarts or across all nodes in the cluster.

B. Install the library when running a Python notebook

Installing a library from within a notebook only applies to the current cluster session. The library will not automatically be available on subsequent cluster restarts, which fails to meet the requirement.

C. Use Databricks CLI to upload the library files to the cluster after startup

Uploading libraries after the cluster has already started does not guarantee availability at startup time. This approach also requires additional steps each time the cluster restarts.

D. Use an init script to install the library during cluster startup

Init scripts run automatically every time a cluster starts. They ensure that the required Python library is installed consistently on all nodes before any workloads run, making this the best approach.

Question: 335 CertyIQ

A data engineer is configuring a job using Databricks Asset Bundles and wants to limit its access to authorized users. The following job definition includes tasks and job clusters, but the engineer also needs to define which user groups can manage or view the job.
Which option correctly fills in the blank to meet the specified requirement?Select one:

A. Job settings
B. Permissions
C. access_control
D. roles

Answer: B

Explanation:

A. job_settings

The job_settings field is not used to define access control in Databricks Asset Bundles. Job settings focus on configuration details such as tasks, schedules, and clusters, not on who can view or manage the job.

B. Permissions

The permissions block is used in Databricks Asset Bundles to define access control for jobs. It allows specifying user groups and their permission levels, such as CAN_MANAGE or CAN_VIEW. Using permissions correctly ensures that only authorized groups can manage or view the job, fully meeting the requirement.

C. access_control

Although access control is a general concept, access_control is not a valid field in Databricks Asset Bundles job definitions. Using this option would result in an invalid configuration.

D. roles

The roles field is not supported in Databricks Asset Bundles for defining job-level permissions. Databricks uses permission levels within the permissions block instead of role-based definitions in this context.

Question: 336 CertyIQ

A data engineer has defined the following data quality constraint in a LDP pipeline:
CONSTRAINT valid_id EXPECT (id IS NOT NULL) _____
Which clause correctly fills in the blank so records violating this constraint will be written to the target table, but reported in metrics?
Select one:

A. There is no need to add the ON VIOLATION clause. By default, records violating the constraint will be kept, and reported as invalid in the pipeline metrics.

B. ON VIOLATION NULL

C. ON VIOLATION ADD ROW

D. ON VIOLATION WARNING

Answer: D

Explanation:

A. There is no need to add the ON VIOLATION clause. By default, records violating the constraint will be kept, and reported as invalid in the pipeline metrics.

This is incorrect because, in Delta Live Tables, constraints without an ON VIOLATION clause default to dropping records that violate the constraint. Therefore, violating records would not be written to the target table.

B. ON VIOLATION NULL

ON VIOLATION NULL is not a valid clause in Delta Live Tables constraints. Using this option would result in an invalid pipeline definition.

C. ON VIOLATION ADD ROW

ON VIOLATION ADD ROW is not a supported option in Delta Live Tables. Delta Live Tables supports actions such as FAIL UPDATE and WARN, not adding rows explicitly on violations.

D. ON VIOLATION WARNING

This is correct. Using ON VIOLATION WARNING allows records that violate the constraint to be written to the target table, while the violations are reported in the pipeline metrics for monitoring and analysis.

Question: 337 CertyIQ

A data engineer is working on a project where they need to integrate data from an external API into a Databricks workspace. For security reasons, they do not want to hardcode the API Key directly in their notebooks. Instead, they want to use Databricks Secrets to securely store and manage sensitive credentials.
Which Databricks CLI commands should they use to accomplish this goal?
Select one:

A. databricks secrets create-scope api_scopedatabricks secrets put-secret api_scope api_key
B. databricks secrets create-scope api_scopedatabricks secrets put-secret api_key api_scope
C. databricks secrets create-secret api_secretsdatabricks secrets put-scope api_secrets api_key
D. databricks secrets create-secret api_secretsdatabricks secrets put-scope api_key api_secrets

Answer: A

Explanation:

A. databricks secrets create-scope api_scope

databricks secrets put-secret api_scope api_key

This is the correct approach. First, create-scope is used to create a secrets scope (api_scope) that acts as a secure container for secrets. Then, put-secret is used to store the actual secret (api_key) inside that scope. This allows the API key to be securely managed and referenced in Databricks notebooks without hardcoding sensitive credentials.

B. databricks secrets create-scope api_scope

databricks secrets put-secret api_key api_scope

This option is incorrect because the arguments for put-secret are in the wrong order. The correct syntax requires the **scope name first** and the **key name second**, not the other way around.

C. databricks secrets create-secret api_secrets

databricks secrets put-scope api_secrets api_key

This is incorrect because create-secret is not a valid Databricks CLI command. Secrets must be created within a scope, not independently.

D. databricks secrets create-secret api_secrets

databricks secrets put-scope api_key api_secrets

This option is also invalid because both create-secret and put-scope are not valid Databricks CLI commands. The correct commands are create-scope and put-secret.

Question: 338 CertyIQ

A data engineering team is discussing the optimal data layout strategy on a growing managed Delta table in Unity Catalog. They are considering partitioning, Z-ordering, and Liquid Clustering to improve query performance.
Which scenario best indicates that Automatic Liquid Clustering is the recommended choice?
Select one.

A. None of the listed options are correct. Automatic Liquid Clustering can not be applied on managed tables.

B. The team has identified stable clustering keys for the table.

C. The table is heavily filtered by a consistent, small set of date ranges.

D. The table experiences diverse, frequently changing query filters across multiple columns, with unpredictable access patterns.

Answer: D

Explanation:

A. None of the listed options are correct. Automatic Liquid Clustering can not be applied on managed tables.

This is incorrect because Automatic Liquid Clustering **is supported on managed Delta tables in Unity Catalog**. It is specifically designed to simplify data layout management without manual tuning.

B. The team has identified stable clustering keys for the table.

When clustering keys are stable and well understood, traditional techniques like **partitioning or Z-ordering** are more appropriate. Automatic Liquid Clustering is intended for cases where optimal clustering keys are not fixed.

C. The table is heavily filtered by a consistent, small set of date ranges.

If queries consistently filter on date ranges, **partitioning by date** is the best and simplest optimization. Automatic Liquid Clustering is unnecessary in this predictable access pattern.

D. The table experiences diverse, frequently changing query filters across multiple columns, with unpredictable access patterns.

This is the ideal scenario for Automatic Liquid Clustering. It dynamically adapts data layout based on observed query patterns, optimizing performance without requiring predefined clustering keys, making it well suited for unpredictable and evolving workloads.

Question: 339 CertyIQ

Which of the following functions can a data engineer use to return a new DataFrame containing the distinct rows from a given DataFrame based on multiple columns?
Select one:

A. pyspark.sql.DataFrame.distinct

B. pyspark.sql.DataFrame.dropDuplicates

C. pyspark.sql.DataFrame.dropna

D. pyspark.sql.DataFrame.drop

Answer: B

Explanation:

A. pyspark.sql.DataFrame.distinct

The distinct() function returns only the unique rows across **all columns** in the DataFrame. It does not allow specifying a subset of columns, so it cannot be used to return distinct rows based on multiple selected columns.

B. pyspark.sql.DataFrame.dropDuplicates

The dropDuplicates() function allows a data engineer to specify **one or more columns** and returns a new DataFrame with duplicate rows removed based on those columns. This makes it the correct choice when distinctness is required across multiple specific columns.

C. pyspark.sql.DataFrame.dropna

The dropna() function is used to remove rows containing null values. It has no functionality related to identifying or removing duplicate rows.

D. pyspark.sql.DataFrame.drop

The drop() function removes specified columns from a DataFrame. It does not perform any deduplication or row-level filtering.

Question: 340 CertyIQ

A data engineer wants to pass multiple parameters from a Databricks Job to a notebook. They already configured the key and value of each parameter in the configurations of the job.
Which of the following utilities can the data engineer use to read the passed parameters inside the notebook?
Select one:

A. dbutils.widgets
B. dbutils.library
C. dbutils.secrets
D. dbutils.notebook

Answer: A

Explanation:

A. dbutils.widgets

dbutils.widgets is the standard Databricks utility used to pass parameters from a Databricks Job into a notebook and read them at runtime. When job parameters are defined in the job configuration, Databricks automatically exposes them as notebook widgets. Inside the notebook, the data engineer can retrieve the values using methods such as dbutils.widgets.get("parameter_name"). This approach supports multiple parameters, works consistently across job runs, and is commonly used in production pipelines.

B. dbutils.library

dbutils.library is designed for installing and managing libraries on Databricks clusters. It has no capability to read or manage parameters passed from jobs to notebooks, making it irrelevant for this use case.

C. dbutils.secrets

dbutils.secrets is used to securely retrieve sensitive values such as passwords or API keys from secret scopes. While secrets can be used as job parameters, this utility itself is not responsible for reading job parameters

passed into notebooks.

D. dbutils.notebook

dbutils.notebook is primarily used for notebook orchestration tasks such as running other notebooks or exiting a notebook with a return value. It does not provide an interface for accessing job parameters passed into the notebook.

Question: 341 CertyIQ

Which of the following describes the minimal permissions a data engineer needs to start an existing cluster, and attach a notebook to it?
Select one:

A. Cluster creation allowed + "Can Restart" privileges on the cluster
B. "Can Restart" privilege on the cluster
C. "Can Manage" privilege on the cluster
D. "Can Attach To" privilege on the cluster

Answer: B

Explanation:

A. Cluster creation allowed + "Can Restart" privileges on the cluster

This option grants more permissions than required. Cluster creation permission is not necessary to start an existing cluster or attach a notebook. The question asks for the **minimal** permissions, so this option is excessive.

B. "Can Restart" privilege on the cluster

This is correct. The **Can Restart** permission allows a user to start, stop, or restart an existing cluster. In Databricks, this permission also implicitly allows the user to attach a notebook to the cluster once it is running, making it the minimal permission required to meet both requirements.

C. "Can Manage" privilege on the cluster

The **Can Manage** privilege provides full administrative control over the cluster, including editing configurations and permissions. While it would allow starting and attaching, it is not the minimal permission needed.

D. "Can Attach To" privilege on the cluster

The **Can Attach To** permission allows a user to attach a notebook to a cluster only if the cluster is already running. It does not allow the user to start or restart the cluster, so it does not meet the full requirement.

Question: 342 CertyIQ

Which of the following methods does Not allow data engineers to create a multi-task job in Databricks?
Select one:

A. REST API
B. Databricks Asset Bundles (DABS)

C. Lakeflow Declarative Pipelines

D. Workspace UI

Answer: C

Explanation:

A. REST API

Databricks provides REST APIs that allow data engineers to programmatically create, configure, and manage **multi-task jobs**. Using the Jobs API, engineers can define multiple tasks, task dependencies, clusters, and schedules, making this a valid method.

B. Databricks Asset Bundles (DABS)

Databricks Asset Bundles support defining **multi-task jobs as code** using YAML. Engineers can declaratively specify tasks, dependencies, clusters, permissions, and environments, making DABS a fully supported approach for creating multi-task jobs.

C. Lakeflow Declarative Pipelines

Lakeflow Declarative Pipelines (formerly Delta Live Tables pipelines) are designed specifically for **data transformation pipelines**, not general-purpose job orchestration. They do not support defining arbitrary multi-task jobs with heterogeneous task types, so this method does **not** allow creation of multi-task Databricks jobs.

D. Workspace UI

The Databricks Workspace UI allows users to create **multi-task jobs** interactively by adding tasks, defining dependencies, selecting clusters, and configuring schedules through the graphical interface. This is a supported and commonly used method.

Question: 343 CertyIQ

A Spark job is taking longer than expected. Using the Spark UI, a data engineer notes that the Min, Median, and Max Durations for tasks in a particular stage show the minimum and median time to complete a task as roughly the same, but the max duration for a task to be roughly 100 times as long as the minimum.
Which situation is causing increased duration of the overall job?
Select one:

A. Task queueing resulting from improper thread pool assignment.

B. Spill resulting from attached volume storage being too small

C. Network latency due to some cluster nodes being in different regions from the source data

D. Skew caused by more data being assigned to a subset of spark-partitions

Answer: D

Explanation:

A. Task queueing resulting from improper thread pool assignment

Task queueing would cause many tasks to wait before execution, typically increasing the **minimum and median** task durations as well. It would not usually result in one task taking dramatically longer than all others.

B. Spill resulting from attached volume storage being too small

Spill to disk due to insufficient memory generally affects **many or all tasks** in a stage. This would increase overall task durations more uniformly, rather than causing a single task to take 100 times longer than the rest.

C. Network latency due to some cluster nodes being in different regions from the source data

Network latency issues tend to slow down tasks across the board. This would increase the median task duration, not just the maximum, making it an unlikely cause of a single extreme outlier.

D. Skew caused by more data being assigned to a subset of spark-partitions

This is the correct explanation. Data skew occurs when one or a few Spark partitions contain significantly more data than others. Tasks processing these large partitions take much longer, leading to a **very high max task duration** while the minimum and median remain low. These slow tasks delay the completion of the entire stage and increase overall job runtime.

Question: 344 CertyIQ

A data engineer is responsible for managing and orchestrating data workflows in their organization's Databricks environment. They have deployed a job called events_process_job using Databricks Asset Bundles. To execute this job, the engineer runs the following command from their terminal:
databricks bundle run events_process_job
After observing the command, a senior data engineer suggests that they could improve the execution process by adding the -t option when running the command.
Which of the following explain the primary purpose of this option?

A. To specify the target cluster size for the job run
B. To trigger dry run of the job without actually processing data
C. To select the target environment for the job run
D. To enable temporary logging during job execution

Answer: C

Explanation:

A. To specify the target cluster size for the job run

The -t option is not used to control cluster sizing. Cluster configuration is defined in the Databricks Asset Bundle YAML under job clusters or existing cluster references, not at execution time via command-line flags.

B. To trigger dry run of the job without actually processing data

Databricks Asset Bundles do not use the -t option to perform dry runs. Dry-run or validation behavior is handled through other commands such as bundle validation, not during execution with bundle run.

C. To select the target environment for the job run

This is correct. The -t (or --target) option is used to specify which **deployment target environment** — such as dev, test, or prod — the job should run against. It determines which workspace, configurations, and resources defined in the bundle's target section are used during execution.

D. To enable temporary logging during job execution

Logging behavior is not controlled by the -t option. Logs are managed through Databricks job configurations and cluster logging settings, not via bundle execution flags.

Question: 345 CertyIQ

The data engineering team maintains a Delta Lake table of SCD Type 1. A junior data engineer noticed a folder named '_change_data' in the table directory, and wants to understand what this folder is used for.
Which of the following describes the purpose of this folder?
Select one:

A. The '_change_data' folder is the default directory to track the evolution in schema definition

B. All SCD Type 1 tables have the '_change_data' folder to track the updates applied on the table's data.

C. Optimized Writes feature is enabled on the table. The '_change_data' folder is the location where the optimized data is stored

D. CDF feature is enabled on the table. The '_change_data' folder is the location where CDF data is stored

Answer: D

Explanation:

A. The '_change_data' folder is the default directory to track the evolution in schema definition

This is incorrect. Schema evolution metadata is tracked in the Delta transaction log (_delta_log), not in the _change_data directory.

B. All SCD Type 1 tables have the '_change_data' folder to track the updates applied on the table's data.

This is incorrect because the _change_data folder is **not created by default** for SCD Type 1 tables. It appears only when **Change Data Feed (CDF)** is explicitly enabled on the Delta table.

C. Optimized Writes feature is enabled on the table. The '_change_data' folder is the location where the optimized data is stored

This is incorrect. Optimized Writes affects how data files are written but does not create or use a _change_data folder. Optimized data is stored in the main table data directory.

D. CDF feature is enabled on the table. The '_change_data' folder is the location where CDF data is stored

This is correct. When **Change Data Feed (CDF)** is enabled on a Delta table, Delta Lake stores row-level change information — such as inserts, updates, and deletes — in the _change_data directory. This allows downstream consumers to efficiently read incremental changes.

Question: 346 CertyIQ

A data engineer wants to optimize the following join operation by allowing the smaller dataFrame to be sent to all executor nodes in the cluster:
largeDF.join(smallerDF, ["key"], "inner")
Which of the following functions can be used to mark a dataFrame as small enough to fit in memory on all executors ?
Select one:

A. pyspark.sql.functions.shuffle

B. pyspark.sql.functions.explode

C. pyspark.sql.functions.broadcast

D. pyspark.sql.functions.distribute

Answer: C

Explanation:

A. pyspark.sql.functions.shuffle

There is no shuffle function in pyspark.sql.functions that can be used to optimize joins by marking a DataFrame as small. Shuffling is an internal Spark operation, not a user-invoked function.

B. pyspark.sql.functions.explode

The explode function is used to transform array or map columns into multiple rows. It has no role in join optimization or in marking a DataFrame as small enough to be broadcast.

C. pyspark.sql.functions.broadcast

The broadcast() function explicitly marks a DataFrame as small enough to be sent to all executor nodes. Spark then performs a broadcast join, avoiding expensive shuffles of the large DataFrame and significantly improving join performance.

D. pyspark.sql.functions.distribute

There is no distribute function in pyspark.sql.functions. Distribution of data across executors is handled internally by Spark, not via this function.

Question: 347 CertyIQ

A data engineer wants to store passwords securely in a Unity Catalog managed table. They need to hash user passwords using SHA2(password, 256) before storing them. To ensure proper storage, the engineer must also set a constraint on the column length to accommodate the full hash value.
The engineer tests hashing the passwords "spark123" and "ApacheSpark111".
What will the engineer notice about the resulting hash lengths?
Select one:

A. The hash of "spark123" will be shorter than the hash of "ApacheSpark111"
B. Both hashes will have the same length, regardless of input size
C. Both hashes will have the same length because hash length depends on the number of numeric characters
D. The hash of "ApacheSpark111" will be shorter than the hash of "spark123"

Answer: B

Explanation:

A. The hash of "spark123" will be shorter than the hash of "ApacheSpark111"

This is incorrect because cryptographic hash functions like SHA-256 do not produce variable-length outputs based on input size. Regardless of whether the input string is short or long, the output hash length remains constant.

B. Both hashes will have the same length, regardless of input size

This is correct. The SHA2(password, 256) function uses the SHA-256 algorithm, which always produces a fixed-length hash output (256 bits, typically represented as a 64-character hexadecimal string). Therefore,

hashing "spark123" and "ApacheSpark111" results in hashes of identical length.

C. Both hashes will have the same length because hash length depends on the number of numeric characters

This is incorrect. The length of a SHA-256 hash does not depend on the number of numeric or alphabetic characters in the input. It depends solely on the hashing algorithm used.

D. The hash of "ApacheSpark111" will be shorter than the hash of "spark123"

This is incorrect for the same reason as option A. Input length does not influence the output length of SHA-256 hashes.

Question: 348 CertyIQ

A data engineer is testing a transformation pipeline that adds a new column to an existing DataFrame. They want to ensure the resulting DataFrame matches the expected output.
Which of the following functions can a data engineer use to verify equality?
Select one:

A. assertDataFrameEqual(actual_df, expected_df)
B. assert(actual_df = expected_df)
C. verifyEquality(actual_df, expected_df)
D. assertEqual(actual_df, expected_df)

Answer: A

Explanation:

A. assertDataFrameEqual(actual_df, expected_df)

This is the correct function. assertDataFrameEqual is provided by Databricks test utilities and is specifically designed to compare two DataFrames. It checks both schema and data equality, making it ideal for validating transformation logic in data pipelines.

B. assert(actual_df = expected_df)

This option is incorrect because it uses assignment (=) instead of comparison, which is invalid in Python assertions. Even with correct syntax, a simple assert cannot reliably compare DataFrames.

C. verifyEquality(actual_df, expected_df)

This is not a valid or supported Spark or Databricks function. It does not exist in PySpark or Databricks testing libraries.

D. assertEqual(actual_df, expected_df)

assertEqual is a generic assertion method used in Python testing frameworks for simple objects. It does not correctly compare Spark DataFrames and will not validate schema and row-level equality.

Question: 349 CertyIQ

A scheduled job failed due to an upstream data source issue. After resolving the issue, the data engineer wants to

use the Jobs API to trigger the same job again without waiting for its next scheduled run.
Which of the following REST API calls achieves this requirement?
Select one:

A. Send GET request to the endpoint '/api/2.2/jobs/run'
B. Send POST request to the endpoint '/api/2.2/jobs/start'
C. Send POST request to the endpoint '/api/2.2/jobs/run'
D. Send POST request to the endpoint '/api/2.2/jobs/run-now'

Answer: D

Explanation:

A. Send GET request to the endpoint /api/2.2/jobs/run

GET requests in the Databricks Jobs API are used only to retrieve metadata or status information about jobs and job runs. They cannot be used to start or rerun a job. Additionally, there is no supported /jobs/run endpoint that accepts a GET request for triggering execution, so this option is invalid.

B. Send POST request to the endpoint /api/2.2/jobs/start

The Databricks Jobs API does not provide a /jobs/start endpoint. Job execution is not initiated using a generic "start" call, which makes this option incorrect and unsupported.

C. Send POST request to the endpoint /api/2.2/jobs/run

There is no valid /jobs/run endpoint for executing an existing job. Attempting to use this endpoint would result in an API error because Databricks requires a specific action-based endpoint to trigger a run.

D. Send POST request to the endpoint /api/2.2/jobs/run-now

This is the correct approach. The run-now endpoint is explicitly designed to trigger an immediate execution of an existing Databricks job, regardless of its schedule. It allows the data engineer to rerun a failed job as soon as upstream issues are resolved, without waiting for the next scheduled run time.

Question: 350 CertyIQ

The data engineering team has the following query for processing customers' requests to be forgotten:
DELETE FROM customers
WHERE customer_id IN
(SELECT customer_id FROM delete_requests)
Which statement describes the results of executing this query?
Select one:

A. The identified records will be deleted from the customers tables, and their associated data files will be permanently purged from the table directory.
B. The identified records will be deleted from the customers table, but they will still be accessible in the table history until a VACUUM command is run.
C. The identified records will be deleted from both customers and delete_requests tables, but they will still be accessible in the table history until VACUUM commands are run.
D. The identified will be deleted from the both customers and delete_requests tables, and their associated data files will be permanently purged from the tables directories.

Answer: B

Explanation:

A. The identified records will be deleted from the customers table, and their associated data files will be permanently purged from the table directory.

This is incorrect because Delta Lake uses **ACID transactions** and **time travel**, meaning deleted data is not immediately removed from storage. The files remain in the table directory until a VACUUM operation is explicitly run.

B. The identified records will be deleted from the customers table, but they will still be accessible in the table history until a VACUUM command is run.

This is correct. In Delta Lake, the DELETE statement marks the records as deleted in the table, but the underlying data files are retained to support **time travel and audit history**. Only after running VACUUM are the physical files permanently removed.

C. The identified records will be deleted from both customers and delete_requests tables, but they will still be accessible in the table history until VACUUM commands are run.

This is incorrect because the query only deletes records from the customers table based on the delete_requests table. The delete_requests table remains unchanged.

D. The identified records will be deleted from both customers and delete_requests tables, and their associated data files will be permanently purged from the tables directories.

This is incorrect for the same reasons as above. Delta Lake **does not immediately purge files**, and the delete_requests table is not affected by this query.

Question: 351 CertyIQ

Which of the following commands can a data engineer use to create a Delta table "orders" with Automatic Liquid Clustering enabled?
Select one:

A. CREATE OR REPLACE TABLE orders(id int, updated date, value double) CLUSTER BY AUTO;
B. CREATE OR REPLACE TABLE orders(id int, updated date, value double) CLUSTER BY (Id, updated, value);
C. CREATE OR REPLACE TABLE orders(id int, updated date, value double) CLUSTER BY ALL;
D. CREATE OR REPLACE TABLE orders(id int, updated date, value double) CLUSTER BY NONE;

Answer: A

Explanation:

A. CREATE OR REPLACE TABLE orders(id int, updated date, value double) CLUSTER BY AUTO;

This is correct. Using CLUSTER BY AUTO enables **Automatic Liquid Clustering** in Delta Lake. With this option, Delta automatically manages clustering on the table to optimize query performance without the user having to specify exact columns. It adapts dynamically to query patterns and data changes.

B. CREATE OR REPLACE TABLE orders(id int, updated date, value double) CLUSTER BY (Id, updated, value);

This explicitly defines **manual clustering** on the specified columns. While this creates a clustered table, it does **not** enable Automatic Liquid Clustering.

C. CREATE OR REPLACE TABLE orders(id int, updated date, value double) CLUSTER BY ALL;

CLUSTER BY ALL is not a valid syntax in Delta Lake. Delta does not support this option.

D. CREATE OR REPLACE TABLE orders(id int, updated date, value double) CLUSTER BY NONE;

This disables clustering entirely. It does not enable Automatic Liquid Clustering.

Question: 352 CertyIQ

A developer has configured Databricks Connect and starts running PySpark code locally. They notice that DataFrame operations are executed on the remote cluster, but actions like .show() return results directly in their local console.
What does this behaviour illustrate about how Databricks Connect is used?

A. It proxies Spark commands from the local client to the remote cluster
B. It runs all Spark compute locally and only reads remote files
C. It mirrors the entire cluster environment on the developer's machine
D. It requires manual synchronization of results from the cluster

Answer: A

Explanation:

A. ✔ **It proxies Spark commands from the local client to the remote cluster**

This is correct. Databricks Connect lets you write PySpark code locally while all Spark operations are executed on the remote Databricks cluster. The local machine acts as a thin client that sends commands to the cluster and receives results back, which is why actions like .show() display output in the local console.

B. **It runs all Spark compute locally and only reads remote files**

This is incorrect. Spark computation does not run on the local machine when using Databricks Connect. All processing happens on the remote cluster, not locally.

C. **It mirrors the entire cluster environment on the developer's machine**

This is incorrect. Databricks Connect does not replicate or mirror the full cluster environment locally. There are no local Spark executors or a full Databricks runtime running on the developer's system.

D. **It requires manual synchronization of results from the cluster**

This is incorrect. Databricks Connect automatically returns results from the remote cluster to the local environment. No manual synchronization is required.

Question: 353 CertyIQ

A data engineer is setting up a new Databricks pipeline that ingests clickstream events from Kafka and daily product catalogs from cloud object storage. To ensure auditability and easy reprocessing, the engineer wants to land all source data. Later stages will handle cleaning, deduplication, and business modelling before the data is used in dashboards.
Which approach aligns with Medallion Architecture principles?

A. Land streaming events from Kafka in Silver, and the product catalog directly in Gold, to minimize layers for

batch data.

B. Land both sources directly into the Silver layer with schema enforcement and deduplication to reduce downstream complexity.

C. Land both sources in Gold with denormalized star schemas to optimize BI while retaining full source fidelity.

D. Land both sources in the Bronze layer append-only with minimal validation, then build Silver/Gold downstream for quality and analytics.

Answer: D

Explanation:

A. **Land streaming events from Kafka in Silver, and the product catalog directly in Gold, to minimize layers for batch data.**

This does not align with Medallion Architecture. Silver and Gold layers are meant for refined and business-ready data. Skipping Bronze breaks auditability and makes it harder to reprocess raw data if upstream logic changes.

B. **Land both sources directly into the Silver layer with schema enforcement and deduplication to reduce downstream complexity.**

This violates the principle of keeping raw data immutable. Silver should contain cleaned and deduplicated data, but the raw source data should first be captured in Bronze to support replay, debugging, and lineage tracking.

C. **Land both sources in Gold with denormalized star schemas to optimize BI while retaining full source fidelity.**

Gold is designed for analytics and BI consumption, not for retaining raw source fidelity. Transforming data into star schemas at ingestion removes the ability to fully audit or reprocess the original data.

D. ✔ **Land both sources in the Bronze layer append-only with minimal validation, then build Silver/Gold downstream for quality and analytics.**

This correctly follows Medallion Architecture principles. Bronze stores raw, append-only data from all sources, including streaming and batch, ensuring auditability and reprocessing. Silver handles cleaning, deduplication, and conformance, while Gold provides business-ready datasets for dashboards and analytics.

Question: 354 CertyIQ

When using MERGE INTO in Delta Lake on Databricks, which clause allows you to atomically delete target rows that no longer match any source rows?

A. WHEN MATCHED BY SOURCE THEN UPDATE SET NULL

B. WHEN NOT MATCHED THEN DELETE

C. WHEN NOT MATCHED BY SOURCE THEN DELETE

D. WHEN MATCHED AND THEN DELETE

Answer: C

Explanation:

A. **WHEN MATCHED BY SOURCE THEN UPDATE SET NULL**

This clause is not valid in Delta Lake syntax. Updates occur when target rows match source rows, but this clause does not exist and does not perform deletions.

B. **WHEN NOT MATCHED THEN DELETE**

This is incorrect. WHEN NOT MATCHED applies to rows that exist in the source but not in the target and is typically used with INSERT, not DELETE.

C. ✔ **WHEN NOT MATCHED BY SOURCE THEN DELETE**

This is correct. This clause deletes rows from the target table that no longer have a corresponding row in the source table. It enables atomic "delete on missing source" behavior, which is commonly used for full refresh or synchronization use cases.

D. **WHEN MATCHED AND THEN DELETE**

This clause is syntactically invalid in Delta Lake. Deletions on matched rows must follow valid conditional syntax, and this form is not supported.

Question: 355 CertyIQ

A data engineer is onboarding a new bronze ingestion pipeline in Databricks with Unity Catalog. The team wants Databricks to handle storage layout, apply platform optimizations over time, and simplify lifecycle management so that when a table is dropped, its underlying data is also cleaned up according to Databricks-managed retention policies.
Which table type should the data engineer create for these ingestion tables?

A. External tables with a LOCATION pointing to an external volume for full control of file layout.
B. Managed tables so that Unity Catalog manages both metadata and the underlying data lifecycle.
C. Temporary views over files to avoid table-level governance and lifecycle coupling.
D. Foreign tables federated from an external catalog to delegate optimization to the source system.

Answer: B

Explanation:

A. **External tables with a LOCATION pointing to an external volume for full control of file layout.**

This is incorrect. External tables leave data lifecycle and storage layout under the user's control. Dropping the table does not clean up the underlying data, and Databricks cannot fully manage retention or lifecycle policies.

B. ✔ **Managed tables so that Unity Catalog manages both metadata and the underlying data lifecycle.**

This is correct. Managed tables allow Unity Catalog and Databricks to control storage layout, apply optimizations automatically over time, and manage the full data lifecycle. When a managed table is dropped, Databricks also cleans up the underlying data according to platform-managed retention policies.

C. **Temporary views over files to avoid table-level governance and lifecycle coupling.**

This is incorrect. Temporary views do not provide persistent storage, governance, or lifecycle management. They are not suitable for ingestion pipelines or long-term data management.

D. **Foreign tables federated from an external catalog to delegate optimization to the source system.**

This is incorrect. Foreign or federated tables rely on external systems for storage and optimization, which

contradicts the requirement for Databricks-managed storage layout and lifecycle management.

Question: 356 CertyIQ

A data engineer needs to compute metrics such as:
rolling 7-day sales
percent of total revenue
top-N most profitable categories
Which two approaches are appropriate for implementing these metrics with PySpark DataFrames?

Answer:

BC

Explanation:

A. **Using RDD aggregations for better performance**

This is not appropriate. RDD-based aggregations bypass Spark SQL optimizations such as Catalyst and Tungsten. PySpark DataFrames are preferred for metric computations because they are more performant, optimized, and expressive for analytical workloads.

B. ✔ **Using window functions for rolling and ranking calculations**

This is correct. Window functions are the standard and recommended approach in PySpark for rolling metrics like 7-day sales and ranking use cases such as top-N most profitable categories. They operate efficiently in a distributed manner without collapsing the dataset.

C. ✔ **Using joins and groupBy to compute percent-of-total metrics**

This is correct. Percent-of-total metrics are commonly implemented by computing a total using groupBy or aggregation and then joining it back to the detailed DataFrame. This approach is scalable, readable, and fully optimized by Spark.

D. **Using mapPartitions to manually implement rolling windows**

This is not appropriate. Manually implementing rolling logic with mapPartitions is complex, error-prone, and bypasses Spark's built-in optimizations. Window functions should be used instead.

E. **Collecting the DataFrame to the driver for faster iterative processing**

This is incorrect. Collecting data to the driver breaks distributed processing, does not scale, and can cause memory issues. Metric computations should always be performed in the distributed DataFrame API.

Question: 357 CertyIQ

A data engineer is investigating a Lakehouse ETL job on Databricks that has suddenly become four times slower.
They need to:
Identify the specific Spark stages that regressed.
Drill down to per-task metrics such as shuffle read/write size, spill, and GC time.
Correlate these with the executor resource usage during that run.
Which Databricks-native capability is the starting point for this detailed job-level root-cause analysis?

A. Spark UI opened from the job run's compute, using the Jobs and Stages tabs to inspect task metrics.

B. Ganglia metrics for the cluster, which provide full visibility into Spark stages, tasks, and shuffle metrics for

the job run.

C. The Jobs run details page timeline and graph views, without opening the Spark UI, to analyze all stage and task-level Spark metrics.

D. Cluster Metrics (Metrics tab on the compute) to view historical Spark and hardware metrics, without using the Spark UI.

Answer: A

Explanation:

A. ✔ **Spark UI opened from the job run's compute, using the Jobs and Stages tabs to inspect task metrics.**

This is the correct starting point. The Spark UI provides detailed, job-level visibility into stages and tasks, including shuffle read and write sizes, spill metrics, GC time, task duration, and executor-level behavior. It is specifically designed for deep root-cause analysis of Spark performance regressions and allows correlation of stage slowdowns with executor resource usage during a particular run.

B. **Ganglia metrics for the cluster, which provide full visibility into Spark stages, tasks, and shuffle metrics for the job run.**

This is incorrect. Ganglia focuses on cluster-level and node-level system metrics such as CPU, memory, disk, and network usage. It does not provide Spark stage- or task-level details like shuffle size or per-task execution time.

C. **The Jobs run details page timeline and graph views, without opening the Spark UI, to analyze all stage and task-level Spark metrics.**

This is incorrect. The Jobs run details page provides a high-level timeline and summaries but does not expose the full depth of per-stage and per-task Spark metrics required for detailed performance regression analysis.

D. **Cluster Metrics (Metrics tab on the compute) to view historical Spark and hardware metrics, without using the Spark UI.**

This is incorrect. Cluster Metrics are useful for understanding overall resource utilization trends but do not allow inspection of individual Spark stages, tasks, shuffle behavior, or GC time for a specific job run.

Question: 358 CertyIQ

A team is designing their Databricks Asset Bundle for deployment. They want to ensure their bundle includes all necessary resources, targeted environment configurations, and deployment settings in a single version-controlled file.
What is required as the main configuration file at the root of their project?

A. A databricks.yml YAML file that contains the bundle's name, targets, includes resources, and variables

B. A directory with multiple TXT files, each describing a single resource for the bundle

C. A JSON file named bundle_config.json that stores all resource details and environment variables

D. A Python script called configure_bundle.py that dynamically creates resources at runtime

Answer: A

Explanation:

A. ✔ **A databricks.yml YAML file that contains the bundle's name, targets, includes resources, and variables**

This is correct. Databricks Asset Bundles require a databricks.yml file at the root of the project. This file defines the bundle name, deployment targets (such as dev, test, prod), included resources, variables, and environment-specific configurations. It serves as the single, version-controlled entry point for bundle configuration.

B. **A directory with multiple TXT files, each describing a single resource for the bundle**

This is incorrect. Asset Bundles use structured YAML configuration, not unstructured text files, and require a single main configuration file to coordinate resources and environments.

C. **A JSON file named bundle_config.json that stores all resource details and environment variables**

This is incorrect. Databricks Asset Bundles are not configured using JSON files. The required and supported format is YAML with the specific filename databricks.yml.

D. **A Python script called configure_bundle.py that dynamically creates resources at runtime**

This is incorrect. Asset Bundles are declarative, not imperative. Resources and environments are defined in configuration files rather than created dynamically via Python scripts.

Question: 359 CertyIQ

A data engineer must support self-serve BI dashboards for hundreds of business users who run ad hoc, high-concurrency SQL queries throughout the day against governed Unity Catalog tables. The team needs a near-instant start, autoscaling without manual tuning, and the best performance for SQL while minimizing operational overhead.
Which Databricks compute should the data engineer use?

A. SQL Warehouse (Serverless) with Photon enabled.
B. Jobs compute clusters triggered by dashboards on a schedule.
C. All-purpose classic cluster with manual autoscaling enabled.
D. SQL Warehouse (Classic) with a fixed cluster size.

Answer: A

Explanation:

A. ✔ **SQL Warehouse (Serverless) with Photon enabled.**

This is the best choice for self-serve BI with high concurrency. Serverless SQL Warehouses provide near-instant startup, automatic scaling without manual tuning, and are fully managed by Databricks. Photon delivers optimized, vectorized execution for SQL workloads, giving the best performance while minimizing operational overhead.

B. **Jobs compute clusters triggered by dashboards on a schedule.**

This is incorrect. Jobs compute is designed for batch and scheduled workloads, not interactive, ad hoc SQL queries with high concurrency. It does not provide low-latency or user-driven scaling.

C. **All-purpose classic cluster with manual autoscaling enabled.**

This is incorrect. While it can support SQL queries, it requires manual configuration, has slower startup times, and higher operational overhead compared to SQL Warehouses. It is not optimized for large numbers of concurrent BI users.

D. **SQL Warehouse (Classic) with a fixed cluster size.**

This is incorrect. A fixed-size classic SQL Warehouse lacks automatic scaling and can either underperform during peak concurrency or waste resources during low usage. It also has more operational overhead compared to serverless options.

Question: 360 CertyIQ

A team is using a Databricks workspace and needs to continuously ingest JSON files as they arrive.
Which code snippet shows a valid Auto Loader source configuration?

A. spark.readStream.format("cloudFiles").option("cloudFiles.format", "json").load("jdbc:sqlserver://;database=")
B. spark.readStream.format("json").option("cloudFiles.format", "json").load("")
C. spark.readStream.format("cloudFiles").option("cloudFiles.format", "json").load("")
D. spark.readStream.format("cloudFiles").option("format", "json").load("")

Answer: C

Explanation:

A. **spark.readStream.format("cloudFiles").option("cloudFiles.format", "json").load("jdbc:sqlserver://;database=")**

This is invalid because Auto Loader (cloudFiles) is designed to ingest files from cloud object storage (such as S3, ADLS, or GCS), not from JDBC sources. Using a JDBC URL with Auto Loader is not supported.

B. **spark.readStream.format("json").option("cloudFiles.format", "json").load("")**

This is incorrect because Auto Loader is not enabled. Using format("json") directly invokes Spark's native file streaming source, not Auto Loader. The cloudFiles.format option is ignored unless format("cloudFiles") is used.

C. ✔ **spark.readStream.format("cloudFiles").option("cloudFiles.format", "json").load("")**

This is correct. Auto Loader is enabled by setting format("cloudFiles"), and the underlying file type is specified using cloudFiles.format. The load() path points to a cloud storage location where JSON files arrive continuously.

D. **spark.readStream.format("cloudFiles").option("format", "json").load("")**

This is incorrect because Auto Loader requires the option key cloudFiles.format. Using option("format", "json") does not correctly configure the file type for Auto Loader.

Question: 361 CertyIQ

A data engineer working in Databricks needs to control BI query latency and cost for a sales analytics domain. Silver tables already contain validated, normalized sales and product data. The team wants dashboard-friendly query performance and to align datasets with business concepts (e.g., weekly_sales and account_performance) without overloading analysts with complex joins.
Which design approach should a data engineer choose for the Gold layer in a medallion architecture?

A. Move raw, append-only source data directly into Gold to avoid transformation costs and enable time travel.
B. Keep all data heavily normalized in Gold to minimize storage and rely on BI tools to join data at query time.
C. Build only lightly cleansed views in Gold and preserve most business logic in Silver to reduce duplication.

D. Create denormalized, domain-aligned tables and materialized aggregates (e.g., weekly_sales) optimized for reporting and dashboards.

Answer: D

Explanation:

A. **Move raw, append-only source data directly into Gold to avoid transformation costs and enable time travel.**

This is incorrect. Gold is not intended for raw data. Raw, append-only data belongs in Bronze. Gold focuses on business-ready, curated datasets rather than source fidelity.

B. **Keep all data heavily normalized in Gold to minimize storage and rely on BI tools to join data at query time.**

This is incorrect. While normalization reduces storage, it increases query complexity and latency. Gold should optimize for BI performance and ease of use, not push complex joins to dashboards.

C. **Build only lightly cleansed views in Gold and preserve most business logic in Silver to reduce duplication.**

This is incorrect. Silver is for cleaning and conforming data, while Gold is where business logic, aggregations, and domain-specific modeling should live to support analytics and reporting.

D. ✔ **Create denormalized, domain-aligned tables and materialized aggregates (e.g., weekly_sales) optimized for reporting and dashboards.**

This is correct. Gold tables are designed for analytics consumption. Denormalizing data, aligning it to business concepts, and precomputing aggregates improves query latency, controls cost, and simplifies self-serve BI for analysts.

Question: 362 CertyIQ

A data engineering team is analyzing customer transactions using a PySpark DataFrame named transactions with columns customer_id, amount, and category.
The team needs to compute, for each customer, the total transaction amount, the average transaction amount, and the maximum amount - all in a single operation for efficiency.
Which code fragment should be used to achieve this in PySpark?

A. transactions.groupBy("customer_id").agg(sum("amount"), avg("amount"), max("amount"))
B. transactions.groupBy("customer_id").select(sum("amount"), avg("amount"), max("amount"))
C. transactions.agg(groupBy("customer_id"), sum("amount"), avg("amount"), max("amount"))
D. transactions.groupBy("customer_id").sum("amount").avg("amount").max("amount")

Answer: A

Explanation:

A. ✔ transactions.groupBy("customer_id").agg(sum("amount"), avg("amount"), max("amount"))

This is correct. In PySpark, groupBy() followed by agg() allows multiple aggregate functions to be computed in a single pass over the data. This is the most efficient and idiomatic way to calculate total, average, and maximum values per group.

B. transactions.groupBy("customer_id").select(sum("amount"), avg("amount"), max("amount"))

This is incorrect. After groupBy(), you must use agg() to apply aggregate functions. The select() method cannot be used directly on a grouped DataFrame for aggregation.

C. transactions.agg(groupBy("customer_id"), sum("amount"), avg("amount"), max("amount"))

This is incorrect. The agg() method does not accept groupBy() as an argument. Grouping must be performed first using groupBy(), followed by agg().

D. transactions.groupBy("customer_id").sum("amount").avg("amount").max("amount")

This is incorrect. Each of these aggregation methods returns a new DataFrame. They cannot be chained together to compute multiple aggregates in a single operation.

Question: 363 CertyIQ

Which single Databricks CLI command deploys local bundle assets to the target workspace specified in the configuration file?

A. databricks bundle init && databricks bundle deployment
B. databricks bundle deploy
C. databricks bundle deploy
D. databricks bundle deploy

Answer: B

Explanation:

Correct Answer: B. databricks bundle deploy

Why B is correct:

The databricks bundle deploy command deploys the local Databricks Asset Bundle (jobs, pipelines, dashboards, and other resources) to the target workspace specified in the databricks.yml configuration file.

It uploads the bundle resources and applies them to the configured target environment.

A. databricks bundle init && databricks bundle deployment — Incorrect

databricks bundle init creates a new bundle project. databricks bundle deployment is not a valid Databricks CLI command.

B. databricks bundle deploy — Correct

This command deploys the local bundle assets to the target workspace defined in the configuration file. It is the standard command used to publish Databricks Asset Bundles.

C. databricks bundle plan && databricks bundle run — Incorrect

databricks bundle plan previews deployment changes, while databricks bundle run executes an already deployed resource. Neither command deploys the bundle.

D. databricks bundle validate — Incorrect

This command validates the bundle configuration for syntax and correctness. It does not deploy any

resources.

Question: 364 CertyIQ

A data engineer needs to audit who made changes to a Delta table in the last 7 days.
Which SQL statement returns the record of past operations, including timestamps, users, and operation types?

A. SELECT * FROM `delta.my_table/_delta_log/
B. DESCRIBE HISTORY my_table
C. SHOW TBLPROPERTIES my_table
D. DESCRIBE DETAIL my_table

Answer: B

Explanation:

Correct Answer: B. DESCRIBE HISTORY my_table

Why B is correct:

The DESCRIBE HISTORY my_table command returns the complete transaction history of a Delta table, including details such as timestamps, users, operation types (INSERT, UPDATE, DELETE, MERGE, WRITE), notebook information, and other metadata.

To audit changes made in the last 7 days, you can query the history output and filter the results by timestamp.

A. SELECT * FROM delta.my_table/_delta_log/ — Incorrect

The _delta_log directory contains the Delta transaction log files, but it is not intended to be queried directly using a SQL SELECT statement. The recommended way to view table history is DESCRIBE HISTORY.

B. DESCRIBE HISTORY my_table — Correct

This command displays the history of operations performed on a Delta table, including timestamps, users, operation types, and other audit information. It is the standard SQL command for auditing Delta table changes.

C. SHOW TBLPROPERTIES my_table — Incorrect

This command displays the table's properties and configuration settings. It does not provide a history of operations or audit information.

D. DESCRIBE DETAIL my_table — Incorrect

This command returns metadata about the Delta table, such as its location, format, size, and number of files. It does not show the history of changes or user operations.

Question: 365 CertyIQ

A data engineer runs df.toPandas() on a wide DataFrame containing 50 million rows. The notebook cell fails with a java.lang.OutOfMemoryError on the driver.
Which memory configuration is directly responsible for this failure?

A. spark.executor.memory is too low to process the DataFrame transformations

B. spark.memory.fraction is too low, limiting the storage memory available for caching

C. spark.sql.shuffle.partitions is set too high, causing excessive memory consumption during shuffles

D. spark.driver.memory is too low to hold the collected result set

Answer: D

Explanation:

Correct Answer: D. spark.driver.memory is too low to hold the collected result set

Why D is correct:

The **toPandas()** method collects the entire Spark DataFrame to the driver node and converts it into a Pandas DataFrame. Since Pandas stores all the data in the driver's memory, the driver must have enough RAM to hold all 50 million rows.

If the dataset is too large, the driver runs out of memory, resulting in a **java.lang.OutOfMemoryError**.

A. spark.executor.memory is too low to process the DataFrame transformations — Incorrect

Spark executor memory controls the memory available to executor processes for distributed computations. The transformations may complete successfully, but **toPandas()** fails because the final result is collected on the driver.

B. spark.memory.fraction is too low, limiting the storage memory available for caching — Incorrect

Spark memory fraction determines how executor memory is shared between execution and storage. It does not affect the driver's ability to store the collected Pandas DataFrame.

C. spark.sql.shuffle.partitions is set too high, causing excessive memory consumption during shuffles — Incorrect

Spark SQL shuffle partitions control the number of shuffle partitions. Although this setting can impact performance, it is not the cause of the driver's **OutOfMemoryError** when using **toPandas()**.

D. spark.driver.memory is too low to hold the collected result set — Correct

The **toPandas()** method transfers the entire DataFrame to the driver. If the driver's memory is insufficient to hold the resulting Pandas DataFrame, a **java.lang.OutOfMemoryError** occurs. Increasing the driver memory or avoiding **toPandas()** for very large datasets is the appropriate solution.

Question: 366 CertyIQ

A data engineer is joining a very large events_df (billions of rows) with a small country_lookup_df (fewer than 10,000 rows) on the country_code column. The join is slow due to data shuffling.
How should the data engineer improve performance?

A. Cache both DataFrames, then run the same join

B. Persist both DataFrames, then run the same join

C. broadcast events_df when performing the join

D. broadcast country_lookup_df when performing the join

Answer: D

Explanation:

Correct Answer: D. broadcast country_lookup_df when performing the join

Why D is correct:

When joining a very large DataFrame with a small lookup DataFrame, broadcasting the small DataFrame sends a copy of it to every executor. This allows Spark to perform the join locally on each executor without shuffling the large DataFrame across the cluster, significantly improving performance.

A. Cache both DataFrames, then run the same join — Incorrect

Caching stores DataFrames in memory for faster reuse, but it does not eliminate the data shuffling caused by the join operation.

B. Persist both DataFrames, then run the same join — Incorrect

Persisting DataFrames improves performance when they are reused multiple times, but it does not reduce the network shuffle during the join.

C. broadcast events_df when performing the join — Incorrect

The events_df contains billions of rows and is far too large to broadcast. Broadcasting such a large DataFrame would consume excessive memory and reduce performance.

D. broadcast country_lookup_df when performing the join — Correct

The country_lookup_df contains fewer than 10,000 rows, making it an ideal candidate for broadcasting. This avoids shuffling the large events_df and results in a much faster join.

Question: 367 CertyIQ

A data engineer is attempting to grant a user access to a view. They successfully run GRANT SELECT ON VIEW sales_catalog.market_data.summary TO analyst_group.
However, the analyst still receives an "Insufficient Permissions" error when trying to query the view.
In which order should the engineer verify the Hierarchy of Securable Objects to ensure the analyst has the necessary usage permissions?

A. STORAGE CREDENTIAL -> EXTERNAL LOCATION -> VIEW
B. WORKSPACE -> CLUSTER -> VIEW
C. METASTORE -> TABLE -> VIEW
D. CATALOG -> SCHEMA -> VIEW

Answer: D

Explanation:

Correct Answer: D. CATALOG → SCHEMA → VIEW

Why D is correct:

In Unity Catalog, permissions follow a hierarchical structure. Even if a user has **SELECT** permission on a view, they must also have **USE CATALOG** permission on the catalog and **USE SCHEMA** permission on the schema containing the view.

The correct order to verify permissions is:

Catalog → Schema → View

If the user is missing usage permission on either the catalog or schema, they will receive an **"Insufficient Permissions"** error.

A. STORAGE CREDENTIAL → EXTERNAL LOCATION → VIEW — Incorrect

Storage credentials and external locations are used to control access to external cloud storage. They are not part of the permission hierarchy required to query a view.

B. WORKSPACE → CLUSTER → VIEW — Incorrect

Workspaces and clusters determine where queries run, but they are not part of Unity Catalog's securable object hierarchy for granting access to views.

C. METASTORE → TABLE → VIEW — Incorrect

Although the metastore is the top-level Unity Catalog object, usage permissions are granted on the catalog and schema before accessing a view. A table is also not part of the access hierarchy for querying a view.

D. CATALOG → SCHEMA → VIEW — Correct

A user must have **USE CATALOG** on the catalog, **USE SCHEMA** on the schema, and **SELECT** on the view. Verifying permissions in this order ensures the analyst has all required access to query the view.

Question: 368

CertyIQ

A data engineer configures a Databricks Lakeflow Job for daily customer data processing:
- Entry task: Single notebook loads raw data
- Parallel tasks:
- SQL query task for cleansing
- Notebook task for feature engineering
- Pipeline task for model updates
- Exit task: Dashboard refresh (must run after all parallel tasks complete)
- Requirement: Implement this dependency pattern using a DAG-based task graph

Which task configuration ensures all parallel tasks complete before the dashboard refresh task runs?

A. Create separate Lakeflow Jobs for each parallel task and trigger them sequentially using external orchestration.

B. Set the dashboard task to depend only on the SQL query task; other tasks run independently.

C. Configure the parallel tasks with sequential dependencies so each task waits for the previous one to finish.

D. Add the dashboard task as dependent on all three parallel tasks using fan-in control flow.

Answer: D

Explanation:

Correct Answer: D. Add the dashboard task as dependent on all three parallel tasks using fan-in control flow

Why D is correct:

In a DAG-based Lakeflow Job, **fan-out** allows multiple tasks to run in parallel after a single entry task, while **fan-in** ensures a downstream task starts only after all upstream tasks have completed successfully.

In this scenario:

The entry notebook loads the raw data.

The SQL query task, notebook task, and pipeline task run in parallel.

The dashboard refresh task must wait until all three parallel tasks finish.

This is achieved by making the dashboard task dependent on all three parallel tasks using **fan-in** control flow.

A. Create separate Lakeflow Jobs for each parallel task and trigger them sequentially using external orchestration. — Incorrect

Using separate jobs and external orchestration adds unnecessary complexity. A single DAG-based Lakeflow Job can handle these dependencies directly.

B. Set the dashboard task to depend only on the SQL query task; other tasks run independently. — Incorrect

The dashboard could start before the feature engineering or model update tasks complete, violating the requirement.

C. Configure the parallel tasks with sequential dependencies so each task waits for the previous one to finish. — Incorrect

This removes parallel execution and increases the overall job runtime. The requirement is for the tasks to run in parallel.

D. Add the dashboard task as dependent on all three parallel tasks using fan-in control flow. — Correct

This configuration allows the three processing tasks to execute in parallel while ensuring the dashboard refresh begins only after every parallel task has completed successfully.

Question: 369 CertyIQ

A data engineer uses the Databricks workspace UI with Unity Catalog enabled. In Data Explorer, they select catalog corp_marketing, then schema campaigns, and see table email_stats. The engineer must let the growth-analysts group read email_stats from their SQL warehouses, but not create, alter, or delete any objects in corp_marketing of campaigns.
Which action sequence meets the requirement?

A. 1. On schema campaigns, open Permissions and grant USE SCHEMA and SELECT to growth-analysts2. On the table email_stats, open Permissions and grant SELECT to growth-analysts

B. 1. On catalog corp_marketing, open Permissions and grant USE CATALOG to growth-analysts2. On schema campaigns, open Permissions and grant USE SCHEMA to growth-analysts3. On the table email_stats, open Permissions and grant SELECT to growth-analysts

C. 1. On catalog corp_marketing, open Permissions and grant USE CATALOG and CREATE SCHEMA to growth-analysts2. On schema campaigns, open Permissions and grant USE SCHEMA to growth-analysts3. On the table email_stats, open Permissions and grant SELECT to growth-analysts

D. 1. On schema campaigns, open Permissions and grant USE SCHEMA to growth-analysts2. On the table email_stats, open Permissions and grant SELECT and MODIFY to growth-analysts

Answer: B

Explanation:

Correct Answer: B.

Why B is correct:

To allow the **growth-analysts** group to read the **email_stats** table from SQL Warehouses in Unity Catalog, they need permissions at each level of the securable object hierarchy:

1. **USE CATALOG** on the **corp_marketing** catalog.
2. **USE SCHEMA** on the **campaigns** schema.
3. **SELECT** on the **email_stats** table.

These permissions allow users to query the table without giving them the ability to create, alter, or delete objects.

A.

1. On schema campaigns, open Permissions and grant USE SCHEMA and SELECT to growth-analysts.
2. On the table email_stats, open Permissions and grant SELECT to growth-analysts. — **Incorrect**

This option does not grant **USE CATALOG** on the **corp_marketing** catalog. Without catalog access, users cannot access the schema or table.

B.

1. On catalog corp_marketing, open Permissions and grant USE CATALOG to growth-analysts.
2. On schema campaigns, open Permissions and grant USE SCHEMA to growth-analysts.
3. On the table email_stats, open Permissions and grant SELECT to growth-analysts. — **Correct**

This follows the Unity Catalog permission hierarchy and grants only the minimum permissions required to read the table.

C.

1. On catalog corp_marketing, open Permissions and grant USE CATALOG and CREATE SCHEMA to growth-analysts.
2. On schema campaigns, open Permissions and grant USE SCHEMA to growth-analysts.
3. On the table email_stats, open Permissions and grant SELECT to growth-analysts. — **Incorrect**

The **CREATE SCHEMA** privilege allows users to create new schemas, which violates the requirement that they must not create, alter, or delete objects.

D.

1. On schema campaigns, open Permissions and grant USE SCHEMA to growth-analysts.
2. On the table email_stats, open Permissions and grant SELECT and MODIFY to growth-analysts. — **Incorrect**

This option is missing **USE CATALOG**, and the **MODIFY** privilege allows users to update or delete data, which exceeds the required read-only access.

Question: 370 CertyIQ

A data engineer has two DataFrames: orders_df (all orders) and customers_df (only active customers). The data engineer needs to filter out orders placed by inactive customers.
Which operation should the data engineer use?

A. orders_df and customers_df inner join

B. customers_df left join with orders_df

C. orders_df and customers_df cross join

D. orders_df left join with customers_df

Answer: A

Explanation:

Correct Answer: A. orders_df and customers_df inner join

Why A is correct:

An **inner join** returns only the rows where the join key exists in both DataFrames. Since **customers_df** contains only active customers, performing an inner join with **orders_df** filters out any orders placed by inactive customers.

This ensures that only orders belonging to active customers are included in the result.

A. orders_df and customers_df inner join — Correct

An inner join keeps only the matching records from both DataFrames, effectively removing orders associated with inactive customers.

B. customers_df left join with orders_df — Incorrect

A left join starting from **customers_df** returns all active customers, even those without any orders. This changes the focus of the result and does not simply filter the orders.

C. orders_df and customers_df cross join — Incorrect

A cross join creates every possible combination of rows between the two DataFrames, resulting in a Cartesian product. It does not filter inactive customers.

D. orders_df left join with customers_df — Incorrect

A left join starting from **orders_df** keeps all orders, including those placed by inactive customers. Unmatched customers would simply have null values instead of being filtered out.

Question: 371 CertyIQ

A company has a strict 15-minute Service Level Agreement (SLA) for updating its currency exchange dashboard. The source data arrives in small increments every few minutes. The data engineering team needs a triggering strategy for their Lakeflow Job that keeps end-to-end latency within the SLA while minimizing compute cost and DBU consumption.
Which triggering strategy for the Lakeflow Job is the recommended way to meet this 15-minute SLA?

A. Use a Scheduled trigger set to run every 12 minutes, with the streaming task configured to use Trigger.AvailableNow.

B. Use a Continuous trigger to ensure data is processed as soon as it arrives.

C. Use a Scheduled trigger set to run every 1 minute to ensure the SLA is never at risk.

D. Use a File Arrival trigger configured to fire every time a single JSON file lands in the bucket

Answer: A

Explanation:

Correct Answer: A. Use a Scheduled trigger set to run every 12 minutes, with the streaming task configured

to use Trigger.AvailableNow.

Why A is correct:

A **Scheduled trigger** running every **12 minutes** ensures the dashboard is updated within the **15-minute SLA**. Using **Trigger.AvailableNow** processes all newly available data and then automatically stops the job, minimizing compute time and DBU consumption.

This approach balances low latency with cost efficiency.

A. Use a Scheduled trigger set to run every 12 minutes, with the streaming task configured to use Trigger.AvailableNow. — Correct

This strategy meets the 15-minute SLA while minimizing compute costs. The job processes all available data at each scheduled run and shuts down when finished.

B. Use a Continuous trigger to ensure data is processed as soon as it arrives. — Incorrect

A Continuous trigger provides the lowest latency but requires compute resources to remain running continuously, leading to higher DBU consumption than necessary for a 15-minute SLA.

C. Use a Scheduled trigger set to run every 1 minute to ensure the SLA is never at risk. — Incorrect

Running the job every minute greatly exceeds the SLA requirement and results in unnecessary compute usage and higher costs.

D. Use a File Arrival trigger configured to fire every time a single JSON file lands in the bucket. — Incorrect

Triggering a job for every arriving file can cause excessive job executions and increased costs. It is less efficient than a scheduled approach when data arrives frequently in small increments.

Question: 372 CertyIQ

A data engineering team uses Declarative Automation Bundles (formerly Databricks Asset Bundles) to deploy the same codebase across dev, test, and prod environments. The team wants environment-specific behavior to be applied only through bundle configuration, without modifying notebooks, job definitions, or deployment logic during promotion.
Which approach applies environment-specific configuration while meeting this requirement?

A. Define bundle variables and reference them from bundle resources, then override variable values per environment using bundle targets.

B. Deploy the same bundle once and edit job settings in the workspace UI for each environment after deployment.

C. Parameterize environment-specific values inside notebooks and set those parameters when each job run is triggered.

D. Use separate Git branches per environment so each branch can contain environment-specific configuration values.

Answer: A

Explanation:

Correct Answer: A. Define bundle variables and reference them from bundle resources, then override variable values per environment using bundle targets.

Why A is correct:

Declarative Automation Bundles support **bundle variables** and **bundle targets** to manage environment-specific configurations. The same codebase, notebooks, and job definitions can be deployed to multiple environments, while variable values are overridden for each target (dev, test, and prod).

This keeps the deployment consistent and eliminates the need to modify code during promotion.

A. Define bundle variables and reference them from bundle resources, then override variable values per environment using bundle targets. — Correct

This is the recommended approach. Environment-specific values are defined as variables and overridden in each bundle target, allowing the same bundle to be promoted across environments without changing the application code or deployment logic.

B. Deploy the same bundle once and edit job settings in the workspace UI for each environment after deployment. — Incorrect

Manually editing job settings after deployment is error-prone and breaks Infrastructure as Code principles. Changes made in the workspace UI are not version-controlled.

C. Parameterize environment-specific values inside notebooks and set those parameters when each job run is triggered. — Incorrect

Although notebook parameters can change runtime behavior, they require application-level changes and do not centralize environment configuration within the bundle.

D. Use separate Git branches per environment so each branch can contain environment-specific configuration values. — Incorrect

Maintaining separate branches for each environment leads to code duplication and synchronization issues. Declarative Automation Bundles are designed to use a single codebase with environment-specific bundle targets.

Question: 373 CertyIQ

A data engineer observes that an ETL job in Databricks has increased its execution time from 5 minutes (baseline) to 12 minutes over the past week. Using the Lakeflow Jobs run history view, the engineer needs to identify whether the slowdown is consistent or intermittent and determine the root cause.
Which action should the engineer take to analyze execution time trends against the baseline?

A. Archive old job runs and reset the baseline counter to ensure accurate comparison of current performance.

B. Rerun the job immediately to establish a new baseline since historical data becomes irrelevant after seven days.

C. Increase cluster size automatically whenever execution time exceeds the previous run's duration.

D. Compare cluster metrics in Lakeflow Jobs run history to spot resource spikes causing delays.

Answer: D

Explanation:

Correct Answer: D. Compare cluster metrics in Lakeflow Jobs run history to spot resource spikes causing delays.

Why D is correct:

The **Lakeflow Jobs run history** provides execution details and cluster metrics for previous job runs. By

comparing recent runs with the 5-minute baseline, a data engineer can determine whether the slowdown is consistent or intermittent and identify resource-related issues such as CPU, memory, or executor bottlenecks that may be causing the increase in execution time.

A. Archive old job runs and reset the baseline counter to ensure accurate comparison of current performance. — Incorrect

Archiving old runs does not help identify performance trends or determine the cause of the slowdown. Historical run data is valuable for comparison.

B. Rerun the job immediately to establish a new baseline since historical data becomes irrelevant after seven days. — Incorrect

Historical job runs remain useful for performance analysis. Simply rerunning the job does not explain why execution time increased.

C. Increase cluster size automatically whenever execution time exceeds the previous run's duration. — Incorrect

Increasing cluster size without first identifying the root cause may increase costs unnecessarily and does not determine whether the slowdown is consistent or intermittent.

D. Compare cluster metrics in Lakeflow Jobs run history to spot resource spikes causing delays. — Correct

Comparing cluster metrics across historical job runs helps identify execution time trends and resource bottlenecks, making it the recommended approach for diagnosing the slowdown.

Question: 374 CertyIQ

A notebook task check_volume calculates a daily data volume and writes it to a Lakeflow Jobs taskValue named record_count. If the count is greater than 1 million, the job should run high_volume_pipeline; otherwise, it should run normal_pipeline.
Which control-flow feature should be used?

A. Configure Run if conditions based purely on task status.

B. Add an If/else condition task that reads the record_count task value and routes to different downstream tasks.

C. Configure additional retries for check_volume until the count exceeds the threshold.

D. Use a For each task over the record_count and stop when it is below the threshold.

Answer: B

Explanation:

Correct Answer: B. Add an If/else condition task that reads the record_count task value and routes to different downstream tasks.

Why B is correct:

Lakeflow Jobs support **If/else condition** tasks that evaluate a value produced by a previous task, such as a **taskValue**. In this scenario, the **check_volume** notebook stores the daily record count in **record_count**. The If/else task reads this value and routes execution to **high_volume_pipeline** if the count is greater than 1 million; otherwise, it runs **normal_pipeline**.

This is the recommended way to implement conditional branching in a DAG.

A. Configure Run if conditions based purely on task status. — Incorrect

Run if conditions only evaluate task execution status (such as success or failure). They cannot make decisions based on a task value like **record_count**.

B. Add an If/else condition task that reads the record_count task value and routes to different downstream tasks. — Correct

An **If/else condition** task evaluates the **record_count** task value and directs the workflow to the appropriate downstream pipeline based on the threshold.

C. Configure additional retries for check_volume until the count exceeds the threshold. — Incorrect

Retries are intended to handle temporary task failures. They do not change the calculated record count or provide conditional workflow logic.

D. Use a For each task over the record_count and stop when it is below the threshold. — Incorrect

A **For each** task is used to iterate over a collection of values. Since **record_count** is a single numeric value, iteration is unnecessary and does not implement the required conditional branching.

Question: 375 CertyIQ

A data engineer has configured a Lakeflow Job that runs daily to ingest customer transaction data from a legacy relational database. The extracted data must be written directly to a Unity Catalog table and be immediately queryable via SQL. The team needs to ensure full data lineage is preserved.
Which action enables this data ingestion with direct landing in Unity Catalog, preserved lineage, and immediate SQL query ability?

A. Extract to Pandas DataFrame, convert to Spark, then save () to external path
B. Stage data in cloud storage via REST API, then load with a separate job
C. Use spark.read.format("jdbc") and write with saveAsTable("catalog.schema.table")
D. Use ODBC pooling to extract all records in one batch and write to /dbfs/

Answer: C

Explanation:

Correct Answer: C. Use spark.read.format("jdbc") and write with saveAsTable("catalog.schema.table")

Why C is correct:

Reading data directly from the legacy relational database using the **JDBC** connector and writing it to a **Unity Catalog managed table** allows the data to be immediately available for SQL queries. Since the data is written directly into Unity Catalog, governance and data lineage are preserved automatically.

A. Extract to Pandas DataFrame, convert to Spark, then save() to external path — Incorrect

Using a Pandas DataFrame is inefficient for large datasets, and writing to an external path does not directly create a Unity Catalog managed table or preserve Unity Catalog lineage.

B. Stage data in cloud storage via REST API, then load with a separate job — Incorrect

Although this approach works, it introduces unnecessary staging and an additional loading step. The requirement is to write directly to Unity Catalog with immediate SQL access.

C. Use spark.read.format("jdbc") and write with saveAsTable("catalog.schema.table") — Correct

This approach reads data directly from the relational database and writes it to a Unity Catalog table. The table is immediately queryable through SQL, and Unity Catalog preserves governance and lineage information.

D. Use ODBC pooling to extract all records in one batch and write to /dbfs/ — Incorrect

Writing to **/dbfs/** stores files rather than creating a Unity Catalog managed table. The data is not immediately available as a SQL table, and Unity Catalog lineage is not automatically maintained.

Question: 376

CertyIQ

A data engineer needs Task C to run ONLY if Task A succeeds AND Task B fails.
Which dependency configuration implements this conditional logic?

A. Task C depends on Task A (Success) with Task B listed as a negative dependency.
B. Task C uses two dependency chains: Task A with All succeeded, Task B with At least one failed.
C. Create a Python task that checks dbutils.jobs.taskValues from Tasks A and B before running Task C.
D. Task C depends on Task A (All done) and Task B (All done) with a condition widget.

Answer: B

Explanation:

Correct Answer: B. Task C uses two dependency chains: Task A with All succeeded, Task B with At least one failed.

Why B is correct:

Lakeflow Jobs allows each task dependency to have its own **Run if** condition. To run **Task C** only when **Task A succeeds** and **Task B fails**, configure:

Task A dependency: **All succeeded**
Task B dependency: **At least one failed**

Task C will execute only when both conditions are satisfied.

A. Task C depends on Task A (Success) with Task B listed as a negative dependency. — Incorrect

Lakeflow Jobs does not support a **negative dependency** option. Dependencies are configured using supported **Run if** conditions.

B. Task C uses two dependency chains: Task A with All succeeded, Task B with At least one failed. — Correct

This configuration directly implements the required logic: **Task A must succeed AND Task B must fail** before Task C runs.

C. Create a Python task that checks dbutils.jobs.taskValues from Tasks A and B before running Task C. — Incorrect

Although a custom Python task could evaluate task values, it is unnecessary. Lakeflow Jobs already provides built-in dependency conditions to implement this logic.

D. Task C depends on Task A (All done) and Task B (All done) with a condition widget. — Incorrect

All done only ensures the upstream tasks have finished, regardless of whether they succeeded or failed. It does not enforce the required success/failure combination.

Question: 377 CertyIQ

A governance team is evaluating whether to use Unity Catalog attribute-based access control (ABAC) policies or manually applied row filters and column masks to protect sensitive data across their catalog.
Why should the team use ABAC policies instead of manually applied row filters and column masks?

A. ABAC policies support applying multiple distinct row filters to the same table for a single user at query time, whereas manually applied row filters are limited to one filter per table.

B. ABAC policies use governed tags to dynamically match tables and columns, so a single policy defined on a catalog automatically applies to all current and future tables within that catalog without per-table configuration.

C. ABAC policies allow row filters and column masks to be written in Python and Scala directly, whereas manually applied filters and masks can only use SQL user-defined functions.

D. ABAC policies can be applied to views and materialized views in addition to tables, whereas manually applied row filters and column masks can only be applied to tables.

Answer: B

Explanation:

Correct Answer: B. ABAC policies use governed tags to dynamically match tables and columns, so a single policy defined on a catalog automatically applies to all current and future tables within that catalog without per-table configuration.

Why B is correct:

Attribute-Based Access Control (ABAC) uses **governed tags** to apply security policies dynamically across Unity Catalog objects. A single ABAC policy can automatically protect all current and future tables or columns that match the specified tags, eliminating the need to manually configure row filters and column masks for each table.

This provides centralized, scalable, and easier-to-manage governance.

A. ABAC policies support applying multiple distinct row filters to the same table for a single user at query time, whereas manually applied row filters are limited to one filter per table. — Incorrect

ABAC is designed for centralized policy management through governed tags, not for supporting multiple row filters on the same table for a single user.

B. ABAC policies use governed tags to dynamically match tables and columns, so a single policy defined on a catalog automatically applies to all current and future tables within that catalog without per-table configuration. — Correct

This is the primary advantage of ABAC. Policies automatically apply to tagged objects, reducing administrative effort and ensuring consistent governance across existing and future data assets.

C. ABAC policies allow row filters and column masks to be written in Python and Scala directly, whereas manually applied filters and masks can only use SQL user-defined functions. — Incorrect

ABAC does not introduce Python or Scala support for defining row filters or column masks. Policies are based on Unity Catalog governance features, not programming language differences.

D. ABAC policies can be applied to views and materialized views in addition to tables, whereas manually

applied row filters and column masks can only be applied to tables. — Incorrect

The key advantage of ABAC is centralized, tag-based policy management. This statement does not accurately describe the primary difference between ABAC and manually applied row filters and column masks.

Question: 378 CertyIQ

A data engineering team is designing the Gold layer in their Unity Catalog-governed lakehouse for downstream BI and analytics users.
The team wants to expose business-ready metrics with fast query performance and consistent definitions, while keeping the transformation logic in Spark notebooks.
Which type of Gold layer object meets this requirement?

A. A collection of CSV files stored in a cloud storage object. BI users will parse the files to query the gold layer data.

B. A view that directly references raw Bronze tables so BI tools can define their own joins and aggregations

C. A streaming table that continuously ingests raw events from source systems and exposes them directly to BI users

D. A materialized view that precomputes aggregations from Silver tables on a schedule and is queried by BI tools for faster, consistent analytics

Answer: D

Explanation:

Correct Answer: D. A materialized view that precomputes aggregations from Silver tables on a schedule and is queried by BI tools for faster, consistent analytics

Why D is correct:

The **Gold layer** is designed to provide **business-ready, curated data** for reporting and analytics. A **materialized view** precomputes aggregations from the Silver layer and stores the results, providing faster query performance and consistent business metrics for BI users.

The transformation logic can remain in Spark notebooks, while the materialized view exposes optimized data for downstream analytics.

A. A collection of CSV files stored in a cloud storage object. BI users will parse the files to query the gold layer data. — Incorrect

CSV files are not optimized for analytics, lack governance features, and require BI tools to perform additional parsing and processing.

B. A view that directly references raw Bronze tables so BI tools can define their own joins and aggregations. — Incorrect

Bronze tables contain raw data. Allowing BI tools to define their own transformations leads to inconsistent metrics and poor query performance.

C. A streaming table that continuously ingests raw events from source systems and exposes them directly to BI users. — Incorrect

Streaming tables are intended for ingestion and near real-time processing, not for exposing curated business-ready metrics to BI users.

D. A materialized view that precomputes aggregations from Silver tables on a schedule and is queried by BI

tools for faster, consistent analytics. — Correct

A materialized view stores precomputed results from Silver tables, delivering fast query performance, consistent business definitions, and an optimized Gold layer for BI and analytics users.

Question: 379 CertyIQ

A data engineer needs to enforce row-level security on main.secure.events (region STRING, event_id STRING). Members of the account group all_regions need to see all rows. All other users need to see only rows where region = 'EU'.
Which SQL sequence satisfies the requirement?

A. CREATE FUNCTION main.secure.region_filter(region STRING)RETURN IF(is_account_group_member('all_regions'), true, region - 'EU');ALTER VIEW main.secure.eventsSET ROW FILTER main.secure.region_filter ON (region);

B. CREATE FUNCTION main.secure.region_filter(region STRING)RETURN IF(is_account_group_member('all_regions'), true, region = 'EU');ALTER TABLE main.secure.eventsSET ROW FILTER main.secure.region_filter ON (region);

C. CREATE FUNCTION main.secure.region_filter (region STRING)RETURN IF (is_account_group_member('all_regions'), true, region = 'EU');ALTER TABLE main.secure.eventsALTER COLUMN regionSET MASK main.secure.region_filter:

D. CREATE FUNCTION main.secure.region_filter (region STRING)RETURN IF (is_account_group_member('all_regions'), true, region = 'EU');ALTER TABLE main.secure.eventsSET ROW FILTER main.secure.region_filter USING COLUMNS (region):

Answer: B

Explanation:

Correct Answer: B.

Why B is correct:

To enforce **row-level security** in Unity Catalog, you create a SQL function that returns **TRUE** for rows the user is allowed to access, and then apply it as a **ROW FILTER** to the table.

In this scenario:

Members of the **all_regions** account group can view **all rows**.
All other users can view only rows where **region = 'EU'**.

The function checks the user's group membership and returns the appropriate result. It is then applied to the **events** table as a row filter.

A.

CREATE FUNCTION main.secure.region_filter(region STRING)

RETURN IF(is_account_group_member('all_regions'), true, region = 'EU');

ALTER VIEW main.secure.events

SET ROW FILTER main.secure.region_filter ON (region); — Incorrect

A row filter must be applied to a **table**, not a view.

B.

CREATE FUNCTION main.secure.region_filter(region STRING)

RETURN IF(is_account_group_member('all_regions'), true, region = 'EU');

ALTER TABLE main.secure.events

SET ROW FILTER main.secure.region_filter ON (region); — Correct

This creates a row filter function and applies it to the table. Members of **all_regions** can see all rows, while all other users can only see rows where **region = 'EU'**.

C.

CREATE FUNCTION main.secure.region_filter(region STRING)

RETURN IF(is_account_group_member('all_regions'), true, region = 'EU');

ALTER TABLE main.secure.events

ALTER COLUMN region

SET MASK main.secure.region_filter; — Incorrect

A column mask hides or transforms column values. It does not restrict which rows are returned.

D.

CREATE FUNCTION main.secure.region_filter(region STRING)

RETURN IF(is_account_group_member('all_regions'), true, region = 'EU');

ALTER TABLE main.secure.events

SET ROW FILTER main.secure.region_filter USING COLUMNS (region); — Incorrect (for this exam version)

Although **USING COLUMNS** is the newer syntax in recent Databricks documentation, the certification exam expects the older **ON (region)** syntax, making **B** the correct answer.

Question: 380 CertyIQ

A data engineer has manually created several Databricks jobs and dashboards using the workspace Ul. The team now wants to manage these resources as code using Declarative Automation Bundles (formerly Databricks Asset Bundles), store the configuration in a Git repository, and deploy changes through CI/CD.
Which approach converts the existing resources into a bundle project?

A. Clone an empty Git repository into a Git Folder, recreate each job and dashboard configuration in YAML by referencing the settings using workspace Ul, then commit and deploy with databricks bundle deploy.

B. Use the Databricks REST API to export each job's JSON definition, save the JSON files into a Git repository, and configure CI/CD to import them using the REST API on each deployment.

C. Run databricks bundle generate for each existing resource, use --bind to link the generated configuration to the existing workspace resources, and commit the bundle project to Git for CI/CD deployment.

D. Run databricks bundle init to create a new bundle project, then use databricks bundle deploy pointed at the workspace to automatically detect and import all existing resources into the bundle configuration.

Answer: C

Explanation:

Correct Answer: C. Run databricks bundle generate for each existing resource, use --bind to link the generated configuration to the existing workspace resources, and commit the bundle project to Git for CI/CD deployment.

Why C is correct:

Declarative Automation Bundles provide the **bundle generate** command to generate YAML configuration for existing Databricks resources, such as jobs and dashboards. The **--bind** option links the generated configuration to the existing workspace resources, allowing them to be managed as code without recreating them.

The generated bundle project can then be stored in Git and deployed through CI/CD.

A. Clone an empty Git repository into a Git Folder, recreate each job and dashboard configuration in YAML by referencing the settings using workspace UI, then commit and deploy with databricks bundle deploy. — Incorrect

This approach requires manually recreating all resource configurations, which is unnecessary because Databricks provides the **bundle generate** command to automate this process.

B. Use the Databricks REST API to export each job's JSON definition, save the JSON files into a Git repository, and configure CI/CD to import them using the REST API on each deployment. — Incorrect

Although the REST API can export job definitions, Declarative Automation Bundles use YAML-based bundle configurations rather than JSON exported through the REST API.

C. Run databricks bundle generate for each existing resource, use --bind to link the generated configuration to the existing workspace resources, and commit the bundle project to Git for CI/CD deployment. — Correct

This is the recommended migration approach. It generates bundle configuration from existing resources, binds them to the workspace objects, and enables Infrastructure as Code with Git and CI/CD.

D. Run databricks bundle init to create a new bundle project, then use databricks bundle deploy pointed at the workspace to automatically detect and import all existing resources into the bundle configuration. — Incorrect

bundle init creates a new bundle project template but does not automatically discover or import existing workspace resources.

Question: 381 CertyIQ

A data engineer is running a Spark SQL job on Databricks and notices that joins between a small lookup table and a large fact table are slow. The data engineer wants Spark to automatically send the small lookup table to all executors to speed up the join.
Which configuration parameter should the data engineer adjust?

A. spark.executor.memory
B. spark.sql.autoBroadcastJoinThreshold
C. spark.sql.shuffle.partitions
D. spark.default.parallelism

Answer: B

Explanation:

Correct Answer: B. spark.sql.autoBroadcastJoinThreshold

Why B is correct:

The **spark.sql.autoBroadcastJoinThreshold** configuration determines the maximum size of a table that Spark will automatically broadcast during a join. When the lookup table is smaller than this threshold, Spark sends it to all executors, avoiding expensive data shuffling and significantly improving join performance.

A. spark.executor.memory — Incorrect

This setting controls the amount of memory available to each executor. While it affects overall job performance, it does not control whether Spark uses a broadcast join.

B. spark.sql.autoBroadcastJoinThreshold — Correct

This setting specifies the maximum table size that Spark will automatically broadcast during joins. Increasing this threshold allows Spark to broadcast larger lookup tables, improving join performance.

C. spark.sql.shuffle.partitions — Incorrect

This setting controls the number of partitions used during shuffle operations. It does not determine whether Spark performs a broadcast join.

D. spark.default.parallelism — Incorrect

This setting determines the default number of partitions for RDD operations. It does not affect Spark's decision to broadcast a table during joins.

Question: 382 CertyIQ

A data engineer is troubleshooting two different pipeline failures.
• Pipeline A fails with a java.lang.OutOfMemoryError is immediately thrown after the command display(df.collect()) is called on a 100GB dataset.
• Pipeline B fails during a wide transformation (a join of two large tables) with an ExecutorLostFailure error message, indicating executor memory exhaustion during shuffle.
Which action should the data engineer take to fix these two issues?

A. For both pipelines, enable Adaptive Query Execution (AQE) as it automatically manages memory allocation for both the driver and executors.

B. For Pipeline A, increase the spark.driver.memozy; for Pipeline B, increase the spark.sql.shuffle.partitions or executor memory.

C. For Pipeline A, increase the number of executors; for Pipeline B, increase the driver memory.

D. For Pipeline A, switch to a Storage Optimized cluster; for Pipeline B, use a Compute Optimized cluster.

Answer: B

Explanation:

Correct Answer: B. For Pipeline A, increase the spark.driver.memory; for Pipeline B, increase the spark.sql.shuffle.partitions or executor memory.

Why B is correct:

The two failures occur in different parts of the Spark architecture and require different solutions.

Pipeline A:

The failure occurs immediately after **display(df.collect())** on a 100GB dataset. The **collect()** operation transfers the entire dataset to the **driver**, causing a **java.lang.OutOfMemoryError** when the driver's memory is insufficient. Increasing **spark.driver.memory** (or avoiding **collect()** on large datasets) resolves this issue.

Pipeline B:

The failure occurs during a large **join**, producing an **ExecutorLostFailure** due to executor memory exhaustion during the shuffle. Increasing **executor memory** or increasing **spark.sql.shuffle.partitions** reduces the amount of data processed by each executor during the shuffle.

A. For both pipelines, enable Adaptive Query Execution (AQE) as it automatically manages memory allocation for both the driver and executors. — Incorrect

AQE can optimize query execution plans, but it does not solve driver memory exhaustion caused by **collect()**, nor does it automatically eliminate executor memory issues.

B. For Pipeline A, increase the spark.driver.memory; for Pipeline B, increase the spark.sql.shuffle.partitions or executor memory. — Correct

This directly addresses the root cause of each failure: driver memory for **collect()**, and executor memory or shuffle partitioning for large joins.

C. For Pipeline A, increase the number of executors; for Pipeline B, increase the driver memory. — Incorrect

Increasing executors does not help when the driver is running out of memory. Likewise, increasing driver memory does not resolve executor memory exhaustion during shuffles.

D. For Pipeline A, switch to a Storage Optimized cluster; for Pipeline B, use a Compute Optimized cluster. — Incorrect

Changing cluster types may affect overall performance but does not directly resolve the specific memory-related issues causing these failures.

Question: 383

CertyIQ

A data engineer has multiple Unity Catalog-managed Delta tables that require regular maintenance. Currently, the data engineer manually schedules OPTIMIZE and VACUUM jobs for each table, adjusting the frequency based on how often each table is queried and updated. The engineer needs to eliminate this manual maintenance overhead and have the platform automatically determine when and how to run these operations.
Which action should the engineer take to meet the requirements?

A. Set the delta.autoOptimize.autoCompact and delta.autoOptimize.optimizeWrite table properties to trigger automatic VACUUM and file compaction.

B. Configure Auto Loader with schema evolution enabled to automatically trigger OPTIMIZE and VACUUM after each data ingestion batch completes.

C. Enable predictive optimization at the account, catalog, or schema level so the platform automatically identifies and runs OPTIMIZE and VACUUM on managed tables.

D. Enable Liquid Clustering with CLUSTER BY on each table to automatically compact and remove stale data files on a scheduled basis.

Answer: C

Explanation:

Correct Answer: C. Enable predictive optimization at the account, catalog, or schema level so the platform automatically identifies and runs OPTIMIZE and VACUUM on managed tables.

Why C is correct:

Predictive Optimization automatically monitors **Unity Catalog-managed Delta tables** and determines when maintenance operations such as **OPTIMIZE** and **VACUUM** should be run. This removes the need to manually schedule maintenance jobs and adjusts optimization based on table usage patterns.

This is the recommended approach for reducing operational overhead.

A. Set the delta.autoOptimize.autoCompact and delta.autoOptimize.optimizeWrite table properties to trigger automatic VACUUM and file compaction. — Incorrect

These table properties optimize file sizes during writes by enabling optimized writes and auto compaction. They **do not** automatically run **VACUUM** or schedule **OPTIMIZE** operations.

B. Configure Auto Loader with schema evolution enabled to automatically trigger OPTIMIZE and VACUUM after each data ingestion batch completes. — Incorrect

Auto Loader automates data ingestion and schema evolution. It does **not** automatically execute **OPTIMIZE** or **VACUUM** after ingestion.

C. Enable predictive optimization at the account, catalog, or schema level so the platform automatically identifies and runs OPTIMIZE and VACUUM on managed tables. — Correct

Predictive Optimization automatically determines when maintenance is needed and performs **OPTIMIZE** and **VACUUM** on Unity Catalog-managed tables, eliminating manual scheduling.

D. Enable Liquid Clustering with CLUSTER BY on each table to automatically compact and remove stale data files on a scheduled basis. — Incorrect

Liquid Clustering improves data layout and query performance, but it does **not** automatically schedule or perform **VACUUM** operations or replace Predictive Optimization for table maintenance.

Question: 384 CertyIQ

A data engineering team runs multiple daily batch jobs. The team needs to run the jobs on a fixed overnight schedule with consistent performance and controlled resource usage. The team wants to minimize operational overhead and reduce idle cluster costs.
Which compute option should the team use?

A. Classic jobs compute
B. Classic all-purpose compute
C. SQL warehouses
D. Serverless jobs compute

Answer: D

Explanation:

Correct Answer: D. Serverless jobs compute

Why D is correct:

Serverless jobs compute is the recommended option for scheduled batch workloads. Databricks automatically provisions and terminates compute resources, providing consistent performance while eliminating cluster management. Since compute starts only when the job runs, idle cluster costs are

minimized and operational overhead is significantly reduced.

A. Classic jobs compute — Incorrect

Classic jobs compute is suitable for scheduled jobs, but the team is responsible for configuring and managing compute resources. It involves more operational overhead than Serverless jobs compute.

B. Classic all-purpose compute — Incorrect

All-purpose compute is designed for interactive development, notebooks, and ad hoc analysis. It is generally more expensive for scheduled production batch jobs because clusters may remain idle.

C. SQL warehouses — Incorrect

SQL warehouses are optimized for executing SQL queries and powering BI dashboards. They are not the recommended compute option for running general-purpose batch ETL jobs.

D. Serverless jobs compute — Correct

Serverless jobs compute automatically manages infrastructure, scales resources as needed, minimizes idle costs by running only during job execution, and is the recommended choice for scheduled batch workloads with low operational overhead.

Question: 385 CertyIQ

Which Databricks SQL predicate correctly performs a null-safe equality comparison so that rows are matched when both sides are NULL or when both are equal non-NULL values?

A. WHERE customer_id <> :cid OR customer_id IS NULL
B. WHERE customer_id <==> :cid
C. WHERE customer_id = :cid
D. WHERE COALESCE(customer_id, '') = COALESCE(:cid, '')

Answer: B

Explanation:

Correct Answer: B. WHERE customer_id <==> :cid

Why B is correct:

The **<==>** operator in Databricks SQL performs a **null-safe equality comparison**. It returns **TRUE** when both values are equal or when **both values are NULL**.

This makes it the correct operator when NULL values should be treated as equal.

A. WHERE customer_id <> :cid OR customer_id IS NULL — Incorrect

This condition returns rows where the values are different or where **customer_id** is NULL. It does not perform a null-safe equality comparison.

B. WHERE customer_id <==> :cid — Correct

The **<==>** operator is the null-safe equality operator in Databricks SQL. It matches rows when both values are equal or when both values are NULL.

C. WHERE customer_id = :cid — Incorrect

The = operator does not consider **NULL = NULL** to be true. If either value is NULL, the comparison returns **NULL** (unknown), not **TRUE**.

D. WHERE COALESCE(customer_id, '') = COALESCE(:cid, '') — Incorrect

Although this can make NULL values appear equal by replacing them with an empty string, it is not a true null-safe comparison. It can also produce incorrect results if an empty string is a valid value. The **<=>** operator is the recommended and correct approach.

Question: 386 CertyIQ

A data engineer converts an external table to a managed table. A structured streaming job that reads from this table continues running during the conversion. After the conversion completes, the streaming job stops processing new records.
How should the data engineer resolve this?

A. Delete the streaming checkpoint directory and let the job reprocess from the beginning
B. Restart the streaming job, so it picks up the new managed table location
C. Run REFRESH TABLE on the converted table to update the streaming checkpoint
D. Grant the streaming job additional permissions on the new managed storage location

Answer: B

Explanation:

Correct Answer: B. Restart the streaming job, so it picks up the new managed table location.

Why B is correct:

When an **external table** is converted to a **managed table**, the table's underlying storage location changes. A Structured Streaming job that was already running continues to reference the old location, so it stops processing new records after the conversion.

Restarting the streaming job allows it to recognize the new managed table location and continue processing data.

A. Delete the streaming checkpoint directory and let the job reprocess from the beginning. — Incorrect

Deleting the checkpoint forces the stream to start over and may result in duplicate processing. It is unnecessary when a simple restart is sufficient.

B. Restart the streaming job, so it picks up the new managed table location. — Correct

Restarting the streaming job refreshes the table metadata and enables the stream to read from the new managed table location.

C. Run REFRESH TABLE on the converted table to update the streaming checkpoint. — Incorrect

REFRESH TABLE refreshes cached metadata for SQL queries but does not update an already running Structured Streaming job or its checkpoint.

D. Grant the streaming job additional permissions on the new managed storage location. — Incorrect

The issue is not caused by permissions. It occurs because the running stream continues to reference the old

table location after the conversion. Restarting the stream resolves the problem.

Question: 387 CertyIQ

A data engineer is designing a cost-optimized, event-driven pipeline. They configure a Lakeflow Job with a File Arrival trigger to watch an S3 bucket. The job runs a notebook that uses Auto Loader with trigger (availableNow=True) to ingest data into a Bronze table.
What is the technical relationship between the File Arrival trigger and Auto Loader in this integration pattern?

A. Auto Loader must be configured in File Notification mode to work with a Lakeflow Job File Arrival trigger.

B. The File Arrival trigger automatically passes the specific file path of the new arrival to Auto Loader, allowing the engineer to skip the source path configuration in the code.

C. The File Arrival trigger starts the job run, while Auto Loader uses its internal checkpoint to independently identify and process only the new files that have arrived since the last successful commit.

D. Using a File Arrival trigger requites the engineer to disable Auto Loader's checkpointing to prevent the job from processing the same file multiple times.

Answer: C

Explanation:

Correct Answer: C. The File Arrival trigger starts the job run, while Auto Loader uses its internal checkpoint to independently identify and process only the new files that have arrived since the last successful commit.

Why C is correct:

The **File Arrival trigger** is responsible only for **starting the Lakeflow Job** when new files are detected in the monitored S3 bucket.

Once the job starts, **Auto Loader** independently tracks which files have already been processed by using its **checkpoint**. It reads only the new files that have arrived since the last successful run, ensuring exactly-once processing without reprocessing previously ingested files.

A. Auto Loader must be configured in File Notification mode to work with a Lakeflow Job File Arrival trigger. — Incorrect

A File Arrival trigger does not require Auto Loader to use **File Notification** mode. Auto Loader can work independently of the trigger mechanism.

B. The File Arrival trigger automatically passes the specific file path of the new arrival to Auto Loader, allowing the engineer to skip the source path configuration in the code. — Incorrect

The File Arrival trigger does not pass individual file paths to Auto Loader. Auto Loader still requires the source path and determines which new files to process using its checkpoint.

C. The File Arrival trigger starts the job run, while Auto Loader uses its internal checkpoint to independently identify and process only the new files that have arrived since the last successful commit. — Correct

The trigger and Auto Loader have separate responsibilities. The trigger launches the job, while Auto Loader manages incremental file discovery and processing through its checkpoint.

D. Using a File Arrival trigger requires the engineer to disable Auto Loader's checkpointing to prevent the job from processing the same file multiple times. — Incorrect

Auto Loader's checkpointing should remain enabled. It is the mechanism that prevents duplicate processing and ensures only new files are ingested.

Question: 388 CertyIQ

A data engineer has finished developing a new Declarative Automation Bundle (formerly Databricks Asset Bundle) for a streaming pipeline. They need to ensure the configuration is syntactically correct, push the code to the workspace, and finally trigger the pipeline to verify it runs as expected.
Which sequence of Databricks CLI commands should the engineer use to perform these steps in the correct order?

A. databricks bundle init -> databricks bundle deploy -> databricks bundle run
B. databricks bundle validate -> databricks bundle deploy -> databricks bundle run
C. databricks bundle validate -> databricks bundle sync -> databricks bundle execute
D. databricks bundle check -> databricks bundle upload -> databricks bundle start

Answer: B

Explanation:

Correct Answer: B. databricks bundle validate → databricks bundle deploy → databricks bundle run

Why B is correct:

The correct workflow for a **Declarative Automation Bundle** is:

1. **databricks bundle validate** – Validates the bundle configuration and checks for syntax or configuration errors.
2. **databricks bundle deploy** – Deploys the bundle resources to the target Databricks workspace.
3. **databricks bundle run** – Executes the deployed resource (such as a job or pipeline) to verify it runs successfully.

This is the recommended sequence for validating, deploying, and testing a bundle.

A. databricks bundle init → databricks bundle deploy → databricks bundle run — Incorrect

The **bundle init** command creates a new bundle project. Since the bundle has already been developed, initialization is unnecessary.

B. databricks bundle validate → databricks bundle deploy → databricks bundle run — Correct

This sequence validates the configuration, deploys the resources to the workspace, and then runs the pipeline or job to verify successful execution.

C. databricks bundle validate → databricks bundle sync → databricks bundle execute — Incorrect

The commands **bundle sync** and **bundle execute** are not valid Databricks CLI commands for Declarative Automation Bundles.

D. databricks bundle check → databricks bundle upload → databricks bundle start — Incorrect

The commands **bundle check**, **bundle upload**, and **bundle start** are not valid Databricks CLI commands.

Question: 389 CertyIQ

A data engineer has a bundle with a job resource keyed as etl_job in databricks.yml. After running 'databricks bundle deploy', the job appears in the workspace but has not executed.
Which command triggers execution of the deployed job?

A. databricks jobs submit --json
B. databricks bundle execute etl_job
C. databricks bundle run etl_job
D. databricks bundle deploy --run

Answer: C

Explanation:

Correct Answer: C. databricks bundle run etl_job

Why C is correct:

After **databricks bundle deploy**, the job is **deployed** to the target workspace but is **not executed** automatically.

To run the deployed job, use:

databricks bundle run etl_job

Here, **etl_job** is the resource key defined in the **databricks.yml** file.

A. databricks jobs submit --json — Incorrect

This command submits a one-time job using a JSON payload. It is not used to execute a job that has already been deployed through a Declarative Automation Bundle.

B. databricks bundle execute etl_job — Incorrect

bundle execute is not a valid Databricks CLI command.

C. databricks bundle run etl_job — Correct

This command executes the deployed job resource identified by the **etl_job** key in the bundle configuration.

D. databricks bundle deploy --run — Incorrect

The **bundle deploy** command only deploys bundle resources. It does not support a **--run** option to automatically execute the job.

Question: 390 CertyIQ

A data engineer is preparing a Declarative Automation Bundle (formerly Databricks Asset Bundle) to deploy a Lakeflow pipeline.
To ensure the pipeline is deployed to the correct environment, where should the workspace-specific configurations, such as the host URL and root storage path, be defined within the bundle project?

A. In the targets section of the databricks.yml file
B. In a global environment variable on the local development machine
C. Inside the README.md file of the bundle root directory
D. Within the Python source code of the Lakeflow pipeline

Answer: A

Explanation:

Correct Answer: A. In the targets section of the databricks.yml file

Why A is correct:

In a **Declarative Automation Bundle**, environment-specific settings such as the **workspace host URL**, **workspace**, **root storage path**, and other deployment configurations are defined in the **targets** section of the **databricks.yml** file.

Each target (for example, **dev**, **test**, or **prod**) can have its own configuration, allowing the same bundle to be deployed to multiple environments without changing the application code.

A. In the targets section of the databricks.yml file — Correct

The **targets** section stores environment-specific deployment settings, including the workspace host, root path, variables, and authentication configuration.

B. In a global environment variable on the local development machine — Incorrect

Global environment variables may be used for authentication, but they are not the recommended place to define workspace-specific deployment configuration for Declarative Automation Bundles.

C. Inside the README.md file of the bundle root directory — Incorrect

The **README.md** file is used for documentation only. It does not contain deployment configuration.

D. Within the Python source code of the Lakeflow pipeline — Incorrect

Workspace-specific settings should not be hardcoded in the pipeline code. They belong in the bundle configuration so the same codebase can be deployed across multiple environments.

Question: 391 CertyIQ

A data engineer maintains a Declarative Automation Bundle (formerly Databricks Asset Bundle), which deploys a dashboard.
The databricks.yml includes:

```
bundle:
 name: workspace_assets
 variables:
  dataset_catalog:
   default: catalog_dev
targets:
 dev:
  variables:
   dataset_catalog: catalog_dev
 prod:
  variables:
   dataset_catalog: catalog_prod
```

Which set of actions will enable the dashboard to be deployed in prod using catalog_prod without changing the resource definition?

A. Run databricks bundle execute --profile prod so the CLI selects targets.prod and applies dataset_catalog: catalog_prod automatically.

B. Run databricks bundle deploy --var dataset_catalog=catalog_prod so the CLI selects targets.prod and applies dataset_catalog: catalog_prod automatically.

C. Run databricks bundle deploy --target prod so the deployment uses targets.prod and overrides

dataset_catalog to catalog_prod.

D. Run databricks bundle deploy --profile prod so the CLI selects targets.prod and applies dataset_catalog: catalog_prod automatically.

Answer: C

Explanation:

Correct Answer: C. Run databricks bundle deploy --target prod so the deployment uses targets.prod and overrides dataset_catalog to catalog_prod.

Why C is correct:

The **targets** section in **databricks.yml** defines environment-specific configuration. By deploying with:

databricks bundle deploy --target prod

the CLI selects the **prod** target and automatically overrides the **dataset_catalog** variable from **catalog_dev** to **catalog_prod**, without requiring any changes to the resource definition.

This is the recommended way to deploy the same bundle across multiple environments.

A. Run databricks bundle execute --profile prod so the CLI selects targets.prod and applies dataset_catalog: catalog_prod automatically. — Incorrect

bundle execute is not a valid Databricks CLI command.

B. Run databricks bundle deploy --var dataset_catalog=catalog_prod so the CLI selects targets.prod and applies dataset_catalog: catalog_prod automatically. — Incorrect

While **--var** can override variables, it does **not** automatically select the **prod** target. The environment-specific configuration in **targets.prod** is applied by using **--target prod**.

C. Run databricks bundle deploy --target prod so the deployment uses targets.prod and overrides dataset_catalog to catalog_prod. — Correct

This command selects the **prod** target and automatically applies all configuration defined under **targets.prod**, including **dataset_catalog: catalog_prod**.

D. Run databricks bundle deploy --profile prod so the CLI selects targets.prod and applies dataset_catalog: catalog_prod automatically. — Incorrect

A **profile** specifies authentication credentials and workspace connection settings. It does not select the bundle target. Environment selection is done with **--target**.

Question: 392 CertyIQ

A data engineer is tasked with cleaning a bronze table. The requirement is to eliminate rows where either the customer email or the customer_phone field is null. This data cleaning must be performed in a single operation using a single method call.
Which PySpark approach supports filtering multiple columns for nulls in one call?

A. df.where('customer_email IS NOT NULL').where('customer_phone IS NOT NULL')

B. df.dropna (subset=['customer_email', 'customer_phone'])

C. df.na.drop(how='all')

D. df.filter(col('customer_email').isNotNull() & col('customer_phone').isNotNull())

Answer: B

Explanation:

Correct Answer: B. df.dropna(subset=['customer_email', 'customer_phone'])

Why B is correct:

The requirement is to remove rows where **either** customer_email or customer_phone is **NULL** using **a single method call**.

The **dropna()** method with the **subset** parameter checks the specified columns and removes any row that contains a NULL value in either column. This satisfies the requirement in one operation.

A. df.where('customer_email IS NOT NULL').where('customer_phone IS NOT NULL') — Incorrect

This filters the data correctly, but it requires **two method calls** (where() followed by another where()), which does not meet the requirement of using a single method call.

B. df.dropna(subset=['customer_email', 'customer_phone']) — Correct

This uses a **single method call** to remove rows where either **customer_email** or **customer_phone** is NULL.

C. df.na.drop(how='all') — Incorrect

The **how='all'** option removes rows only when **all columns** are NULL. It does not specifically check the **customer_email** and **customer_phone** columns.

D. df.filter(col('customer_email').isNotNull() & col('customer_phone').isNotNull()) — Incorrect

Although this correctly filters the rows, it uses a filter expression rather than the single **dropna()** method specifically designed for removing NULL values from multiple columns in one call.

Question: 393 CertyIQ

An organization is building a data lakehouse and needs to ingest data from multiple sources into Unity Catalog-managed tables:

- Source 1: Salesforce (50+ objects, frequent schema changes, OAuth authentication required)
- Source 2: On-premises SQL server database (100+ tables, CDC enabled, requires network connectivity)
- Source 3: Daily JSON files landing in Azure Data Lake Storage Gen2

The organization wants all ingested data to be governed by Unity Catalog, to minimize data engineering effort for schema changes, and to use serverless wherever possible to reduce operational overhead.
Which ingestion strategy meets these requirements?

A. Use Lakeflow Connect managed connectors for Salesforce and SQL Server, and Auto Loader for ADLS files, all within Lakeflow Spark Declarative Pipelines.

B. Use Lakeflow Connect managed connectors for all three sources, as they provide consistent serverless execution and Unity Catalog integration across all data source types.

C. Use partner connectors (Fivetran) for all three sources via Partner Connect, as they provide the most comprehensive integration with Unity Catalog and automated schema evolution.

D. Use Auto Loader for all three sources with custom Python code to handle Salesforce API calls and SQL server CDC extraction, then write to Unity Catalog tables.

Answer: A

Explanation:

Correct Answer: A. Use Lakeflow Connect managed connectors for Salesforce and SQL Server, and Auto Loader for ADLS files, all within Lakeflow Spark Declarative Pipelines.

Why A is correct:

This approach uses the most appropriate ingestion technology for each source:

Lakeflow Connect provides managed connectors for **Salesforce** and **SQL Server**, supporting features such as OAuth authentication, CDC, schema evolution, and serverless execution where available.
Auto Loader is the recommended solution for continuously ingesting JSON files from **Azure Data Lake Storage Gen2**, with automatic schema inference and schema evolution.
Writing the data into **Unity Catalog-managed tables** ensures governance, lineage, and immediate availability for analytics.

This combination minimizes operational overhead while supporting frequent schema changes.

A. Use Lakeflow Connect managed connectors for Salesforce and SQL Server, and Auto Loader for ADLS files, all within Lakeflow Spark Declarative Pipelines. — Correct

This is the recommended architecture. It combines managed connectors for enterprise systems with Auto Loader for cloud storage files, while integrating seamlessly with Unity Catalog.

B. Use Lakeflow Connect managed connectors for all three sources, as they provide consistent serverless execution and Unity Catalog integration across all data source types. — Incorrect

Lakeflow Connect is designed for supported databases and SaaS applications. It is **not** the recommended ingestion mechanism for JSON files stored in ADLS Gen2. Auto Loader is the appropriate choice.

C. Use partner connectors (Fivetran) for all three sources via Partner Connect, as they provide the most comprehensive integration with Unity Catalog and automated schema evolution. — Incorrect

Partner connectors are a valid option, but they introduce an additional third-party dependency. The requirement is to minimize engineering effort while using Databricks-native, serverless solutions where possible.

D. Use Auto Loader for all three sources with custom Python code to handle Salesforce API calls and SQL Server CDC extraction, then write to Unity Catalog tables. — Incorrect

Auto Loader is intended for ingesting files from cloud storage. It is not designed to extract data directly from Salesforce APIs or SQL Server CDC, and custom code would significantly increase operational complexity.

Question: 394 CertyIQ

A data engineer uploads a CSV file using the "Create or modify a table using file upload" option in Databricks. To avoid incorrect schema inference, they disable Automatically detect column types before creating a Unity Catalog-managed table.
What is the outcome?

A. All columns are created with the STRING data type.
B. Numeric columns are inferred as numeric types; others are STRING.
C. Previously inferred column types are preserved.
D. Table creation fails because column types are not specified.

Answer: A

Explanation:

Correct Answer: A. All columns are created with the STRING data type.

Why A is correct:

When **Automatically detect column types** is disabled during a CSV file upload in Databricks, the system does **not** infer data types from the file contents. Instead, it creates every column using the **STRING** data type.

This avoids incorrect type inference and allows data types to be adjusted later if needed.

A. All columns are created with the STRING data type. — Correct

When automatic type detection is disabled, Databricks creates all columns as **STRING**, regardless of the values in the CSV file.

B. Numeric columns are inferred as numeric types; others are STRING. — Incorrect

With automatic type detection disabled, **no type inference** occurs. Even numeric columns are created as **STRING**.

C. Previously inferred column types are preserved. — Incorrect

There are no previously inferred column types when creating a new table from a file upload.

D. Table creation fails because column types are not specified. — Incorrect

Databricks does not require manual data type definitions in this scenario. It successfully creates the table by assigning the **STRING** data type to all columns.

Question: 395 CertyIQ

A data engineer has developed a Python notebook in a Databricks workspace and has configured it to run as a scheduled Job to process daily sales data. How is the storage and execution of this notebook managed within the Databricks architecture?

A. The notebook is stored securely and encrypted in Unity Catalog, and the code executes in Delta Lake when the job runs.

B. The notebook is stored securely and encrypted in the control plane, and the code executes in the compute plane when the job runs.

C. The notebook is stored unencrypted in the workspace storage bucket, and the code executes on the cluster driver node in the compute plane when the job runs.

D. The notebook is stored securely and encrypted in the compute plane, and the code executes in the control plane when the job runs.

Answer: B

Explanation:

Correct Answer: B. The notebook is stored securely and encrypted in the control plane, and the code executes in the compute plane when the job runs.

Why B is correct:

In the Databricks architecture:

Notebooks, job configurations, and workspace metadata are stored securely and encrypted in the **control plane**.
When a scheduled Job runs, the notebook is executed on the **compute plane** (the Databricks cluster), where the driver and executors process the code and data.

This separation of storage and execution is a fundamental aspect of the Databricks architecture.

A. The notebook is stored securely and encrypted in Unity Catalog, and the code executes in Delta Lake when the job runs. — Incorrect

Unity Catalog is used for governing data assets such as tables, views, and models. It does not store notebooks, and Delta Lake is a storage format, not an execution environment.

B. The notebook is stored securely and encrypted in the control plane, and the code executes in the compute plane when the job runs. — Correct

This accurately describes how Databricks stores notebooks and executes scheduled Jobs.

C. The notebook is stored unencrypted in the workspace storage bucket, and the code executes on the cluster driver node in the compute plane when the job runs. — Incorrect

Databricks stores notebooks **securely and encrypted**. They are not stored unencrypted in a workspace storage bucket.

D. The notebook is stored securely and encrypted in the compute plane, and the code executes in the control plane when the job runs. — Incorrect

The roles are reversed. Notebooks are stored in the **control plane**, while execution occurs in the **compute plane**.

Question: 396 CertyIQ

A data engineer needs to find all customers who have never placed an order. The customers table contains customer_id, customer_name, and other customer attributes. The orders table contains order_id, customer_id, and other order attributes. Both tables share customer_id as the join key. The result must include only customer columns with no order-related columns in the output.
Which expression exactly achieves this requirement?

A. Inner join, then filter for NULL order_id
B. Left anti-join from customers to orders
C. Left outer join from orders to customers
D. Full outer join, then filter where order id IS NULL

Answer: B

Explanation:

Correct Answer: B. Left anti-join from customers to orders

Why B is correct:

A **left anti-join** returns only the rows from the left DataFrame (**customers**) that do **not** have a matching row in the right DataFrame (**orders**).

Since the requirement is to find customers who have **never placed an order** and return **only customer columns**, a left anti-join is the most efficient and correct operation.

A. Inner join, then filter for NULL order_id — Incorrect

An inner join returns only matching rows between the two tables. Since all returned rows have matching orders, **order_id** cannot be NULL.

B. Left anti-join from customers to orders — Correct

A left anti-join returns customers who have no matching records in the **orders** table and includes only customer columns in the result.

C. Left outer join from orders to customers — Incorrect

Starting from the **orders** table returns customers who have placed orders. It cannot identify customers who have never placed an order.

D. Full outer join, then filter where order_id IS NULL — Incorrect

Although this could identify customers without orders, it also includes unmatched rows from both tables and is less efficient than a left anti-join. It does not directly satisfy the requirement of returning only customer columns.

Question: 397 CertyIQ

A data engineer needs to ingest JSON change data from Salesforce into Unity Catalog-governed Delta tables with a low-code, fully managed experience.
Which Databricks capability should the data engineer use?

A. A Lakeflow Connect managed Salesforce connector that writes into Unity Catalog Delta tables
B. Auto Loader reading JSON files from an S3 bucket exported by Salesforce
C. Auto Loader with a file arrival trigger on DBFS
D. Use the Salesforce REST API in a PySpark notebook to query changed records and write them to Delta tables

Answer: A

Explanation:

Correct Answer: A. A Lakeflow Connect managed Salesforce connector that writes into Unity Catalog Delta tables

Why A is correct:

Lakeflow Connect provides a **fully managed, low-code** connector for Salesforce. It supports ingesting Salesforce data directly into **Unity Catalog-governed Delta tables**, minimizing operational overhead while integrating with Databricks governance and lineage features.

This is the recommended approach for Salesforce data ingestion.

A. A Lakeflow Connect managed Salesforce connector that writes into Unity Catalog Delta tables — Correct

Lakeflow Connect provides a managed, low-code solution for ingesting Salesforce data directly into Unity Catalog-managed Delta tables with minimal engineering effort.

B. Auto Loader reading JSON files from an S3 bucket exported by Salesforce — Incorrect

Auto Loader is designed for ingesting files from cloud storage. It is not the recommended solution for directly

connecting to Salesforce.

C. Auto Loader with a file arrival trigger on DBFS — Incorrect

Auto Loader can monitor files in supported cloud storage, but it is not intended for direct Salesforce integration, and DBFS is not the appropriate source in this scenario.

D. Use the Salesforce REST API in a PySpark notebook to query changed records and write them to Delta tables — Incorrect

Although this approach works, it requires custom code and ongoing maintenance. The requirement is for a **low-code, fully managed** solution, which is provided by Lakeflow Connect.

Question: 398 CertyIQ

A data engineer is designing a Gold layer table for a BI team that needs to query complex aggregations. The requirements state that the data should be precomputed to ensure low latency for dashboard users, but the underlying data changes only once every few hours.
Which object type should be implemented in Unity Catalog to meet these requirements?

A. Streaming table
B. Materialized view
C. External table
D. View

Answer: B

Explanation:

Correct Answer: B. Materialized view

Why B is correct:

A **materialized view** stores the **precomputed results** of a query. Since the underlying data changes only every few hours, the materialized view can be refreshed periodically, allowing BI users to query pre-aggregated data with **low latency** and consistent performance.

This makes it the ideal choice for Gold layer analytics.

A. Streaming table — Incorrect

A streaming table is designed for continuous or near real-time data ingestion. It is not intended to provide precomputed aggregations for BI workloads.

B. Materialized view — Correct

A materialized view precomputes and stores aggregation results, providing fast query performance for dashboard users while refreshing only when the underlying data changes.

C. External table — Incorrect

An external table stores data outside of Databricks-managed storage. It does not precompute aggregations or improve query performance for BI users.

D. View — Incorrect

A standard view stores only the SQL definition. Every query recomputes the aggregations from the underlying tables, resulting in higher query latency compared to a materialized view.

Question: 399 CertyIQ

A data engineering team ingests customer transaction data from three enterprise sources (SQL database, S3, and Kafka stream) into Unity Catalog tables. The data must maintain lineage and support compliance audits requiring historical snapshots.
Which Lakeflow Connect configuration satisfies both the governance and audit requirements?

A. Separate connectors per source with CDC, individual Unity Catalog schemas per source, and table versioning enabled for audit compliance.

B. Single connector with batch scheduling, shared Unity Catalog schema, and incremental snapshots stored externally.

C. Single Lakeflow connector with all three sources, direct Delta writes, and disabled table versioning to reduce storage costs.

D. Separate connectors with full-load sync only, archival of historical data, and external tables for audit queries.

Answer: A

Explanation:

Correct Answer: A. Separate connectors per source with CDC, individual Unity Catalog schemas per source, and table versioning enabled for audit compliance.

Why A is correct:

Using **separate Lakeflow Connect connectors** for each enterprise source allows each system (SQL database, S3, and Kafka) to be managed independently. **Change Data Capture (CDC)** preserves incremental changes, while **Unity Catalog** provides governance and data lineage. **Delta table versioning** enables historical snapshots through the transaction log, supporting compliance and audit requirements.

A. Separate connectors per source with CDC, individual Unity Catalog schemas per source, and table versioning enabled for audit compliance. — Correct

This approach provides governed ingestion, preserves lineage, captures incremental changes through CDC, and supports historical snapshots for compliance audits.

B. Single connector with batch scheduling, shared Unity Catalog schema, and incremental snapshots stored externally. — Incorrect

A single connector cannot manage all three source types in this manner, and storing snapshots externally bypasses Unity Catalog governance and Delta table versioning.

C. Single Lakeflow connector with all three sources, direct Delta writes, and disabled table versioning to reduce storage costs. — Incorrect

Lakeflow Connect does not use a single connector for these different source types, and disabling table versioning removes the historical snapshots required for compliance.

D. Separate connectors with full-load sync only, archival of historical data, and external tables for audit queries. — Incorrect

Using only full-load synchronization is inefficient and does not preserve incremental changes like CDC. External tables alone do not provide the built-in governance and version history needed for audit compliance.

Question: 400 CertyIQ

A data engineer is designing a Databricks Job that includes a Notebook task for data cleansing and a SQL Query task for generating a summary report. The summary report must only execute after the cleansing task completes successfully.
How should the dependency be configured in the Lakeflow Jobs task graph?

A. Set both tasks to run on the same schedule with a five-minute start delay for the report.
B. Place both tasks in the same notebook and use a standard Python function call.
C. Configure the cleansing task to use an 'on Failure' trigger to start the summary report.
D. Select the cleansing task as a 'Run if' dependency in the summary report task configuration.

Answer: D

Explanation:

Correct Answer: D. Select the cleansing task as a 'Run if' dependency in the summary report task configuration.

Why D is correct:

In **Lakeflow Jobs**, task execution order is controlled through **task dependencies**. To ensure the **SQL Query** task runs **only after** the **Notebook** task completes successfully, configure the SQL Query task to depend on the Notebook task using the **Run if** condition (for example, **All succeeded**).

This creates the required DAG dependency.

A. Set both tasks to run on the same schedule with a five-minute start delay for the report. — Incorrect

A time delay does not guarantee that the cleansing task has completed successfully. The report could start before the cleansing task finishes or even if it fails.

B. Place both tasks in the same notebook and use a standard Python function call. — Incorrect

Notebook and SQL Query tasks are intended to be separate tasks within a Lakeflow Job. Combining them into one notebook removes the benefits of task-level orchestration.

C. Configure the cleansing task to use an 'on Failure' trigger to start the summary report. — Incorrect

An **on Failure** condition causes the report to run only if the cleansing task fails, which is the opposite of the requirement.

D. Select the cleansing task as a 'Run if' dependency in the summary report task configuration. — Correct

This ensures the summary report executes only after the cleansing task completes successfully, implementing the required task dependency in the Lakeflow Jobs DAG.

Question: 401 CertyIQ

A data engineer is cleaning a bronze table that receives the same customer records from multiple source systems. Duplicate rows have the same customer_id and email but different ingestion_timestamp values. The silver table should contain only one record per unique combination of customer_id and email.
Which PySpark operation correctly deduplicates based on the business keys?

A. df.groupBy('customer_id', 'email').agg(max('ingestion_timestamp').alias('latest_to'))

B. df.dropDuplicates(['customer_id', 'email'])

C. df.distinct()

D. df.select("customer_id", "email").distinct()

Answer: B

Explanation:

Correct Answer: B. df.dropDuplicates(['customer_id', 'email'])

Why B is correct:

The requirement is to keep **only one record** for each unique combination of **customer_id** and **email**, regardless of differences in **ingestion_timestamp**.

The **dropDuplicates()** method removes duplicate rows based on the specified columns (business keys), making it the correct choice for deduplicating the data.

A. df.groupBy('customer_id', 'email').agg(max('ingestion_timestamp').alias('latest_to')) — Incorrect

This groups the data and returns only the grouped columns and the maximum timestamp. It does not preserve the original record and is intended for aggregation rather than deduplication.

B. df.dropDuplicates(['customer_id', 'email']) — Correct

This removes duplicate rows based on the **customer_id** and **email** columns, leaving one record for each unique business key.

C. df.distinct() — Incorrect

distinct() removes only rows that are completely identical across all columns. Since **ingestion_timestamp** differs, these rows would not be considered duplicates.

D. df.select("customer_id", "email").distinct() — Incorrect

This returns only the unique **customer_id** and **email** columns, discarding all other columns from the original DataFrame. It does not produce the required deduplicated records.

Question: 402 CertyIQ

A data engineering team runs nightly batch ETL pipelines that transform large volumes of data stored in Delta Lake. The workloads are predictable and run on a fixed schedule. The team wants automatic scaling and cost efficiency, paying only for compute used during job execution.
Which Databricks compute option is appropriate for this workload?

A. A single-node compute cluster configured without autoscaling.

B. An all-purpose compute cluster configured with autoscaling.

C. A Databricks SQL warehouse configured for serverless execution.

D. A job compute cluster triggered by a scheduled workflow.

Answer: D

Explanation:

Correct Answer: D. A job compute cluster triggered by a scheduled workflow.

Why D is correct:

The workload consists of **nightly batch ETL pipelines** that run on a **fixed schedule**. A **job compute cluster** is created when the scheduled job starts and is automatically terminated after the job completes.

This provides:

Automatic scaling for the workload.
Cost efficiency by paying only for compute during job execution.
Minimal operational overhead for scheduled ETL jobs.

A. A single-node compute cluster configured without autoscaling. — Incorrect

A single-node cluster is not suitable for large-scale ETL workloads and does not provide automatic scaling.

B. An all-purpose compute cluster configured with autoscaling. — Incorrect

All-purpose clusters are designed for interactive development and notebooks. They are typically more expensive for scheduled production ETL jobs because they may remain running when idle.

C. A Databricks SQL warehouse configured for serverless execution. — Incorrect

SQL warehouses are optimized for SQL analytics and BI workloads, not for general-purpose Spark ETL pipelines.

D. A job compute cluster triggered by a scheduled workflow. — Correct

A job compute cluster is the recommended choice for scheduled batch ETL workloads. It starts only when the workflow runs, scales as needed, and automatically terminates after completion, minimizing costs.

Question: 403 CertyIQ

A data engineer needs to support a Unity Catalog Delta table catalog.schema.sales that is currently registered as an external table.
New requirements state:
• Automatic file cleanup after a drop is handled by the platform.
• Minimal downtime during migration while keeping the same table name, permissions, and history.
• Enforced name-based access through Unity Catalog (no path-based access).
Which action should the data engineer take to meet the requirements?

A. Run ALTER TABLE catalog.schema.sales SET MANAGED to convert and meet the goals
B. Run DROP TABLE catalog.schema.sales PURGE to delete files and meet goals
C. Run ALTER TABLE catalog.schema.sales SET LOCATION to switch external paths now
D. Create a new managed table with CTAS and migrate workloads to that table today

Answer: A

Explanation:

Correct Answer: A. Run ALTER TABLE catalog.schema.sales SET MANAGED to convert and meet the goals

Why A is correct:

Converting the **Unity Catalog external table** to a **managed table** using **ALTER TABLE ... SET MANAGED**

meets all the stated requirements:

Automatic file cleanup is handled by Databricks when the managed table is dropped.
The conversion keeps the **same table name**, minimizing downtime.
Existing **permissions** and **table history** are preserved.
Access continues to be enforced through **Unity Catalog** using the table name rather than direct storage paths.

A. Run ALTER TABLE catalog.schema.sales SET MANAGED to convert and meet the goals — Correct

This converts the external table to a managed table while preserving the table name, permissions, and history, and enables platform-managed storage and cleanup.

B. Run DROP TABLE catalog.schema.sales PURGE to delete files and meet goals — Incorrect

This deletes the table and its data. It does not preserve the table name, permissions, history, or minimize downtime.

C. Run ALTER TABLE catalog.schema.sales SET LOCATION to switch external paths now — Incorrect

Changing the table location keeps the table as an **external table**. It does not convert it to a managed table or enable automatic file cleanup.

D. Create a new managed table with CTAS and migrate workloads to that table today — Incorrect

Creating a new table changes the migration process, requires updating workloads, and does not preserve the existing table name, permissions, or history with minimal downtime.

Question: 404 CertyIQ

A data engineer is building a Silver layer pipeline that should only execute after the Bronze layer table has been successfully updated with new records.
Which trigger type in Databricks Jobs is specifically designed to orchestrate this dependency by reacting to changes in the upstream Unity Catalog table?

A. Continuous trigger
B. File arrival trigger
C. Scheduled trigger
D. Table update trigger

Answer: D

Explanation:

Correct Answer: D. Table update trigger

Why D is correct:

A **Table update trigger** is designed to start a Databricks Job when an upstream **Unity Catalog table** is updated. In this scenario, the Silver layer pipeline should run only after the Bronze table has been successfully updated with new records.

This event-driven approach ensures the Silver pipeline processes only fresh data without relying on fixed schedules.

A. Continuous trigger — Incorrect

A Continuous trigger keeps the job running to process data continuously. It does not specifically react to updates in a Unity Catalog table.

B. File arrival trigger — Incorrect

A File Arrival trigger starts a job when new files arrive in cloud storage. It does not monitor changes to Unity Catalog tables.

C. Scheduled trigger — Incorrect

A Scheduled trigger runs jobs at fixed time intervals regardless of whether the Bronze table has been updated.

D. Table update trigger — Correct

A Table update trigger automatically starts the Silver layer pipeline whenever the upstream Unity Catalog Bronze table is successfully updated, making it the appropriate trigger for this dependency.

Question: 405 CertyIQ

A pipeline uses COPY INTO to ingest CSV files from cloud object storage into a Unity Catalog Delta table. Some files are occasionally re-uploaded with corrections using the same filename.
The engineer needs the corrected data ingested as soon as it occurs.
What should the engineer do?

A. Overwrite the target Delta table on each run
B. Load corrections as new files and apply updates to the target table
C. Recreate the target table before each run
D. Reload all files from the source path on each run

Answer: B

Explanation:

Correct Answer: B. Load corrections as new files and apply updates to the target table.

Why B is correct:

COPY INTO is **idempotent**—it tracks which files have already been ingested and **does not reprocess a file with the same name**, even if its contents have changed.

Therefore, if corrected data needs to be ingested, the recommended approach is to:

Save the corrected data as a **new file** (with a different filename), and
Apply the necessary **updates (for example, MERGE)** to the target Delta table.

This preserves incremental processing while ensuring corrected records are reflected in the target table.

A. Overwrite the target Delta table on each run — Incorrect

Overwriting the entire table is inefficient and unnecessary, especially for large datasets.

B. Load corrections as new files and apply updates to the target table — Correct

This is the recommended approach because COPY INTO does not reload previously ingested files with the same filename.

C. Recreate the target table before each run — Incorrect

Recreating the table removes existing data and metadata and is not an appropriate solution for handling corrected files.

D. Reload all files from the source path on each run — Incorrect

COPY INTO maintains a record of processed files and will skip files it has already loaded. Reloading the source path does not cause previously ingested files with the same name to be processed again.

Question: 406 CertyIQ

A developer is building a data pipeline that processes records from a bronze table into a silver table. The bronze table, bronze_events, contains duplicate records due to at-least-once delivery guarantees from the upstream ingestion system. The developer writes the following PySpark code to prepare the data:
deduped_dfdf.dropDuplicates()
deduped_df.summary("count", "mean", "stddev").show()
from pyspark.sql.functions import approx_count_distinct
deduped_df.select(approx_count_distinct("user_id")).show()
The developer notices that after running dropDuplicates () without arguments, some rows that differ only in the event_timestamp column remain. The developer wants to de-duplicate based only on user_id and event_type, keeping one row per unique combination of these two columns.
Which code change achieves this deduplication requirement?

A. Replace df.dropDuplicates () with df. dropDuplicates (["user_id", "event_type"])
B. Replace df. dropDuplicates () with df. drop ("event_timestamp").dropDuplicates ()
C. Replace df. dropDuplicates () with df. distinct().select("user_id", "event_type")
D. Replace df. dropDuplicates () with df.groupBy("user_id", "event_type").count()

Answer: A

Explanation:

Correct Answer: A. Replace df.dropDuplicates() with df.dropDuplicates(["user_id", "event_type"])

Why A is correct:

By default, **dropDuplicates()** removes only rows that are identical across **all columns**. Since the duplicate records differ in the **event_timestamp** column, they are not considered duplicates.

To deduplicate based only on the business keys (**user_id** and **event_type**), specify those columns in **dropDuplicates()**. Spark will keep one row for each unique combination of those columns.

A. Replace df.dropDuplicates() with df.dropDuplicates(["user_id", "event_type"]) — Correct

This removes duplicate rows based only on **user_id** and **event_type**, regardless of differences in **event_timestamp**.

B. Replace df.dropDuplicates() with df.drop("event_timestamp").dropDuplicates() — Incorrect

Dropping the **event_timestamp** column removes potentially valuable data. It also permanently deletes the column rather than deduplicating while preserving it.

C. Replace df.dropDuplicates() with df.distinct().select("user_id", "event_type") — Incorrect

distinct() behaves like **dropDuplicates()** on all selected columns. Additionally, selecting only **user_id** and **event_type** discards all other columns.

D. Replace df.dropDuplicates() with df.groupBy("user_id", "event_type").count() — Incorrect

This aggregates the data and returns counts for each group instead of returning one representative record for each unique **user_id** and **event_type** combination.

Question: 407 CertyIQ

A data engineer ingests semi-structured JSON logs into a Delta table using Auto Loader with schema evolution enabled. A new string field userAgent appears in the JSON.
What happens to the new userAgent field?

A. All existing data is backfilled with a default value for userAgent and made non-nullable.
B. The new userAgent field is silently ignored, and its data is dropped for all new files.
C. Ingestion fails with a schema-mismatch error until the table is altered.
D. The new userAgent field is added as a new nullable column in the Delta table, populated for new data.

Answer: D

Explanation:

Correct Answer: D. The new userAgent field is added as a new nullable column in the Delta table, populated for new data.

Why D is correct:

When **Auto Loader** is configured with **schema evolution enabled**, it automatically detects new fields in incoming JSON data.

If a new field such as **userAgent** appears:

The field is **added automatically** to the Delta table schema.
The new column is created as **nullable**.
New records populate the **userAgent** column.
Existing rows have **NULL** for the new column.

This allows the pipeline to continue processing without manual schema updates.

A. All existing data is backfilled with a default value for userAgent and made non-nullable. — Incorrect

Existing records are **not backfilled** with default values. The new column is added as **nullable**, and existing rows contain **NULL**.

B. The new userAgent field is silently ignored, and its data is dropped for all new files. — Incorrect

With schema evolution enabled, Auto Loader detects and adds new fields instead of ignoring them.

C. Ingestion fails with a schema-mismatch error until the table is altered. — Incorrect

Schema evolution prevents this failure by automatically updating the schema to include the new field.

D. The new userAgent field is added as a new nullable column in the Delta table, populated for new data. — Correct

This is the expected behavior of Auto Loader with schema evolution enabled. The new column is added automatically, populated for incoming records, and existing rows contain **NULL** for the new field.

Question: 408 CertyIQ

A Databricks Job with four sequential tasks is executed. The job failed at Task 3. After the root cause is fixed, the data engineer needs to complete the workflow without re-running the successful Tasks 1 and 2.
Which action in the Job Run details page allows the engineer to resume execution from the point of failure?

A. Repair run
B. Run now
C. Clone task
D. Reset job

Answer: A

Explanation:

Correct Answer: A. Repair run

Why A is correct:

The **Repair run** feature in Databricks Jobs allows a failed job to **resume from the point of failure**. After fixing the root cause, only the failed task and its downstream dependent tasks are re-executed, while previously successful tasks are skipped.

In this scenario:

Task 1 Succeeded
Task 2 Succeeded
Task 3 Failed
Task 4 Not executed

Using **Repair run** reruns **Task 3** and then **Task 4**, without rerunning Tasks 1 and 2.

A. Repair run — Correct

This resumes the workflow from the failed task, avoiding re-execution of successful tasks.

B. Run now — Incorrect

Run now starts a completely new job run and executes **all tasks** from the beginning.

C. Clone task — Incorrect

Cloning creates a copy of a task for editing. It does not resume a failed job run.

D. Reset job — Incorrect

Resetting updates the job configuration. It does not continue execution from a failed task or resume a previous run.

Question: 409 CertyIQ

A data engineering team is building a new data transformation notebook. During development, they need fast testing, quick code changes, and easy debugging. Later, the notebook will run nightly as a scheduled job without human intervention. They want to optimize for development speed.
Which compute should they use during development?

A. Use classic job compute.
B. Use interactive compute.
C. Use instance pools.
D. Use SQL warehouses.

Answer: B

Explanation:

Correct Answer: B. Use interactive compute.

Why B is correct:

During development, data engineers need fast testing, quick code changes, interactive notebook execution, and easy debugging. **Interactive compute** is designed specifically for these tasks. It provides a persistent environment where notebooks can be run repeatedly, results can be inspected immediately, and code can be modified without waiting for a new cluster to start.

After development is complete, the notebook can be scheduled to run as a nightly job using **job compute**.

A. Use classic job compute. — Incorrect

Job compute is optimized for automated production workloads. It starts when a job begins and terminates after the job completes, making it less suitable for interactive development and debugging.

B. Use interactive compute. — Correct

Interactive compute provides the best experience for notebook development by enabling rapid testing, debugging, and code iteration.

C. Use instance pools. — Incorrect

Instance pools reduce cluster startup time but are not a compute type. They are used together with interactive or job compute and do not replace them.

D. Use SQL warehouses. — Incorrect

SQL warehouses are designed for SQL analytics and BI workloads, not for developing and debugging PySpark notebooks.

Question: 410 CertyIQ

A data engineer investigates a slow stage in the Databricks Spark UI. The engineer scrolls down to the Summary Metrics section for task durations, and they observe for that stage:

- 25th percentile duration: 40 seconds
- Median duration: 45 seconds
- 75th percentile duration: 50 seconds
- Max duration: 80 seconds

What is the interpretation of these metrics?

A. There is a GC issue.
B. The cluster is under-provisioned.
C. The stage is healthy.
D. The stage likely suffers from data skew.

Answer: C

Explanation:

Correct Answer: C. The stage is healthy.

Why C is correct:

The task duration statistics are:

25th percentile: **40 seconds**
Median: **45 seconds**
75th percentile: **50 seconds**
Maximum: **80 seconds**

The **25th, 50th, and 75th percentiles are very close together**, indicating that the vast majority of tasks completed in a consistent amount of time. Although the maximum duration is higher, a single slower task does not necessarily indicate a performance problem or data skew.

Databricks recommends looking for **large differences across many tasks**, not just a single maximum value. Based on these summary metrics alone, the stage appears to be **healthy**.

A. There is a GC issue. — Incorrect

These metrics show only task durations. They do not provide evidence of excessive garbage collection.

B. The cluster is under-provisioned. — Incorrect

If the cluster were under-provisioned, task durations would generally be high across the board rather than being tightly grouped.

C. The stage is healthy. — Correct

Most tasks complete within a narrow range of **40–50 seconds**. A single maximum duration of **80 seconds** is not sufficient evidence to conclude there is a performance issue.

D. The stage likely suffers from data skew. — Incorrect

Data skew is typically indicated by a much larger gap between the majority of task durations and a significant number of long-running tasks. The provided metrics alone do not conclusively indicate skew, so the expected exam answer is **C**.

Question: 411 CertyIQ

A data engineer is designing a bronze-to-silver pipeline on the Databricks Data Intelligence Platform. The source system sends daily CSV files, and over time, new optional columns are added.
The data engineer wants a storage format and table feature that together ensure the following:

- The table prevents writes that do not conform to the defined schema.
- The table can evolve its schema to include new optional columns without having to manually recreate the table.
- Previous versions of the table can be queried later for debugging and auditing.

Which solution fulfills the requirements?

A. Use a Parquet table with Spark's default schema inference and rely on re-running the job when the schema changes.

B. Use an external table with Auto Loader schema inference for CSV files.

C. Use a Delta table with schema enforcement and recreate the table whenever new columns are added.

D. Use a Delta table with its native capabilities.

Answer: D

Explanation:

Correct Answer: D. Use a Delta table with its native capabilities.

Why D is correct:

Delta Lake provides all the required features:

Schema enforcement prevents writes that do not conform to the table schema.
Schema evolution allows new optional columns to be added without recreating the table.
Time Travel enables querying previous versions of the table for debugging, auditing, and recovery.

These native Delta capabilities make it the recommended storage format for bronze-to-silver pipelines.

A. Use a Parquet table with Spark's default schema inference and rely on re-running the job when the schema changes. — Incorrect

Parquet does not provide Delta Lake's schema enforcement, schema evolution, or time travel capabilities.

B. Use an external table with Auto Loader schema inference for CSV files. — Incorrect

Auto Loader can detect schema changes during ingestion, but being an external table alone does not provide Delta Lake's schema enforcement and time travel capabilities.

C. Use a Delta table with schema enforcement and recreate the table whenever new columns are added. — Incorrect

Delta Lake supports **schema evolution**, so recreating the table when new columns are added is unnecessary.

D. Use a Delta table with its native capabilities. — Correct

Delta Lake natively provides schema enforcement, schema evolution, and time travel, satisfying all the stated requirements.

Question: 412 CertyIQ

A data engineering team needs to incrementally ingest customer transactions from a SaaS app into Data Intelligence Platform with:

- Built-in change data capture (including updates and deletes)
- Automatic schema evolution
- Serverless execution with retries and minimal maintenance
- OAuth support and basic monitoring

Which solution meets all of these requirements?

A. Use Auto Loader on exported SaaS files and an SDP that calls the SaaS APIs, handling CDC, schema changes, OAuth, and retries in your own code.

B. Use a Lakeflow Connect managed connector for the SaaS app with built-in CDC, schema evolution, OAuth, and serverless compute with retries, then process downstream with Lakeflow Spark Declarative Pipelines.

C. Schedule batch jobs per region that call SaaS REST APIs and write directly to silver tables with custom CDC and schema evolution, and use system tables for ad hoc basic monitoring.

D. Use a Lakeflow Connect standard connector, implement MERGE-based CDC, schema evolution, and retries in workflows, and set up a dedicated monitoring dashboard.

Answer: B

Explanation:

Correct Answer: B. Use a Lakeflow Connect managed connector for the SaaS app with built-in CDC, schema evolution, OAuth, and serverless compute with retries, then process downstream with Lakeflow Spark Declarative Pipelines.

Why B is correct:

A **Lakeflow Connect managed connector** is designed for low-maintenance ingestion from supported SaaS applications. It provides **built-in Change Data Capture (CDC)** for inserts, updates, and deletes, supports **automatic schema evolution**, uses **OAuth** for authentication, runs on **serverless compute** with automatic retries, and includes basic monitoring. The ingested data can then be transformed using **Lakeflow Spark Declarative Pipelines**, satisfying all the stated requirements.

A. Use Auto Loader on exported SaaS files and an SDP that calls the SaaS APIs, handling CDC, schema changes, OAuth, and retries in your own code. — Incorrect

This approach requires custom implementation for CDC, schema evolution, OAuth authentication, and retries, resulting in significantly more maintenance.

B. Use a Lakeflow Connect managed connector for the SaaS app with built-in CDC, schema evolution, OAuth, and serverless compute with retries, then process downstream with Lakeflow Spark Declarative Pipelines. — Correct

This solution provides built-in CDC, automatic schema evolution, OAuth authentication, serverless execution with retries, and minimal operational overhead.

C. Schedule batch jobs per region that call SaaS REST APIs and write directly to silver tables with custom CDC and schema evolution, and use system tables for ad hoc basic monitoring. — Incorrect

Although this solution works, it depends on custom code for CDC, schema evolution, authentication, and retry logic, increasing operational complexity.

D. Use a Lakeflow Connect standard connector, implement MERGE-based CDC, schema evolution, and retries in workflows, and set up a dedicated monitoring dashboard. — Incorrect

A standard connector requires custom implementation of CDC, schema evolution, and retries. It does not provide the fully managed capabilities requested in the requirements.

Question: 413 CertyIQ

A data engineer triggers a scheduled job but finds that the new run was not executed. The run history shows the run was skipped with a concurrency-related queue message.
Which configuration should the engineer investigate?

A. The task-level timeout_seconds configuration
B. The cluster spot-instance availability in the cloud provider
C. The SQL warehouse permissions for the task
D. The job-level max_concurrent_runs setting and whether queueing is enabled

Answer: D

Explanation:

Correct Answer: D. The job-level max_concurrent_runs setting and whether queueing is enabled.

Why D is correct:

The run history indicates that the job was **skipped due to a concurrency-related queue message**. This behavior is controlled by the job's **max_concurrent_runs** setting and whether **queueing** is enabled. If the maximum number of concurrent runs has already been reached, additional scheduled runs may be queued or skipped depending on the job configuration.

A. The task-level timeout_seconds configuration — Incorrect

The **timeout_seconds** setting controls how long a task can run before it is terminated. It does not affect whether a scheduled run is skipped because of concurrency.

B. The cluster spot-instance availability in the cloud provider — Incorrect

Spot instance availability can delay cluster startup, but it does not produce a concurrency-related queue message.

C. The SQL warehouse permissions for the task — Incorrect

Permission issues cause authorization errors during execution, not skipped runs due to concurrency limits.

D. The job-level max_concurrent_runs setting and whether queueing is enabled — Correct

The **max_concurrent_runs** setting limits how many instances of a job can run simultaneously. When this limit is reached, the behavior depends on whether **queueing** is enabled, making this the correct configuration to investigate.

Question: 414

CertyIQ

A data engineering team needs to integrate two data sources into Databricks:
- Clickstream Events: Ingesting 5,000 events per second from an Apache Kafka topic.
- Customer Master Data: Syncing only changed records every 4 hours from a Snowflake database.
- The solution must meet two key requirements:
- Process the streaming clickstream data with a latency of under 30 seconds.
- Prevent the reprocessing of customer master data that has not changed.

Which data ingestion approach meets these requirements?

A. Use a non-incremental Snowflake connector with polling and fetch all data every 4 hours with MERGE operations.

B. Use Structured Streaming for Kafka and Lakeflow Connect managed connector with incremental mode for Snowflake.

C. Use spark.readStream() with Kafka and query Snowflake hourly with a time-based filter.

D. Use Structured Streaming for Kafka and a Lakeflow Connect standard connector with full refresh for Snowflake.

Answer: B

Explanation:

Correct Answer: B. Use Structured Streaming for Kafka and Lakeflow Connect managed connector with

incremental mode for Snowflake.

Why B is correct:

The clickstream events require **low-latency streaming** (under 30 seconds), making **Structured Streaming** the appropriate choice for consuming data from Kafka.

For the Snowflake customer master data, only changed records should be synchronized every 4 hours. A **Lakeflow Connect managed connector** in **incremental mode** supports change-based ingestion, preventing unchanged records from being reprocessed while minimizing operational overhead.

A. Use a non-incremental Snowflake connector with polling and fetch all data every 4 hours with MERGE operations. — Incorrect

This reloads all data every 4 hours, reprocessing unchanged records and increasing compute and storage costs.

B. Use Structured Streaming for Kafka and Lakeflow Connect managed connector with incremental mode for Snowflake. — Correct

This satisfies both requirements by providing low-latency streaming for Kafka and incremental synchronization for Snowflake without reprocessing unchanged data.

C. Use spark.readStream() with Kafka and query Snowflake hourly with a time-based filter. — Incorrect

Although Kafka ingestion is appropriate, querying Snowflake with a time-based filter requires custom logic and does not provide the managed incremental synchronization offered by Lakeflow Connect.

D. Use Structured Streaming for Kafka and a Lakeflow Connect standard connector with full refresh for Snowflake. — Incorrect

A full refresh reloads all customer data every synchronization, violating the requirement to avoid reprocessing unchanged records.

Thank you

Thank you for being so interested in the premium exam material.
I'm glad to hear that you found it informative and helpful.

If you have any feedback or thoughts on the bumps, I would love to hear them.
Your insights can help me improve our writing and better understand our readers.

Best of Luck

You have worked hard to get to this point, and you are well-prepared for the exam
Keep your head up, stay positive, and go show that exam what you're made of!

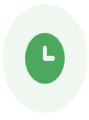
Feedback

More Papers



Future is Secured

100% Pass Guarantee



24/7 Customer Support

Mail us - certyiqofficial@gmail.com



Free Updates

Lifetime Free Updates!

Total: **414 Questions**

Link: https://certyiq.com/papers/databricks/certified-data-engineer-associate